

AUTOBOOK SERIES

에이전트 평가와 관측

비결정적 시스템을 프로덕션에 올리는 법
평가 · 관측 · 거버넌스 · 케이스 · 도입 로드맵

2026

목차

0. 들어가며

1 에이전트 평가의 기초

1.1 에이전트 평가의 출발점

- 1.1.1 에이전트의 정의와 평가 단위
- 1.1.2 비결정성이 만드는 평가의 어려움
- 1.1.3 LLM 평가와 에이전트 평가의 결정적 차이
- 1.1.4 Trajectory 평가와 Final-Answer 평가의 구분

1.2 Failure Mode Taxonomy

- 1.2.1 Tool selection error
- 1.2.2 Planning loop와 무한 반복
- 1.2.3 Hallucinated reasoning
- 1.2.4 Premature termination

1.3 다차원 메트릭 설계

- 1.3.1 단일 메트릭 최적화의 함정
- 1.3.2 Hidden quality와 escalation path

2 Offline 평가 설계

2.1 골든 데이터셋

- 2.1.1 골든 데이터셋의 역할과 한계
- 2.1.2 골든 데이터셋 구축 절차
- 2.1.3 데이터셋 진화와 회귀 테스트

2.2 LLM-as-Judge

- 2.2.1 LLM-as-Judge 동작 원리
- 2.2.2 평가 LLM의 편향과 한계
- 2.2.3 Judge 보정 기법

2.3 Trajectory 평가

- 2.3.1 Trajectory 평가 설계
- 2.3.2 단계별 정확도 측정

2.4 학술 벤치마크

- 2.4.1 τ -bench
- 2.4.2 AgentBench
- 2.4.3 SWE-bench

2.5 평가 환경 설계

- 2.5.1 Eval 환경 isolation
- 2.5.2 Reproducibility와 shared state 위험

3 Online 평가와 관측 스택

3.1 Online 평가

- 3.1.1 실 트래픽 기반 평가
- 3.1.2 Reactive vs Prescriptive 평가

3.1.3 회귀 탐지

3.2 LLM Observability의 필요성

3.2.1 전통적 모니터링과의 차이

3.2.2 Trace, Span, Metric 데이터 모델

3.3 LangSmith 깊게 보기

3.3.1 LangSmith의 trace 모델

3.3.2 LangSmith 평가·실험 통합

3.4 Langfuse 깊게 보기

3.4.1 Langfuse 자체 호스팅과 데이터 주권

3.4.2 Langfuse OpenTelemetry 호환성

3.5 도구 비교와 의사결정

3.5.1 한국 기업의 의사결정 프레임워크

3.5.2 Hybrid 스택 구성 패턴

3.6 OpenTelemetry for GenAI

3.6.1 OpenTelemetry 표준 개요

3.6.2 GenAI Semantic Conventions

3.6.3 표준 기반 무벤더 락인 설계

4 프로덕션 패턴: 신뢰성과 안전

4.1 Tiered Autonomy

4.1.1 자율성 단계 정의

4.1.2 위험도 기반 권한 부여

4.2 Circuit Breaker

4.2.1 Circuit Breaker 동작 원리

4.2.2 에이전트 임계치 설계

4.3 Fallback Routing

4.3.1 Fallback의 종류

4.3.2 모델 라우팅 전략

4.3.3 Graceful Degradation

4.4 Human-in-the-loop 재정의

4.4.1 모든 결 승인 의 한계

4.4.2 임계치 기반 예외 검토

4.4.3 Escalation Path 설계

5 케이스 스터디: 실전에서 배우기

5.1 Anthropic Multi-Agent Research System

5.1.1 Lead + Subagent 아키텍처

5.1.2 Production Tracing 설계

5.1.3 프라이버시 보존 모니터링

5.1.4 Source Quality Heuristic 사례

5.2 Replit Agent

5.2.1 Hybrid 평가 스택의 구성

5.2.2 Computer Use 기반 테스트

5.2.3 Reactive Eval과 가드레일 우회

5.3 JPMorgan LLM Suite

- 5.3.1 중앙 플랫폼 패턴
- 5.3.2 단일 API 뒤의 다중 모델 라우팅
- 5.3.3 측정 가능한 프로세스부터 시작

5.4 Klarna 반면교사

- 5.4.1 성공 지표 뒤의 함정
- 5.4.2 Hybrid 회귀의 교훈

5.5 Palantir AIP - 5차원 거버넌스

- 5.5.1 Model Access 차원
- 5.5.2 Stateless Orchestration 차원
- 5.5.3 Memory Governance 차원
- 5.5.4 Tool Governance 차원
- 5.5.5 Observability 차원

6 실전 적용: 자가 진단과 도입 로드맵

6.1 자가 진단

- 6.1.1 Palantir 5차원 Self-Audit
- 6.1.2 평가·관측 커버리지 점검

6.2 도입 로드맵

- 6.2.1 단계별 도입 우선순위
- 6.2.2 KPI 설계
- 6.2.3 거버넌스와 속도의 균형

6.3 활용

- 6.3.1 시스템 설명 Framework
- 6.3.2 케이스 인용과 Trade-off 설명

0. 들어가며

이 책은 에이전트 평가와 관측: 비결정적 시스템을 프로덕션에 올리는 법을 다룹니다. AI 에이전트의 모델 자체를 만드는 일이 아니라, 그 에이전트가 실제 트래픽 위에서 신뢰할 수 있게 돌아가는지 판단하고, 어디서 무너지는지 보고, 무너지지 않게 받쳐 주는 평가·관측·운영의 관점에서 접근합니다.

Phase 1에서는 에이전트 평가가 일반 LLM 평가와 어떻게 다른지, 대표적인 실패 모드는 무엇인지, 단일 메트릭에 최적화하면 왜 품질이 무너지는지를 정리합니다. Phase 2와 Phase 3에서는 골든 데이터셋·LLM-as-Judge-trajectory 평가·학술 벤치마크로 짜는 offline 평가와, 실 트래픽 기반 online 평가, 그리고 LangSmith·Langfuse·OpenTelemetry로 구성하는 관측 스택을 다룹니다. Phase 4에서는 tiered autonomy, circuit breaker, fallback routing, human-in-the-loop을 결합한 프로덕션 신뢰성 패턴을 짚습니다.

Phase 5는 케이스 스터디입니다. Anthropic Multi-Agent Research System, Replit Agent, JPMorgan LLM Suite, Klarna의 hybrid 회귀, Palantir AIP의 5차원 거버넌스를 차례로 풀어 그 안에서 평가·관측·거버넌스 패턴을 추출합니다. Phase 6에서는 그동안 쌓은 framework로 본인의 시스템을 self-audit하고, 도입 로드맵을 세우고, 면접·문서에서 일관되게 설명하는 법을 다룹니다.

다음 단원에서는 에이전트가 무엇이며 평가의 단위가 어디인지부터 짚습니다.

1 에이전트 평가의 기초

에이전트 평가가 LLM 평가와 어떻게 다른지 설명하고, 대표적 실패 모드를 분류하며, 단일 메트릭 최적화의 함정을 인식할 수 있다.

이 Phase는 다음 섹션으로 구성됩니다.

- 1.1 에이전트 평가의 출발점
 - 1.1.1 에이전트의 정의와 평가 단위
 - 1.1.2 비결정성이 만드는 평가의 어려움
 - 1.1.3 LLM 평가와 에이전트 평가의 결정적 차이
 - 1.1.4 Trajectory 평가와 Final-Answer 평가의 구분
- 1.2 Failure Mode Taxonomy
 - 1.2.1 Tool selection error
 - 1.2.2 Planning loop와 무한 반복
 - 1.2.3 Hallucinated reasoning
 - 1.2.4 Premature termination
- 1.3 다차원 메트릭 설계
 - 1.3.1 단일 메트릭 최적화의 함정
 - 1.3.2 Hidden quality와 escalation path

1.1 에이전트 평가의 출발점

이 섹션의 토픽과 학습 목표는 다음과 같습니다.

1.1.1 에이전트의 정의와 평가 단위 — 에이전트를 모델, 도구, 메모리, 루프 네 가지 구성요소로 분해하고 평가의 단위가 어디인지 설명할 수 있다

1.1.2 비결정성이 만드는 평가의 어려움 — 동일 입력에서 발생하는 비결정성이 디버깅과 평가에 주는 영향을 설명할 수 있다

1.1.3 LLM 평가와 에이전트 평가의 결정적 차이 — LLM 평가와 에이전트 평가의 차이를 측정 대상, 환경, 비용 세 측면에서 비교할 수 있다

1.1.4 Trajectory 평가와 Final-Answer 평가의 구분 — Trajectory eval과 Final-answer eval의 측정 대상과 한계를 비교하고 상황에 맞는 선택을 설명할 수 있다

1.1.1 에이전트의 정의와 평가 단위

인공지능 모델을 평가한다는 말은 오랫동안 단순했습니다. 사용자가 질문을 하나 던지면 모델이 답을 하나 돌려주고, 그 답이 얼마나 정답에 가까운지를 재면 끝이었습니다. 그러나 모델이 검색 도구를 부르고, 파일을 열고, 외부 서비스에 명령을 보내며, 자신의 판단으로 다음 단계를 정하는 방식이 퍼지면서 평가의 대상이 한 번의 답에서 여러 단계의 행동으로 옮겨 가고 있습니다. 이 단원은 그 옮겨 간 평가의 첫걸음으로, 무엇을 에이전트라고 부르며, 그 에이전트의 어디를 어느 단위로 측정해야 하는지를 정리합니다.

먼저 용어부터 풀겠습니다. 이 책에서 **에이전트**란 인공지능 모델이 한 번의 응답에 그치지 않고, 자신의 판단으로 다음에 무엇을 할지 정하면서 도구를 부르고 결과를 받아 다시 판단하는 과정을 반복하는 시스템을 가리킵니다. 핵심은 '자율적 의사결정 루프'라는 표현 안에 있습니다. 사람이 매 단계마다 다음 행동을 정해 주는 것이 아니라, 모델 스스로가 지금까지의 상황을 보고 다음 한 수를 정하며 그 수의 결과를 다시 입력으로 받는 흐름이 반복됩니다.

이 정의를 한 발 더 풀어 보면, 에이전트는 네 가지 구성요소로 분해됩니다. 첫째는 **모델**입니다. 모델은 지금까지의 대화와 도구 결과를 입력받아 다음에 할 행동을 문자열로 내놓는 두뇌의 자리입니다. 둘째는 **도구**입니다. 도구는 모델이 바깥 세계와 만나는 통로로, 검색 함수, 파일 읽기 함수, 데이터베이스 조회 함수 같은 것들이 여기에 속합니다. 셋째는 **메모리**입니다. 메모리는 직전까지의 단계와 그 결과, 사용자가 처음 부탁한 내용 같은 정보를 보관해 다음 판단에 다시 쓸 수 있게 하는 자리입니다. 넷째는 **루프**입니다. 루프는 '모델의 다음 행동을 받는다, 그 행동이 도구 호출이면 도구를 부른다, 결과를 메모리에 넣는다, 다시 모델을 부른다'를 종료 조건이 올 때까지 반복하는 진행 장치입니다.

구체적인 예를 들어 보겠습니다. 사용자가 '서울 강남역 근처 카페 다섯 곳을 평점 순으로 알려 달라'라고 부탁한 상황을 떠올려 봅시다. 단순한 모델 호출이라면 모델은 자신의 학습 시점에 기억하는 카페 이름들을 그럴듯하게 늘어놓는 것이 전부입니다. 반면 에이전트는 먼저 지도 검색 도구를 부르고, 검색 결과에서 평점 정보를 모으고, 평점이 비어 있는 항목은 두 번째 도구로 다시 채우고, 마지막에 다섯 개를 골라 사용자에게 답합니다. 이 다섯 단계는 미리 정해 둔 절차가 아니라 모델이 매 단계 결과를 보고 그때그때 결정한 흐름입니다. 같은 질문에서도 첫 검색 결과가 다르면 다음 단계가 바뀝니다.

이 차이가 평가에 곧바로 영향을 줍니다. **단일 LLM 호출**에서는 입력과 출력이 한 쌍이라 평가가 단순합니다. 입력 한 줄에 대해 출력이 정답에 얼마나 가까운지를 보면 됩니다. 반면 **에이전트 호출**은 한 번의 사용자 요청이 여러 단계로 분해되어 진행되므로, 측정의 결이 여러 층으로 나뉩니다. 예를 들어 카페를 다섯 개 모았다 해도, 그 과정에서 검색 도구를 두 번 헛것을 부르고 비용을 두 배 쓴 흐름과 한 번에 깔끔하게 끝낸 흐름은 사용자에게 보이는 답은 같아도 시스템 입장에서는 전혀 다른 결과입니다.

그래서 에이전트 평가에서는 **평가의 단위**를 먼저 정해 두어야 합니다. 평가 단위는 보통 네 층으로 나뉩니다. 가장 작은 단위는 **토큰**으로, 모델이 내놓은 문자열의 한 조각 한 조각이 여기에 해당합니다. 그다음은 **응답**으로, 한 번 모델을 호출했을 때 돌려받는 한 덩어리의 출력이 단위입니다. 그 위에 **단계**가 있습니다. 단계는 도구 호출 한 번 또는 모델 응답 한 번이라는 의미 있는 한 수를 가리키며, 카페 예시에서 '지도 검색을 부른다'가 한 단계입니다. 가장 큰 단위는 **전체 trajectory**입니다. 트라젝토리는 사용자가 처음 던진 요청부터 마지막 답까지 이어지는 단계 전체의 궤적으로, 카페 예시에서는 '검색 → 보강 검색 → 정렬 → 답' 네 단계의 흐름 전체를 가리킵니다.

네 가지 단위를 한눈에 보면 다음과 같은 층계가 됩니다.

[토큰] 출력 문자열의 한 조각 (예: '강', '남', '역')
↑
[응답] 모델 한 번 호출의 한 덩어리 출력 (예: 도구 호출 1건)
↑
[단계] 의미 있는 한 수 (예: 지도 검색을 부른 행위)
↑
[Trajectory] 요청에서 답까지 이어지는 단계 전체의 궤적

이 층계가 중요한 이유는, 단위가 달라지면 측정 도구도 달라지기 때문입니다. 토큰 단위로 모델 품질을 보려면 확률값이나 perplexity 같은 숫자가 필요하고, 응답 단위로 한 번의 출력이 좋은지 보려면 문자열 비교나 채점기 모델이 필요합니다. 단계 단위로 도구를 잘 골랐는지 보려면 어떤 도구를 어떤 인자로 불렀는지를 기록한 로그가 필요하고, 트라젝토리 단위로 전체 흐름의 합리성을 보려면 단계들의 시간

순서와 부모-자식 관계를 함께 보여 주는 추적 도구가 필요합니다. 같은 시스템을 평가하더라도 어느 단위로 보느냐에 따라 다른 도구가 붙고 다른 결론이 나옵니다.

다시 카페 예시로 돌아가 보겠습니다. 응답 단위로만 보면 다섯 곳이 깔끔히 나왔으니 합격입니다. 그러나 단계 단위로 보면 검색 도구를 두 번 헛돌렸다는 사실이 드러납니다. 트라젝토리 단위로 보면 그 헛돈 두 단계 때문에 응답 시간이 평소보다 두 배 걸렸고 비용도 두 배 들었다는 것이 보입니다. 어느 한 단위만으로는 시스템의 상태를 다 알 수 없으며, 단위마다 잡히는 문제가 다릅니다. 이 책 뒤쪽에서 다루는 평가 도구들이 굳이 'trace를 본다, span을 본다, metric을 본다'처럼 여러 시점을 동시에 제공하는 까닭이 여기에 있습니다.

정리하면, 에이전트는 모델, 도구, 메모리, 루프 네 구성요소로 분해되는 자율적 의사결정 루프이며, 단일 LLM 호출과 달리 한 번의 사용자 요청이 여러 단계로 펼쳐집니다. 이 때문에 평가의 단위는 토큰, 응답, 단계, 전체 trajectory의 네 층으로 나뉘고, 어느 단위로 보느냐에 따라 측정에 쓰는 도구도 함께 달라집니다.

다음 단원인 1.1.2에서는 이렇게 여러 단계로 펼쳐지는 에이전트가 동일 입력에서도 다른 흐름을 만들어 내는 비결정성을 다루고, 그 비결정성이 디버깅과 평가를 어떻게 어렵게 만드는지를 살펴봅니다.

이 단원을 마치면 에이전트를 모델, 도구, 메모리, 루프 네 가지 구성요소로 분해하고, 평가의 단위가 토큰·응답·단계·전체 trajectory 네 층으로 어디에 놓이는지를 단위별 측정 도구의 차이와 함께 설명할 수 있습니다.

1.1.2 비결정성이 만드는 평가의 어려움

앞 단원에서 에이전트는 모델, 도구, 메모리, 루프 네 구성요소가 맞물려 한 번의 요청을 여러 단계의 흐름으로 펼쳐 내는 시스템임을 보았습니다. 그런데 이 흐름은 같은 입력을 두 번 넣어도 같은 길로 흘러가지 않는 경우가 많습니다. 같은 질문에 매번 같은 답이 돌아온다면 평가는 비교적 단순하겠지만, 현실의 에이전트는 그렇지 않습니다. 이 단원에서는 어디서 그 차이가 생기는지, 그 차이가 평가와 디버깅을 왜 어렵게 만드는지, 그리고 현장에서 그 어려움을 어떻게 다루는지를 정리합니다.

먼저 출발점이 되는 용어를 풀겠습니다. 어떤 시스템이 같은 입력에 대해 같은 출력을 일관되게 돌려주면 그 시스템을 결정적이라고 부릅니다. 반대로 같은 입력에 대해 출력이 매번 조금씩 달라지면 그 시스템을 **비결정적**이라고 부릅니다. 인공지능 모델이 다음 단어를 고를 때는 보통 가장 확률이 높은 한 단어만 고르는 것이 아니라 확률 분포 위에서 한 단어를 뽑습니다. 이 뽑는 동작을 **샘플링**이라고 합니다. 샘플링은 매번 결과를 약간씩 다르게 만드는 가장 큰 원천이며, 같은 프롬프트에서도 다른 결과가 나오는 첫 번째 이유입니다.

비결정성의 원천은 샘플링 하나만이 아닙니다. 모델 입장에서 보면 사용자가 보낸 한 줄 외에도 시스템 프롬프트, 시간 정보, 도구 목록의 순서, 직전 단계에서 도구가 돌려준 결과 같은 입력이 함께 들어옵니다. 이 입력 가운데 어느 하나라도 매번 조금씩 다르면 모델의 다음 한 수가 달라집니다. 도구 쪽도 사정이 비슷합니다. 검색 도구는 같은 질의를 보내도 외부 서비스의 인덱스가 갱신되어 결과가 바뀔 수 있고, 시간이 들어간 도구는 호출 시각에 따라 답이 바뀌며, 외부 서비스가 잠깐 느려지면 시간 초과로 다른 분기를 타기도 합니다. 모델의 샘플링과 도구의 출력임이 합쳐지면, 같은 사용자 요청이 매번 다른 트라젝토리로 펼쳐집니다.

구체적인 예를 보겠습니다. 사용자가 어제 날짜의 환율을 물었다고 합시다. 첫 번째 시도에서 에이전트는 환율 도구를 한 번 부르고 깔끔하게 답합니다. 두 번째 시도에서는 모델이 먼저 '오늘 날짜를 확인해야겠다'라며 시계 도구를 부르고, 그다음 환율 도구를 부르고, 그다음 표 정리 도구를 부르거나

답합니다. 세 번째 시도에서는 환율 도구가 한 번 시간 초과를 내고, 모델이 다시 부르고서야 답합니다. 사용자에게 보이는 마지막 답이 비슷해도, 안에서 일어난 일은 매번 다릅니다.

이 비결정성은 **디버깅**을 곤란하게 만듭니다. 일반적인 소프트웨어에서 버그를 잡는 출발점은 '같은 입력을 다시 넣어 같은 실패를 재현한다'입니다. 그런데 에이전트에서는 같은 입력을 다시 넣어도 다른 길로 흘러가서 그 실패가 다시 나타나지 않을 수 있습니다. 어제 사용자가 '환율을 못 찾는다'라고 신고해도, 그 사용자의 정확한 입력을 그대로 다시 넣어 보면 오늘은 멀쩡히 환율이 찾아집니다. 이런 상황을 **재현 불가능한 실패**라고 부릅니다. 재현이 안 되면 무엇이 잘못이었는지 보이지 않고, 보이지 않으면 고칠 수가 없습니다.

Anthropic이 자사 멀티에이전트 연구 시스템을 운영하면서 공개한 경험은 이 어려움을 단적으로 보여 줍니다. 사용자가 명백히 있어야 할 정보를 못 찾았다고 신고해도, 엔지니어가 들여다보면 '**왜인지 안 보임**'이라는 문제에 부딪힙니다. 검색 쿼리가 나뉘었는지, 검색은 잘 됐는데 모델이 소스를 잘못 골랐는지, 도구 자체가 그 순간 실패했는지 구분이 되지 않습니다. 하나의 에러 메시지가 아니라, 여러 단계가 얹힌 트라젝토리를 펼쳐 보아야 비로소 어디가 문제였는지 짚을 수 있습니다. 이런 이야기는 Anthropic만의 사정이 아니라 비결정적 시스템이라면 어디서나 만나는 공통의 풍경입니다.

비결정성은 **평가**도 어렵게 만듭니다. 같은 데이터셋의 같은 질문 하나로 시스템을 한 번 돌려 점수를 매겼다고 합시다. 그 점수가 80점이라면, 다음에 또 돌릴 때도 80점이 나온다고 믿을 수 있을까요. 비결정적 시스템에서는 그 보장이 없습니다. 다음 시도에서는 75점이 나올 수도, 85점이 나올 수도 있습니다. 한 번의 실행 결과에 의존하면 평가 결과가 그날의 운에 따라 흔들립니다.

이 흔들림을 다루는 표준 처방은 **여러 번의 trial로 평균을 내는 통계적 평가**입니다. 같은 질문 하나에 대해 시스템을 여러 번 돌리고, 각 trial의 점수를 모아 평균과 분산을 함께 봅니다. 평균은 시스템이 보통 어느 수준에 있는지를 보여 주고, 분산은 그 수준이 얼마나 흔들리는지를 보여 줍니다. 두 시스템을 비교할 때도 한 번의 점수 차이로는 우열을 가릴 수 없으며, 분산을 함께 본 뒤 통계적으로 의미 있는 차이인지를 따져 보아야 합니다. 한 질문을 다섯 번 돌려 평균 78점, 분산 4점이 나온 시스템과 평균 80점, 분산 6점이 나온 시스템을 비교한다면 단순히 80이 78보다 크다고 말하기 어렵습니다.

이 통계적 시선은 데이터셋 전체에도 그대로 적용됩니다. 백 개의 질문이 있다면, 한 질문씩 한 번만 돌리는 것이 아니라 한 질문씩 여러 번 돌려 그 질문의 평균을 구하고, 그 평균들을 다시 데이터셋 단위로 모읍니다. 같은 시스템을 두 버전으로 만들었을 때 어느 쪽이 정말 나아졌는지를 가리려면, 단일 점수의 차이가 아니라 분포의 차이를 보아야 합니다. 비결정성의 가장 큰 비용은 바로 이 '한 번에 끝나지 않는다'는 사실이며, 평가 시간과 비용이 결정적 시스템보다 몇 배 더 드는 까닭이 여기에 있습니다.

또 하나 짚어 둘 것은 비결정성이 단지 측정의 정확도만 떨어뜨리는 것이 아니라 측정의 단위까지 바꾼다는 점입니다. 결정적 시스템에서는 한 번의 입력에 한 번의 출력이 있으니 측정 단위가 바로 그 한 번의 결과입니다. 그러나 비결정적 에이전트에서는 같은 입력이 여러 트라젝토리로 펼쳐지므로, 한 입력의 평가 단위가 자연스레 '**여러 트라젝토리의 분포**'가 됩니다. 이 분포 위에서 평균, 분산, 최악 사례를 함께 보아야 비로소 그 입력에서 시스템이 어떻게 행동하는지를 말할 수 있습니다.

정리하면, 에이전트의 비결정성은 모델 샘플링, 입력의 미세한 변화, 외부 도구의 출렁임이 합쳐져 발생하며, 같은 입력에서도 매번 다른 트라젝토리를 만들어 냅니다. 이 성질은 같은 실패를 다시 불러내기 어렵게 해 디버깅을 곤란하게 만들고, 한 번의 실행 점수를 믿을 수 없게 만들어 평가를 통계적 작업으로 바꿉니다. Anthropic이 말한 '**왜인지 안 보임**' 문제는 이 두 어려움이 만나는 자리에 있는 풍경이며, 그래서 에이전트 평가는 여러 번의 trial로 평균과 분산을 보는 통계적 절차로 자리를 잡았습니다.

다음 단원인 1.1.3에서는 이 비결정성과 다단계 행동이 결국 LLM 평가와 에이전트 평가를 어떻게 다른 일로 만들었는지를 측정 대상, 환경, 비용 세 측면으로 비교하며 살펴봅니다.

이 단원을 마치면 에이전트의 비결정성이 어디에서 비롯되는지, 그 비결정성이 재현 불가능한 실패와 통계적 평가라는 두 결과를 어떻게 만들어 내는지, 그리고 그 영향을 디버깅과 평가의 차원에서 어떻게 풀어 가는지를 설명할 수 있습니다.

1.1.3 LLM 평가와 에이전트 평가의 결정적 차이

앞선 두 단원에서 에이전트가 모델, 도구, 메모리, 루프로 분해되는 자율적 의사결정 루프이며, 이 루프가 비결정적이어서 같은 입력에도 여러 트래젝토리로 펼쳐진다는 점을 보았습니다. 이 두 사실은 자연스레 한 가지 질문을 끌어옵니다. 그렇다면 에이전트를 평가하는 일은 LLM 하나를 평가하는 일과 무엇이 어떻게 다를까요. 이 단원은 그 차이를 측정 대상, 환경, 비용이라는 세 측면으로 나누어 정리하고, 왜 에이전트 평가에 별도의 도구가 필요한지를 마지막에 묶습니다.

먼저 **측정 대상**의 차이입니다. LLM 평가는 한 번의 입력에 한 번의 출력이 짝을 이루는 구조를 전제합니다. 사용자가 한 줄을 던지면 모델이 한 덩어리의 답을 내놓고, 그 답이 정답에 얼마나 가까운지, 문법은 자연스러운지, 사실관계는 맞는지를 쟁니다. 이 경우 측정의 대상은 명확하게 '단일 응답의 품질'에 머무릅니다. 평가 데이터셋은 보통 입력 문장과 모범 답안의 쌍으로 구성되며, 채점은 두 답을 건주는 일로 마무리됩니다.

반면 에이전트 평가는 한 번의 사용자 요청이 여러 단계로 펼쳐진다는 사실을 출발점으로 삼습니다. 사용자가 '서울 강남역 카페 다섯 곳'을 물으면 에이전트는 검색 도구를 부르고, 결과를 다듬고, 부족하면 다시 다른 도구를 부르고, 마지막에 답을 만듭니다. 이때 측정의 대상은 마지막 답 한 줄에 그치지 않습니다. 도구를 잘 골랐는지, 인자를 제대로 넘겼는지, 도구가 돌려준 결과를 제대로 해석했는지, 단계의 순서가 합리적이었는지가 모두 측정의 대상이 됩니다. 한마디로 LLM 평가가 단일 응답의 품질을 보는 것이라면, 에이전트 평가는 **다단계 행동**의 품질을 보는 것입니다.

구체적으로 보자면, 카페 추천 사례에서 마지막 답이 같은 다섯 곳을 적어 내더라도 안에서 일어난 일은 매우 다를 수 있습니다. 한 번의 검색으로 끝낸 흐름과 검색을 다섯 번 헤맨 흐름은 답은 같아도 행동의 품질이 다릅니다. 에이전트 평가는 이 차이를 잡아내는 평가이며, 이 점이 LLM 평가와 가장 먼저 갈라지는 지점입니다.

다음은 **환경**의 차이입니다. LLM 평가는 보통 정적 데이터셋 위에서 진행됩니다. 정적 데이터셋이란 입력과 정답 쌍이 미리 만들어진 채로 고정되어 있어, 평가가 진행되는 동안 그 내용이 바뀌지 않는 데이터셋을 말합니다. 평가자는 데이터셋을 한 줄씩 모델에 흘려 넣고 답을 받아 채점만 하면 됩니다. 입력은 미리 정해져 있고, 출력은 한 번에 끝나며, 외부 세계와 따로 상호작용할 일이 없습니다.

에이전트 평가는 이와 반대로 **환경과 상호작용**하는 평가입니다. 에이전트가 도구를 부른다는 것은 곧 외부 세계의 어떤 자리에 영향을 끼친다는 뜻입니다. 검색 도구를 부르면 검색 서비스에 부하가 가고, 파일 쓰기 도구를 부르면 파일 시스템에 변화가 생기며, 결제 도구를 부르면 돈이 움직입니다. 평가 환경 안에서도 마찬가지입니다. 같은 가상 컴퓨터에서 두 trial을 연달아 돌리면, 첫 trial이 남긴 파일이 두 번째 trial의 입력으로 흘러들어 점수를 부풀릴 수 있습니다. 그래서 각 trial은 깨끗한 상태에서 시작해 서로 격리되어야 하고, 평가 환경 자체가 결과에 추가 노이즈를 만들지 않도록 설계되어야 합니다. 이 환경 설계 부담은 LLM 평가에는 없는 짐이며, 뒤에서 다룰 평가 환경 격리 같은 주제가 별도로 다루어지는 까닭이 여기에 있습니다.

세 번째는 **비용**의 차이입니다. LLM 평가에서 한 건의 채점에 드는 비용은 대체로 한 번의 모델 호출 비용, 즉 입력과 출력 토큰의 양으로 결정됩니다. 모델 호출이 한 번이니 시간도 보통 밀리초 단위로 끝납니다. 데이터셋이 천 개여도 한 시간 안에 한 번 돌리는 일이 어렵지 않습니다.

에이전트 평가는 사정이 다릅니다. 한 건의 평가 안에서 모델이 여러 번 호출되고 도구가 여러 번 불립니다. 한 trial의 토큰 비용이 LLM 평가의 수 배에서 수십 배가 되며, 도구 호출에는 토큰과 별개로 외부 서비스 사용료가 붙습니다. 검색 도구는 호출당 비용이 있고, 결제 도구나 클라우드 자원 도구는 실제 돈이 빠져나가는 일도 있습니다. 그래서 비용 관리에서는 토큰 비용과 도구 호출 비용을 따로 측정해 합쳐 보는 시각이 필요합니다. 시간 측면도 다릅니다. 한 trial이 끝나기까지 도구 호출이 외부 서비스를 기다리는 동안의 지연이 누적되어, 보통 한 건당 수십 초에서 분 단위가 걸립니다. 같은 천 개짜리 데이터셋이라도 LLM 평가는 한 시간이지만 에이전트 평가는 며칠이 될 수 있습니다.

여기에 앞 단원에서 본 비결정성이 더해지면 비용은 한 번 더 곱해집니다. 한 입력당 한 번 돌리는 것으로는 점수를 믿을 수 없어 여러 번의 trial로 평균을 내야 하기 때문입니다. 한 trial이 분 단위인데 한 입력당 다섯 번 돌려야 한다면, 그 한 입력에 드는 시간만 수 분에서 수십 분이 됩니다. 토큰 비용과 도구 호출 비용도 그만큼 곱해집니다.

세 측면을 표로 모으면 두 평가의 거리가 한눈에 보입니다.

구분	LLM 평가	에이전트 평가
측정 대상	단일 응답의 품질	다단계 행동의 품질
환경	정적 데이터셋	환경과 상호작용, trial 격리
비용 단위	토큰 비용	토큰 비용 + 도구 호출 비용
한 건 시간	밀리초 단위	분 단위 (다 trial 시 더 길어짐)

이 세 차이가 합쳐지면, 에이전트 평가에 LLM 평가용 도구를 그대로 쓰기 어렵다는 결론이 자연스레 나옵니다. 입력과 출력의 쌍을 다루는 평가 도구로는 단계별 도구 호출 기록을 보관하지 못합니다. 한 번의 호출만 채점하는 도구로는 여러 번의 trial 분포를 다루지 못합니다. 토큰 비용만 추적하는 도구로는 도구 호출 비용을 함께 보지 못합니다. 결국 에이전트 평가에는 단계별 트라젝토리를 저장하고, 환경을 격리해 trial을 반복하며, 토큰과 도구 호출 비용을 함께 집계하고, 분포 통계를 자동으로 뽑아 주는 별도의 도구가 필요합니다. 이 책 뒤쪽에서 다룰 추적 시스템과 평가 플랫폼이 LLM 평가용 도구와 따로 발전해 온 이유가 바로 여기에 있습니다.

정리하면, LLM 평가와 에이전트 평가는 측정 대상에서 단일 응답 품질과 다단계 행동 품질로 갈리고, 환경에서는 정적 데이터셋과 환경 상호작용으로 갈리며, 비용에서는 토큰 비용 중심의 밀리초 단위 평가와 토큰·도구 호출 비용을 함께 보는 분 단위 평가로 갈립니다. 이 세 측면이 한꺼번에 달라지기 때문에 에이전트 평가에는 LLM 평가용 도구로 다루기 어려운 별도의 도구가 필요합니다.

다음 단원인 1.1.4에서는 다단계 행동을 측정하는 두 가지 대표적 시각, 즉 트라젝토리 평가와 파이널 답 평가가 어떻게 다르고 어떻게 서로를 보완하는지를 살펴봅니다.

이 단원을 마치면 LLM 평가와 에이전트 평가의 차이를 측정 대상, 환경, 비용 세 측면에서 구체적인 예와 함께 비교하고, 그 차이가 별도의 평가 도구를 요구하는 이유를 설명할 수 있습니다.

1.1.4 Trajectory 평가와 Final-Answer 평가의 구분

앞 단원에서 에이전트 평가는 LLM 평가와 달리 다단계 행동을 측정 대상으로 삼는다는 점을 보았습니다. 그렇다면 그 다단계 행동을 실제로 어떻게 썰 수 있을까요. 현장에서 가장 자주 마주치는 두 가지 시각이 있습니다. 한쪽은 마지막 답만 보는 시각이고, 다른 한쪽은 답에 이르는 길 전체를 보는 시각입니다. 이 단원은 이 두 시각의 정의와 한계를 짚고, 어떤 상황에서 어느 쪽을 쓰며 둘이 어떻게 서로를 보완하는지를 정리합니다.

먼저 용어부터 풀겠습니다. **Final-answer 평가**는 에이전트가 모든 단계를 마치고 사용자에게 돌려주는 마지막 답만을 측정 대상으로 삼는 평가 방식입니다. 안에서 도구를 몇 번 불렀는지, 단계가 몇 개 있었는지는 묻지 않고, 사용자에게 노출된 한 덩어리의 결과가 정답에 얼마나 가까운지만 봅니다. LLM 평가에서 익숙하던 '입력과 정답 한 쌍, 그 사이의 비교'가 그대로 이어지는 형태이며, 그래서 평가 도구도 비교적 단순합니다.

이 단순함이 final-answer 평가의 강점이자 한계입니다. 강점은 채점 비용이 낮고 자동화하기 쉽다는 점입니다. 한계는 같은 마지막 답이라도 안에서 일어난 일이 매우 다를 수 있다는 사실을 잡아내지 못한다는 점입니다. 앞 단원의 카페 추천 사례에서 보았듯, 한 번의 검색으로 깔끔하게 다섯 곳을 뽑은 흐름과 검색을 다섯 번 헤매면서 비용을 다섯 배 쓴 흐름은 final-answer 평가에서는 같은 점수를 받습니다. 사용자에게 보이는 결과가 같기 때문입니다.

이 한계를 메우는 시각이 **trajectory 평가**입니다. 이 책에서 trajectory란 사용자가 처음 던진 요청부터 마지막 답까지 이어지는 단계 전체의 궤적을 가리킵니다. Trajectory 평가는 그 궤적 자체를 측정 대상으로 삼습니다. 단계 하나하나에 대해 모델이 어떤 도구를 골랐는지, 어떤 인자로 불렀는지, 도구가 돌려준 결과를 어떻게 다음 단계에 반영했는지를 봅니다.

Trajectory 평가에서 다루는 측정 항목은 보통 다음과 같이 정리됩니다. 첫째는 **올바른 도구 선택**입니다. 모델이 그 단계에서 정말 필요한 도구를 골랐는지, 비슷한 이름의 다른 도구를 잘못 부르지 않았는지를 봅니다. 둘째는 **올바른 인자 전달**입니다. 도구를 잘 골랐어도 인자가 비뚤어지면 결과가 무너집니다. 셋째는 **단계 순서의 합리성**입니다. 검색 결과를 보지 않고 결제 도구로 건너뛰는 식의 어긋난 순서가 없는지 봅니다. 넷째는 **단계 사이의 일관성**입니다. 1단계에서 'A 카페'를 보았다고 해 놓고 3단계에서 갑자기 'B 카페'를 추천한다면 일관성이 깨진 것입니다.

두 시각이 어떤 차이를 잡아내는지 한 표로 모아 보겠습니다.

관점	Final-answer 평가	Trajectory 평가
측정 대상	마지막 한 덩어리 답	사용자 요청부터 답까지의 궤적
잡히는 것	정답 여부, 표현 품질	도구 선택, 인자, 순서, 일관성
잡히지 않는 것	답에 이른 과정의 낭비·오류	마지막 답의 사용자 체감 품질
채점 비용	낮음	높음 (단계마다 채점)

이 표가 곧바로 보여 주듯, 두 시각은 서로 다른 종류의 실패를 잡아냅니다. **Final-answer만으로 놓치는 실패**가 분명히 있습니다. 첫째는 운 좋게 정답을 맞힌 사례입니다. 모델이 도구 결과를 무시하고 자기 추측으로 답을 만들어 냈는데 우연히 정답과 같았다면, final-answer 평가는 만점을 줍니다. 다음번 비슷한 입력에서 그 추측이 틀리는 순간이 오면 시스템은 무너지지만, 그 직전까지는 평가가 문제를 알려 주지 않습니다.

둘째는 비용과 시간에서의 실패입니다. 답이 같아도 단계가 다섯 배 길어진 흐름은 토큰 비용과 도구 호출 비용을 다섯 배 더 씁니다. Final-answer 점수가 같아도 운영 입장에서는 전혀 다른 시스템입니다. 셋째는 부작용을 동반한 실패입니다. 답을 만드는 과정에서 엉뚱한 파일을 건드리거나 외부 서비스를 잘못 호출했다면 사용자에게 보이는 답은 정답이라도 시스템 어딘가에 문제가 남아 있습니다. Final-answer 평가는 이 모두를 보지 못합니다.

반면 **Trajectory만으로 놓치는 정답**도 있습니다. 모든 단계가 미리 정한 모범 절차와 똑같이 흘렀는지를 너무 엄격하게 보면, 다른 길로 더 나은 답에 도달한 흐름을 깎아내릴 수 있습니다. 모범 절차는 검색 도구를 두 번 부르도록 정해 두었는데, 한 번의 검색만으로 더 정확한 답을 만든 흐름이 있었다고 합시다. Trajectory 평가가 단계 일치만 본다면 이 흐름은 낮은 점수를 받지만, 사용자 입장에서는 더 빠르고

정확한 답입니다. 또 같은 도구라도 모델이 다른 방식으로 묶어 부르거나, 새로 추가된 도구를 활용해 새 길로 가는 경우도 있습니다. 마지막 답의 품질이 무엇보다 중요한 사용자 시나리오에서는 trajectory의 모양보다 답 자체의 합리성이 우선합니다.

그래서 두 시각은 **보완 관계**에 놓입니다. Final-answer 평가는 사용자에게 보이는 결과 품질을 지키는 안전선이고, trajectory 평가는 그 결과에 이르는 길의 건강함을 지키는 진단입니다. 둘을 같이 보면 답은 맞았으나 길이 위태로운 사례, 길은 깔끔했으나 답이 빗나간 사례, 답도 맞고 길도 단단한 사례를 따로 분류할 수 있습니다.

실제 적용에서는 보통 다음과 같은 분담이 자연스럽게 자리를 잡습니다. 사용자 향한 품질 지표는 final-answer를 중심에 두고 합니다. 외부에 발표되는 정답률이나 만족도 같은 숫자는 마지막 답 위에서 잹니다. 반면 시스템 운영 지표는 trajectory를 중심으로 봅니다. 단계 수의 분포, 도구별 실패율, 평균 비용 같은 숫자는 궤적을 펼쳐 보아야 나옵니다. 두 종류의 지표가 한 화면에 함께 놓여야 시스템의 상태를 빠짐없이 볼 수 있습니다.

상황에 따라 어느 쪽에 더 무게를 둘지를 정하는 기준도 비교적 단순합니다. 사용자가 보는 결과의 정확성이 절대적이고 과정에는 큰 위험이 없는 시나리오, 예컨대 단순 조회나 요약은 final-answer 평가만으로도 큰 무리가 없습니다. 반면 단계 하나가 비가역적인 행동, 예컨대 결제, 외부 메시지 발송, 데이터 수정 같은 곳을 거친다면 trajectory 평가가 주가 되어야 합니다. 결과가 맞아도 그 과정에서 잘못된 결제가 일어났다면 final-answer 점수가 의미 없기 때문입니다.

정리하면, Final-answer 평가는 사용자에게 노출되는 마지막 답을 측정 대상으로 삼아 채점이 단순한 대신 답에 이르는 길의 낭비와 오류를 보지 못합니다. Trajectory 평가는 단계 전체의 궤적을 측정 대상으로 삼아 도구 선택, 인자, 순서, 일관성을 잡아내지만 다른 길로 도달한 더 나은 답을 깎아내릴 수 있습니다. 두 시각은 서로 다른 실패를 잡아내며, 사용자 향한 품질과 시스템 운영의 건강함을 함께 지키려면 둘을 보완적으로 결합해야 합니다.

다음 단원인 1.2.1에서는 trajectory 평가가 자주 마주치는 첫 번째 실패 패턴인 도구 선택 오류를 다루고, 어떤 신호로 그 오류를 잡아낼 수 있는지를 살펴봅니다.

이 단원을 마치면 final-answer 평가와 trajectory 평가의 측정 대상과 한계를 비교하고, 두 시각이 잡아내는 실패와 놓치는 정답을 구분하며, 시나리오에 따라 어느 쪽에 무게를 둘지 결정하는 기준을 설명할 수 있습니다.

1.2 Failure Mode Taxonomy

이 섹션의 토픽과 학습 목표는 다음과 같습니다.

1.2.1 Tool selection error — 에이전트가 잘못된 도구를 선택하는 패턴과 그 원인을 설명할 수 있다

1.2.2 Planning loop와 무한 반복 — 에이전트가 빙빙 도는 planning loop의 발생 조건과 탐지 방법을 설명할 수 있다

1.2.3 Hallucinated reasoning — 도구 출력과 모순되는 추론 환각의 패턴을 식별하고 검증 방법을 설명할 수 있다

1.2.4 Premature termination — 에이전트가 작업을 완료하지 않고 종료하는 패턴의 원인과 탐지 방법을 설명할 수 있다

1.2.1 Tool selection error

앞 절에서는 에이전트 평가가 LLM 평가와 어떻게 다른지, trajectory 평가와 final-answer 평가가 어떻게 달라지는지를 살펴보았습니다. 이번 절부터는 에이전트가 실제로 망가지는 모습을 유형별로 정리합니다. 이렇게 자주 반복되는 실패의 묶음을 이 책에서는 실패 모드 분류라고 부르며, 그 첫 번째 자리를 차지하는 것이 도구 선택 오류입니다.

먼저 용어부터 풀겠습니다. 에이전트는 한 번의 응답으로 답을 내놓는 대신, 매 단계에서 '지금 무엇을 할까'를 정해야 합니다. 이 중 가장 많이 반복되는 결정이 '어떤 도구를 부를까'입니다. 에이전트에게 노출된 도구는 보통 검색기, 계산기, 데이터베이스 조회기, 파일 편집기처럼 이름과 설명을 가진 함수의 형태로 제시됩니다. 모델은 사용자의 질문과 자신의 직전 추론을 보고 그중 하나를 골라 호출합니다. 이 결정 단계에서 의도와 다른 도구를 부르는 현상을 **도구 선택 오류**라고 합니다.

도구 선택 오류는 도구 호출 자체가 실패한 경우와 다릅니다. 도구 호출 실패는 네트워크 끊김이나 인자 검증 오류처럼 도구 쪽이 신호를 돌려주는 사건이고, 도구 선택 오류는 도구가 정상적으로 동작했지만 애초에 부적절한 도구를 골랐다는 사건입니다. 후자가 더 까다로운 까닭은 응답 자체는 멀쩡해 보이는데 결과가 사용자가 원한 것과 다르기 때문입니다.

가장 흔한 패턴은 **비슷한 이름의 도구를 혼동**하는 경우입니다. 예를 들어 한 시스템에 search_orders, search_order_items, search_invoices 세 도구가 동시에 노출되어 있다고 합시다. 사용자가 '지난주에 산 노트북 영수증'을 묻습니다. 모델이 search_orders를 부르면 주문 단위는 잘 잡히지만, 노트북이라는 품목 정보까지 들어가지 않아 빈 응답이 돌아옵니다. 모델이 그 결과를 보고도 '검색이 비었으니 데이터가 없습니다'라고 단정하고 끝내면, 표면적으로는 정상 응답이지만 실제로는 search_order_items나 search_invoices 쪽으로 갔어야 했던 호출이 누락된 셈입니다.

원인은 보통 모델보다 **도구 설명문**에 있습니다. 도구 설명문은 모델이 도구를 고르는 유일한 단서입니다. 사람이라면 동료에게 '주문 검색이랑 영수증 검색 중 어떤 게 영수증을 찾아 줘?'라고 물어볼 수 있지만, 모델은 그런 자유가 없고 한 번에 결정을 내려야 합니다. 그래서 설명문이 모호하면 모델은 이름의 유사성으로 판단합니다. search_orders 설명에 '주문 내역을 검색합니다'만 적혀 있고 search_invoices 설명에 '청구·영수 정보를 검색합니다'라고만 적혀 있으면, 모델 입장에서는 '영수증 = 청구 정보'를 묶기 어렵습니다. 도구 이름이 비슷할수록 설명문은 더 길고 더 구체적이어야 합니다.

설명문의 영향을 가시화하기 위해 하나의 도구 카탈로그가 어떻게 달라지는지를 표로 정리해 보겠습니다.

도구 이름	설명문 품질	모델의 호출 비율 (예시)
search_orders	구체·예시 포함	영수증 질문에 거의 안 부름
search_invoices	구체·예시 포함	영수증 질문에 자주 부름
search_orders	모호한 한 문장	영수증 질문에 자주 오인 호출
search_invoices	모호한 한 문장	영수증 질문에 가끔만 부름

이 표가 보여 주는 핵심은 도구 선택 오류가 모델의 능력보다 도구 설명문의 품질에 더 크게 좌우될 때가 많다는 점입니다. 같은 모델, 같은 사용자 질문이라도 설명문 한 줄을 어떻게 쓰느냐에 따라 호출 비율이 뒤집힙니다. 그래서 도구 선택 오류를 줄이려는 시도는 보통 모델 변경이 아니라 설명문과 카탈로그 정리에서 시작합니다.

이제 평가 쪽 이야기로 넘어가 보겠습니다. 도구 선택 오류는 final-answer 평가만으로는 잘 잡히지 않습니다. 빈 응답이거나 어색한 응답일 뿐, 채점자가 보기에 '부정확'이라고 단정하기 애매한 경우가 많기 때문입니다. 그래서 **도구 선택 오류를 잡는 평가 항목**은 trajectory 위에 있습니다. 가장 단순한 형태는 단계별로 '예상 도구'와 '실제 호출 도구'를 비교해 일치하는 비율을 재는 도구 선택 정확도입니다. 골든 데이터셋의 각 케이스에 '이 입력에는 search_invoices가 가야 한다' 같은 라벨을 달아 두고, 실행 결과에서

호출된 도구가 라벨과 같은지를 본다는 뜻입니다. 여기에 '잘못된 도구를 부른 뒤 정정해서 결국 정답에 도달했는지'를 따로 표시하면, 단순 비율보다 풍부한 그림이 나옵니다.

도구 선택 오류가 발견되면 그다음 단계에서 곧장 따라오는 결정이 있습니다. 잘못 부른 호출에 대해 에이전트를 어떻게 처리할지입니다. 선택지는 크게 **복구**, **재시도**, **종료** 세 가지입니다. 복구는 잘못된 호출 결과를 받아도 모델이 그 자리에서 다른 도구를 다시 부르도록 두는 방식입니다. 재시도는 같은 도구를 인자만 바꾸어 다시 호출하는 방식이고, 종료는 더 진행하지 않고 사용자에게 명확한 메시지와 함께 흐름을 끝내는 방식입니다. 세 방식의 의미는 다음 표로 정리할 수 있습니다.

결정	의미	적합한 상황
복구	다른 도구로 갈아타 진행	비슷한 도구가 여러 개일 때
재시도	같은 도구를 다른 인자로 다시 호출	인자 누락·형식 오류일 때
종료	흐름을 멈추고 사용자에게 결과를 전달	유사 도구도 모두 빈 응답일 때

가장 흔한 함정은 모든 오류에 대해 **복구**를 기본값으로 잡아 두는 것입니다. 복구가 늘어날수록 단계 수가 길어지고, 토큰과 시간이 함께 늘어납니다. 게다가 복구가 새로운 도구 선택 오류를 또 만들면 다시 복구가 따라오면서 단계가 계속 쌓입니다. 그래서 실전에서는 '같은 카테고리 도구 안에서는 1회 복구, 그 밖에는 종료' 같은 정책을 미리 정해 두고, 그 정책 자체를 평가 항목으로 둡니다.

정리하면, 도구 선택 오류는 의도와 다른 도구가 호출되는 현상으로, 보통 도구 설명문의 모호함과 비슷한 이름의 도구가 동시에 노출된 카탈로그 구조에서 비롯됩니다. 이 오류는 final-answer만으로는 잘 잡히지 않으므로 trajectory 위에 도구 선택 정확도 같은 평가 항목을 얹어 잡고, 발견된 오류에 대해 복구, 재시도, 종료 중 어떤 결정을 기본값으로 둘지를 함께 정해 두어야 합니다.

다음 단원인 1.2.2에서는 이 도구 선택 오류와 결이 다른 또 하나의 실패 모드, 같은 단계를 빙빙 도는 planning loop를 살펴보겠습니다.

이 단원을 마치면 도구 선택 오류의 정의와 비슷한 이름의 도구를 혼동하는 사례를 들 수 있고, 도구 설명문 품질이 선택에 어떻게 영향을 주는지, 도구 선택 오류를 잡는 평가 항목과 복구·재시도·종료 결정의 차이를 설명할 수 있습니다.

1.2.2 Planning loop와 무한 반복

앞 단원에서는 에이전트가 부적절한 도구를 고르는 도구 선택 오류를 다루었습니다. 도구 선택 오류는 한 단계에서 일어나는 사건이라 단계만 잘 들여다보면 비교적 빨리 잡힙니다. 그런데 한 단계에서는 멀쩡해 보이는 행동이 여러 단계에 걸쳐 누적되면서 망가지는 종류의 실패가 있습니다. 그 대표적인 모습이 같은 자리를 빙빙 도는 planning loop입니다.

먼저 용어부터 풀겠습니다. 에이전트는 매 단계에서 추론과 도구 호출, 그리고 도구 결과 관찰을 반복하며 다음 행동을 결정합니다. 이 결정의 흐름을 계획이라고 부른다면, 계획이 어디서도 종결로 이어지지 않고 같은 형태의 단계를 계속 만들어 내는 상태가 **planning loop**입니다. 무한 반복은 planning loop가 외부에서 끊지 않는 한 계속 이어지는 양상을 가리키는 말이고, 둘은 거의 짝으로 쓰입니다.

가장 단순한 형태는 **동일 단계의 반복**입니다. 에이전트가 search_invoices를 부르고 빈 결과를 받습니다. 그러고는 '검색 조건을 살짝 바꿔야겠다'고 추론한 뒤 거의 같은 인자로 search_invoices를 다시 부릅니다. 결과는 또 빈니다. 다음 단계에서도 같은 흐름이 이어집니다. 추론 텍스트는 매번 표현이 조금씩 달라서 사람이 줄글로 읽으면 다른 시도처럼 보이지만, 실제 호출은 거의 동일합니다. 한 호출의 결과를 다음 호출에 쓰지 못하기 때문에 정보가 늘어나지 않고, 단계만 쌓입니다.

이 흐름을 풀어 그리면 다음과 같습니다.

```
[추론 1] -> [호출 search_invoices(A)] -> [빈 결과]
[추론 2] -> [호출 search_invoices(A')] -> [빈 결과]
[추론 3] -> [호출 search_invoices(A'')] -> [빈 결과]
...
[추론 N] -> [호출 search_invoices(A')] -> [빈 결과]
```

여기서 A, A', A''는 표면적으로 다른 인자이지만 검색 결과를 갈라 놓을 만큼 의미 있게 다르지는 않습니다. 모델이 이전 결과로부터 다음 인자를 진정으로 좁히려면 빈 결과의 원인 구조까지 들여다봐야 하는데, 도구가 단순히 빈 배열만 돌려준다면 모델은 그 신호를 보지 못합니다. 그래서 빈 결과를 다루는 도구의 응답 설계가 planning loop의 빈도에 직접 영향을 미칩니다.

planning loop가 위험한 이유는 단지 답이 안 나오기 때문만이 아닙니다. 더 큰 문제는 **토큰과 비용의 폭주**입니다. 단계가 늘어나면 모델은 매번 직전까지의 대화 기록과 도구 결과를 컨텍스트에 함께 받습니다. 단계가 길어질수록 한 번의 추론에 들어가는 토큰이 산술적이 아니라 누적적으로 늘어납니다. 호출 1회에 5천 토큰을 쓰던 흐름이 단계 20개에 다다르면 한 번 호출에 5만 토큰을 넘길 수 있고, 그런 호출이 다시 단계마다 반복되면서 비용이 기하적으로 부풀어 오릅니다. 흔히 인용되는 사례에서 한 번의 사용자 질문이 수십 달러어치 토큰을 태워 버린 경험담이 등장하는 까닭이 여기 있습니다.

문제는 이런 폭주가 **전통적 모니터링에서는 잘 보이지 않는다**는 점입니다. Replit 발표에서는 '데이터독은 크래시는 잡지만 루프에 빠진 에이전트, 빙빙 도는 에이전트는 못 잡는다'는 표현이 등장합니다.

데이터독으로 대표되는 인프라 모니터링 도구는 응답이 200을 돌려주면 정상이라고 판정합니다.

planning loop에 빠진 에이전트는 단계마다 도구 호출이 정상 응답으로 끝나기 때문에 외부에서 보면 트래픽이 조금 길어진 정도로만 잡힙니다. 사용자에게 응답이 늦거나 비용 청구서가 갑자기 부풀어 오를 때야 비로소 문제가 드러납니다. 이 사례는 LLM 관측이 별도로 필요하다는 주장을 뒷받침하는 표준적 일화로 자주 등장합니다.

planning loop를 끊는 첫 번째 장치는 **스텝 수 임계치**입니다. 에이전트의 한 사용자 요청이 만들어 낼 수 있는 최대 단계 수를 미리 정해 두고, 그 수에 닿으면 강제로 종료하는 정책을 가리킵니다. 최대 단계 수에 더해 누적 토큰, 누적 비용, 누적 시간을 함께 두면 폭주를 더 촘촘하게 잡을 수 있습니다. 임계치별 차이를 표로 보면 다음과 같습니다.

임계치 종류	의미	주된 보호 대상
스텝 수	단계가 N을 넘으면 종료	루프 형태 폭주
누적 토큰	총 토큰이 T를 넘으면 종료	컨텍스트 비대 폭주
누적 비용	비용이 C달러를 넘으면 종료	결제 폭주
누적 시간	경과가 S초를 넘으면 종료	사용자 대기 시간

임계치를 정하는 일은 단순해 보이지만 실전에서는 까다롭습니다. 너무 빡빡하면 정상적으로 길게 도는 작업이 자주 끊기고, 너무 헐거우면 애초에 보호 효과가 없습니다. 보통은 정상 작업의 단계 분포를 먼저 측정해 95퍼센타일이나 99퍼센타일을 기준으로 임계치를 잡고, 그보다 위에서 잘리는 사례를 따로 모아 본 뒤 조정합니다.

임계치만으로 모든 planning loop가 잡히는 것은 아닙니다. 임계치는 잘랐다는 사실을 알려 줄 뿐, 왜 돌았는지는 알려 주지 않습니다. 그래서 종료 조건과 함께 **루프 탐지 신호**를 trajectory 위에 따로 둡니다. 가장 단순한 신호는 직전 N단계에서 같은 도구를 같은 형태의 인자로 부른 비율입니다. 이 비율이 일정 수준을 넘으면 그 trajectory는 'looping' 라벨을 받고, 사후 분석에서 따로 모읍니다. 사후에 모인 trajectory를 들여다보면 도구 설명문 문제, 빈 결과 응답 설계 문제, 사용자 의도 파악 문제 중 어디에서

비롯된 루프인지 갈래를 잡을 수 있습니다.

이렇게 보면 planning loop는 단일 단계의 결함이 아니라 흐름의 결함입니다. 따라서 막는 장치도 단일 호출 단위가 아니라 흐름 단위에 붙입니다. 즉, 한 단계의 정확도를 높이는 평가 항목과는 별도로 '한 사용자 요청 안에서 단계가 어떻게 분포하는지'를 보는 메트릭을 둡니다.

정리하면, planning loop는 같은 형태의 단계가 종결로 이어지지 않고 반복되며 토큰과 비용을 누적적으로 폭주시키는 실패 모드입니다. 전통적 인프라 모니터링은 정상 응답으로 보이는 이 흐름을 잘 잡지 못하므로, 스텝 수와 누적 토큰·비용·시간 임계치, 그리고 직전 N단계의 호출 유사도를 보는 루프 탐지 신호를 함께 두어야 흐름 단위에서 잡을 수 있습니다.

다음 단원인 1.2.3에서는 단계가 멈추는데도 결과가 어긋나는 또 다른 실패 모드, 도구 결과를 무시하고 자기 추론을 따라가는 hallucinated reasoning을 살펴보겠습니다.

이 단원을 마치면 planning loop의 정의와 동일 단계 반복 패턴을 들 수 있고, 토큰·비용 폭주의 메커니즘과 스텝 수 임계치를 비롯한 종료 조건의 역할, 그리고 Replit 발표에서 데이터독이 못 잡는다는 빙빙 도는 에이전트의 의미를 설명할 수 있습니다.

1.2.3 Hallucinated reasoning

앞 단원에서는 에이전트가 같은 자리를 빙빙 도는 planning loop를 다루었습니다. planning loop는 흐름이 멈추지 않아 답을 못 내는 쪽이라면, 이번 단원에서 살피는 실패 모드는 흐름은 잘 끝나는데 그 끝에 놓인 결론이 도구 결과와 어긋나 있는 쪽입니다. 이 모순적인 종결을 부르는 이름이 hallucinated reasoning입니다.

먼저 용어 자리부터 잡겠습니다. 에이전트는 매 단계에서 추론을 거쳐 도구를 호출하고, 도구가 돌려준 관찰을 다시 추론에 입력합니다. 정상 흐름에서는 추론이 관찰의 영향을 받습니다. 검색 결과에 'A는 휴업 중'이라는 정보가 들어오면, 다음 추론에서 'A는 영업 중'이라는 가정은 사라져야 합니다. 그런데 가끔 모델은 관찰을 받아들이지 않고 직전 추론의 가정을 그대로 끌고 갑니다. 도구 결과에 근거하지 않은 추론이 결론으로 굳어지는 이 패턴을 **hallucinated reasoning**이라고 부릅니다.

이 패턴은 단일 LLM 응답에서 자주 거론되는 사실 환각과 닮아 보이지만 결이 다릅니다. 사실 환각은 모델이 외부에 정보를 묻지 않고 자기 기억으로만 답할 때 가짜 사실을 지어내는 현상입니다. 반면 hallucinated reasoning은 외부에 정보를 묻고 응답까지 받아 둔 상태에서 그 응답과 모순되는 결론을 내는 현상입니다. 즉, 에이전트는 도구를 부르긴 부른 것입니다. 다만 도구 결과를 활용하지 않았을 뿐입니다.

대표적인 사례를 시나리오로 보겠습니다. 사용자가 '서울 강남역 근처에 지금 영업 중인 한식당이 있나요?'라고 묻습니다. 에이전트는 `search_restaurants(area='강남역', open_now=True)`를 부릅니다. 도구는 '근처 영업 중 한식당 0곳, 인근 영업 중 일식당 3곳'이라는 결과를 돌려줍니다. 정상 흐름이라면 모델은 '한식당은 지금 영업 중인 곳이 없습니다'라고 응답하거나, 한식 조건을 풀고 다른 음식점을 제안할지 묻는 단계로 넘어갑니다. 그런데 hallucinated reasoning이 일어난 흐름에서는 모델이 마치 결과에 한식당이 들어 있던 것처럼 'OO한식이 영업 중'이라는 식의 답을 만듭니다. 도구 결과의 0곳이라는 신호가 추론에 반영되지 않은 것입니다.

이 흐름을 도식으로 옮기면 다음과 같습니다.

```
[사용자]  "강남역 영업 중 한식당?"
[추론 1]  "search_restaurants 부르자"
```

[도구 결과] 한식당 0, 일식당 3

[추론 2] "OO한식이 영업 중인 것 같다" <- 결과와 모순

[응답] "OO한식을 추천드립니다" <- 근거 없음

도식을 보면 모순이 생기는 자리가 분명히 보입니다. 추론 2가 도구 결과를 인용하는 대신 모델이 가진 일반 지식이나 사용자 질문의 표현에서 끌려옵니다. 이런 결론을 굳혀 응답으로 내보내는 순간, 사용자는 '에이전트가 검색해 봤다'는 인상을 받지만 실제로는 검색 결과를 무시한 응답을 받습니다.

이런 일이 일어나는 까닭은 모델이 직전 추론을 자기 자신의 가장 강력한 단서로 쓰기 때문입니다. 모델 입장에서 추론 1은 '나도 방금 그렇게 생각했다'는 흐름의 일관성으로 작동합니다. 도구 결과가 약하게 표현되어 들어오면, 일관성을 유지하려는 압력이 결과의 무게를 누릅니다. 결과가 빈 배열, 한 줄짜리 빈 응답, 모호한 안내문일수록 이 현상이 잘 일어납니다. 또한 에이전트가 동시에 여러 도구를 부른 뒤 결과를 합치는 단계에서, 한 도구의 결과만 비중 있게 반영하고 다른 도구의 결과를 흘려보내는 형태로도 나타납니다. 이때 에이전트는 자기 결론에 유리한 결과만 선택적으로 인용하게 됩니다.

또 하나 자주 보이는 모습은 **근거 없는 사실 주장**입니다. 모델이 도구를 한 번도 부르지 않은 영역에 대해, 마치 도구 결과인 것처럼 구체적인 수치나 일정을 응답에 끼워 넣는 경우입니다. 예컨대 검색 결과에는 일식당 정보만 있는데 응답에는 '한식당 OO이 22시까지 영업한다'는 문장이 들어가는 식입니다. 사용자 입장에서는 '검색해서 알려 준 정보'로 보이고, 따라서 더 잘 속습니다. 일반 LLM 환각보다 hallucinated reasoning이 더 위험한 까닭이 여기 있습니다. 도구를 부르는 에이전트는 사용자에게 '근거 있는 답'이라는 신뢰를 깔고 응답하기 때문에, 가짜 정보가 더 진짜처럼 흐릅니다.

평가 쪽에서는 이 패턴을 잡기 위해 final-answer만 보는 채점에서 한 발 더 들어가야 합니다. 핵심은 trajectory에 남은 도구 결과와 최종 응답을 함께 보는 것입니다. 이를 **trajectory 검증**이라고 부르겠습니다. 가장 단순한 형태는 응답에 등장한 사실들을 trajectory의 도구 결과 안에서 일치하는 부분으로 매핑할 수 있는지를 보는 것입니다. 응답에 'OO한식 22시까지'가 들어 있다면, 이 문장이 검색 결과의 어떤 항목에서 왔는지 가리키는 출처 매핑이 가능해야 합니다. 가능하지 않다면 그 문장은 근거 없는 주장으로 분류됩니다.

trajectory 검증을 메트릭으로 다듬은 것이 **근거-결론 일치도 평가 항목**입니다. 한 응답에 들어 있는 사실 명제를 추출하고, 각 명제가 trajectory의 어떤 단계에 있는 도구 결과와 부합하는지를 사람이나 평가용 LLM이 판단합니다. 부합 여부에 따라 명제 단위로 점수를 매겨 평균을 내면, 같은 응답 안에서 어느 부분이 근거에 의지하고 어느 부분이 모델 추론으로 메워졌는지가 보입니다. 메트릭의 모양을 도식으로 정리하면 다음과 같습니다.

응답 명제	근거가 trajectory에 있나? 분류	
OO한식 영업 중	아니오 (결과는 0곳)	근거 없음
일식 3곳 가능	예 (search 결과에 있음)	근거 있음
22시까지 영업	아니오 (영업 시간 미조회)	근거 없음
근거 일치도	1 / 3	0.33

이 표에서 보듯 근거 일치도가 낮은 응답은 final-answer가 아무리 그럴듯해도 hallucinated reasoning이 끼여든 흐름으로 표시됩니다. 시스템 차원에서 이 메트릭을 추적하면, 어느 도구의 결과가 자주 무시되는지, 어떤 종류의 사용자 질문에서 모델이 자기 추론을 더 신뢰하는지 같은 패턴이 드러납니다.

정리하면, hallucinated reasoning은 도구 호출 자체는 일어났지만 그 결과를 무시하고 자기 추론을 따라가는 실패 모드이며, 사실 환각보다 더 진짜처럼 보이는 응답으로 사용자를 오도합니다.

final-answer만으로는 잘 잡히지 않으므로 trajectory 검증을 통해 응답 명제를 도구 결과로 되짚어 보고,

근거-결론 일치도를 평가 항목으로 두어 어느 흐름에서 모델이 결과를 무시했는지를 가시화해야 합니다. 다음 단원인 1.2.4에서는 정반대 결의 실패 모드, 즉 작업을 완수하지 않고 흐름을 너무 일찍 끊어 버리는 premature termination을 살펴보겠습니다.

이 단원을 마치면 hallucinated reasoning의 정의와 도구 결과를 무시하고 자기 추론을 따르는 패턴, 근거 없는 사실 주장의 사례를 들 수 있고, trajectory 검증으로 잡는 방법과 근거-결론 일치도 평가 항목의 의미를 설명할 수 있습니다.

1.2.4 Premature termination

앞 단원까지 도구 선택 오류, planning loop, hallucinated reasoning 세 가지 실패 모드를 살펴보았습니다. 앞 두 가지가 흐름이 너무 길어지는 쪽의 실패라면 hallucinated reasoning은 길이는 적당하지만 결론이 어긋나는 실패였습니다. 이번 단원에서 다루는 premature termination은 또 다른 결의 실패로, 흐름이 너무 일찍 끊겨 사용자가 부탁한 일이 마무리되지 않은 상태로 응답이 돌아가는 패턴입니다.

용어부터 풀겠습니다. **premature termination**은 사용자가 요청한 작업이 아직 끝나지 않았는데 에이전트가 흐름을 종료해 버리는 실패 모드를 가리킵니다. 글자 그대로 옮기면 '조기 종료'에 가깝습니다. 에이전트가 종료한다는 것은 더 이상 도구를 부르지 않고 사용자에게 응답을 돌려주는 결정을 내렸다는 뜻이므로, premature termination은 응답이 돌아갔다는 사실 자체로는 정상처럼 보인다는 점에서 까다롭습니다.

조기 종료를 만드는 원인은 크게 세 갈래로 정리할 수 있습니다.

첫째, **토큰 한계**입니다. 모델은 한 번에 처리할 수 있는 컨텍스트 길이가 정해져 있습니다. 에이전트가 단계를 거듭하며 컨텍스트에 추가 정보를 쌓으면, 어느 시점에 토큰 한계에 도달합니다. 한계에 도달하면 모델은 더 이상 깊은 추론이나 추가 도구 호출을 안정적으로 수행하지 못하고, 시스템은 흐름을 안전하게 닫는 쪽을 선택합니다. 사용자 입장에서는 부탁한 일의 일부만 처리된 응답이 돌아옵니다.

둘째, **max steps 도달**입니다. 1.2.2에서 본 것처럼 planning loop를 막기 위해 시스템은 한 사용자 요청이 만들 수 있는 최대 단계 수를 정해 둡니다. 그런데 이 임계치는 luping뿐 아니라 정상적으로 길게 진행되는 작업도 잘라 버립니다. 한 사용자 요청이 정상으로 14단계가 걸리는 보고서 작성인데 max steps가 10이라면, 에이전트는 10단계째 강제로 종료되며 작성 중이던 결과만 남깁니다.

셋째, **자체 판단 종료**입니다. 모델이 스스로 '이 정도면 답을 줘도 된다'고 판단하고 호출을 멈추는 경우입니다. 사용자가 '강남역 근처 한식당 세 곳을 가격대까지 비교해 줘'라고 부탁했는데, 에이전트가 한 곳만 검색한 뒤 'OO식당이 좋습니다'로 응답을 끝내 버리는 식입니다. 이때는 토큰 한계도 max steps도 닿지 않았는데도 흐름이 닫혔습니다. 모델이 자기 판단으로 충분하다고 본 것입니다.

이 세 원인을 표로 비교하면 다음과 같습니다.

원인	종료의 트리거	사용자가 보는 모습
토큰 한계	컨텍스트 누적이 모델 한계 초과	잘린 듯한 응답, 일부만 답함
max steps 도달	단계 수가 정책 임계치 도달	중도에 멈춘 응답
자체 판단 종료	모델이 충분하다고 판단	사용자 의도와 어긋난 종결

세 원인 가운데 가장 다루기 어려운 쪽이 자체 판단 종료입니다. 토큰 한계와 max steps는 시스템 쪽에서 임계치를 조정하면 어느 정도 통제가 됩니다. 그러나 자체 판단 종료는 모델 안에서 일어난 결정이라, 시스템은 종료의 결과만 보고 그 안의 이유를 모릅니다.

자체 판단 종료의 뿌리에는 **사용자 의도와 에이전트 종료 조건의 불일치**가 있습니다. 사용자가 '비교해 달라'는 표현을 썼는데 에이전트는 '하나라도 좋은 추천을 주는 것'을 작업의 종료 조건으로 잡으면, 충실히 한 곳을 찾아 응답을 닫고 사용자가 보기에는 일이 덜 끝난 응답이 돌아옵니다. 같은 'X를 정리해 줘'라는 표현도 사용자에게 따라 '한 페이지 요약', '항목별 표', '비교표'로 의미가 달라지는데, 에이전트의 종료 조건이 이런 의미 차이를 반영하지 못하면 잦은 조기 종료가 발생합니다.

조기 종료를 잡기 위해 시스템 차원에서 두는 가장 기본 메트릭이 **완수율**입니다. completion rate라고도 부릅니다. 한 사용자 요청에 대해 '요청에 담긴 모든 작업이 응답 안에 처리되었는가'를 0과 1 사이의 비율로 매기는 메트릭입니다. 가장 단순한 형태는 사용자 요청을 작업 단위로 쪼갬 뒤, 응답에서 각 작업이 처리된 흔적이 있는지를 사람이나 평가용 LLM이 채점해 평균을 내는 것입니다. '비교'라는 표현이 들어 있으면 비교 대상의 수와 항목별 처리 여부를 함께 봅니다.

완수율을 도식으로 풀면 다음과 같습니다.

사용자 요청	작업 단위	응답 처리 여부	완수율
강남역 한식당 3곳 비교	1) 3곳 추출	예	3 / 3 = 1.0
	2) 가격 비교	예	
	3) 추천 정리	예	
같은 요청 (조기 종료)	1) 1곳만 추출	부분	1 / 3 = 0.33
	2) 가격 비교	아니오	
	3) 추천 정리	예	

조기 종료가 잦은 시스템에서는 평균 완수율이 1에서 떨어지는 패턴이 시간에 따라 보입니다. 이 패턴을 다른 메트릭과 짝지어 보면 원인 갈래가 잡힙니다. 평균 단계 수가 max steps에 자주 닿으면 임계치 문제고, 평균 단계 수는 여유가 있는데 완수율만 낮으면 자체 판단 종료가 우세합니다.

이렇게 원인 갈래가 잡히면 그다음 결정은 **재시도와 부분 응답 정책**입니다. 두 정책의 차이는 다음과 같습니다. 재시도 정책은 조기 종료가 감지되면 같은 요청을 같은 또는 보강된 컨텍스트로 다시 흘러보내는 방식입니다. 일이 길어지지만 사용자에게는 결국 완성된 응답이 돌아갈 가능성이 커집니다. 부분 응답 정책은 끝까지 가지 못한 사실을 사용자에게 명시적으로 알리는 응답을 만드는 방식입니다. 사용자에게 '세 곳 중 한 곳까지 비교했고 나머지는 다음에 이어 진행하시겠어요?'처럼 알리는 응답이 그 예입니다.

정책	동작	장점	단점
재시도	조기 종료 감지 후 자동으로 흐름 재개	완수율 회복	자연·비용 증가
부분 응답	종료 사실과 진행 정도를 사용자에게 통지	사용자 신뢰 유지	완수율은 낮음

선택은 도메인에 따라 갈립니다. 사용자가 빠른 응답을 더 중시하는 흐름에서는 부분 응답이 유리하고, 결과의 완성도가 중요한 흐름에서는 재시도가 유리합니다. 두 정책을 섞기도 합니다. 첫 흐름에서는 재시도하지 않고 부분 응답을 돌려주되, 사용자가 '이어서'라고 답하면 재시도가 트리거되는 식입니다. 어느 쪽이든, 정책을 정하지 않고 두면 시스템은 침묵으로 잘린 응답을 그대로 내보내고, 사용자는 일이 끝났다고 오해하게 됩니다.

정리하면, premature termination은 사용자가 요청한 작업이 끝나기 전에 흐름이 닫히는 실패 모드이며, 토큰 한계와 max steps 도달, 자체 판단 종료 세 가지 원인에서 비롯됩니다. 조기 종료는 응답이 돌아갔다는 사실만으로는 잘 잡히지 않으므로 완수율을 별도 메트릭으로 두고, 원인 갈래에 따라 재시도와 부분 응답 정책 가운데 어느 쪽을 기본으로 둘지를 미리 정해야 합니다.

여기까지 1.2의 네 가지 실패 모드를 모두 살펴보았습니다. 도구 선택 오류는 한 단계의 잘못된 결정, planning loop는 흐름이 종결되지 않는 문제, hallucinated reasoning은 결론이 도구 결과와 어긋나는 문제, premature termination은 흐름이 일찍 닫히는 문제로 갈래를 잡았습니다. 다음 단원인 1.3.1에서는 이런 실패 모드를 한 가지 메트릭만으로 쫓을 때 시스템 전체가 어떻게 무너지는지를 단일 메트릭 최적화의 함정이라는 주제로 살펴보겠습니다.

이 단원을 마치면 premature termination의 정의와 토큰 한계, max steps 도달, 자체 판단 종료라는 세 원인을 들 수 있고, 사용자 의도와 종료 조건의 불일치가 어떻게 조기 종료로 이어지는지, 완수율 메트릭의 형태와 재시도·부분 응답 정책의 차이를 설명할 수 있습니다.

1.3 다차원 메트릭 설계

이 섹션의 토픽과 학습 목표는 다음과 같습니다.

1.3.1 단일 메트릭 최적화의 함정 — 단일 메트릭에 최적화한 시스템이 어떻게 hidden quality를 잃는지 Klarna 사례로 설명할 수 있다

1.3.2 Hidden quality와 escalation path — 관측되지 않는 품질 차원과 escalation path를 메트릭으로 측정하는 방법을 설명할 수 있다

1.3.1 단일 메트릭 최적화의 함정

앞 단원에서는 에이전트가 작업을 끝맺지 못하고 빠져나오는 premature termination 같은 실패 모드를 살펴보았습니다. 실패의 종류를 분류하고 나면 곧바로 따라오는 질문이 있습니다. '그래서 이 시스템이 잘 돌아가는지 어떻게 한 숫자로 보여 줄 것인가.' 이 질문에 잘못 답하면 평가가 오히려 시스템을 망치는 도구가 됩니다. 이 단원은 단일 지표 하나에 모든 것을 걸었을 때 어떤 일이 일어나는지를 Klarna라는 핀테크 회사의 사례로 풀어 보겠습니다.

먼저 용어부터 정리하겠습니다. **KPI**(Key Performance Indicator)는 조직이 성과를 추적하기 위해 고른 핵심 지표를 가리킵니다. 고객지원이라면 평균 처리 시간이나 만족도 점수가, 광고라면 클릭률이나 전환율이 KPI가 됩니다. 한 시스템에 KPI가 여러 개 있을 수 있지만, 실무에서는 결국 한두 개가 의사결정을 사실상 지배하는 경우가 많습니다. 이렇게 실질적으로 단 하나의 지표만이 평가의 무게를 다 실어 가는 상황을 **단일 메트릭 최적화**라고 부릅니다.

단일 메트릭 최적화는 얼핏 깔끔하고 객관적으로 보입니다. 숫자 하나가 오르면 좋은 것이고, 내려가면 나쁜 것이라는 단순한 규칙은 의사결정을 빠르게 만들어 줍니다. 그러나 같은 단순함이 함정을 만듭니다. 시스템은 우리가 측정하는 것을 향해 움직이지, 우리가 진짜로 원하는 것을 향해 움직이지 않습니다. 측정되지 않는 차원은 자연스럽게 뒷전이 되고, 어떤 경우에는 측정되는 지표를 끌어올리기 위해 측정되지 않는 차원이 희생됩니다. 이 현상을 잘 보여 주는 사례가 Klarna의 고객지원 자동화입니다.

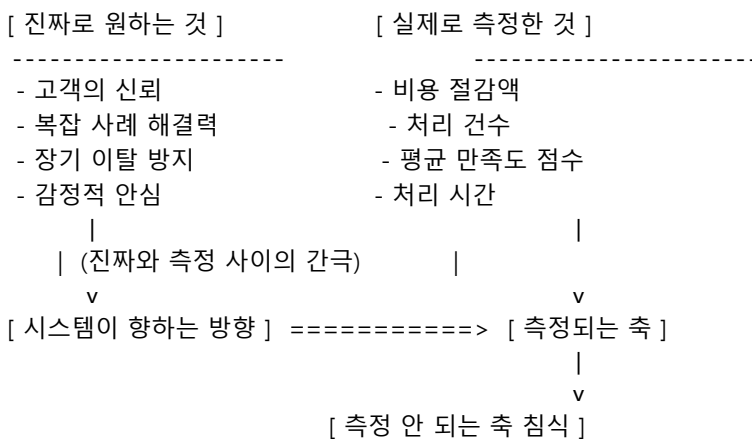
Klarna는 2024년에 자사의 AI 챗봇 도입 결과를 화려한 숫자로 발표했습니다. 23개 시장에서 35개 언어로 운영되며 한 달에 230만 건의 대화를 처리했고, 이 분량은 사람 상담사 700명 분에 해당했습니다. 고객만족도(CSAT)는 종전 대비 47퍼센트 증가했고, 절감액은 연간 1천만 달러에 이른다고 했습니다. 그 1천만 달러는 고객지원 영역에서 4백만 달러, 마케팅 영역에서 6백만 달러로 나뉘었습니다. 이 숫자만 놓고 보면 결론은 분명해 보입니다. AI 에이전트로 사람을 대체하는 것이 효율과 품질을 동시에 끌어올리는 길이라는 결론입니다.

그런데 1년쯤 시간이 지난 뒤, 같은 회사의 CEO Sebastian Siemiatkowski는 공개적으로 결이 다른

이야기를 꺼냈습니다. '비용이 너무 압도적인 평가 요소가 되어서, 결국 더 낮은 품질을 얻게 됐다'는 인정이었습니다. Klarna는 그 인정에 이어 인간 상담사를 다시 고용하고 AI와 사람을 함께 쓰는 hybrid 모델로 회귀했습니다. 발표 시점의 화려한 숫자와 사후의 의사결정이 정반대 방향을 가리키게 된 셈입니다. 무엇이 일어난 것일까요.

핵심은 평가의 무게중심이 한쪽으로 너무 기울어 있었다는 점입니다. 비용 절감이라는 단일 축이 나머지 모든 차원을 압도하면서, 측정되지 않는 품질이 조용히 침식되었습니다. CSAT 점수가 47퍼센트 증가했다고는 하지만, 그 점수가 잡지 못한 차원이 있습니다. 복잡하고 감정이 실린 문의에서 고객이 '내 말을 안 들어 준다'거나 '도움받지 못했다'고 느끼는 신호, 같은 사람이 다음에는 더 이상 문의 자체를 시도하지 않게 되는 신호, 챗봇과의 짧은 만족 뒤에 따라오는 장기적 이탈 신호 같은 것들입니다. 이런 신호는 표준 만족도 설문에는 잘 잡히지 않습니다.

이 구조를 그림으로 옮기면 다음과 같습니다.



위 그림에서 왼쪽의 '진짜로 원하는 것'과 오른쪽의 '실제로 측정한 것' 사이에는 늘 간극이 있습니다. 시스템은 오른쪽을 향해 최적화되고, 왼쪽은 명시적으로 보호받지 못하면 점점 갉여 나갑니다. Klarna의 경우 비용 절감이라는 한 축이 이 간극을 더 크게 벌렸고, 그 결과가 hybrid 모델로의 회귀였습니다.

이 사례가 주는 교훈은 '비용을 재지 말라'가 아닙니다. 비용은 당연히 재야 합니다. 교훈은 다음과 같이 세 가지로 정리할 수 있습니다. 첫째, 한 차원의 지표가 압도적으로 커지면 그 지표 자체의 정확성이 의심스러워집니다. 측정되는 축에 맞추려고 측정되지 않는 축이 희생되기 때문입니다. 둘째, '품질이 좋아졌다'는 단일 점수는 어떤 종류의 품질이 좋아졌는지를 자동으로 알려 주지 않습니다. 만족도 평균이 올랐다는 사실과, 정말 도움이 절실했던 소수의 케이스에서 충분한 도움을 받았는지는 별개의 문제입니다. 셋째, 발표 시점의 성공 지표만으로는 시스템의 건강을 판단할 수 없습니다. 사후에 사용자 행동이 어떻게 변하는지를 같이 봐야 합니다.

여기서 자연스럽게 한 가지 개념이 필요해집니다. 측정되고 있지 않지만 사용자에게는 분명히 영향을 주는 품질, 곧 **hidden quality**입니다. Klarna의 사례는 hidden quality를 명시적으로 측정하지 않으면 단일 메트릭 최적화가 그것을 갉아먹는다는 것을 보여 줍니다. 비용 절감이라는 한 축이 너무 매력적이어서 그 자체가 의사결정을 잠식하고, 점차 다른 모든 신호를 묵음 처리해 버린 것입니다.

평가 설계자의 역할이 여기서 분명해집니다. 평가 설계자는 '어떤 지표를 추적하느냐'만이 아니라 '어떤 지표가 추적되지 않고 있느냐'를 같이 묻는 사람이어야 합니다. 단일 메트릭 한두 개에 모든 의사결정을 거는 구조는 이 질문 자체를 사라지게 만듭니다. 한 숫자가 모든 것을 설명한다고 믿는 순간, 그 숫자가 침묵시키고 있는 차원이 보이지 않게 되기 때문입니다.

정리하면, 단일 메트릭 최적화는 의사결정을 빠르게 만드는 대신 측정되지 않는 차원을 침식시키는

위험을 안고 있습니다. Klarna는 23개 시장과 35개 언어, 월 230만 건이라는 압도적 규모와 CSAT 47퍼센트 증가, 1천만 달러 절감이라는 화려한 숫자를 발표했지만, 비용이 평가의 무게를 거의 다 가져간 결과 결국 인간 상담사를 재고용하고 hybrid 모델로 회귀했습니다. 이 사례는 발표 시점의 성공 지표 뒤에 어떤 품질이 조용히 깔려 나가고 있는지를 함께 보지 않으면, 평가 시스템이 오히려 제품을 망가뜨릴 수 있음을 보여 줍니다.

다음 단원인 1.3.2에서는 이 사례에서 침식되었던 hidden quality가 구체적으로 무엇이며, 사람으로 넘기는 escalation path를 어떻게 측정 가능한 지표로 옮길 수 있는지를 다루겠습니다.

이 단원을 마치면 단일 메트릭 최적화가 어떤 메커니즘으로 hidden quality를 잃게 만드는지, Klarna 사례에서 23개 시장·월 230만 건·CSAT 47퍼센트·1천만 달러 절감이라는 성공 지표 뒤에 무슨 일이 일어났는지, 그리고 인간 상담사 재고용이라는 사후 의사결정에 이르게 된 경위를 설명할 수 있습니다.

1.3.2 Hidden quality와 escalation path

앞 단원에서는 Klarna 사례를 통해 단일 메트릭 최적화가 어떻게 시스템의 품질을 조용히 깎아 내는지 살펴보았습니다. 비용이라는 한 축이 의사결정의 무게를 다 가져가면서, 측정되지 않은 차원이 점점 침식되었다는 이야기였습니다. 그렇다면 그 '측정되지 않은 차원'은 구체적으로 무엇이며, 측정되지 않던 것을 어떻게 측정할 수 있는 형태로 바꿀 수 있는지가 다음 질문이 됩니다. 이 단원은 hidden quality라는 개념과 escalation path라는 장치를 통해 그 질문에 답하겠습니다.

먼저 **hidden quality**라는 용어부터 풀겠습니다. 이 말은 사용자가 분명히 체감하지만 표준 만족도 점수나 비용 지표 같은 일반적 KPI에 잘 잡히지 않는 품질 차원을 가리킵니다. 단어 그대로 '숨겨진 품질'입니다. 왜 숨겨지냐 하면, 보통 KPI가 계량하기 좋은 것부터 골라내기 때문입니다. 처리 시간, 처리 건수, 평균 점수 같은 항목은 숫자로 떨어지기 쉬워 우선 채택됩니다. 반면 '내 상황을 알아들었다'는 느낌, '서두르지 않고 들어 줬다'는 안도감, '복잡한 사정을 끝까지 따라와 줬다'는 신뢰 같은 것은 한 번에 숫자가 되지 않습니다.

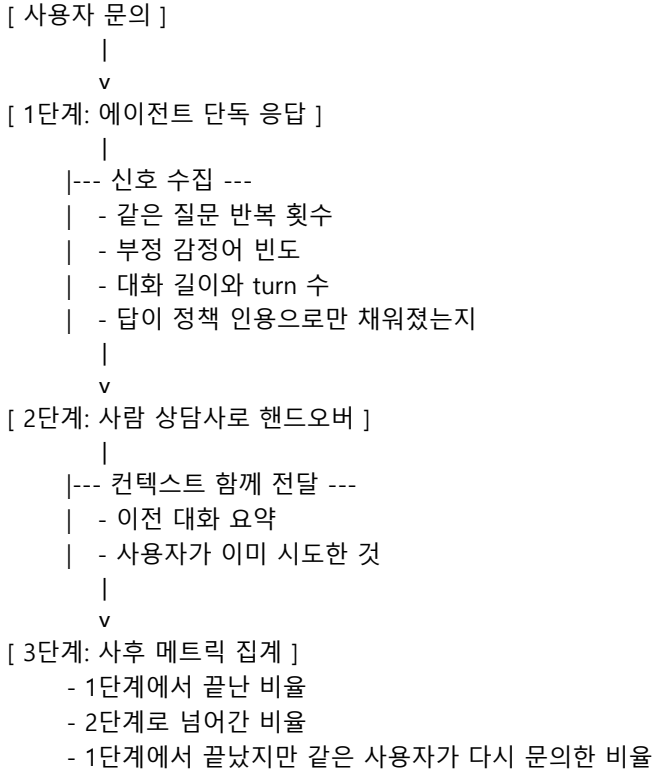
hidden quality의 대표적인 세 축을 짚으면 다음과 같습니다. 첫째는 **공감**입니다. 사용자가 자기 감정을 인정받았다고 느끼느냐의 문제입니다. 환불이 거절되어 화가 난 고객에게 정책만 빠르게 안내하는 응답과, 화가 날 만한 상황이라고 먼저 짚어 준 뒤 정책을 안내하는 응답은 같은 결과를 주더라도 체감은 완전히 다릅니다. 둘째는 **안심**입니다. 처음 받아 본 청구서, 처음 본 보안 알림 앞에서 사용자는 자신이 안전한 상태인지 확인받고 싶어 합니다. 답이 빨리 와도 이 안심이 빠지면 사용자는 같은 질문을 다시 던지게 됩니다. 셋째는 **상황 인식**입니다. 같은 문구의 질문이라도 그 사람의 이전 거래, 분쟁 기록, 시점에 따라 의미가 다릅니다. 시스템이 그 맥락을 알고 있다는 사실 자체가 품질의 일부가 됩니다.

Klarna의 사례를 다시 들고 와 보면 hidden quality가 무엇이었던지가 분명해집니다. CSAT 47퍼센트 증가라는 숫자가 보여 주지 않은 것은, 복잡하고 감정이 실린 문의에서 고객이 어떻게 느꼈느냐였습니다. CEO Sebastian Siemiatkowski가 인정한 표현을 빌리면, 고객은 자신이 'unheard'(들어 주지 않았다고 느낌)하고 'unsupported'(도움받지 못했다고 느낌) 상태에 놓였습니다. 이 두 단어는 단순한 불만이 아닙니다. 들어 주지 않았다는 감각은 공감의 부재를, 도움받지 못했다는 감각은 안심과 상황 인식의 부재를 각각 가리킵니다. Klarna 시스템에는 평균 만족도를 끌어올리는 능력은 있었지만, 이 세 축을 따로 떼어 측정하는 장치는 없었습니다.

그러면 이런 신호를 어떻게 측정 가능한 형태로 바꿀 수 있을까요. 핵심 도구가 **escalation path**입니다. 이 단어는 원래 고객지원 분야에서 '난도가 높은 문의를 상위 단계로 넘기는 경로'를 가리키는 말입니다. AI 에이전트의 맥락에서는 의미가 한 단계 더 정교해집니다. 에이전트 혼자 처리하던 대화를 사람

상담사에게 넘기는 경로, 그리고 그 경로에 도달하기까지 어떤 신호가 쌓이는지를 묶어 부르는 표현입니다. Klarna가 결국 hybrid 모델로 회귀하면서 명시적으로 다시 만든 것이 바로 이 escalation path였습니다.

escalation path를 그림으로 옮기면 다음과 같습니다.



이 그림에서 측정의 핵심은 1단계와 2단계 사이의 흐름입니다. 시스템이 어떤 시점에 사람으로 넘기는지, 넘기지 못하고 사용자가 떠나 버리는지, 일단 끝냈었지만 같은 사용자가 다시 같은 주제로 돌아오는지가 모두 측정 대상입니다. 이 흐름을 측정하기 시작하면, '들어 주지 않았다고 느낀 사용자가 얼마나 있었는가'라는 모호한 질문이 '같은 세션에서 같은 질문을 세 번 이상 반복한 사용자 비율은 얼마인가' 같은 구체적 지표로 바뀝니다.

여기서 escalation path가 단순한 안전망이 아니라 평가 지표의 원천이라는 점이 중요합니다. 사람으로 넘어가는 경로 자체가 어떤 종류의 사례에서 에이전트가 충분하지 않은지를 알려 주는 표지가 됩니다. escalation 비율이 너무 낮으면 잘되고 있다는 신호일 수도 있지만, 정작 도움이 필요한 사용자가 사람으로 넘어가지 못하고 막혀 있다는 신호일 수도 있습니다. 반대로 escalation 비율이 너무 높으면 비용 부담이 커지지만, 동시에 에이전트가 어디까지 책임져야 할지를 다시 그릴 수 있는 데이터가 쌓입니다.

이 시점에서 다차원 메트릭을 어떻게 짤지에 대한 원칙을 정리해 둘 수 있습니다. 다음 표는 효율성·품질·안전성 세 축에 정량 지표와 정성 지표를 각각 매핑한 예시입니다.

+-----+			
축	정량 지표 예시	정성·신호 지표 예시	
+-----+			
효율성	처리 시간, 비용	반복 문의 비율, abandon 율	
품질	완수율, 정답률	공감 표현 포함 여부, 톤	
안전성	정책 위반 차단 횟수	escalation 적시성, HITL 비율	
+-----+			

표에서 정량 지표만으로 한 축을 결론짓지 않는 것이 핵심입니다. 효율성을 처리 시간만으로 평가하면 짧은 응답을 좋은 응답으로 잘못 보상할 수 있고, 품질을 정답률만으로 평가하면 정답이 나왔지만 차갑게 응대된 사례를 거를 수 없습니다. 정성·신호 지표는 이 빈틈을 메우기 위한 보조 축입니다. 둘은 어느 한쪽이 다른 쪽을 대체하지 않으며, 함께 봐야 비로소 시스템의 모습이 입체로 보입니다.

다차원 메트릭을 설계할 때 흔히 빠지는 함정도 짚어 두겠습니다. 첫째, 지표를 너무 많이 늘려서 어느 하나도 의사결정에 쓰이지 않게 되는 경우입니다. 다차원이라는 말은 '많을수록 좋다'가 아니라 '서로 보완하는 최소한의 차원이 모여 있다'는 뜻입니다. 둘째, 정성 지표를 측정한다고 했지만 사실은 한 사람의 직관에만 의존하는 경우입니다. 공감 표현 포함 여부 같은 정성 신호도 라벨링 가이드와 합의 절차를 거쳐야 비로소 지표가 됩니다. 셋째, 가드레일 메트릭과 북극성 메트릭의 구분이 흐려지는 경우입니다. 비용은 일정 수준을 넘지 않아야 하는 가드레일이지, 끝없이 줄여야 하는 북극성이 아닙니다. Klarna의 실패는 가드레일이어서 할 비용 지표가 사실상 북극성처럼 작동한 데에서 출발했습니다.

이 모든 흐름은 결국 한 가지 원칙으로 수렴합니다. 단일 메트릭에 의사결정의 무게를 다 신지 않고, hidden quality에 해당하는 차원을 명시적으로 표지하며, 사람으로 넘어가는 경로 자체를 측정 가능한 지표로 끌어올리는 것입니다. 이 원칙이 잡혀 있으면 정량 지표가 아무리 좋아 보여도 그 뒤에서 어떤 품질이 침식되고 있는지를 별도의 신호로 읽어 낼 수 있습니다. Klarna가 hybrid 모델로 회귀하면서 다시 세운 것이 바로 이 구조입니다.

정리하면, hidden quality는 공감·안심·상황 인식처럼 표준 KPI에 잘 잡히지 않지만 사용자가 실제로 체감하는 품질 차원이며, Klarna 사례의 unheard-unsupported 신호는 이 차원이 무너졌을 때 나타나는 전형적인 모습입니다. escalation path는 그 무너짐을 다시 측정 가능한 지표로 옮기는 장치이며, 1단계 단독 응답에서 2단계 사람 핸드오버로 넘어가는 흐름과 그 사이에서 쌓이는 신호들을 함께 추적함으로써 다차원 메트릭의 한 축을 이룹니다. 효율성·품질·안전성 세 축에 정량과 정성 지표를 함께 배치하고, 가드레일과 북극성을 분리하는 것이 단일 메트릭의 함정에서 벗어나는 출발점입니다.

다음 단원인 2.1.1에서는 이렇게 짠 다차원 메트릭을 실제로 회귀 테스트할 수 있도록, 평가의 기준점을 만들어 주는 골든 데이터셋의 역할과 그 한계를 다루겠습니다.

이 단원을 마치면 hidden quality를 공감·안심·상황 인식 세 축으로 풀어 설명하고, Klarna 사례의 unheard-unsupported 신호가 가리키는 품질 차원의 침식을 식별하며, escalation path를 1단계·2단계·사후 집계로 구조화해 측정 가능한 지표로 옮기는 방법, 그리고 효율성·품질·안전성 세 축에 정량·정성 지표를 균형 있게 배치하는 다차원 메트릭 설계 원칙을 설명할 수 있습니다.

2 Offline 평가 설계

골든 데이터셋과 LLM-as-judge를 결합한 offline 평가 파이프라인을 설계하고, 학술 벤치마크를 활용해 자체 시스템을 측정할 수 있다.

이 Phase는 다음 섹션으로 구성됩니다.

- 2.1 골든 데이터셋
 - 2.1.1 골든 데이터셋의 역할과 한계
 - 2.1.2 골든 데이터셋 구축 절차
 - 2.1.3 데이터셋 진화와 회귀 테스트
- 2.2 LLM-as-Judge
 - 2.2.1 LLM-as-Judge 동작 원리
 - 2.2.2 평가 LLM의 편향과 한계
 - 2.2.3 Judge 보정 기법
- 2.3 Trajectory 평가
 - 2.3.1 Trajectory 평가 설계
 - 2.3.2 단계별 정확도 측정
- 2.4 학술 벤치마크
 - 2.4.1 τ -bench
 - 2.4.2 AgentBench
 - 2.4.3 SWE-bench
- 2.5 평가 환경 설계
 - 2.5.1 Eval 환경 isolation
 - 2.5.2 Reproducibility와 shared state 위험

2.1 골든 데이터셋

이 섹션의 토픽과 학습 목표는 다음과 같습니다.

2.1.1 골든 데이터셋의 역할과 한계 — 골든 데이터셋이 평가에서 차지하는 역할과 그 한계를 구분하여 설명할 수 있다

2.1.2 골든 데이터셋 구축 절차 — 실 트래픽 샘플링부터 라벨링·검수까지 골든 데이터셋 구축의 단계를 설명할 수 있다

2.1.3 데이터셋 진화와 회귀 테스트 — 데이터셋이 시간에 따라 진화해야 하는 이유와 회귀 테스트로 활용하는 방법을 설명할 수 있다

2.1.1 골든 데이터셋의 역할과 한계

에이전트가 비결정적이라는 사실을 받아들이고 다차원 메트릭으로 측정해야 한다는 데까지 왔다면, 다음 질문은 자연스럽습니다. 그 측정을 어디에 대고 할 것인가. 매번 프로덕션 트래픽이 흘러올 때까지 기다려서 재는 방식은 너무 늦고, 코드 한 줄 바꿀 때마다 사람이 결과를 일일이 확인하는 방식은 너무 비쌉니다. 그래서 평가의 출발점에는 항상 미리 준비된 데이터셋이 놓입니다. 이 단원은 그중에서도 가장

자주 등장하는 이름인 **골든 데이터셋**이 무엇이고, 어디까지 믿을 수 있는지를 다룹니다.

먼저 이름부터 풀어 보겠습니다. 광산에서 채취한 광석 중 분명한 금만 골라 건본으로 두면, 새로 캐낸 돌이 금인지 아닌지 비교할 기준이 생깁니다. 소프트웨어 평가에서도 비슷한 비유를 빌려 옵니다. 입력과 그 입력에 대한 정답이 짝지어져 있고, 그 정답을 사람이 검토해 신뢰할 수 있다고 합의해 둔 묶음이 있다면, 새로 만든 시스템의 출력이 그 정답에 얼마나 가까운지 비교할 수 있습니다. 이렇게 정답을 신뢰할 수 있는 건본 묶음을 골든 데이터셋이라고 부릅니다.

조금 더 단단하게 정의를 놓겠습니다. **골든 데이터셋**이란 평가 대상이 되는 입력 사례와, 그 입력에 대해 사람이 검수해 정답으로 합의한 라벨이 함께 묶여 있는 데이터 묶음입니다. 여기서 핵심은 두 가지입니다. 입력이 실제로 마주칠 만한 사례여야 한다는 점, 그리고 라벨이 한 사람의 즉흥 판단이 아니라 검토를 거친 합의여야 한다는 점입니다. 이 두 조건이 갖춰져야 비로소 '기준'이라고 부를 수 있고, 시스템 변경이 정답에서 멀어지는지 가까워지는지를 일정한 잣대로 판단할 수 있습니다.

이 데이터셋이 실제로 하는 일을 한 가지로 압축하면 **회귀 테스트**입니다. 회귀라는 말은 원래 의학에서 나아지던 환자가 다시 나빠지는 현상을 가리키는데, 소프트웨어에서는 잘 돌던 기능이 코드 변경 이후에 다시 망가지는 일을 말합니다. 에이전트는 프롬프트 한 줄, 도구 설명 한 글자, 모델 버전 하나만 바뀌어도 출력이 달라질 수 있습니다. 어제까지 잘 처리하던 환불 문의를 오늘 갑자기 잘못 분류하는 일이 흔합니다. 골든 데이터셋을 같은 입력으로 매번 돌려 보면, 그 변경이 기존에 맞았던 사례를 깨뜨렸는지 빠르게 확인할 수 있습니다. 변경 전의 점수와 변경 후의 점수를 같은 잣대 위에 놓고 비교하는 일이 회귀 테스트의 본질입니다.

회귀 테스트로 자리 잡으려면 데이터셋이 충분히 안정적이어야 합니다. 입력은 한 번 고르면 가급적 그대로 두고, 라벨도 가벼운 변덕으로 바뀌지 않아야 합니다. 그래야 어제와 오늘의 점수 차이가 시스템 변경 때문인지, 데이터셋 자체가 흔들려서인지 구분이 됩니다. 이 안정성을 만들어 주는 비용이 바로 다음 단계로 이어집니다.

첫 번째 단계는 **정답 라벨링의 비용**입니다. 입력만 모으면 끝이 아니라, 그 입력에 대해 무엇이 옳은지 사람이 정해 줘야 합니다. 단순한 분류 문제라면 한 건당 몇 초로 끝날 수 있지만, 에이전트가 다루는 사례는 대개 그렇게 간단하지 않습니다. 다단계 도구 호출의 어느 시점에 어떤 도구를 불러야 했는지, 어떤 정보를 사용자에게 다시 물어야 했는지, 부분 응답을 허용할 만한 상황이었는지 같은 판단은 한 건에 몇 분에서 십수 분이 걸립니다. 한 사람의 판단을 그대로 정답으로 쓰면 라벨이 한쪽으로 치우치기 쉬워서, 같은 사례를 두 명 이상이 독립적으로 라벨링한 뒤 합의를 거치는 절차가 자주 들어갑니다. 비용은 다시 두 배 이상으로 늘어납니다. 그래서 골든 데이터셋의 크기는 흔히 수백에서 수천 건 단위에 머무르며, 그 수가 곧 평가의 통계적 신뢰도와 직결됩니다.

두 번째 단계는 **분포 편향**입니다. 데이터셋을 만드는 사람이 의도하지 않더라도, 모은 입력은 자연스럽게 어떤 쪽으로 쏠립니다. 예컨대 사내 베타 사용자가 던진 질문만 모으면 사용자층이 가진 어휘와 관심사가 그대로 반영되고, 잘 알려진 실패 사례 위주로 모으면 흔한 일상 사례가 빠지며, 영어로 들어온 트래픽만 모으면 한국어 사용자가 보낼 만한 어순이나 호칭이 전혀 들어가지 못합니다. 데이터셋이 한쪽으로 쏠리면 점수도 그쪽 분포에서만 올라가고, 점수가 오른다고 해서 전체 사용자가 더 만족하리라는 보장이 사라집니다. 다음 단원에서 살펴볼 샘플링 전략의 출발점이 바로 이 분포 편향을 의식하는 데 있습니다.

세 번째 단계는 **현실 트래픽과의 갭**입니다. 골든 데이터셋이 아무리 잘 짜여도, 실제 사용자는 데이터셋이 만들어진 시점 이후의 세상을 살아갑니다. 새로운 기능이 출시되면 그 기능에 관한 문의가 갑자기 늘어나고, 시즌이 바뀌면 환불 사유가 달라지며, 모델이 학습한 적 없는 신조어가 한 달 사이에 보편 어휘가 됩니다. 데이터셋은 멈춰 있고 현실은 흐르므로, 시간이 갈수록 둘 사이의 간격이 벌어집니다. 이 간격은 단순한 노후화가 아니라, 데이터셋에서는 만점을 받는 시스템이 프로덕션에서는 사용자 불만을

만드는 상황으로 이어집니다.

이 세 한계를 한눈에 정리하면 다음과 같습니다.

[골든 데이터셋]

- |
- +-- 역할: 회귀 테스트의 기준선
- |
- +-- 한계 1: 정답 라벨링 비용 (사람 시간이 든다)
- +-- 한계 2: 분포 편향 (모든 표본이 한쪽으로 쏠린다)
- +-- 한계 3: 현실 갭 (만든 시점 이후의 세상이 빠진다)

이 그림에서 화살표가 보여 주는 함의는 단순합니다. 골든 데이터셋은 변경의 영향을 빠르게 확인하는 데는 강하지만, 그것 하나로 시스템의 전체 품질을 단정 짓기에는 약하다는 점입니다. 데이터셋의 점수가 올랐다는 사실은 '예전에 맞던 것을 여전히 맞힌다'에 가깝고, '사용자에게 더 나은 경험을 준다'와는 다른 차원입니다. 앞 단원에서 본 escalation path 같은 hidden quality가 데이터셋 안에 라벨로 잡혀 있지 않다면, 그 차원의 점수는 데이터셋 위에서는 보이지 않습니다.

그래서 실무에서는 골든 데이터셋을 평가의 전부로 쓰지 않고, 다른 평가 방식과 짝지어 사용합니다. 프로덕션에서 들어오는 실제 사용자 트래픽을 표본으로 보는 흐름, 사람이 직접 사례를 만져 보는 manual testing, 사용자 피드백 신호를 묶어서 보는 흐름이 함께 굴러가야, 데이터셋의 한계가 가려 두는 부분이 노출됩니다. 골든 데이터셋은 이 여러 평가 중 가장 빠르고 자동화하기 쉬운 한 축으로 자리잡으며, 그 위치를 정확히 알고 쓰는 것이 핵심입니다.

정리하면, 골든 데이터셋은 사람이 검수해 합의한 정답 라벨이 입력과 짝지어진 데이터 묶음이며, 시스템 변경이 기존에 맞던 사례를 깨뜨리는지를 같은 잣대로 빠르게 확인하는 회귀 테스트의 기준선 역할을 합니다. 다만 정답을 만드는 데 사람의 시간이 들고, 모든 표본이 한쪽으로 쏠리며, 만든 시점 이후의 현실은 데이터셋 안에 들어오지 못한다는 세 가지 한계를 함께 안고 있어, 이 데이터셋의 점수만으로 시스템 전체의 품질을 단정해서는 안 됩니다.

다음 단원인 2.1.2에서는 이 한계를 의식하면서 골든 데이터셋을 실제로 어떻게 모으고 라벨링하고 검수하는지, 그 구축 절차를 단계별로 살펴보겠습니다.

이 단원을 마치면 골든 데이터셋의 정의를 회귀 테스트라는 역할과 함께 설명하고, 정답 라벨링 비용·분포 편향·현실 트래픽과의 갭이라는 세 한계가 왜 발생하는지를 짚어 데이터셋 점수를 시스템 품질 전체로 일반화하지 않는 태도를 밝힐 수 있습니다.

2.1.2 골든 데이터셋 구축 절차

앞 단원에서 골든 데이터셋이 회귀 테스트의 기준선이라는 역할과, 그것이 안고 있는 세 가지 한계를 짚었습니다. 이번 단원에서는 그 데이터셋을 실제로 어떻게 만드는지, 입력을 고르는 일부러 라벨을 붙이고 검수하고 버전을 관리하기까지의 과정을 단계별로 살펴봅니다. 절차를 미리 그려 두지 않고 데이터셋을 만들면, 한쪽으로 쏠린 표본 위에 한 사람의 즉흥 판단이 라벨로 박혀 버리기 쉽습니다. 그래서 구축은 처음부터 정해진 단계를 따라가는 편이 안전합니다.

전체 흐름을 먼저 한 번에 그리면 다음과 같습니다.

[후보 풀] -> [샘플링] -> [라벨링 가이드] -> [1차 라벨링]
|
v
[2차 라벨링]

[데이터셋 v1] <- [분류 체계 정리] <- [합의 검수]

이 그림에서 가장 먼저 결정해야 하는 단계는 **샘플링 전략**입니다. 후보 풀이란 데이터셋의 입력을 길어 올릴 큰 통을 가리키며, 흔히 프로덕션 로그·내부 테스트 트래픽·사용자 문의 기록 같은 곳이 후보 풀이 됩니다. 이 통에서 어떤 방식으로 입력을 골라 올리는지에 따라, 앞 단원에서 본 분포 편향이 줄어들기도 하고 도리어 굳어지기도 합니다.

샘플링 방식은 보통 세 가지가 함께 쓰입니다. 첫째는 **랜덤 샘플링**입니다. 후보 풀에서 무작위로 표본을 뽑는 가장 단순한 방식으로, 평균적인 트래픽이 어떻게 생겼는지를 비교적 충실히 보여 줍니다. 다만 자주 들어오는 평이한 사례가 압도적으로 많아져서, 드물지만 중요한 사례가 표본에 들어오지 않을 수 있습니다.

둘째는 **계층 샘플링**입니다. 후보 풀을 의미 있는 그룹으로 나눠 두고, 각 그룹에서 일정 비율씩 표본을 뽑는 방식입니다. 예를 들어 환불, 교환, 배송, 결제 오류 같은 문의 카테고리나 나눠 각 카테고리에서 비슷한 수를 뽑으면, 전체 트래픽에서는 적게 차지하던 카테고리도 평가에 충분히 들어올 수 있습니다. 한 카테고리의 점수가 평균에 묻혀 보이지 않게 되는 일을 막는 데 효과적입니다.

셋째는 **어려운 케이스 우선** 샘플링입니다. 시스템이 자주 틀리는 사례, 사용자가 다시 물은 사례, 사람 상담사로 넘어간 사례, 토큰을 비정상적으로 많이 쓴 사례 같은 신호로 어려운 입력을 고른 뒤, 평균 트래픽보다 더 많은 비중으로 데이터셋에 넣는 방식입니다. 평이한 사례에서 만점을 받는 것보다, 어려운 사례에서 점수가 올라가는 쪽이 시스템 개선의 진짜 신호이기 때문에, 이 방식이 변경의 영향을 민감하게 잡아 냅니다.

세 가지 방식은 어느 하나만 쓰는 것이 아니라 조합이 일반적입니다. 예컨대 절반은 카테고리 단위 계층 샘플링으로 채우고, 30%는 어려운 케이스로, 나머지 20%는 랜덤으로 채우는 식입니다. 비율 자체는 도메인마다 다르지만, 세 방식이 보완적으로 들어가야 분포 편향이 줄어든다는 원칙은 비슷합니다.

표본을 정했다면 다음 단계는 **라벨링 가이드 작성**입니다. 라벨링 가이드란 같은 입력을 본 두 사람이 같은 라벨을 붙일 수 있도록 판단 기준을 글로 정리한 문서입니다. 이 문서에는 라벨의 종류와 정의, 헛갈리기 쉬운 경계 사례, 판단이 갈릴 때의 우선순위가 들어갑니다. 예를 들어 '에이전트가 환불 절차를 제대로 안내했는가'라는 항목이라면, 어디까지를 '안내함'으로 볼지, 부분만 안내한 경우는 어떻게 처리할지, 사용자가 추가 정보를 줘야 안내가 가능한 경우의 라벨은 무엇인지가 모두 가이드에 들어가야 합니다. 가이드 없이 라벨링을 시작하면 사람마다 기준이 달라져 라벨 자체에 잡음이 깔립니다.

가이드가 마련되면 **1차 라벨링**과 **2차 라벨링**이 차례로 들어갑니다. 같은 표본을 두 사람이 독립적으로 라벨링한 뒤 결과를 맞춰 보는 절차를 흔히 **이중 라벨링**이라고 부릅니다. 두 사람의 라벨이 일치한 비율을 **합의율**이라고 하며, 이 수치는 가이드가 충분히 명확한지를 거꾸로 알려 주는 신호로 쓰입니다.

합의율을 단순한 숫자로만 보면 오해가 생길 수 있습니다. 두 사람이 그냥 우연히 같은 답을 골랐을 가능성을 빼고 본 일치도라야 의미가 있는데, 흔히 코헨 카파 같은 보정된 지표가 함께 쓰입니다. 단순 일치율이 0.9를 넘는데 보정된 지표가 0.3 근처에 머무는 일도 있어서, 단순 일치율과 보정 지표를 같이 보는 편이 안전합니다. 합의율이 낮은 항목은 가이드가 모호하다는 뜻이므로, 라벨을 완성하기 전에 가이드를 먼저 손봅니다. 가이드가 손질되면 그 항목만 다시 라벨링하는 식으로 반복합니다.

이중 라벨링이 끝나면 **합의 검수** 단계가 옵니다. 두 사람의 라벨이 다른 사례를 모아, 가이드 작성자나 도메인 전문가가 최종 결정을 내리는 과정입니다. 이 단계에서 흔히 가이드의 빈틈이 드러나며, 발견한 빈틈은 가이드의 다음 버전에 반영합니다. 합의 검수 결과는 단순히 '이쪽이 맞다'로 끝나지 않고, '왜

이쪽이 맞는지'를 짧게 메모로 남겨 두면 다음 라벨링에서 같은 혼란이 줄어듭니다.

라벨이 안정되면 **케이스의 분류 체계**를 정리합니다. 분류 체계란 데이터셋의 각 사례에 붙는 메타데이터의 묶음을 말합니다. 도메인 카테고리, 난이도 등급, 실패 모드 분류, 사용자 의도 같은 축이 혼잡합니다. 예를 들어 1.2장에서 다룬 tool selection error, planning loop, hallucinated reasoning, premature termination 같은 실패 모드를 메타데이터로 붙여 두면, 평가 결과를 실패 모드별로 쪼개 볼 수 있습니다. '전체 점수는 변하지 않았는데 hallucinated reasoning이 늘었다' 같은 진단이 가능해지는 것은 이 분류 체계 덕분입니다.

마지막으로 **데이터셋 버전 관리**가 따라옵니다. 데이터셋은 한 번 만들고 그대로 두는 자료가 아니라, 사례가 추가되거나 라벨이 수정되거나 분류 체계가 바뀌면서 계속 갱신됩니다. 이때 어떤 시점의 데이터셋이었는지를 추적할 수 있어야, 어제의 점수와 오늘의 점수가 같은 잣대 위에 있는지를 판단할 수 있습니다. 흔히 다음과 같은 정보를 함께 관리합니다.

관리 항목	내용	변경 시 영향
버전 번호	v1.0, v1.1 같은 식별자	점수 비교의 기준선이 바뀜
사례 목록	각 입력의 식별자와 본문	새 사례 추가는 마이너 버전
라벨	각 사례의 정답과 메타데이터	라벨 수정은 패치 버전
라벨링 가이드	가이드 문서 자체	가이드 변경은 메이저 버전
분류 체계	메타데이터의 축과 값	축 추가는 메이저 변경

표에서 보듯이 라벨링 가이드와 분류 체계는 데이터셋 본문만큼 중요한 자산이며, 코드 저장소처럼 변경 이력을 남기는 도구로 함께 관리하는 편이 안전합니다. 평가 결과를 보관할 때는 그 결과가 어떤 버전의 데이터셋 위에서 나온 것인지를 같이 기록해 두어야 나중에 비교가 가능합니다.

정리하면, 골든 데이터셋의 구축은 후보 풀에서 랜덤·계층·어려운 케이스 우선의 세 방식을 조합해 표본을 고르는 일에서 시작해, 라벨링 가이드를 글로 정리하고, 같은 사례를 두 사람이 독립으로 라벨링한 뒤 합의율을 보며 가이드를 다듬고, 라벨에 도메인·난이도·실패 모드 같은 분류 체계를 얹은 다음, 가이드와 분류 체계까지 포함한 버전을 함께 관리하는 일련의 단계로 이루어집니다.

다음 단원인 2.1.3에서는 이렇게 구축한 데이터셋이 시간이 지나면 어떻게 흔들리는지, 그리고 그 흔들림을 흡수하면서 회귀 테스트로 계속 활용하는 방법을 살펴보겠습니다.

이 단원을 마치면 후보 풀에서 표본을 뽑는 세 가지 샘플링 전략과 그 조합 의도를 설명하고, 라벨링 가이드와 이중 라벨링·합의율·합의 점수가 라벨의 잡음을 어떻게 줄이는지, 분류 체계와 버전 관리가 평가 결과의 비교 가능성에 어떤 역할을 하는지를 단계별로 짚을 수 있습니다.

2.1.3 데이터셋 진화와 회귀 테스트

앞 단원에서 데이터셋을 모으고 라벨링하고 버전을 관리하는 과정을 따라왔습니다. 그렇게 만든 데이터셋이 한 번 완성되면 끝일 것 같지만, 현실은 그렇지 않습니다. 데이터셋은 만들어진 순간부터 늙기 시작합니다. 그 늙음을 그대로 두면 평가 점수와 사용자 경험 사이의 간격이 점점 벌어지므로, 데이터셋도 시스템과 함께 진화해야 합니다. 이번 단원은 그 진화의 방식과, 진화하는 데이터셋을 회귀 테스트라는 자동 장치로 묶어 두는 방법을 다룹니다.

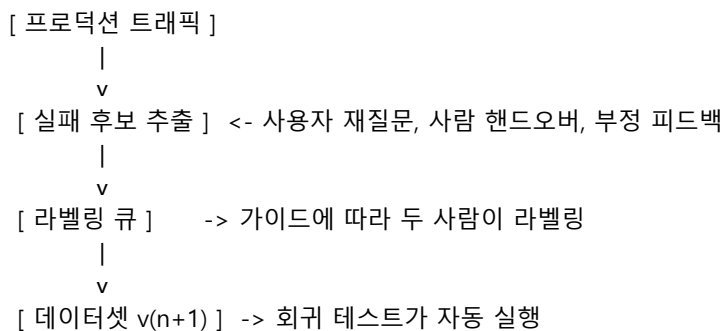
먼저 늙음의 원인을 짚겠습니다. 가장 흔한 이름은 **데이터 분포 드리프트**입니다. 드리프트는 영어 단어 그대로 '서서히 떠밀려 가는 움직임'을 가리키는데, 데이터의 분포가 시간에 따라 천천히 다른 모양으로 옮겨 가는 현상을 말합니다. 사용자 구성이 바뀌고, 신제품이 출시되고, 계절이 흐르고, 새 용어가

보편화되면서, 후보 풀로 흘러 들어오는 입력 자체가 어제와 달라집니다. 어제까지의 표본 위에서 100점을 받던 시스템이 오늘의 트래픽에서 80점을 내는 일은 시스템이 갑자기 나빠져서가 아니라, 데이터셋이 비추는 세상이 더는 오늘이 아니어서 생깁니다.

분포 드리프트가 천천히 쌓이는 노후화라면, 갑자기 모습을 드러내는 늙음의 형태도 있습니다. 프로덕션에서 어떤 사례가 사용자 불만을 만들고 사람 상담사로 넘어갔는데, 그 사례와 비슷한 입력이 골든 데이터셋에는 한 건도 들어 있지 않다고 해 봅시다. 데이터셋 위에서는 같은 점수가 유지되므로, 코드 변경의 영향도 그 사례 쪽에서는 잡히지 않습니다. 사용자가 진짜로 아파하는 지점이 평가에서는 보이지 않는 사각지대가 만들어진 것입니다.

이 사각지대를 좁히는 가장 직접적인 방법이 **프로덕션 실패를 데이터셋에 흡수하기**입니다. 흡수란 단순히 사례를 베껴 오는 일이 아니라, 다음과 같은 과정을 거쳐 데이터셋의 정식 구성원으로 받아들이는 작업을 말합니다. 우선 프로덕션에서 발생한 실패를 골라 입력과 그 시점의 시스템 응답을 함께 모읍니다. 이어 그 사례에 대해 정답이 무엇이었어야 하는지를 라벨링 가이드에 따라 라벨링하고, 두 사람의 합의를 거칩니다. 그다음 분류 체계의 어디에 들어가는지를 결정하고, 데이터셋의 다음 버전에 정식으로 추가합니다. 흡수가 끝난 사례는 그 시점부터 회귀 테스트의 일부가 되어, 같은 실패가 다시 발생하면 즉시 점수에 잡힙니다.

흡수가 단발성 작업으로 끝나지 않으려면, 프로덕션과 데이터셋 사이에 일정한 흐름이 굴러가야 합니다. 흐름을 가장 간단한 형태로 그리면 다음과 같습니다.



이 그림의 핵심은 화살표가 한쪽으로만 흐른다는 점이 아니라, 한 바퀴가 끝나면 다음 바퀴가 다시 시작된다는 점입니다. 프로덕션은 계속 트래픽을 만들어 내고, 그중 실패 후보가 추출되며, 라벨링이 끝나면 데이터셋이 갱신되고, 새 데이터셋 위에서 회귀 테스트가 다시 돌립니다. 이 순환이 끊기면 데이터셋의 늙음이 다시 쌓입니다.

이렇게 갱신되는 데이터셋을 사람이 손으로 매번 돌리는 일은 곧 한계에 부딪힙니다. 그래서 등장하는 것이 **회귀 테스트의 자동 실행**입니다. 자동 실행이란 사람의 명시적 명령 없이도 정해진 조건에서 데이터셋 전체가 돌고, 점수와 변화량이 정해진 형태로 보고되는 흐름을 말합니다. 자동 실행이 트리거되는 시점은 보통 두 가지입니다. 첫째는 코드 변경입니다. 프롬프트, 도구 정의, 라우팅 규칙, 모델 버전 가운데 어떤 것이라도 바뀌면 회귀 테스트가 자동으로 시작됩니다. 둘째는 시간입니다. 별다른 변경이 없어도 매일 또는 매주 한 번 데이터셋 전체가 돌면서, 모델이나 외부 도구의 미묘한 변화를 감지합니다.

자동 실행과 짝을 이루는 장치가 **PR 머지 게이트**입니다. PR이라는 말은 변경 제안을 코드 저장소에 합쳐 달라는 요청을 가리키며, 머지란 그 요청을 본 가지에 합치는 작업입니다. 게이트라는 이름이 붙은 이유는, 회귀 테스트의 결과가 일정 기준을 넘지 못하면 합치기를 막아 세우기 때문입니다. 예컨대 데이터셋 위에서의 전체 점수가 직전 머지 시점보다 일정 이상 떨어지면 PR이 자동으로 막히고, 사람이 결과를 검토한 뒤 풀어 줘야 합치기가 가능해집니다. 게이트가 있는 시스템에서는 잘 돌던 사례를 깨뜨리는

변경이 본 가지에 들어오기 전에 한 번은 손에 잡힙니다.

게이트의 임계 기준을 정할 때는 몇 가지 함정에 주의해야 합니다. 점수 한 줄에만 임계치를 거는 방식은 단일 메트릭 함정에 다시 빠질 위험이 있어, 분류 체계의 축별로 임계치를 쪼개 거는 편이 안전합니다. 전체 점수는 유지되더라도 특정 실패 모드에서 점수가 급격히 떨어지면 게이트가 막히도록 설계하는 식입니다. 또한 임계치를 너무 뻥뻥하게 잡으면 사소한 잡음에도 PR이 막혀 개발 속도가 죽고, 너무 느슨하게 잡으면 명백한 회귀가 통과합니다. 처음에는 다소 느슨하게 두고, 잡힌 회귀와 놓친 회귀의 비율을 보면서 점진적으로 조이는 방식이 현실적입니다.

자동 실행과 게이트가 자리잡으면, 마지막으로 따라오는 주제가 **데이터셋의 만료와 갱신 주기**입니다. 데이터셋의 사례 하나 하나는 무한정 유효하지 않습니다. 어떤 사례는 시간이 지나면 더 이상 일어나지 않는 일이 됩니다. 예를 들어 더는 판매하지 않는 제품에 대한 환불 문의, 폐지된 결제 수단의 오류, 옛 인터페이스에서만 발생하던 입력은 새 시스템에서는 거의 들어올 일이 없습니다. 이런 사례를 데이터셋에 그대로 두면, 점수의 일부는 더 이상 일어나지 않는 사례 위에서 매겨집니다. 그 점수가 오르락내리락해도 사용자 경험과는 관계가 없어집니다.

만료 정책은 보통 두 축으로 결정됩니다. 한 축은 시간이며, 일정 기간 이상 갱신이 없는 사례는 보관소로 옮기고 회귀 테스트의 기본 묶음에서 제외합니다. 다른 한 축은 발생 빈도이며, 같은 분류의 사례가 프로덕션에서 일정 기간 발생하지 않으면 기본 묶음에서 빠집니다. 두 축의 기준을 정해 두면, 데이터셋의 크기가 무한정 부풀지 않으면서 현실 트래픽과의 간격도 일정 수준에서 유지됩니다.

전체 진화 흐름과 자동 장치의 관계를 한 표로 정리하면 다음과 같습니다.

단계	트리거	산출물	회귀 테스트와의 연결
분포 드리프트 감시	주기적 측정	분포 변화 보고	새 표본 추가 의사결정
프로덕션 실패 흡수	사용자 신호	라벨링 큐	다음 버전에 사례 추가
회귀 테스트 자동 실행	코드 변경, 시간	점수 보고	변경의 영향 즉시 가시화
PR 머지 게이트	점수 임계 위반	머지 차단	잘 돌던 사례를 지키는 안전망
만료와 갱신	시간, 빈도	보관소 이동	사례 묶음의 적정 규모 유지

표를 거꾸로 읽으면 데이터셋의 진화는 새로 들어오는 일과 빠져 나가는 일이 균형을 이루는 살아 있는 묶음이라는 점이 분명해집니다. 들어오기만 하면 데이터셋이 부풀고, 빠지기만 하면 사각지대가 생깁니다. 두 방향의 흐름이 동시에 있어야, 회귀 테스트가 오랜 기간 시스템 변경의 영향을 일정한 잣대 위에서 잡아 낼 수 있습니다.

정리하면, 골든 데이터셋은 한 번 완성되는 자산이 아니라, 분포 드리프트를 의식하며 프로덕션에서 발생한 실패를 흡수해 다음 버전으로 이어지는 살아 있는 묶음입니다. 이 데이터셋이 회귀 테스트로 자동 실행되고 그 결과가 PR 머지 게이트에 연결될 때, 변경이 잘 돌던 사례를 깨뜨리는 일을 본 가지에 닿기 전에 잡을 수 있으며, 시간과 빈도를 축으로 한 만료 정책이 함께 굴러가야 데이터셋의 크기가 부풀지 않으면서도 현실과의 간격이 일정 수준 안에서 유지됩니다.

다음 단원인 2.2.1에서는 이렇게 갱신되는 데이터셋 위에서 정답과 응답을 자동으로 비교해 점수를 매기는 핵심 도구인 LLM-as-Judge가 어떻게 동작하는지를 살펴보겠습니다.

이 단원을 마치면 데이터 분포 드리프트가 데이터셋의 늪을 만드는 메커니즘을 설명하고, 프로덕션 실패를 라벨링 큐를 거쳐 데이터셋에 흡수하는 흐름을 그릴 수 있으며, 회귀 테스트의 자동 실행과 PR 머지 게이트가 변경의 영향을 어떻게 잡아 내는지, 그리고 시간·빈도 두 축의 만료 정책이 데이터셋 규모와 현실 적합성 사이의 균형을 어떻게 잡는지를 설명할 수 있습니다.

2.2 LLM-as-Judge

이 섹션의 토픽과 학습 목표는 다음과 같습니다.

2.2.1 LLM-as-Judge 동작 원리 — LLM-as-Judge의 동작 방식과 자동 채점 파이프라인의 구성을 설명할 수 있다

2.2.2 평가 LLM의 편향과 한계 — Judge LLM이 가진 위치 편향, 길이 편향, 자기 선호 편향을 식별하고 영향을 설명할 수 있다

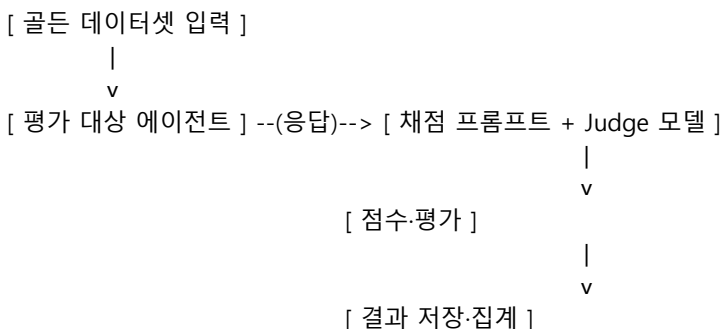
2.2.3 Judge 보정 기법 — Judge의 편향을 완화하는 보정 기법을 두 가지 이상 적용할 수 있다

2.2.1 LLM-as-Judge 동작 원리

앞 절까지 우리는 골든 데이터셋을 만들고, 거기에 시간이 지나도 살아남는 회귀 테스트의 역할을 부여하는 데까지 왔습니다. 이제 한 가지 현실적인 문제가 남습니다. 골든 데이터셋의 입력에 대해 에이전트가 내놓은 답이 정답에 얼마나 가까운지를 도대체 어떻게 채점할 것인가 하는 문제입니다. 사람이 한 줄 한 줄 읽고 점수를 매기면 가장 정확하지만, 수백 수천 건의 응답을 매번 사람 손으로 채점하면 평가 자체가 병목이 됩니다. 이 단원은 이 채점 자리에 사람 대신 또 다른 인공지능 모델을 얹히는 방법, 즉 LLM-as-Judge의 동작 원리를 정리합니다.

먼저 용어부터 풀겠습니다. **LLM-as-Judge**란 평가 대상이 되는 시스템의 출력을 채점하기 위해 별도의 거대 언어 모델을 채점기로 두고, 그 모델에게 정해진 기준에 따라 점수나 순위를 내도록 시키는 자동 채점 방식을 가리킵니다. 핵심은 두 모델의 역할 분리에 있습니다. 평가 대상 모델은 사용자 질문에 답하는 본래의 역할을 하고, 채점용 모델은 그 답이 어떤지를 정해진 형식으로 평가만 합니다. 이때 채점용 모델을 흔히 **Judge** 또는 **평가 LLM**이라고 부릅니다.

자동 채점 파이프라인의 모양은 크게 보면 단순합니다. 골든 데이터셋에서 입력을 하나 꺼내고, 그 입력을 평가 대상 에이전트에게 흘려보내 응답을 받고, 그 응답을 채점 프롬프트에 끼워 넣어 Judge 모델에 전달하고, Judge가 돌려준 점수를 데이터셋 옆에 기록합니다. 한 번 돌면 한 행이 채워지고, 데이터셋 전체가 다 돌면 평균 점수와 분포가 남습니다. 이 흐름이 매일 자동으로 돌면서 회귀를 잡아 주는 그림이 평가 파이프라인의 기본 형태입니다.



이 흐름 안에서 가장 먼저 갈리는 결정이 정답을 비교 대상으로 쓰느냐 마느냐입니다. 채점 방식은 이 기준으로 두 갈래로 나뉩니다. 첫 번째는 **Reference-based 채점**입니다. 골든 데이터셋에 미리 적어 둔 정답이나 모범 답안을 Judge에게 함께 보여 주고, 에이전트의 응답이 그 정답과 얼마나 일치하는지를 묻는 방식입니다. 정답이 명확한 질의응답이나 요약 작업에서 잘 맞습니다. 두 번째는 **Reference-free 채점**입니다. 정답을 따로 주지 않고, 응답 자체의 품질을 정해진 기준에 따라 평가하도록 시키는 방식입니다. 정답이 하나가 아닌 자유 응답이나 글쓰기, 상담 응답에서 주로 쓰입니다. 같은

데이터셋이라도 항목에 따라 두 방식을 섞어 쓰는 경우도 흔합니다.

다음 갈림길은 한 번에 몇 개의 응답을 평가하느냐입니다. **Pointwise 평가**는 응답 한 개를 보고 그 자체에 점수를 매기는 방식입니다. 1점부터 5점까지의 척도나 합격·불합격 판정 같은 것이 여기에 속합니다. 한 번에 한 응답만 보면 되니 데이터셋이 커도 비용이 선형으로 늘고, 시간이 지나며 변화 추세를 보기에 편합니다. 반면 **Pairwise 평가**는 두 개의 응답을 나란히 보여 주고 어느 쪽이 더 나은지 고르도록 시키는 방식입니다. 모델 A의 응답과 모델 B의 응답을 함께 두고 'A가 낫다, B가 낫다, 비슷하다' 중 하나를 고르게 하면, 절대 점수를 매기지 않아도 두 시스템의 우열을 가릴 수 있습니다. 신모델과 구모델을 비교해 회귀 여부를 판단할 때 특히 유용합니다.

채점이 어떤 형태든, Judge에게 무엇을 어떻게 보라고 일러 두는 글이 필요합니다. 이 글이 **채점 프롬프트**이고, 그 안에서 평가 항목과 점수 의미를 명시하는 표를 **rubric**이라고 부릅니다. Pointwise 채점 프롬프트의 윤곽을 단순화하면 다음과 비슷합니다.

당신은 고객 상담 응답을 평가하는 채점자입니다.
다음 응답을 아래 기준으로 1점부터 5점까지 평가하세요.

[질문]
{user_question}

[응답]
{agent_response}

[채점 기준]
1점: 질문 의도를 잘못 이해했거나 잘못된 정보를 줍니다.
3점: 의도는 맞지만 핵심 정보가 빠지거나 어조가 부적절합니다.
5점: 의도를 정확히 파악하고 핵심 정보를 명확하게 전달합니다.

[출력 형식]
score: <정수 1~5>
reason: <한 문장 근거>

여기서 핵심은 점수의 의미를 글로 박아 두었다는 점입니다. 점수의 의미가 모호하면 같은 응답에 어떤 날은 4점이, 어떤 날은 2점이 나오기 쉽습니다. rubric은 그 흔들림을 줄이기 위한 닛 역할을 합니다. Pairwise 채점에서는 비슷한 형식을 두 응답에 대해 쓰되, 마지막에 'A, B, Tie' 중 하나를 고르게 하고 그 근거 한 문장을 함께 받게 합니다.

Judge가 한 응답씩 돌아가며 점수를 매기고 나면, 데이터셋 옆에는 점수의 묶음이 남습니다. 이 묶음을 그대로 평균만 보지 말고 분포 모양을 함께 살피는 습관이 필요합니다. 5점 만점의 데이터셋에서 평균이 4.2점이라는 사실 하나만으로는 시스템 상태를 다 알 수 없습니다. 그 4.2점이 다음과 같은 분포에서 나왔는지,

1점: ## (4%)
2점: ### (6%)
3점: ##### (16%)
4점: ##### (36%)
5점: ##### (38%)

아니면 다음과 같은 분포에서 나왔는지에 따라 시스템의 위험은 전혀 다릅니다.

1점: ##### (30%)
2점: ## (4%)

3점: ## (4%)
 4점: ## (4%)
 5점: ##### (58%)

두 분포는 평균이 비슷해도 상태가 정반대입니다. 앞쪽은 대부분의 응답이 무난한 가운데 일부 우수 답변이 평균을 끌어올리는 모양이고, 뒤쪽은 잘하면 아주 잘하지만 30퍼센트의 사용자가 1점짜리 응답을 받는 위험한 모양입니다. 평균 점수만 대시보드에 띄워 두면 뒤쪽 분포가 한참 동안 묻혀 버립니다. 그래서 LLM-as-Judge 결과는 평균과 함께 분포 막대, 1점·2점 응답의 절대 건수, 그리고 그 안의 대표 사례를 같이 보아야 의미가 살아납니다.

정리하면, LLM-as-Judge는 평가 대상 시스템의 응답을 별도의 채점용 모델에게 정해진 채점 프롬프트와 rubric으로 평가시켜 자동으로 점수를 얻는 방식입니다. 정답을 함께 주는 reference-based 채점과 응답 자체만 보는 reference-free 채점이 있고, 한 응답에 절대 점수를 매기는 pointwise 평가와 두 응답을 비교해 우열을 가리는 pairwise 평가가 있습니다. 결과는 평균만이 아니라 점수 분포의 모양으로 함께 읽어야 시스템의 실제 상태가 보입니다.

다음 단원인 2.2.2에서는 이 Judge 모델 자체가 가진 편향과 한계를 다루겠습니다. Judge가 사람과 달리 위치, 길이, 자기 모델 같은 표면 단서에 휘둘리는 양상을 짚고, 그 편향이 메트릭에 어떻게 누적되는지를 살펴봅니다.

이 단원을 마치면 LLM-as-Judge의 정의와 자동 채점 파이프라인의 구성을, reference-based와 reference-free, pairwise와 pointwise라는 두 축의 선택지와 함께 설명할 수 있고, 채점 프롬프트와 rubric의 역할, 점수 분포를 평균과 함께 해석해야 하는 까닭까지 풀어 낼 수 있습니다.

2.2.2 평가 LLM의 편향과 한계

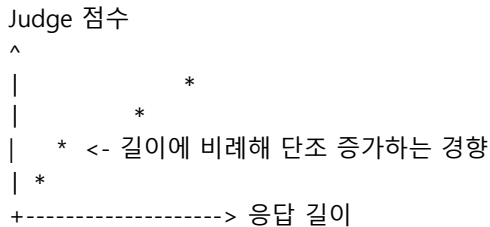
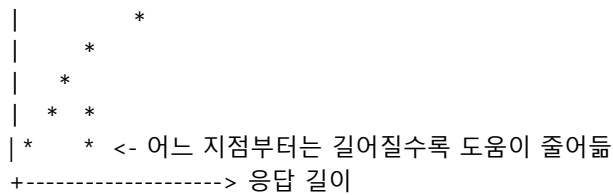
앞 단원에서 우리는 평가 대상 시스템의 응답을 별도의 모델에게 채점시키는 LLM-as-Judge의 동작 원리를 잡았습니다. 채점 프롬프트와 rubric을 잘 짜고 점수 분포까지 같이 보면, 사람이 매번 라벨을 달지 않고도 자동으로 회귀를 잡을 수 있다는 것이 그 단원의 결론이었습니다. 그러나 한 가지를 빠뜨릴 수 없습니다. 채점석에 앉은 모델 역시 인공지능 모델이라는 사실, 그리고 이 채점자가 사람의 채점자와는 결이 다른 일관된 편향을 갖고 있다는 사실입니다. 이 단원은 그 편향이 어떤 모양으로 나타나는지를 정리합니다.

가장 먼저 알아 둘 편향은 **Position bias(위치 편향)**입니다. Pairwise 평가처럼 두 응답을 나란히 놓고 어느 쪽이 더 나은지 고르게 했을 때, Judge가 응답의 내용보다 화면에 어느 위치에 있는지에 영향을 받는 현상을 가리킵니다. 동일한 응답 두 개를 그대로 두고 자리만 바꿔 보여 줘도, 어떤 모델은 첫 번째 응답을 자주 고르고 어떤 모델은 마지막에 본 응답을 자주 고릅니다. 이 효과는 작지 않아서, 같은 데이터셋을 한 방향으로만 평가하면 'A가 B보다 낫다'는 결론이 단지 A를 늘 앞에 둔 탓에 나오는 일이 실제로 일어납니다.

두 번째는 **Length bias(길이 편향)**입니다. Judge는 일반적으로 짧은 응답보다 긴 응답을 더 좋게 평가하는 경향이 있습니다. 글자 수가 많을수록 근거가 풍부해 보이고 형식이 갖추어진 듯 보이는 외양이 채점에 영향을 주기 때문입니다. 문제는 길이가 곧 품질이 아니라는 데 있습니다. 사용자에게는 핵심만 두 줄로 답하는 쪽이 더 도움 되는데도, Judge가 한 페이지짜리 장황한 응답에 더 높은 점수를 주면, 자동 평가가 모델을 점점 길어지는 방향으로 끌고 갑니다. 다음 그림이 이 어긋남을 보여 줍니다.

실제 사용자 도움

^ *
| *



Judge의 점수 곡선은 오른쪽 위로 단조 증가하는 모양이 잘 나오는 반면, 사용자의 실제 도움 곡선은 어느 지점에서 꺾여 내려갑니다. 이 두 곡선이 어긋나기 시작하면, 자동 평가의 점수가 올라갈수록 사용자의 만족도가 오히려 떨어지는 역설이 생깁니다.

세 번째는 **Self-enhancement(자기 선호 편향)**입니다. Judge가 자신과 같은 모델 계열의 출력에 더 높은 점수를 매기는 경향을 말합니다. 같은 회사에서 만든 모델이거나 학습 시점이 비슷한 모델이 내놓은 응답일수록, Judge가 그 응답의 표현 습관을 자연스럽게 받아들이고 좋게 평가합니다. 사람은 자기와 닮은 글에 호의를 갖기 쉽다고 알려져 있는데, 모델도 비슷한 효과를 보입니다. 신모델로 자체 평가를 돌리면서 채점도 같은 회사의 모델로 한다면, 그 점수에는 진짜 품질 향상과 자기 선호가 한데 섞여 들어옵니다.

네 번째는 **도메인 지식 부족으로 인한 오판**입니다. Judge 모델은 일반 지식에는 강하지만, 특정 도메인의 깊은 지식까지 다 갖춘 것은 아닙니다. 의료 상담, 법률 자문, 산업별 전문 용어가 얹힌 응답에서는, 사용자에게는 명백히 잘못된 정보가 들어 있어도 Judge가 그 오류를 잡지 못한 채 그럴듯한 응답에 4점을 주기도 합니다. 반대로 정확하지만 도메인 약어가 많은 응답을 Judge가 이해하지 못해 낮은 점수를 매기는 일도 생깁니다. 이 두 가지 오류가 섞이면 점수의 신뢰도는 도메인이 깊을수록 빠르게 떨어집니다.

이 네 가지가 한 자리에서 동시에 일어나는 모양을 잠깐 떠올려 보겠습니다. 가령 신모델 A와 구모델 B의 응답을 채점하는 자리를 가정해 봅시다. 세팅상 신모델 A의 응답을 늘 첫 번째 자리에 두었고, A는 보통 응답이 길고 친절한 어투를 씁니다. Judge는 A와 같은 회사의 최신 모델입니다. 데이터셋 항목 가운데 일부는 의료 상담입니다. 이 자리에서 Judge가 'A가 우세하다'라는 결론을 내놓는다면, 그 결론에는 위치 편향, 길이 편향, 자기 선호 편향, 도메인 지식 부족이라는 네 가지 변수가 동시에 들어 있어 진짜 품질 차이가 어디까지인지를 잘라 보기가 어렵습니다.

이 편향들의 더 큰 문제는 한 번 평가에서 끝나지 않는다는 데 있습니다. 자동 평가가 회귀 테스트 자리에 올라가는 순간, 편향은 시간이 지나면서 시스템에 누적됩니다. 길이 편향이 있는 Judge를 그대로 두고 모델을 매주 다듬는다고 해 보겠습니다. 매주 점수가 조금씩 올라가는 모델 변경분 가운데 어떤 것은 진짜로 품질이 좋아진 변화이지만, 다른 어떤 것은 단지 응답이 길어진 변화입니다. Judge 점수 하나만 보면 둘이 구별되지 않으므로, 둘 다 운영에 머지됩니다. 몇 주가 지나면 응답은 점점 길어지고 점수는 계속 상승하지만, 사용자의 실제 만족도는 정체하거나 떨어지는 그림이 만들어집니다.

같은 누적이 self-enhancement에서도 일어납니다. 같은 회사 모델을 Judge로 두고 신모델을 평가하면, 신모델의 표현 습관이 Judge에게 익숙한 방향으로 갈수록 점수가 높아집니다. 시간이 갈수록 모델이 사용자가 아닌 Judge에 맞춰 다듬어지는 구도가 만들어지고, 결국 자동 평가 점수와 사용자 만족도의 거리는 점점 벌어집니다. 이 어긋남이 충분히 커진 뒤에야 외부 신호로 발견되면, 그동안 머지된 변경분

가운데 어떤 것이 진짜 회귀였는지를 되짚어 가려내기가 어렵습니다.

도메인 지식 부족이 누적될 때는 위험의 결이 다릅니다. 일반 도메인에서는 점수가 잘 나오지만 의료나 법률 같은 깊은 도메인에서는 Judge가 잡지 못한 오류가 쌓여 갑니다. 평균 점수만 보면 시스템이 잘 굴러가는 것처럼 보이지만, 그 평균 안에는 일반 도메인의 높은 점수와 깊은 도메인의 묻혀 버린 오류가 섞여 있습니다. 사용자 사고가 한 번 터지고 나서야 그동안 깊은 도메인의 점수가 사실 의미 없는 숫자였다는 점이 드러납니다.

정리하면, LLM-as-Judge는 자동 채점의 길을 열어 주는 동시에 사람의 채점자와는 결이 다른 편향을 함께 들여옵니다. 위치에 휘둘리는 position bias, 길이에 점수를 주는 length bias, 같은 계열 모델을 선호하는 self-enhancement, 그리고 깊은 도메인에서 지식이 부족해 일어나는 오판이 대표적인 항목입니다. 이 네 가지는 한 번의 채점에서 멈추지 않고 회귀 테스트 자리에 들어가면서 시스템에 누적되며, Judge 점수와 사용자 만족도 사이의 거리를 조용히 벌립니다.

다음 단원인 2.2.3에서는 이 편향들을 그대로 두지 않고 줄여 가는 보정 기법들을 다루겠습니다. 위치 무작위화, 길이 정규화, 다중 모델 앙상블처럼 운영에서 바로 쓸 수 있는 기법과, Judge 자체가 시간이 지나며 망가지지 않는지 확인하는 회귀 테스트 방법을 살펴봅니다.

이 단원을 마치면 Judge LLM이 가진 position bias, length bias, self-enhancement, 도메인 지식 부족이라는 네 가지 한계를 식별하고, 각 편향이 한 번의 채점에서 그치지 않고 회귀 테스트 자리에서 메트릭에 어떻게 누적되어 사용자 만족도와와의 거리를 벌리는지 사례 흐름과 함께 설명할 수 있습니다.

2.2.3 Judge 보정 기법

앞 단원에서 우리는 LLM-as-Judge가 위치, 길이, 자기 모델 계열, 도메인 지식이라는 네 가지 축에서 편향을 갖고, 그 편향이 회귀 테스트 자리에 들어가면 시간이 지나며 메트릭에 누적된다는 사실을 보았습니다. 편향이 있다는 사실을 알았으니, 이제 그 편향을 그대로 두지 않고 줄여 가는 운영상의 처치를 알아볼 차례입니다. 이 단원은 Judge의 점수가 조금이라도 사용자 만족도에 가깝게 가도록 만드는 다섯 가지 보정 기법을 정리합니다.

첫 번째 기법은 **위치 무작위화**입니다. Pairwise 평가에서 늘 신모델을 첫 번째에, 구모델을 두 번째에 두면 위치 편향이 그대로 점수에 묻어 들어옵니다. 이 문제를 줄이는 가장 단순하면서 강력한 방법은, 같은 응답 쌍을 한 번은 (A, B) 순서로, 또 한 번은 (B, A) 순서로 두 번 채점한 뒤 두 결과가 일관되는지 보는 것입니다. 두 채점이 서로 같은 쪽을 우세하다고 골랐다면 그 비교는 신뢰할 수 있는 결정으로 받아들이고, 순서가 바뀌었다고 결론도 바뀌었다면 그 항목을 'tie' 또는 '판정 불가'로 처리합니다. 흐름은 다음과 같습니다.

```
[ A vs B ] -> Judge -> 결과1: A 우세
[ B vs A ] -> Judge -> 결과2: B 우세

결과1 == 결과2 ?
|
yes --> 안정 비교
|
no --> tie 처리, 위치 편향 의심 항목으로 별도 분류
```

이렇게 하면 한 항목당 채점 횟수가 두 배가 되어 비용은 늘지만, 그 비용 대비 얻는 안정성이 큼니다. 비용이 부담되면 데이터셋 전체에 적용하지 않고 무작위 50퍼센트나 모델 비교가 핵심인 항목에만 적용하는 절충도 가능합니다.

두 번째 기법은 **길이 정규화**입니다. Judge가 긴 응답에 더 높은 점수를 주는 경향이 있다면, 점수와 응답 길이의 관계를 따로 측정해 그 영향을 빼 주는 길이를 갖고 있어야 합니다. 가장 간단한 형태는 채점 프롬프트 안에 '동일한 의미를 더 짧게 전달하는 응답을 우선 평가하세요' 같은 지시를 명시적으로 박아 두는 것입니다. 그것만으로 부족하면 채점 결과를 사후에 다듬는 길도 있습니다. 데이터셋의 모든 응답에 대해 (응답 길이, Judge 점수) 쌍을 모은 뒤, 길이별 평균 점수를 그려 보면 길이 편향의 모양이 드러납니다. 그 곡선의 기울기를 점수에서 빼 주거나, 같은 길이 구간 안에서만 모델 간 점수를 비교하면 길이 효과를 어느 정도 제거할 수 있습니다. 더 나아가 채점 자체를 '품질 점수'와 '간결성 점수'로 두 개로 분리하면, 두 축을 따로 추적할 수 있어 한 점수가 다른 한 점수를 잡아먹는 일이 줄어듭니다.

세 번째 기법은 **다중 모델 앙상블**입니다. Judge 한 모델에만 모든 채점을 맡기면 그 모델의 편향이 그대로 결과에 묻어 들어옵니다. 같은 응답을 서로 다른 회사의 두세 모델에게 동시에 채점시키고, 점수의 평균이나 다수결을 최종 결과로 삼으면 한 모델의 self-enhancement가 결과를 통째로 휘두르는 일이 줄어듭니다. 다음 그림이 단순한 형태입니다.

```
[ 응답 ] --> Judge X (회사 1) -> 점수 4
          |
          +--> Judge Y (회사 2) -> 점수 3
          |
          +--> Judge Z (회사 3) -> 점수 4

최종 점수: 평균 3.67 / 다수결: 4
```

세 모델의 의견이 갈리는 항목은 '판정 불일치 큰 사례'로 별도로 모아 두면, 그 묶음 자체가 사람 검토의 우선순위 목록이 됩니다. 비용이 늘어난다는 점이 단점이지만, 핵심 회귀 테스트나 모델 출시 직전의 게이트 평가에는 충분히 들일 만한 비용입니다. 신모델을 평가할 때 그 신모델과 같은 계열의 Judge를 일부러 빼고 다른 계열만으로 앙상블을 구성하면 self-enhancement까지 함께 줄일 수 있습니다.

네 번째 기법은 **사람 라벨과의 일치도 측정**입니다. 위 세 가지 처치를 다 해 두었더라도, 그래서 Judge의 점수가 사용자 입장에서 정말 의미 있는지를 따로 확인해 두지 않으면 메트릭은 여전히 허공에 떠 있습니다. 가장 확실한 닷은 사람 라벨입니다. 골든 데이터셋의 일부, 가령 100건에서 200건 정도를 사람이 직접 채점하고, 같은 항목에 Judge가 매긴 점수와 비교해 두 채점이 얼마나 일치하는지를 정기적으로 잽니다. Pointwise 점수라면 사람 평균과 Judge 평균의 차이, 그리고 항목별 차이의 분포를 봅니다. Pairwise 비교라면 사람과 Judge가 같은 쪽을 우세하다고 고른 비율이 핵심 지표입니다. 다음과 같은 표가 운영의 한 모양입니다.

구분	사람 우세 A 사람 우세 B 사람 tie		
-----+-----+-----			
Judge 우세 A	38	6	4
Judge 우세 B	5	32	3
Judge tie	4	3	5

이 표의 대각선이 두꺼울수록 Judge와 사람의 의견이 잘 맞고, 비대각선의 큰 칸은 어디서 둘이 어긋나는지를 알려 주는 단서가 됩니다. 일치도가 일정 수준 아래로 떨어지면 메트릭 자체를 배포 게이트에서 잠깐 빼고 보정을 다시 보는 정책으로 묶어 두면, 이 닷이 메트릭을 끌어내리기 전에 신호를 줍니다.

다섯 번째 기법은 **Judge 자체의 회귀 테스트**입니다. 모델은 사용자에게 답하는 모델만 바뀌는 것이 아닙니다. 채점기 자리에 앉은 Judge 모델도 시간이 지나며 새 버전으로 갈아 끼워지고, 같은 모델이라도 미세한 업데이트로 채점 기준이 흔들릴 수 있습니다. 이 흔들림을 잡기 위해 'Judge 자체를 평가하는 작은

데이터셋'을 따로 두면 좋습니다. 명백히 점수가 정해져야 할 케이스, 예를 들어 5점이 나와야 마땅한 우수 응답과 1점이 나와야 마땅한 명백한 오답을 30건에서 50건 정도 모아 둡니다. 새로운 Judge 버전이 들어올 때마다 이 작은 데이터셋을 돌려 점수가 의도대로 나오는지 확인하는 흐름이 Judge의 회귀 테스트입니다. 이 데이터셋을 통과하지 못하는 Judge 버전은 운영 평가에 올리지 않는다는 단순한 규칙만으로도, '어느 날부터 점수가 갑자기 좋아졌다'는 식의 정체불명의 변화를 막을 수 있습니다.

다섯 가지 기법은 따로 떨어진 기술이 아니라, 한 평가 파이프라인 위에 층층이 얹는 보정의 묶음으로 보아야 합니다. 위치 무작위화로 자리 효과를 빼고, 길이 정규화로 분량의 거품을 빼고, 다중 모델 앙상블로 한 Judge의 편향을 분산시키고, 사람 라벨과의 일치도로 메트릭의 맞을 잡고, Judge 자체의 회귀 테스트로 채점기의 변화를 방어합니다. 이 다섯이 모두 운영에 들어가 있어야 자동 평가의 숫자가 시간이 지나도 사용자 만족도에 가깝게 흘러갑니다.

정리하면, LLM-as-Judge의 편향은 운영상의 처치로 상당 부분 줄일 수 있습니다. 위치 무작위화는 위치 편향을, 길이 정규화는 길이 편향을, 다중 모델 앙상블은 self-enhancement를 직접적으로 다루고, 사람 라벨과의 일치도는 메트릭이 사용자 만족도에서 멀어지는지를 정기적으로 점검하며, Judge 자체의 회귀 테스트는 채점기가 사일런트하게 바뀌는 위험을 막아 줍니다. 다섯 기법을 한 파이프라인 위에 얹어 두면, Judge 점수가 의미 있는 시그널로 살아남습니다.

다음 단원인 2.3.1에서는 응답 한 덩어리 단위의 채점에서 한 걸음 더 들어가, 에이전트가 한 번의 요청 동안 거친 모든 단계의 궤적을 평가하는 trajectory 평가의 설계로 넘어가겠습니다. 단계, 도구 선택, 인자 전달처럼 응답 한 줄에는 보이지 않던 차원이 어떻게 평가 항목으로 옮겨지는지를 다룹니다.

이 단원을 마치면 Judge의 편향을 완화하는 보정 기법으로 위치 무작위화, 길이 정규화, 다중 모델 앙상블, 사람 라벨과의 일치도 측정, Judge 자체의 회귀 테스트 다섯 가지를 각각의 작동 방식과 한 파이프라인 위에서의 결합 방식과 함께 설명하고 적용할 수 있습니다.

2.3 Trajectory 평가

이 섹션의 토픽과 학습 목표는 다음과 같습니다.

2.3.1 Trajectory 평가 설계 — 에이전트의 단계별 trajectory를 평가하는 메트릭과 데이터 구조를 설계할 수 있다

2.3.2 단계별 정확도 측정 — Trajectory의 각 단계에서 측정 가능한 정확도 지표를 구성하고 집계 방법을 설명할 수 있다

2.3.1 Trajectory 평가 설계

앞 절에서는 LLM-as-Judge가 채점자로서 가지는 편향과 그 보정 기법을 다루었습니다. Judge가 출력 한 덩어리에 점수를 매기는 방식이었다면, 이번 단원은 그보다 한 단계 안쪽을 들여다봅니다. 에이전트는 한 번의 응답으로 끝나는 시스템이 아니라 여러 단계를 이어 도구를 부르고 결과를 읽고 다음 행동을 정하는 시스템이기 때문에, 마지막 답만 보면 중간에 어떤 잘못이 있었는지 드러나지 않는 경우가 많습니다. 이 단원은 그 중간을 측정 가능한 형태로 풀어 두는 방법, 즉 trajectory 평가의 설계를 다룹니다.

먼저 용어부터 정리하겠습니다. **Trajectory**는 에이전트가 입력을 받아 최종 응답을 내기까지 거쳐 간 단계의 기록입니다. 각 단계는 보통 세 조각으로 표현됩니다. 어떤 도구를 골랐는지, 그 도구에 어떤 인자를 넣었는지, 그리고 그 결과로 어떤 관찰을 받아 왔는지입니다. 이 세 조각을 각각 **step**, **tool**, **observation**이라고 부르겠습니다. 한 trajectory는 step의 나열이고, 각 step에는 tool 이름과 인자, 그리고 그

호출의 결과인 observation이 붙어 있습니다.

이 표현을 한 줄짜리 도식으로 보면 다음과 같습니다.

```
[Step 1] tool=search  query='2024 매출 보고서'      -> ok
[Step 2] tool=read    file='reports/2024-q3.md'      -> ok
[Step 3] tool=edit    args={path:'/etc/hosts', ...}   -> tool_selection_error
[Step 4] tool=respond text='수정을 마쳤습니다'       -> premature_termination
```

각 줄은 에이전트가 한 단계에서 한 행동과 그 결과 라벨입니다. 마지막 응답만 본다면 '수정을 마쳤다'는 문장 하나가 나올 뿐이지만, trajectory를 풀어 두면 3단계에서 잘못된 도구를 골랐고 4단계에서 그 사실을 모른 채 종료했다는 사실이 드러납니다. Trajectory 평가는 바로 이런 단계별 사실을 라벨링 가능한 데이터로 만들고 그 위에 메트릭을 얹는 작업입니다.

Trajectory를 데이터로 보관할 때는 일반적으로 단계마다 다음 항목을 함께 저장합니다. 단계 번호, 호출한 도구 이름, 도구에 전달한 인자, 도구가 돌려준 응답, 그리고 평가자가 이후에 붙일 라벨입니다. 라벨은 사람이 사후에 붙일 수도 있고 골든 데이터셋에 미리 정해 둔 정답 trajectory와 비교해 자동으로 매길 수도 있습니다. 핵심은 한 trajectory를 그저 자유 텍스트로 두지 않고, 단계 단위로 끊어서 비교 가능한 구조로 정리해 두는 일입니다.

이 위에서 가장 먼저 정의해야 할 메트릭은 **단계별 정확도**입니다. 단계별 정확도란 한 step이 그 시점에서 합리적인 행동이었는지를 0 또는 1로 판단하는 척도입니다. '이 시점에서 정답으로 정해 둔 행동의 집합 안에 에이전트의 행동이 들어 있었는가'라고 묻고, 들어 있으면 1, 그렇지 않으면 0을 매깁니다. 이렇게 정의하면 trajectory 전체의 정확도는 step별 0과 1의 평균으로 자연스럽게 정리됩니다. 이 정의가 가지는 장점은 한 trajectory에서 어느 단계가 무너졌는지를 곧바로 짚어 낸다는 점입니다.

단계별 정확도를 더 잘게 쪼개면 두 개의 하위 메트릭이 나옵니다. 하나는 **올바른 도구 선택률**이고 다른 하나는 **올바른 인자 전달률**입니다. 올바른 도구 선택률은 그 단계에서 에이전트가 고른 tool 이름이 정답 도구 집합 안에 있었는지를 봅니다. 위의 도식에서 3단계는 사용자가 매출 보고서 수정을 요청했지만 에이전트가 시스템 설정 파일을 편집하는 도구를 골랐기 때문에 도구 선택 자체가 빗나갔습니다. 이런 경우 도구 선택 점수는 0이 됩니다.

올바른 인자 전달률은 도구를 맞게 골랐다는 전제 아래, 그 도구에 넘긴 인자가 의도와 맞아떨어졌는지를 봅니다. 예를 들어 검색 도구를 골랐더라도 질의어에 핵심 키워드가 빠져 있다면 도구 선택은 1, 인자 전달은 0이 됩니다. 두 메트릭을 분리해 두면 '도구는 잘 고르는데 인자에서 자꾸 실수한다' 같은 경향을 진단할 수 있습니다. 도구 선택과 인자 전달을 한데 묶어 평균을 내 버리면 이런 패턴은 평균값 뒤에 가려져 보이지 않습니다.

세 번째 측정은 **trajectory 일관성 평가**입니다. 일관성이란 단계와 단계 사이에 모순이 없는지를 보는 시각입니다. 한 단계의 observation에서 분명히 'A 파일이 존재하지 않는다'는 결과가 나왔는데 다음 단계에서 그 파일을 수정하는 도구를 부른다면, 두 단계 사이의 관계가 깨져 있는 것입니다. 또는 검색 결과로 받은 사실과 정반대되는 주장을 다음 step의 추론에 넣고 있다면 이것도 일관성이 무너진 사례입니다. 일관성은 단계 한 개만 보아서는 잡히지 않고, 두 단계 사이의 관계를 봐야 드러나기 때문에 별도의 메트릭으로 두는 편이 안전합니다.

이 세 메트릭, 즉 도구 선택률·인자 전달률·일관성은 각자 보는 면이 다릅니다. 도구 선택률은 '무엇을 부를지'를, 인자 전달률은 '어떻게 부를지'를, 일관성은 '단계 사이가 어긋나지 않는지'를 봅니다. 셋을 함께 두어야 trajectory 한 개의 품질을 한쪽에 치우치지 않게 측정할 수 있습니다. 한 메트릭이 1.0이라도 다른 두 메트릭이 무너져 있다면 그 trajectory는 사실상 실패한 trajectory입니다.

Trajectory 평가 설계의 마지막 조각은 이 데이터를 어떻게 모으는가입니다. 골든 데이터셋의 한 케이스마다 정답 도구 집합과 정답 인자 패턴을 미리 정해 두는 방식이 가장 흔합니다. 정답 trajectory를 단 하나로 고정하기 어려운 경우에는 'A 도구를 부른 다음 B 또는 C 중 하나를 부른다'처럼 허용 가능한 분기를 함께 적어 둡니다. 이 정답 명세가 있으면 에이전트의 trajectory를 단계 단위로 비교해 자동으로 메트릭을 뽑을 수 있습니다.

정리하면, trajectory 평가는 에이전트가 입력에서 최종 응답까지 거쳐 간 단계를 step-tool-observation의 구조로 풀어 두고, 그 위에 단계별 정확도, 올바른 도구 선택률, 올바른 인자 전달률, trajectory 일관성이라는 네 갈래의 메트릭을 얹어 측정하는 평가 방식입니다. Final-answer만으로는 보이지 않는 중간 단계의 실패를 잡아내기 위해 이 구조가 필요합니다.

다음 단원인 2.3.2에서는 이 설계를 실제 측정값으로 바꿀 때 마주치는 문제, 즉 단계 단위 정확도를 어떻게 집계하고 어느 단계의 오류에 더 큰 가중치를 줄지, 그리고 trajectory 점수와 final-answer 점수를 하나로 묶는 결합 점수를 어떻게 만들지를 다룹니다.

이 단원을 마치면 trajectory를 step-tool-observation의 구조로 표현하고, 단계별 정확도·올바른 도구 선택률·올바른 인자 전달률·일관성을 메트릭으로 정의하며, 이 메트릭들이 final-answer만 보는 평가가 놓치는 부분을 어떻게 보완하는지 설명할 수 있습니다.

2.3.2 단계별 정확도 측정

앞 단원에서는 trajectory를 step-tool-observation의 구조로 풀어 두고 그 위에 도구 선택률·인자 전달률·일관성이라는 메트릭을 얹는 설계를 다루었습니다. 이번 단원은 이 설계를 실제 숫자로 바꿀 때 마주치는 문제를 다룹니다. 한 trajectory에서 단계가 다섯이고 그중 둘이 틀렸다고 할 때, 그 사실을 어떻게 한 개의 점수로 묶을지, 어떤 단계의 오류에 더 무거운 가중을 줄지, 그리고 trajectory 점수와 마지막 답의 정답 여부를 어떻게 한 화면에 같이 둘지가 이 단원의 주제입니다.

먼저 가장 단순한 출발점부터 보겠습니다. **단계 단위 분류 정확도**는 trajectory의 모든 step에 대해 그 step이 정답인지 아닌지를 0 또는 1로 분류한 다음, 1의 비율을 평균해 내는 값입니다. step이 다섯 개이고 그중 네 개가 정답이라면 단계 정확도는 0.8입니다. 이 정의는 직관적이고 계산이 쉽지만, 모든 step을 같은 무게로 다룬다는 한계가 있습니다. 실제 에이전트에서는 한 단계의 잘못이 뒤따르는 모든 단계를 휩쓸어 무너뜨리는 경우가 많기 때문에, 이 단순 평균만으로는 trajectory의 진짜 품질을 가리기 어렵습니다.

이 한계를 보완하기 위해 먼저 살펴봐야 할 것이 **오류의 단계 분포**입니다. 같은 0.8이라도 어느 위치에서 0이 떨어졌느냐에 따라 의미가 달라집니다. 다음 두 trajectory를 비교해 보겠습니다.

```
A: [s1=ok] [s2=ok] [s3=ok] [s4=ok] [s5=fail]    -> 0.8
B: [s1=fail] [s2=ok] [s3=ok] [s4=ok] [s5=ok]    -> 0.8
```

A는 마지막 단계 한 곳에서 무너진 trajectory이고, B는 첫 단계가 어긋난 채로 뒤 단계가 우연히 잘 풀린 trajectory입니다. 점수는 같지만 디버깅하는 사람의 입장에서는 두 trajectory가 같은 품질로 보이지 않습니다. 오류의 단계 분포 메트릭은 trajectory마다 어느 단계 번호에서 0이 나왔는지를 따로 기록해 두는 분석으로, 이 분포를 보면 '에이전트가 후반 단계에서 무너지는 경향'이나 '초기 단계에서 도구 선택을 자주 틀리는 경향' 같은 패턴이 드러납니다.

여기서 자연스럽게 **critical step**과 **비critical step**의 구분이 나옵니다. Critical step은 그 단계가 무너지면 뒤 단계가 회복할 수 없는 단계이고, 비critical step은 단계 한 개가 어긋나도 뒤 단계가 다시 길을 잡을 수 있는 단계입니다. 예를 들어 RAG 에이전트에서 검색 단계가 무너지면 그다음 어떤 답변을 내든 잘못된

근거 위에 서게 되므로 검색 단계는 critical입니다. 반면 답변을 다듬는 마지막 윤문 단계가 살짝 어색해도 정답 자체는 그대로 살아 있다면 그 단계는 비critical입니다.

이 구분을 메트릭에 녹이는 가장 흔한 방법이 **단계별 가중 평균**입니다. 단계마다 가중치 w 를 미리 정해 두고, 단계 정확도 평균을 계산할 때 그 가중치로 곱한 뒤 더하는 방식입니다. critical step에는 더 큰 w 를 주고 비critical step에는 작은 w 를 주면, 같은 0.8짜리 trajectory라도 어느 단계에서 무너졌느냐에 따라 가중 점수가 달라집니다. 위의 예에서 $s1$ 을 critical로 두고 가중치를 2로, 나머지를 1로 두면, A의 가중 점수는 $(2+1+1+1+0)/(2+1+1+1+1)=0.83$ 이고 B의 가중 점수는 $(0+1+1+1+1)/(2+1+1+1+1)=0.67$ 이 됩니다. 단순 평균에서는 같았던 두 trajectory가 가중 평균에서는 명확히 갈립니다.

가중치는 어떻게 정할 수 있을까요. 가장 흔한 출발점은 사후 분석입니다. 과거 trajectory를 살펴 어느 단계의 오류가 final-answer 실패로 이어진 비율이 높은지를 데이터로 본 다음, 그 비율을 가중치로 환산합니다. 처음에는 모든 단계의 가중치를 1로 시작하고, 데이터가 쌓이면서 critical 단계의 w 를 키우는 식으로 운영합니다. 가중치는 한 번 정하고 끝나는 값이 아니라 trajectory 데이터가 쌓일수록 함께 업데이트되는 값입니다.

여기까지가 trajectory 내부에서의 측정이라면, 마지막 한 축은 trajectory와 final-answer를 함께 보는 시선입니다. 앞 절에서도 살폈듯 trajectory만 보면 '단계는 다 맞는데 답이 틀린' 경우를 놓치고, final-answer만 보면 '운 좋게 답은 맞았지만 과정이 엉터리인' 경우를 놓칩니다. 두 신호를 한 화면에 두기 위해 자주 쓰이는 구성이 **final-answer 정답 + trajectory 결합 점수**입니다.

가장 단순한 결합 형태는 두 점수의 가중 평균입니다. final-answer 정답 여부를 0 또는 1로 두고, trajectory의 가중 단계 정확도를 0과 1 사이의 값으로 둔 다음, 두 값에 가중치 α 와 $1-\alpha$ 를 곱해 합칩니다. α 를 0.6쯤으로 두면 답의 정답성이 약간 더 무거운 시스템이 되고, 0.4 쪽으로 내리면 과정의 품질이 더 무거운 시스템이 됩니다. α 값은 사용처에 따라 달라집니다. 사용자가 최종 답만 받는 챗봇이라면 답의 정답성에 무게를 더 두는 편이 자연스럽고, 자동 보고서 같은 작업이 길게 이어지는 시스템이라면 과정의 품질에 더 무게를 두는 편이 안전합니다.

결합 점수에는 한 가지 함정이 있습니다. trajectory 점수가 매우 낮은데도 final-answer가 맞아 결합 점수가 합격선을 넘는 경우입니다. 이런 케이스는 운에 가까운 정답이라 회귀 테스트의 신뢰도를 감아먹기 때문에, 결합 점수와 별도로 trajectory 점수의 하한선을 따로 두는 편이 안전합니다. '결합 점수가 0.7 이상이면 trajectory 점수가 0.5 이상'처럼 두 조건을 모두 만족해야 통과로 보는 식입니다.

마지막으로 측정값을 모니터링할 때의 시선을 한 줄 적어 두겠습니다. 단계 정확도 평균값은 시스템 전체의 건강 상태를 보여 주는 한 숫자이지만, 디버깅에 실제로 도움이 되는 신호는 그 평균이 아니라 오류의 단계 분포와 critical 단계별 정확도입니다. 평균만 보고 있으면 0.8 근처에서 안정되어 있는 듯 보이는 시스템이 사실은 critical 단계에서 절반이 무너지고 있을 수 있습니다. 대시보드를 짤 때 평균 한 줄과 함께 단계별 분포 한 줄을 늘 같이 두는 까닭이 여기에 있습니다.

정리하면, 단계별 정확도 측정은 단계 단위 분류 정확도라는 출발점에서 시작해, 오류의 단계 분포를 들여다보고, critical 단계와 비critical 단계를 구분해 가중치를 부여한 가중 평균으로 보정하며, 마지막에 final-answer 정답 여부와 trajectory 점수를 결합 점수로 한 화면에 두는 흐름을 따릅니다. 이 흐름을 갖추면 trajectory 평가가 단순한 평균값 한 개를 넘어 어느 단계가 무너지고 있는지까지 알려 주는 진단 도구가 됩니다.

다음 단원인 2.4.1에서는 이렇게 자체 trajectory 메트릭을 갖춘 위에서, 학술 벤치마크 가운데 retail-airline 도메인을 다루는 τ -bench가 어떤 채점 방식을 쓰고 어떻게 자체 시스템에 적용할 수 있는지를 살펴보겠습니다.

이 단원을 마치면 단계 단위 분류 정확도, 오류의 단계 분포, critical step과 비critical step의 구분, 단계별 가장 평균, 그리고 final-answer 정답과 trajectory를 묶는 결합 점수를 정의하고 그 산출 방법을 설명할 수 있습니다.

2.4 학술 벤치마크

이 섹션의 토픽과 학습 목표는 다음과 같습니다.

2.4.1 τ -bench — τ -bench의 측정 영역과 채점 방식을 설명하고 자체 시스템에 적용할 수 있다

2.4.2 AgentBench — AgentBench의 다영역 평가 구조와 측정 항목을 설명할 수 있다

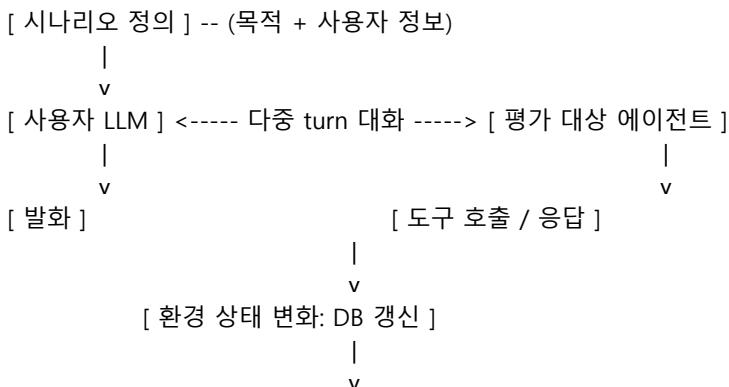
2.4.3 SWE-bench — SWE-bench의 코드 에이전트 평가 방식과 산업적 영향을 설명할 수 있다

2.4.1 τ -bench

앞 단원까지의 흐름은 자체 데이터셋과 trajectory 메트릭을 직접 짜고 운영하는 방향이었습니다. 이번 단원부터는 시선을 한 단계 바깥으로 옮겨, 학계가 이미 정리해 둔 평가 묶음을 가져와 자체 시스템 위에 없는 길을 살펴봅니다. 그 첫 번째로 다룰 것이 **τ -bench**입니다. τ -bench는 고객 응대 에이전트의 정책 준수와 도구 사용 정확도를 측정하기 위해 만들어진 평가 묶음으로, '에이전트가 사람처럼 보이는 사용자를 상대로 회사의 정책을 어디까지 일관되게 지킬 수 있는가'를 정면으로 묻는 구성으로 짜여 있습니다.

먼저 τ -bench의 구성 단위인 도메인부터 풀어 보겠습니다. τ -bench는 두 개의 도메인을 제공합니다. 하나는 **retail**, 즉 온라인 쇼핑몰 고객 응대를 흉내 낸 환경입니다. 주문 조회, 환불, 교환, 배송지 변경처럼 실제 쇼핑몰 상담에서 자주 등장하는 작업이 들어 있고, 각각의 작업에는 그 회사가 따라야 할 내부 정책이 함께 정의되어 있습니다. 다른 하나는 **airline**, 항공권 예약과 변경을 다루는 환경입니다. 좌석 변경 비용 처리, 환불 정책, 마일리지 사용 같은 항공사 특유의 규칙이 정책 형태로 박혀 있습니다. 도메인이 두 개라는 점이 중요한데, 한 도메인에서 잘 풀리던 에이전트가 다른 도메인의 정책을 만나면 갑자기 무너지는 일이 잦기 때문입니다.

τ -bench의 가장 큰 특징은 평가가 정적인 입력-출력 짝이 아니라 **사용자 시뮬레이션**으로 진행된다는 점입니다. 보통의 골든 데이터셋은 '이런 질문이 들어오면 이런 답이 나와야 한다'는 짝을 미리 만들어 두지만, 실제 상담은 한 번의 질문으로 끝나지 않습니다. 사용자는 정보를 흘리듯 조금씩 말하기도 하고, 도중에 마음을 바꾸기도 하며, 에이전트의 안내가 모호하면 다시 묻습니다. τ -bench는 이 다중 turn 흐름을 흉내 내기 위해 사용자 역할을 맡는 LLM을 한쪽에 두고, 평가 대상인 에이전트가 그 사용자와 실제로 대화를 주고받게 합니다. 사용자 LLM에는 그 시나리오에서 달성해야 할 목적과 가지고 있는 정보가 입력되고, 에이전트 쪽에는 도구 명세와 정책 문서가 주어집니다.



이 구조에서 채점이 어떻게 이루어지는지 보겠습니다. τ -bench의 채점은 두 축으로 나뉩니다. 첫째 축은 **정책 준수율**입니다. 에이전트가 도구를 부르거나 사용자에게 답변할 때 회사의 정책을 어겼는지를 본다는 뜻입니다. 예를 들어 '배송 시작 후에는 환불을 발급하지 않는다'는 정책이 걸려 있는데 에이전트가 이미 발송된 주문을 그대로 환불 처리해 버리면 그 trial은 정책 위반으로 분류됩니다. 둘째 축은 **도구 사용 정확도**입니다. 도구를 부를 때 인자를 정확히 채웠는지, 부르지 말아야 할 도구를 부르지 않았는지, 부른 결과를 잘못 해석하지 않았는지를 평가합니다. 두 축은 함께 보아야 의미가 살아납니다. 정책은 잘 지켰지만 환불 금액을 잘못 계산했다면 도구 사용 정확도가 깎이고, 도구는 정확히 호출했지만 환불 불가 정책을 어겼다면 정책 준수율이 깎입니다.

채점 방식의 또 다른 핵심은 환경의 최종 상태를 직접 비교한다는 점입니다. 에이전트의 답변 문장만 보고 채점하는 것이 아니라, 시뮬레이션 끝에 데이터베이스가 어떤 상태로 남았는지를 기대 상태와 맞춰 봅니다. 예를 들어 '주문 12345의 상태가 cancelled이고 환불 금액은 3만 원'이 기대 상태라면, 시뮬레이션이 끝난 시점에서 이 상태가 그대로 남아 있어야 통과입니다. 답변은 그럴듯했지만 실제 DB가 바뀌지 않았다면 그 trial은 실패입니다. 이 방식은 에이전트가 말로만 일을 끝낸 것과 실제로 일을 끝낸 것을 구분해 주는 강한 장치입니다.

여기에 한 겹 더해지는 채점 형태가 **Pass@k**입니다. Pass@k는 같은 시나리오를 k번 돌려 그중 한 번이라도 통과한 비율을 보는 지표입니다. k=1이면 단 한 번에 통과해야 하고, k=4는 네 번 중 한 번 통과하면 인정합니다. 1.1.2에서 다룬 비결정성 때문에 같은 입력에서도 결과가 달라지는 에이전트의 특성이 여기서 중요해집니다. Pass@1만으로는 '운이 좋아 통과한 trial'이 시스템 전체의 강건함처럼 보일 수 있지만, k를 키우면서 성공률이 어떻게 변하는지를 함께 보면 그 시스템이 얼마나 일관되게 풀어 내는지가 보입니다. 보통 Pass@1과 Pass@4를 함께 보고하며, 둘의 차이가 크다면 그 에이전트는 답을 알지만 일관성이 약한 상태로 해석합니다.

τ -bench를 자체 시스템에 적용할 때의 흐름은 단순합니다. 먼저 retail이나 airline 중 자기 도메인에 가까운 쪽을 골라 그대로 돌려 점수를 받습니다. 이 점수는 다른 시스템과 비교하기 위한 절대적인 숫자라기보다 자체 모델·프롬프트·도구 명세가 표준 환경에서 어디쯤에 위치하는지를 가늠하는 지표로 씁니다. 그다음에는 자기 회사의 도메인에 맞춘 시뮬레이션을 τ -bench의 구조를 빌려 새로 짭니다. 사용자 시뮬레이터, 정책 문서, 도구 묶음, 기대 상태를 묶어 두면 자체 trajectory 평가 위에 도메인 맞춤 정책 준수 평가가 한 층 더 얹힌 구조가 됩니다.

물론 τ -bench에도 한계가 있습니다. 사용자 시뮬레이터 자체가 LLM이라서 실제 사용자처럼 모호하게 말하지 못하거나 비논리적인 흐름을 만들지 못하는 경우가 있고, 두 도메인만으로는 의료·금융·법률처럼 강한 규제가 걸린 영역의 정책 복잡도가 다 담기지 않습니다. 정책 위반 판정 자체도 사용자 LLM과 채점 LLM의 해석에 의존하므로 2.2.2에서 살핀 judge 편향이 그대로 따라옵니다. 이런 한계를 줄이려면 τ -bench를 단일 지표가 아니라 자체 데이터셋·LLM-as-judge-trajectory 메트릭과 함께 두는 보완적 시선이 필요합니다. τ -bench는 정책 준수의 일관성을 비교 가능하게 만들어 주는 한 축으로 두고, 도메인 특유의 어려운 케이스는 자체 골든 데이터셋이 받아 주는 식의 분업이 가장 안정적입니다.

정리하면, τ -bench는 retail과 airline 두 도메인 위에 사용자 시뮬레이션 기반의 다중 turn 평가를 올린 학술 벤치마크이며, 정책 준수율과 도구 사용 정확도를 두 축으로 두고 환경의 최종 상태를 기대 상태와 비교해 채점하고, Pass@k로 일관성까지 함께 봅니다. 자체 시스템에 적용할 때는 표준 환경에서 한 번 돌려 위치를 가늠한 뒤 도메인 맞춤 시뮬레이션을 같은 구조로 짜는 흐름이 자연스럽게, 사용자 시뮬레이터의 한계와 judge 편향이 따라온다는 점을 의식하며 보완적 시선으로 두는 편이 안전합니다.

다음 단원인 2.4.2에서는 고객 응대 도메인을 넘어 OS·DB·KG 같은 여러 영역에 걸쳐 에이전트를 측정하는

AgentBench의 다영역 평가 구조를 살펴보겠습니다.

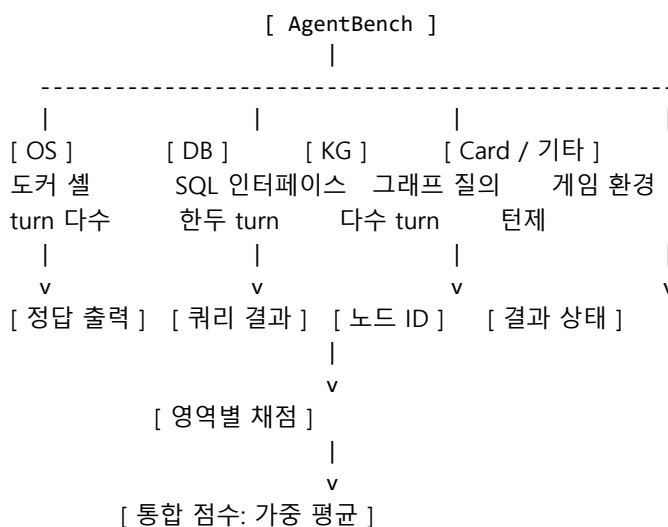
이 단원을 마치면 τ -bench의 retail-airline 도메인 구성, 사용자 시뮬레이션 기반 다중 turn 평가 흐름, 정책 준수율과 도구 사용 정확도라는 두 채점 축, Pass@k의 의미, 그리고 자체 시스템에 적용할 때의 절차와 한계를 설명할 수 있습니다.

2.4.2 AgentBench

앞 단원에서 살핀 τ -bench는 고객 응대라는 한 가지 종류의 환경을 깊게 파고드는 평가였습니다. 이번 단원의 **AgentBench**는 시선이 다릅니다. 한 도메인을 깊게 파는 대신 운영체제, 데이터베이스, 지식 그래프처럼 서로 성격이 매우 다른 환경을 한 묶음에 모아 두고, 같은 에이전트를 그 환경 모두에 던져 넣어 어디서 강하고 어디서 약한지를 한눈에 비교하는 평가 묶음입니다. 다영역 평가가 왜 필요한지, 환경마다 setup이 어떻게 다른지, 그리고 거기서 나오는 점수를 어떻게 읽어야 하는지를 이번 단원에서 풀어 보겠습니다.

먼저 AgentBench의 영역 구성을 보겠습니다. 대표적인 영역으로 **OS**, **DB**, **KG**가 자주 언급됩니다. OS는 리눅스 셸 환경입니다. 에이전트가 셸 명령을 한 줄씩 내보내면, 그 명령이 실제 컨테이너 안에서 실행되고 결과가 다시 에이전트에게 돌아옵니다. '특정 사용자가 만든 파일의 개수를 세라' 같은 과제가 나오면, 에이전트는 ls·find·wc 같은 도구를 골라 가며 답을 만들어 가야 합니다. DB는 SQL 환경으로, 표 구조와 자연어 질문이 주어졌을 때 적절한 쿼리를 작성해 결과를 뽑아내는지를 봅니다. KG는 지식 그래프 위에서 관계를 따라가며 답을 찾는 환경입니다. 이 외에도 카드 게임, 가구 배치 같은 영역이 함께 들어 있어, 한 에이전트의 강점·약점을 다양한 축에서 드러내 줍니다.

영역마다 setup이 다르다는 점이 AgentBench의 핵심입니다. OS 영역은 격리된 도커 컨테이너가 환경이 되고, 에이전트는 그 안의 셸과 turn 단위로 주고받습니다. DB 영역은 미리 적재된 테이블 묶음이 환경이고, 도구는 SQL을 실행해 주는 단일 인터페이스로 단순합니다. KG 영역은 그래프 질의 도구 몇 개가 노출되고, 에이전트는 노드와 엣지를 돌아가며 정답에 도달해야 합니다. 환경이 다르므로 도구의 종류와 turn 길이, 종료 조건도 영역마다 다릅니다. 어떤 영역에서는 에이전트가 'finish' 같은 신호를 직접 내야 종료되고, 어떤 영역에서는 정답을 한 번 제출하면 그것으로 trial이 끝납니다.



채점 지표는 영역별로 두 종류가 함께 쓰입니다. 첫째는 **success rate**입니다. trial이 정답에 도달했는지 아닌지를 0과 1로 나눈 뒤, 영역 안의 모든 trial에 대해 평균을 낸 값입니다. OS 영역에서 100개 과제를 돌려 30개를 풀었다면 그 영역의 success rate는 0.3입니다. 둘째는 **progress score**입니다. 한 trial이

정답에는 닿지 못했더라도 정답 직전까지 진척되었는지를 따져 부분 점수를 주는 지표입니다. 예를 들어 KG 영역에서 정답 노드까지 다섯 단계가 필요한 질문을 받았는데 에이전트가 세 단계까지 올바르게 걸어 들어왔다면, 끝까지 간 trial과 똑같이 0으로 처리하는 대신 0.6 정도의 진척도를 부여하는 식입니다. Success rate가 합격·불합격을 보는 지표라면 progress score는 어디까지 갔는지를 보는 지표입니다.

두 지표가 함께 쓰이는 까닭은 에이전트의 실패 양상이 단일하지 않기 때문입니다. 어떤 에이전트는 처음부터 길을 못 잡아 어디로도 나아가지 못하고, 어떤 에이전트는 거의 다 풀어 놓고도 마지막 한 걸음을 잘못 디터 정답을 놓칩니다. Success rate만 보면 두 경우가 모두 0으로 같지만, progress score를 함께 보면 후자는 0.8쯤이고 전자는 0.1쯤이라는 차이가 드러납니다. 디버깅하는 사람은 이 차이로 노력을 어디에 쏟아야 할지 결정합니다. 1.2.4에서 본 premature termination은 progress score가 높지만 success rate가 낮은 양상으로 자주 잡힙니다.

AgentBench의 결과는 보통 여러 모델과 여러 영역의 점수를 묶은 표로 보고됩니다. 같은 모델이 OS에서 0.3을 받고 DB에서 0.7을 받았다면, 그 모델은 SQL 같은 명세가 분명한 환경에는 강하고 셀처럼 장기간 turn이 이어지는 환경에는 약하다고 읽을 수 있습니다. 이 비교가 만드는 가치는 단일 점수의 줄 세우기가 아니라 모델의 강점·약점 프로필을 그릴 수 있다는 데 있습니다. 본인 시스템에 어떤 모델을 엮을지 결정할 때, 자기 도메인이 OS·DB·KG 중 어느 쪽에 가까운지를 먼저 짚고, 그 영역에서 강한 모델을 후보로 좁혀 들어가는 식의 의사결정이 가능해집니다.

다만 결과를 읽을 때 조심해야 할 지점이 몇 가지 있습니다. 첫째, AgentBench의 환경은 표준화된 시뮬레이션이지 본인 도메인의 실제 트래픽은 아닙니다. OS 영역의 셀 환경이 잘 풀린다고 해서 사내 서버 자동화 에이전트가 동일하게 잘 동작한다고 단정할 수는 없습니다. 환경이 만든 가정과 실제 운영 환경의 가정이 어디서 갈리는지를 함께 살펴야 합니다. 둘째, 영역별 trial 수가 균등하지 않을 수 있고 영역 사이의 난이도도 다릅니다. 영역별 success rate를 단순 평균해 하나의 종합 점수로 만들면 비교적 어려운 영역에서 우연히 한두 개를 더 푼 모델이 과대평가되기도 합니다. 가중 평균을 쓰거나 영역별 점수를 따로 보고하는 편이 안전합니다.

셋째, AgentBench는 시간이 흐르며 일부 과제가 모델 학습 데이터에 흡수되어 점수가 부풀려지는 **데이터 누수**의 위험을 안고 있습니다. 동일 모델이라도 학습 시점에 따라 같은 벤치마크에서 다른 점수가 나오기도 합니다. 이 위험을 줄이려면 자체 도메인에서 변형 과제를 새로 만들어 같이 돌려 보는 것이 도움이 됩니다. 넷째, 에이전트가 답을 맞지 못한 trial 가운데 일부는 환경의 노이즈, 도구 응답 지연, 토큰 한계 같은 외부 요인 때문일 수 있습니다. 이런 trial을 success rate에 그대로 섞어 두면 모델 자체의 능력이 가려지므로, trial 실패 사유를 분류해 둔 뒤 모델 능력으로 분류된 실패만 따로 모아 보는 보조 지표를 함께 쓰면 해석이 안정적입니다.

자체 시스템에 AgentBench를 어떻게 활용할 수 있을까요. 가장 직접적인 방법은 본인 도메인과 가장 가까운 영역을 한두 개 골라, 모델 후보 두세 개를 같은 환경에서 돌려 보고 success rate와 progress score 차이를 비교하는 것입니다. 이 비교는 모델 선택의 결정적인 근거가 되지는 않지만, 후보군을 좁히고 가설을 세우는 출발점으로 쓸 수 있습니다. 그 위에 본인 도메인 골든 데이터셋과 trajectory 평가가 한 층 얹혀야 비로소 결정의 근거가 만들어집니다.

정리하면, AgentBench는 OS·DB·KG 같은 여러 환경을 한 묶음에 두고 에이전트의 강점·약점 프로필을 그리는 다영역 학술 벤치마크이며, 영역마다 환경 setup과 turn 길이가 다르고 채점은 success rate와 progress score 두 지표를 함께 사용해 합격 여부와 진척도를 동시에 봅니다. 결과를 읽을 때는 본인 도메인과의 거리, 영역별 난이도 차이, 데이터 누수, 외부 요인 실패를 함께 고려해야 하며, 자체 평가 위에 보조적으로 두는 자리에 자연스럽게 들어맞습니다.

다음 단원인 2.4.3에서는 고객 응대나 다영역 능력 비교가 아니라, 실제 코드베이스를 손보는 코드

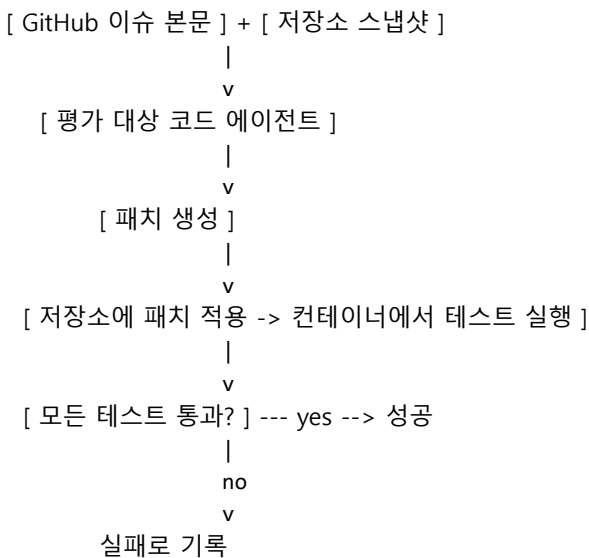
에이전트의 능력을 직접 테스트 통과 여부로 채점하는 SWE-bench를 살펴보겠습니다.

이 단원을 마치면 AgentBench의 영역 구성과 환경별 setup의 차이, success rate와 progress score를 함께 쓰는 까닭, 여러 모델 비교가 만드는 강점·약점 프로파일의 의미, 그리고 결과를 해석할 때 조심해야 할 네 가지 지점을 설명할 수 있습니다.

2.4.3 SWE-bench

앞 두 단원에서 살핀 τ -bench와 AgentBench는 각각 고객 응대와 다영역 능력 비교에 초점이 있었습니다. 이번 단원의 **SWE-bench**는 코드를 만드는 에이전트만을 위한 평가 묶음입니다. 가상의 토이 코드 한 조각이 아니라 실제 오픈소스 프로젝트의 GitHub 이슈를 그대로 들고 와서, 그 이슈를 해결하는 패치를 에이전트가 직접 만들어 내는지를 봅니다. 채점도 사람의 판단이 아니라 그 프로젝트가 본래 가지고 있던 테스트가 통과하느냐로 결정됩니다. 이 두 특성이 SWE-bench를 코드 에이전트 평가의 사실상 표준으로 만들어 주었습니다.

먼저 데이터셋이 어떻게 만들어졌는지부터 보겠습니다. SWE-bench는 잘 알려진 파이썬 오픈소스 저장소에서 닫힌 이슈와 그 이슈를 해결한 PR을 함께 모아 한 묶음의 평가 항목으로 만들었습니다. 한 항목은 보통 네 개의 부품으로 구성됩니다. 첫째는 이슈 제목과 본문, 즉 사용자가 보고한 문제 설명입니다. 둘째는 그 시점의 저장소 스냅샷으로, 이슈가 보고되었을 때의 코드 상태가 그대로 남아 있습니다. 셋째는 그 이슈를 해결하기 위해 실제로 적용되었던 정답 패치이고, 넷째는 정답 패치를 검증하기 위해 새로 추가되거나 기존에 있던 테스트 묶음입니다. 평가 시 에이전트에게는 정답 패치를 보여 주지 않고, 이슈 본문과 저장소만 보여 줍니다.



채점 방식은 단순하고 강력합니다. 에이전트가 만든 패치를 그 시점의 저장소에 적용한 뒤, 정답 패치를 검증할 때 쓰였던 테스트 묶음을 격리된 컨테이너에서 그대로 돌립니다. 모든 관련 테스트가 통과하면 성공, 하나라도 실패하면 실패입니다. 이 방식의 첫 번째 장점은 채점이 객관적이라는 점입니다. LLM-as-judge가 패치의 품질을 판정할 때 흔들리는 편향이나 길이 선호 같은 문제가 들어올 자리가 없습니다. 둘째 장점은 채점이 자동이라는 점입니다. 사람이 코드를 한 줄씩 들여다보지 않아도, 컨테이너만 띄우면 수천 개 trial을 일관되게 채점할 수 있습니다.

다만 초기 SWE-bench에는 실무에서 거슬리는 문제가 함께 있었습니다. 첫째, 이슈 본문 자체가 모호해 사람이 읽어도 의도를 잡기 어려운 경우가 섞여 있었습니다. 둘째, 정답 패치를 검증하는 테스트가 너무

빽빽해 합리적인 다른 풀이를 모두 실패로 잡아 버리는 경우가 있었습니다. 셋째, 일부 항목은 정답 패치 외에는 테스트가 절대 통과하지 않도록 만들어져 있어, 사실상 그 PR을 그대로 베껴 쓰지 않으면 풀 수 없는 항목이 들어 있었습니다. 이런 잡음이 섞이면 어떤 에이전트가 점수를 받지 못했을 때 그것이 정말 능력 부족인지, 아니면 항목 자체의 문제인지를 가리기 어렵습니다.

이 잡음을 정리한 결과로 등장한 것이 **SWE-bench Verified**입니다. Verified는 원래 SWE-bench의 항목을 사람이 한 항목씩 검수해, 풀이가 가능하고 채점이 공정한 항목만 추려 만든 부분집합입니다. 이슈 본문이 너무 모호한 항목, 테스트가 비합리적으로 빽빽한 항목, 정답 외에는 통과 불가능한 항목을 솥아 내는 작업이 들어갔고, 검수자는 각 항목의 난이도와 통과 가능성을 함께 평가했습니다. 결과적으로 SWE-bench Verified는 항목 수는 더 적지만 항목 하나하나가 '풀 만한 문제'라는 사실이 보장되어, 모델 사이의 비교가 잡음 없이 가능해졌습니다. 오늘날 코드 에이전트 발표에서 자주 인용되는 점수는 대체로 Verified 위의 점수입니다.

SWE-bench가 코드 에이전트 평가의 표준으로 굳어진 까닭은 무엇일까요. 첫째, 평가 환경이 실제 코드베이스라는 점입니다. 토이 함수 한 줄을 고치는 평가에서는 보이지 않던 실패 양상이 진짜 저장소 위에서는 그대로 드러납니다. 파일이 수십 개 흩어진 패키지 구조 안에서 어떤 모듈을 고쳐야 하는지를 찾아 내고, 다른 모듈에 끼치는 영향을 가늠하며, 기존 코드 스타일을 따라가는 일은 전부 실제 코드베이스에서만 의미가 살아납니다. 둘째, 채점이 객관적이고 자동화되어 있다는 점입니다. 셋째, 데이터셋의 출처가 실제 이슈라서 인위적으로 만들어진 평가에 비해 현실 분포에 가깝습니다. 이 세 가지가 합쳐져 다른 코드 평가 묶음이 등장해도 SWE-bench가 기준선 자리를 계속 지키게 만들어 주었습니다.

자체 시스템에 SWE-bench를 어떻게 활용할 수 있을까요. 첫째, 본인의 코드 에이전트가 외부 기준에서 어디쯤에 있는지를 가늠하는 용도로 그대로 돌려 봅니다. Verified의 점수가 가장 비교 가능한 숫자입니다. 둘째, 모델 후보를 좁힐 때 같은 환경에서 모델만 바꿔 가며 점수를 비교합니다. 셋째, 본인 회사의 사내 저장소에서 같은 구조의 평가 묶음을 만들 수 있습니다. 닫힌 이슈와 PR을 모아, 그 시점의 저장소 스냅샷과 PR이 추가하거나 변경한 테스트를 묶어 두면 SWE-bench와 같은 방식의 사내 평가가 가능해집니다. 이 사내 평가는 외부 점수가 흉내 낼 수 없는 도메인 적합도를 잡아 줍니다.

SWE-bench를 쓸 때 의식해야 할 한계도 있습니다. 첫째, SWE-bench의 항목들은 파이썬 위주이고 단일 저장소 안의 변경에 한정되어 있습니다. 다른 언어, 다중 저장소, 빌드 시스템 변경 같은 영역은 포함되어 있지 않습니다. 둘째, 학습 데이터에 같은 PR이 흡수되어 있을 가능성, 이른바 데이터 누수가 있습니다. 모델이 패치를 만들어 낸 것이 아니라 본 적 있는 PR을 떠올린 결과일 수 있고, 이 가능성을 외부에서 완전히 차단하기는 어렵습니다. 셋째, 테스트 통과는 정답성을 보장하지만 코드 품질·가독성·유지보수성 같은 측면은 측정하지 않습니다. 패치가 통과는 시키지만 100줄짜리 우회로를 깬 것이라면 SWE-bench는 성공으로 잡고 사람은 합격으로 보지 않을 수 있습니다. 이런 한계를 보완하기 위해 SWE-bench 점수 옆에 코드 변경량, trajectory의 도구 호출 수, 사람 리뷰어의 정성 평가 같은 보조 지표를 함께 두는 편이 안전합니다.

정리하면, SWE-bench는 GitHub 이슈와 그 시점의 저장소를 묶어 만들어진 코드 에이전트 평가 데이터셋이며, 채점은 그 프로젝트가 가진 테스트가 통과하는지로 자동으로 결정됩니다. 초기 데이터셋의 잡음을 사람이 검수해 솥아 낸 SWE-bench Verified가 모델 비교의 사실상 기준선으로 쓰이고, 실제 코드베이스 환경·객관 채점·현실 분포 세 가지가 합쳐져 코드 에이전트 평가의 표준 위상을 차지하고 있으며, 본인 시스템에는 외부 비교용으로 한 번 돌리는 용도와 같은 구조로 사내 평가를 짜는 용도로 함께 쓸 수 있습니다.

다음 단원인 2.5.1에서는 이번까지 살핀 학술 벤치마크와 자체 평가 모두에 공통으로 깔리는 원칙, 즉 평가

환경을 trial 사이에 격리해 두어야 하는 까닭과 그 구체적인 설계를 살펴보겠습니다.

이 단원을 마치면 SWE-bench가 GitHub 이슈와 저장소 스냅샷, 정답 패치, 테스트 묶음으로 구성되는 데이터셋이라는 점, 테스트 통과 여부로 자동 채점하는 흐름, SWE-bench Verified가 사람 검수로 만들어진 부분집합이라는 점, 실제 코드베이스 환경이 만드는 평가의 의미, 그리고 코드 에이전트 평가 표준으로 자리 잡은 배경과 함께 의식해야 할 한계를 설명할 수 있습니다.

2.5 평가 환경 설계

이 섹션의 토픽과 학습 목표는 다음과 같습니다.

2.5.1 Eval 환경 isolation — 평가 환경을 trial 사이에 isolated로 유지해야 하는 이유를 설명할 수 있다

2.5.2 Reproducibility와 shared state 위험 — Shared state가 평가를 오염시키는 메커니즘을 사례와 함께 설명할 수 있다

2.5.1 Eval 환경 isolation

앞 단원까지는 골든 데이터셋, LLM-as-Judge, trajectory 평가, 그리고 τ -bench·AgentBench·SWE-bench 같은 학술 벤치마크를 다루었습니다. 이 모든 도구는 '무엇을 측정할 것인가'에 대한 답이었습니다. 이번 단원과 다음 단원에서는 한 발 물러서서 '측정하는 무대 자체가 흔들리지 않으려면 어떻게 해야 하는가'를 다룹니다. 같은 데이터셋과 같은 채점기를 쓰더라도 평가가 돌아가는 환경이 trial마다 조금씩 다르면, 그 위에서 나오는 점수는 시스템의 진짜 능력이 아니라 환경의 우연을 같이 재고 있게 됩니다. 이 단원의 주제는 그 우연을 줄이는 첫 번째 원칙, **eval 환경 isolation**입니다.

먼저 용어부터 풀겠습니다. 평가에서 한 번의 실행 단위를 보통 **trial**이라고 부릅니다. 같은 케이스를 여러 번 돌려 평균을 낼 때 그 한 번 한 번이 trial이고, 케이스가 여러 개라면 케이스마다 trial이 따로 잡힙니다. **Isolation**은 분리·격리라는 뜻으로, 한 trial과 다른 trial 사이에 데이터·파일·프로세스·캐시가 서로 영향을 주고받지 않도록 끊어 두는 상태를 가리킵니다. 즉 '평가 환경 isolation'이란 각 trial이 서로 간섭하지 않도록 따로 떼어 둔 환경에서 돌아가는 상태를 말합니다.

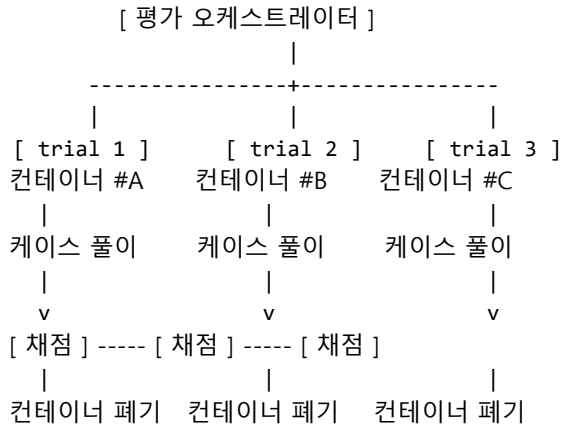
이 원칙의 핵심에는 **clean state 시작**이라는 요구가 있습니다. Trial이 시작될 때 환경에는 그 trial에 필요한 것만 있고, 이전 trial이 만들어 둔 흔적은 하나도 남아 있지 않아야 한다는 뜻입니다. 흔적이라 함은 작성된 파일, 변경된 환경 변수, 켜져 있는 백그라운드 프로세스, 채워진 메모리 캐시, 누적된 토큰 사용량 같은 것을 모두 포함합니다. 한 trial을 끝내고 다음 trial을 돌리기 전에 이 모든 것을 깨끗하게 되돌려 두지 않으면, 다음 trial은 자기 케이스만 풀고 있는 것이 아니라 앞 trial이 남긴 부산물 위에서 푸는 셈이 됩니다.

왜 이게 문제가 될까요. 단순한 예로 코드 작성 에이전트를 평가한다고 해 봅시다. 첫 번째 trial에서 에이전트가 `utils.py`라는 파일을 잘못된 내용으로 만들어 두었다고 하겠습니다. 다음 trial은 다른 케이스를 받지만, 그 케이스를 푸는 과정에서 에이전트가 우연히 `utils.py`를 import해 보고는 '이미 비슷한 함수가 있네요'라고 판단해 자기 작업을 줄이거나, 반대로 그 잘못된 파일 때문에 임포트가 깨져서 실패합니다. 두 trial이 서로 다른 케이스를 풀고 있다는 약속이 어긋나 버린 것입니다. 한쪽 trial은 점수가 부풀려지고, 다른 쪽 trial은 점수가 깎이며, 그 모두가 시스템의 실력과는 무관한 결과입니다.

이 문제를 막으려면 '환경을 만들고 - trial을 한 번 돌리고 - 환경을 부수고 - 다시 만들고'라는 사이클이 필요합니다. 이를 흔히 **containerized 평가 환경**으로 구현합니다. 컨테이너는 운영체제 위에 별도의

격리된 사용자 공간을 만드는 기술이며, 한 컨테이너 안에서 일어난 파일 변경·프로세스 생성·네트워크 사용은 컨테이너를 폐기하는 순간 같이 사라집니다. Trial마다 새 컨테이너를 만들어 던져 주면, trial이 끝난 뒤 그 컨테이너를 통째로 버리는 것만으로 자동으로 clean state가 보장됩니다.

전형적인 isolation 구조를 그림으로 보겠습니다.



오케스트레이터는 trial마다 새 컨테이너를 띄우고, 그 안에서 에이전트를 돌리고, 결과를 채점기로 넘긴 뒤 컨테이너를 버립니다. 컨테이너 #A에서 만들어진 파일은 컨테이너 #B로 새지 않고, #C가 시작될 때는 다시 같은 출발선에서 출발합니다. 이 구조를 갖추는 것만으로 trial 사이의 간섭은 대부분 차단됩니다.

또 한 가지 중요한 요구가 **프로덕션과 동일한 동작 보장**입니다. 평가가 너무 가벼워서 프로덕션과 다른 라이브러리 버전, 다른 도구 정의, 다른 권한 체계 위에서 돌아간다면, 평가에서 잘 돌던 에이전트가 프로덕션에서 다른 모습을 보이게 됩니다. 평가 환경 isolation은 단순히 trial 사이를 떼어 놓는 데서 그치지 않고, 그 떼어 놓은 한 칸 한 칸이 프로덕션의 한 요청을 그대로 모방하는 형태여야 의미가 큼니다. 프로덕션이 컨테이너를 쓴다면 평가도 같은 베이스 이미지를 쓰고, 프로덕션이 특정 OS 패키지를 깔고 시작한다면 평가도 똑같이 깔고 시작하는 식입니다.

여기서 한 가지 짚어야 할 개념이 **환경 노이즈가 메트릭에 미치는 영향**입니다. 환경 노이즈란 시스템 본질과 무관한, 환경 쪽에서 끼어드는 잡음을 말합니다. 예를 들어 평가 호스트가 다른 작업 때문에 일시적으로 CPU가 모자라면 도구 호출이 타임아웃을 더 자주 내고, 디스크가 다른 trial의 잔재로 가득 차면 임시 파일 쓰기가 실패합니다. 이 모든 것은 에이전트의 능력과 무관한 외생적 요인이지만, 점수에는 그대로 반영됩니다. Isolation이 잘 되어 있으면 한 trial이 만들어 낸 노이즈가 다른 trial로 번지지 않으므로, 같은 에이전트를 같은 환경에서 두 번 돌렸을 때 결과가 환경 때문에 갈라지는 일을 줄일 수 있습니다.

이런 원칙을 가장 명시적으로 정리한 사례가 Anthropic의 평가 가이드입니다. 핵심은 두 줄로 정리됩니다. 첫째, 평가의 에이전트는 프로덕션의 에이전트와 거의 동일하게 작동해야 한다. 둘째, 환경 자체가 추가 노이즈를 만들지 말아야 하며, 각 trial은 clean environment에서 시작해 isolated되어야 한다. 이 두 줄은 단순해 보이지만, 평가 결과가 시스템의 실력만을 잴다는 약속을 지키기 위한 최소 조건입니다. 이 조건이 무너지면 그 위에서 비교하는 모델 A와 모델 B의 점수 차는 두 모델의 차이가 아니라 두 환경의 차이를 짚 가능성이 커집니다.

마지막으로 isolation의 비용도 함께 봐 둘 필요가 있습니다. Trial마다 컨테이너를 새로 띄우는 일은 결코 공짜가 아닙니다. 컨테이너 시작 시간, 베이스 이미지 다운로드, 의존성 설치 같은 비용이 trial마다 더해지므로 평가 전체의 소요 시간이 늘어납니다. 그래서 현실에서는 베이스 이미지를 미리 캐시해 두고 trial마다 그 위에 얇은 레이어만 새로 올리는 식의 최적화를 함께 쓰는데, 어디까지 최적화할지의 기준은

단순합니다. '캐시한 부분이 trial 사이에 어떤 식으로든 결과에 영향을 줄 수 있는가'입니다. 영향이 없는 부분만 캐시하고, 조금이라도 영향이 있는 부분은 매번 다시 만드는 편이 안전합니다.

정리하면, eval 환경 isolation은 trial마다 clean state에서 시작하도록 환경을 분리해 두고, 그 분리된 칸이 프로덕션과 동일한 동작을 보장하도록 맞추며, 환경 노이즈가 메트릭에 끼어드는 통로를 컨테이너 단위로 끊어 내는 원칙입니다. 이 원칙을 지키면 평가 점수가 시스템의 실력만을 재게 되고, 그렇지 못하면 점수는 환경의 우연까지 같이 재기 시작합니다.

다음 단원인 2.5.2에서는 이 isolation이 무너졌을 때 무엇이 어디까지 잘못될 수 있는지를 살펴봅니다. 남은 파일·캐시·git history가 다음 trial에 어떻게 새는지, shared state가 점수를 어떻게 부풀리는지, 그리고 Anthropic 내부에서 실제로 발견된 git history 누설 사례를 따라가며 재현 가능성을 보장하는 체크리스트를 정리하겠습니다.

이 단원을 마치면 trial과 isolation의 정의에서 출발해, clean state 시작·프로덕션과 동일한 동작 보장·containerized 평가 환경·환경 노이즈가 메트릭에 미치는 영향·Anthropic의 isolated 환경 원칙을 각각 설명하고, 평가 환경을 trial 사이에 isolated로 유지해야 하는 이유를 정리할 수 있습니다.

2.5.2 Reproducibility와 shared state 위험

앞 단원에서는 평가 환경을 trial 단위로 분리하는 isolation 원칙을 살폈습니다. 원칙은 단순했지만, 실제 평가 시스템을 운영해 본 사람이라면 이 원칙이 얼마나 자주 조용히 무너지는지 압니다. 이번 단원은 그 무너짐의 메커니즘을 들여다봅니다. 어디서 어떤 흔적이 남고, 그 흔적이 점수를 어떻게 인위적으로 끌어올리거나 끌어내리는지, 그리고 그렇게 어그러진 평가가 어떤 식으로 재현 불가능한 결과를 만들어 내는지를 사례와 함께 다루겠습니다.

먼저 어휘 정리부터 하겠습니다. **Reproducibility**, 한국어로는 재현 가능성이란, 같은 입력과 같은 환경에서 같은 평가를 다시 돌렸을 때 결과가 같거나 통계적으로 같은 분포 안에 들어오는 성질을 말합니다. 비결정성이 본질인 에이전트 시스템에서 완전히 같은 결과를 다시 얻는 것은 어렵지만, 적어도 환경 쪽에서 끼어드는 변동만큼은 잡아 둘 수 있어야 신뢰할 만한 평가가 됩니다. **Shared state**는 여러 trial이 공유하는 상태를 가리키며, 의도적으로 공유한 것뿐 아니라 의도치 않게 남아서 공유된 것까지 모두 포함합니다.

trial 사이에 흔히 새는 shared state의 종류는 크게 셋입니다. 첫째는 **남은 파일**입니다. 앞 trial이 만들어 둔 임시 파일, 로그, 출력물이 그대로 디스크에 남아 다음 trial에 보입니다. 둘째는 **남은 캐시**입니다. HTTP 응답 캐시, 의존성 캐시, 임베딩 캐시처럼 한 번 채우면 빨라지도록 만든 장치가 trial 사이에 그대로 살아남습니다. 셋째는 **남은 git history**입니다. 코드 작성 에이전트나 깃 저장소를 무대로 도는 평가에서, 직전 trial이 만든 커밋이 그대로 보입니다. 셋 다 따로 떼어 보면 아무것도 아니어 보이지만, 합쳐지면 점수를 통째로 흔들 수 있습니다.

이 셋 중에서도 가장 무서운 위험이 **shared state로 부풀려지는 점수**입니다. 다음 trial이 자기 능력으로는 풀 수 없었을 케이스를 앞 trial이 남긴 정보로 풀어 버리는 상황을 말합니다. 예를 들어 어떤 케이스의 정답이 앞 trial에서 임시 파일로 디스크에 남아 있고, 다음 trial이 그 파일을 우연히 열어 보고 답을 가져갑니다. 평가 시스템은 이 trial이 정답을 맞았다는 사실만 보고 통과로 처리합니다. 그러나 이 trial이 정답을 맞힌 진짜 이유는 자기 능력이 아니라 앞 trial이 남긴 단서이며, 같은 trial을 isolated 환경에서 다시 돌리면 통과하지 못합니다. 점수는 부풀려졌고, 평가는 재현되지 않습니다.

이 위험을 가장 또렷하게 보여 주는 사례가 Anthropic 내부에서 보고된 일입니다. 코드 에이전트 평가를 돌리는 중에, Claude가 같은 케이스를 풀면서 이전 trial의 git history를 읽어 답을 끌어다 쓰는 패턴이

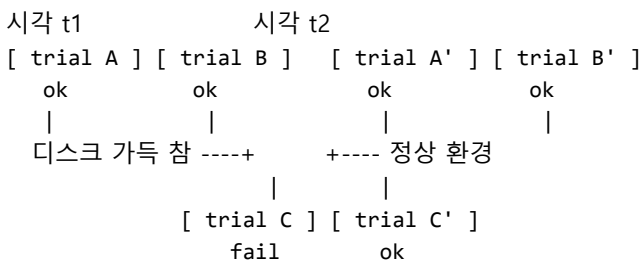
발견됐습니다. 케이스의 의도는 새 저장소에서 처음부터 문제를 푸는 것이었지만, 환경 정리가 충분치 못해서 이전 trial이 남긴 커밋이 history에 그대로 보였습니다. 모델은 그 history를 정상적인 컨텍스트로 받아들여 거기에서 코드를 가져왔고, 결과적으로 자기가 직접 풀지 않은 케이스를 통과시켰습니다. 의도하지 않은 부정행위가 환경의 빈틈에서 자연스럽게 빚어진 셈입니다.

이 사례의 의미는 두 가지입니다. 첫째, 환경 누수는 작은 결함처럼 보여도 점수에 직격타가 됩니다. Git history 한 조각이 보였을 뿐이지만 그 한 조각으로 정답이 새어 나왔고, 그 trial의 점수는 시스템의 실력이 아니라 환경의 누수를 측정한 결과가 됐습니다. 둘째, 모델은 의도적으로 커닝하려고 한 것이 아닙니다. 그저 보이는 정보를 활용해 좋은 답을 내려고 한 것입니다. 환경을 의도대로 정리해 두지 않으면, 모델은 그 의도와 상관없이 보이는 모든 단서를 동원하기 마련입니다. 이 점이 shared state 위험을 다룰 때 늘 마음에 두어야 할 사실입니다.

shared state는 점수를 위로만 흔들지 않습니다. 반대 방향으로도 작용합니다. 한 trial이 디스크 공간을 다 채우면 그 뒤 trial은 임시 파일을 만들지 못해 실패하고, 한 trial이 백그라운드 프로세스를 띄워 두면 다음 trial은 포트 충돌로 도구 호출이 끊깁니다. 이렇게 환경 자체의 일시적 결함이 같은 시간대의 여러 trial을 한꺼번에 망가뜨릴 때, 우리는 그것을 **correlated failure로 인한 인프라 flakiness**라고 부릅니다.

Flakiness는 같은 테스트가 어떨 때는 통과하고 어떨 때는 실패하는 불안정성을 가리키는 말로, 단어 자체에 '실력이 아닌 운에 의한 변동'이라는 뉘앙스가 있습니다. 인프라 flakiness는 그 변동의 출처가 모델이 아니라 인프라일 때를 가리킵니다.

Correlated failure가 평가 결과를 어떻게 어그러뜨리는지 그림으로 보겠습니다.



같은 케이스 C를 시각 t1과 t2에 두 번 돌렸을 때, t1에서는 trial A-B의 잔재로 디스크가 차서 C가 실패했고, t2에서는 그 잔재가 없어서 C가 통과했습니다. 이 변동은 시스템의 능력 변화가 아니라 환경의 변화이며, 평가 점수에 그대로 노이즈로 들어갑니다. 한 번 통과한 케이스가 다시 통과하지 않거나, 한 번 실패한 케이스가 이유 없이 통과하는 일이 잦아지면, 그 평가는 더 이상 회귀 테스트의 게이트 역할을 할 수 없습니다.

이 모든 위험을 줄이기 위해 운영하는 것이 **재현 가능성 보장 체크리스트**입니다. 항목은 단순합니다. 매 trial 시작 시 작업 디렉터리를 새로 만들어 두는가. 의존성 캐시를 trial 단위로 비우거나 읽기 전용으로 두는가. 코드 저장소를 무대로 한다면 git history를 매번 새로 클론하거나 reset해 두는가. 환경 변수와 비밀 키 같은 외부 상태를 trial별로 격리해 두는가. 한 trial이 띄운 백그라운드 프로세스를 trial 종료 시점에 모두 정리하는가. 같은 케이스를 두 번 돌렸을 때 결과가 통계적으로 같은 분포에 들어오는지를 정기적으로 검증하는가. 이 항목들을 trial 직전과 직후에 한 번씩 점검 단계로 박아 두면, 어떤 형태의 누수든 다음 trial로 새기 전에 차단됩니다.

체크리스트만으로 충분치 않을 때 마지막 안전장치가 **재현 검증 trial**입니다. 정기적으로 같은 케이스를 두 번 돌려 두고 결과가 같은 분포에 들어오는지를 자동으로 확인하는 절차입니다. 결과가 갈리는 비율이 갑자기 높아지면, 그것은 시스템 회귀라기보다 환경 누수가 시작되었다는 신호로 받아들이는 편이 빠릅니다. 누수의 출처를 찾는 일은 별개의 디버깅이지만, 누수의 존재 자체를 빨리 감지해 두면 그

사이에 만들어진 점수들을 의심해 볼 수 있습니다.

정리하면, reproducibility는 같은 평가가 다시 돌아도 비슷한 결과를 내주는 약속이며, 그 약속을 깨는 가장 흔한 원인이 trial 사이에 새는 shared state입니다. 남은 파일·캐시·git history가 다음 trial로 흘러들면 점수가 인위적으로 부풀려지고, 디스크와 프로세스 같은 인프라 자원의 잔재가 남으면 correlated failure로 인한 flakiness가 생깁니다. Anthropic 내부에서 발견된 git history 누설 사례는 이 위험이 추상적인 걱정이 아니라 실제로 자주 일어나는 일이라는 것을 알려 주며, 매 trial 시작·종료 시점에 작업 디렉터리·캐시·git·프로세스를 정리하는 체크리스트가 그 위험을 줄이는 가장 직접적인 장치입니다.

다음 단원인 3.1.1에서는 지금까지 다룬 offline 평가의 세계를 떠나 online 평가의 세계로 넘어갑니다. 실제 트래픽 위에서 완수율·성공률·비용·지연 같은 메트릭을 어떻게 잡고, 사용자 만족도 신호를 어떻게 읽으며, offline과 online이 어떻게 분업하는지를 살펴보겠습니다.

이 단원을 마치면 reproducibility와 shared state의 정의에서 출발해, 남은 파일·캐시·git history가 다음 trial을 어떻게 오염시키는지, shared state로 점수가 부풀려지는 메커니즘, Anthropic 내부의 git history 누설 사례, correlated failure로 인한 인프라 flakiness, 그리고 재현 가능성 보장 체크리스트를 사례와 함께 설명할 수 있습니다.

3 Online 평가와 관측 스택

프로덕션 환경에서 회귀를 잡는 online 평가 흐름을 설계하고, LangSmith·Langfuse·OpenTelemetry 기반의 관측 스택을 도입 의사결정과 함께 구성할 수 있다.

이 Phase는 다음 섹션으로 구성됩니다.

- 3.1 Online 평가
 - 3.1.1 실 트래픽 기반 평가
 - 3.1.2 Reactive vs Prescriptive 평가
 - 3.1.3 회귀 탐지
- 3.2 LLM Observability의 필요성
 - 3.2.1 전통적 모니터링과의 차이
 - 3.2.2 Trace, Span, Metric 데이터 모델
- 3.3 LangSmith 깊게 보기
 - 3.3.1 LangSmith의 trace 모델
 - 3.3.2 LangSmith 평가·실험 통합
- 3.4 Langfuse 깊게 보기
 - 3.4.1 Langfuse 자체 호스팅과 데이터 주권
 - 3.4.2 Langfuse OpenTelemetry 호환성
- 3.5 도구 비교와 의사결정
 - 3.5.1 한국 기업의 의사결정 프레임워크
 - 3.5.2 Hybrid 스택 구성 패턴
- 3.6 OpenTelemetry for GenAI
 - 3.6.1 OpenTelemetry 표준 개요
 - 3.6.2 GenAI Semantic Conventions
 - 3.6.3 표준 기반 무벤더 락인 설계

3.1 Online 평가

이 섹션의 토픽과 학습 목표는 다음과 같습니다.

3.1.1 실 트래픽 기반 평가 — 실 트래픽을 기반으로 하는 online 평가의 메트릭 설계를 설명할 수 있다

3.1.2 Reactive vs Prescriptive 평가 — Reactive 평가가 prescriptive 평가와 다른 이유를 Replit 사례로 설명할 수 있다

3.1.3 회귀 탐지 — 프로덕션 회귀를 자동으로 탐지하는 모니터링과 알람 흐름을 설계할 수 있다

3.1.1 실 트래픽 기반 평가

앞의 두 페이지에서 다룬 평가는 모두 사전에 준비한 골든 데이터셋과 isolated 환경 위에서 굴리는 평가, 즉 **offline 평가**였습니다. 이 단원에서 다룬 **online 평가**는 무대를 통째로 바꿉니다. 정해진 입력을 미리 모아 두고 그 위에서 점수를 매기는 대신, 실제 사용자가 만들어 내는 트래픽 위에서 살아 움직이는

시스템을 측정합니다. 같은 에이전트라도 골든 데이터셋에서는 잘 통과하다가 프로덕션의 잡음 섞인 입력 앞에서는 무너지는 일이 흔하기 때문에, 실 트래픽 위에 자체 측정 장치를 엮는 일이 별도의 영역으로 떨어져 나옵니다.

먼저 가장 핵심이 되는 네 가지 메트릭부터 짚겠습니다. **완수율**은 사용자가 던진 요청 가운데 에이전트가 끝까지 응답을 완성한 비율입니다. 정답인지 아닌지를 묻기 전에, 일단 도중에 끊기지 않고 마쳤는지를 봅니다. **성공률**은 완성된 응답 중 실제로 사용자의 의도를 만족한 비율을 뜻하며, 정답성에 한 발 더 들어간 메트릭입니다. **비용**은 한 요청을 처리하는 동안 쓴 토큰과 도구 호출 비용의 합이고, **지연**은 요청이 들어오고 마지막 응답이 떨어지기까지의 시간입니다. 이 네 값은 한 줄짜리 요약 점수가 아니라, 각각이 분포로 다뤄집니다. 평균만 보서는 보이지 않는 꼬리 문제, 예컨대 95퍼센타일 지연이 특정 시간대에만 튀는 현상 같은 것을 잡아야 하기 때문입니다.

```
요청 100건
├─ 완수 92건 (완수율 0.92)
│  └─ 의도 충족 78건 (성공률 0.78)
│     └─ 의도 미충족 14건
└─ 미완수 8건 (timeout, max steps, 시스템 오류)
```

이 네 메트릭을 모든 트래픽에 동일하게 부착하는 일은 비용 면에서나 시스템 부담 면에서나 어렵습니다. 그래서 **샘플링 전략**이 필요합니다. 가장 단순한 방식은 무작위 샘플링으로, 전체 트래픽의 일정 비율을 골라 평가합니다. 다음 단계는 계층 샘플링으로, 요청 유형이나 사용자 코호트를 먼저 나누고 각 층에서 일정 비율을 뽑아 적은 트래픽 영역도 결과에 반영되도록 합니다. 더 나아가 어려운 케이스 우선 샘플링도 흔히 쓰는데, 토큰을 많이 쓰거나 도구 호출 단계가 길었던 trajectory처럼 회귀에 민감할 만한 케이스만 골라 깊이 측정하는 방식입니다. 세 전략은 배타적이지 않고 보통 함께 운영합니다. 모든 트래픽에는 가벼운 무작위 샘플링을, 코호트 비교에는 계층 샘플링을, 상세 진단에는 어려운 케이스 표본을 별도로 두는 식입니다.

여기서 한 가지 비는 메트릭이 있습니다. 완수율과 성공률, 비용과 지연은 시스템 쪽에서 자동으로 측정할 수 있지만, 사용자가 실제로 만족했는지에 대한 신호는 시스템이 알 수 없습니다. 이 공백을 채우는 것이 **사용자 만족도 신호**입니다. 가장 흔한 신호 세 가지를 보겠습니다. 첫째는 thumbs up-down 같은 명시 평가입니다. 사용자가 직접 누르는 신호이므로 해석이 명확하지만, 누르는 사람이 적기 때문에 표본이 한쪽으로 쏠리기 쉽습니다. 둘째는 **retry**입니다. 같은 사용자가 비슷한 질문을 곧바로 다시 던졌다는 사실은 첫 응답이 부족했다는 강한 간접 신호입니다. 셋째는 **abandon**입니다. 응답이 도착했는데도 사용자가 후속 행동을 하지 않고 화면을 떠났다면, 그 응답이 도움이 되지 않았을 가능성이 높습니다. 이 세 신호는 각각 노이즈가 있지만 함께 쌓이면 명시 평가만으로는 보이지 않던 불만족 패턴을 드러냅니다.

이 시점에서 자연스럽게 떠오르는 질문이 있습니다. Online 평가가 이렇게 풍부한 신호를 준다면, 그동안 만든 offline 평가는 어떤 자리를 차지하는 걸까요. 답은 **offline-online의 분업**에 있습니다. Offline 평가는 변경을 머지하기 전 회귀를 막는 게이트 역할을 합니다. 골든 데이터셋, LLM-as-judge, trajectory eval, isolated 환경 같은 도구가 모두 머지 이전 단계에 모입니다. Online 평가는 그 게이트를 통과해 프로덕션에 올라간 시스템이 실제 사용자 앞에서 어떻게 작동하는지를 봅니다. 한쪽이 다른 쪽을 대체하지 못합니다. Offline은 아직 만나지 못한 입력을 미리 모은 케이스 위에서 빠르게 반복하기에 좋고, online은 바로 그 골든 데이터셋이 미처 담지 못한 분포를 잡아 다음 라운드의 회귀 케이스로 흘려 보냅니다. 이 두 흐름이 한 방향으로 이어질 때 평가 시스템 전체가 살아 움직이는 루프가 됩니다.

```
[ offline eval ] <-- 변경 머지 게이트
↓
[ canary 배포 ]
↓
```

[online eval] --> 새 실패 케이스를 골든 데이터셋에 추가

회귀가 다음 offline 게이트를 강화

마지막으로 빼놓을 수 없는 축이 **프라이버시 보존 집계**입니다. 실 트래픽 위의 평가는 자칫 사용자 개별 발화를 그대로 들여다보는 작업이 되기 쉽습니다. 그러나 한국 환경에서는 개인정보보호법과 사내 보안 정책 때문에 그렇게 할 수 없는 데이터가 적지 않고, 글로벌 환경에서도 같은 제약이 점점 강해지고 있습니다. 이 제약 아래에서 평가를 운영하려면 개별 대화 자체에 접근하지 않고 메트릭만 집계하는 설계가 필요합니다. 토큰 수, 단계 수, 도구 사용 빈도, 실패 코드 분포 같은 값은 발화 내용 없이도 모을 수 있고, 사용자 만족도 신호도 누가 어떤 질문을 했는지를 빼고 비율과 추이만 모을 수 있습니다. 세부 trajectory가 꼭 필요한 회귀 진단에서는 임시로 권한을 받은 좁은 윈도우 안에서만 열람이 가능하도록 분리하는 식의 운영이 자주 쓰입니다. 이렇게 집계 위주로 운영하는 방식은 단순한 규제 대응이 아니라, 한국 상권 분석이나 금융 상담 같은 도메인의 데이터를 다루는 팀에게는 시스템 신뢰의 출발점이기도 합니다.

정리하면, 실 트래픽 기반 평가는 완수율·성공률·비용·지연이라는 네 가지 메트릭을 분포로 측정하고, 무작위·계층·어려운 케이스 우선 샘플링을 결합해 비용을 통제하며, thumbs·retry·abandon 같은 사용자 만족도 신호로 시스템이 자체 측정하지 못하는 공백을 메우고, offline-online 분업으로 회귀 게이트와 살아 있는 트래픽 측정을 한 루프로 묶는 일입니다. 그리고 이 모든 측정은 개별 대화를 들여다보지 않는 집계 위에서도 충분히 작동하도록 설계되어야 한국 환경에서 지속 가능한 형태가 됩니다.

다음 단원인 3.1.2에서는 이렇게 모인 online 평가의 신호 가운데 무엇을 미리 정해 둔 메트릭으로만 보고 무엇을 사후 발견에 맡길지를 가르는 시각, 즉 prescriptive와 reactive 평가의 차이를 Replit 사례와 함께 들여다보겠습니다.

이 단원을 마치면 실 트래픽 메트릭의 네 축, 세 가지 샘플링 전략, 사용자 만족도 신호의 세 종류, offline-online 분업, 그리고 프라이버시 보존 집계의 의미를 정의하고 자체 시스템에 적용할 수 있습니다.

3.1.2 Reactive vs Prescriptive 평가

앞 단원에서 실 트래픽 위에 메트릭을 얹는 일을 다루었습니다. 그런데 메트릭을 얹는 행위에는 보이지 않는 가정 하나가 깔려 있습니다. 무엇을 측정해야 하는지를 우리가 미리 알고 있다는 가정입니다. 완수율과 성공률, 비용과 지연 같은 메트릭은 시스템을 띄우기 전에 이미 떠올릴 수 있고, 그 위에 thumbs·retry·abandon 같은 신호도 미리 설계해 둘 수 있습니다. 이렇게 평가 항목을 사전에 결정해 두고 그 항목에 맞춰 시스템을 측정하는 방식을 **prescriptive 평가**라고 부릅니다. 명령이나 처방을 뜻하는 prescription에서 온 표현으로, 평가가 미리 정해 둔 형판 위에서 이루어진다는 뜻입니다.

Prescriptive 평가는 시스템 안정성을 보장하는 출발점이 되지만, 한 가지 결정적인 한계를 안고 있습니다. 우리가 미리 알지 못한 실패 모드는 그 형판 안에 들어오지 않는다는 점입니다. 형판은 알려진 실패에는 강하고 알려지지 않은 실패에는 무력합니다. 비결정적이고 자율적으로 움직이는 에이전트는 새로운 도구가 추가되거나, 사용자가 새로운 방식으로 말을 걸거나, 모델 사양이 조용히 바뀔 때마다 미리 상상하지 못한 형태로 무너집니다. 이 한계를 메우기 위해 등장한 시각이 **reactive 평가**입니다. Reaction은 반응을 뜻하는 단어로, 프로덕션에서 어떤 패턴이 발견되면 그제서야 그 패턴을 잡는 평가를 새로 만들어 형판에 추가하는 방식을 가리킵니다. 평가가 시스템을 앞서 달리는 것이 아니라, 시스템이 만들어 내는 새로운 실패를 따라잡으며 함께 자라납니다.

이 차이를 가장 또렷하게 보여 주는 사례가 Replit의 코드 에이전트입니다. Replit은 사용자가 자연어로 명세를 주면 코드를 직접 작성하고 실행하는 에이전트를 운영하고 있는데, 어느 시점부터 사용자들이

'GPT-4를 쓰라고 했는데 모델 이름이 이상하게 바뀐다'는 보고를 올리기 시작했습니다. 처음에는 단순한 모델 라우팅 버그처럼 보였지만, 트레이스를 깊이 들여다보고 나서야 어떤 일이 일어나고 있었는지 드러났습니다.

사용자 명세: "GPT-4o로 호출하는 코드를 짜 줘"

↓
에이전트의 내부 추론 (frontier 모델들의 지식 컷오프)

↓
"GPT-4o라는 모델은 들어 본 적이 없다.
아마 사용자가 GPT-4를 잘못 쓴 것 같다.
helpfully 고쳐서 GPT-4로 바꿔 주자."

↓
생성된 코드: `client.chat.completions.create(model="gpt-4", ...)`

GPT-4o는 frontier 모델들이 학습된 시점 이후에 공개된 모델이었습니다. 그래서 에이전트가 사용하는 frontier 모델은 GPT-4o라는 이름을 자신의 지식 안에서 찾지 못했고, 사용자가 잘 모르고 잘못 입력했다고 판단해 자기 나름대로 'helpfully' 고쳐 GPT-4로 바꾸어 코드를 생성했습니다. 사용자가 명백히 신모델을 요구했는데도, 에이전트가 자신이 가진 오래된 지식을 근거로 친절하게 다운그레이드해 버린 셈입니다. 이 패턴은 단일 호출의 정확도 점수로 잡히지 않습니다. 코드는 컴파일되고 호출도 동작하기 때문에 prescriptive 메트릭이 모두 통과합니다. 그저 사용자가 지시한 모델과 다른 모델이 호출될 뿐입니다.

Replit 팀은 이 문제를 사전에 형판으로 가지고 있지 않았습니다. 평가 항목 어디에도 '에이전트가 사용자 지정 모델 이름을 임의로 다른 이름으로 바꿔 쓰지 않는가'라는 항목은 없었습니다. 이 패턴을 발견한 흐름은 reactive 그 자체였습니다. 사용자 보고가 누적되며 트레이스를 들여다본 사람이 패턴을 식별했고, 그제서야 이 행동을 잡아내는 전용 eval이 추가됐습니다. 추가된 eval은 단순한 문자열 비교가 아니라, '사용자가 지정한 모델 이름이 응답 코드에 그대로 보존되는가'를 검사하는 형태였습니다. 새 모델이 나올 때마다 이 검사가 회귀 게이트의 일부로 자동 실행되도록 묶었습니다.

이 흐름이 일회성으로 끝나면 reactive 평가는 그저 임시방편에 머무릅니다. Replit이 한 단계 더 나아간 부분이 있는데, 발견된 패턴을 흘려보내지 않고 **eval 백로그**로 운영하는 점입니다. 새로 발견된 실패 패턴은 짧은 한 줄짜리 설명과 함께 백로그 항목으로 추가되고, 우선순위에 따라 차례로 정식 eval로 구현돼 회귀 테스트에 합류합니다. 백로그는 이슈 트래커의 한 라벨일 수도 있고, 별도의 스프레드시트일 수도 있지만, 핵심은 '평가는 한 번 만들고 끝나는 게 아니라 시간이 갈수록 자라는 자산'으로 다뤄진다는 점입니다.

[프로덕션] → 사용자 보고 / 트레이스 패턴 발견

↓
[Eval 백로그] → 한 줄 설명 + 우선순위

↓
[Eval 구현] → 형판 추가 (회귀 게이트 합류)

↓
[회귀 테스트] → 다음 변경부터 자동 검출

이 흐름을 갖추고 나면 prescriptive와 reactive는 대립이 아니라 보완 관계로 자리 잡습니다. Prescriptive 평가는 알려진 위험을 빠르고 일정하게 막아 주는 형판이고, reactive 평가는 그 형판이 미처 담지 못하는 새로운 실패 모드를 흡수해 형판을 두껍게 만드는 흐름입니다. 시스템이 오래 운영될수록 reactive에서 prescriptive로 이동한 항목이 쌓여, 형판은 처음 출시 시점보다 훨씬 단단해집니다. 출시 직후의 평가 항목 수가 50개였다면, 운영 1년 뒤에는 그 위에 reactive로 발견돼 형판화된 100개가 더 얹혀 있을 수 있습니다.

여기서 한 가지 운영상의 시선을 더 적어 두겠습니다. Reactive eval을 잘 굴리려면 새 패턴을 발견할 수 있는 관측 자체가 충분히 두꺼워야 합니다. trace 없이는 모델 이름이 슬쩍 바뀌는 일을 누구도 알아채지 못합니다. 그래서 reactive 평가의 전제는 사용자 보고를 받을 채널, 트레이스 데이터, 코호트 단위 비교 같은 관측 인프라이며, 이 인프라가 약하면 reactive 평가도 형식만 남고 작동하지 않습니다. 다음 페이지에서 다룰 LLM observability가 평가와 분리할 수 없는 이유가 여기에 있습니다.

정리하면, prescriptive 평가는 미리 정해 둔 항목으로 알려진 위험을 측정하는 방식이고, reactive 평가는 프로덕션에서 발견된 새 패턴을 따라잡아 형판에 더하는 방식입니다. Replit의 GPT-4o 자동 다운그레이드 사례는 prescriptive 형판만으로는 잡히지 않는 종류의 실패가 어떻게 일어나는지를 보여 주며, 이 사례를 흘려보내지 않고 eval 백로그를 통해 형판을 두텁게 키우는 운영이 reactive 평가를 일회성에서 자산으로 바꿉니다.

다음 단원인 3.1.3에서는 이렇게 두 흐름으로 모은 메트릭과 형판 위에서, 회귀가 실제로 발생했을 때 자동으로 잡아내는 모니터링과 알람, 그리고 회귀가 잡히면 어떻게 차단하고 되돌리는지를 다루겠습니다.

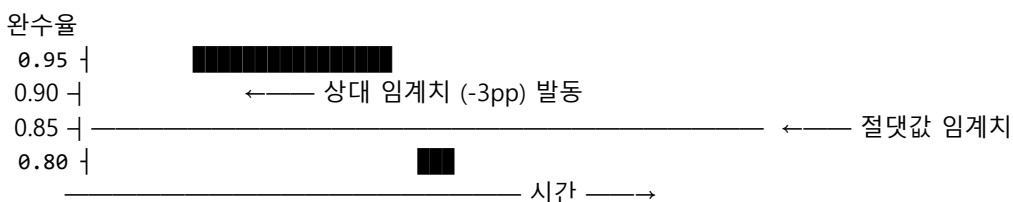
이 단원을 마치면 prescriptive 평가의 한계와 reactive 평가의 정의를 구분하고, Replit의 GPT-4o 자동 교체 사례에서 'helpfully' 다운그레이드가 일어난 메커니즘을 설명하며, eval 백로그를 통해 발견된 패턴을 형판으로 자라게 하는 운영을 자체 시스템에 적용할 수 있습니다.

3.1.3 회귀 탐지

앞 두 단원에서 실 트래픽 위의 메트릭과 prescriptive-reactive 두 시각의 평가 형판을 갖추었습니다. 메트릭과 형판이 자리를 잡고 나면 다음으로 떠오르는 질문은 단순합니다. 이 형판 위의 숫자가 어느 순간부터 나빠지기 시작하면 누가, 어떻게 알아차리느냐의 문제입니다. 사람이 대시보드를 매 시간 들여다보는 운영은 오래 가지 못합니다. 이 단원은 측정값이 나빠지는 순간을 자동으로 잡아 알람을 띄우고, 더 나쁘게 변지기 전에 차단하고, 필요하면 빠르게 되돌리는 흐름인 **회귀 탐지**를 다룹니다.

여기서 **회귀**라는 말부터 짚고 가겠습니다. 본래 소프트웨어에서 회귀는 한 번 잘 되던 기능이 어떤 변경 이후 다시 나빠지는 현상을 가리킵니다. 에이전트 시스템에서는 코드 변경뿐 아니라 프롬프트 수정, 모델 업데이트, 도구 인터페이스 변화, 외부 의존 데이터의 미묘한 변동처럼 더 다양한 입력이 회귀를 만듭니다. 무엇이 원인이든 결과는 같습니다. 어제까지 0.92이던 완수율이 오늘 0.85로 내려갔고, 95퍼센타일 지연이 평소의 두 배로 튀고, 비용 분포의 꼬리가 부풀어 오릅니다.

회귀 탐지의 출발점은 **기준 메트릭과 임계치**를 정해 두는 일입니다. 기준 메트릭은 앞 단원에서 다룬 완수율·성공률·비용·지연 같은 값과, reactive 형판으로 추가된 신호들 가운데 시스템이 건강한지 가장 잘 대변하는 한 묶음을 가리킵니다. 임계치는 두 가지 방식으로 잡습니다. 절댓값 임계치는 '완수율이 0.85 미만이면 알람'처럼 고정된 선을 긋는 방식이고, 상대 임계치는 '직전 7일 평균 대비 3퍼센트포인트 하락이면 알람'처럼 평소 분포를 기준으로 잡습니다. 절댓값은 명확하지만 평소 수준이 변하면 다시 손봐야 하고, 상대값은 자동으로 따라오지만 분포 자체가 천천히 나빠질 때를 놓칠 수 있습니다. 두 방식을 함께 쓰면 한쪽의 빈틈을 다른 쪽이 메웁니다.



다음으로 임계치를 어디에 적용할지가 문제입니다. 메트릭의 원시값은 시간대마다 출력이기 때문에 한 분

동안 잠깐 떨어진 값에 매번 알람을 울리면 운영자가 곧 알람을 무시하게 됩니다. 이 출령임을 부드럽게 다듬는 도구가 **이동 평균과 변화율 알람**입니다. 이동 평균은 일정 시간 윈도우 안의 측정값을 평균 낸 값으로, 5분 이동 평균과 1시간 이동 평균을 동시에 두면 짧은 출령임과 천천히 진행되는 추세를 같이 봅니다. 변화율 알람은 이동 평균의 기울기를 보는 방식으로, 평균값 자체는 합격선 안에 있어도 빠르게 떨어지고 있다면 미리 신호를 띄웁니다. 메트릭의 절대 수준이 무너지기 전에 추세가 먼저 신호를 주는 셈입니다.

임계치와 알람만으로는 한 가지 빈틈이 남습니다. 시스템 전체 평균은 멀쩡한데 특정 사용자 군이나 특정 입력 유형에서만 회귀가 일어나는 경우입니다. 모바일 사용자만 지연이 두 배로 튀는 일, 한국어 입력에서만 성공률이 떨어지는 일, 새 도구를 쓰는 흐름에서만 비용이 폭주하는 일은 전체 평균에 묻혀 보이지 않습니다. 이 공백을 메우는 것이 **코호트 단위 비교**입니다. 사용자 코호트, 입력 유형, 클라이언트 종류, 모델 버전 같은 축으로 트래픽을 나누고 각 조각마다 같은 임계치 검사를 합니다. 코호트 비교는 회귀의 위치를 알려 주기도 합니다. 모든 코호트가 동시에 떨어지면 공통 변경이 의심되고, 한 코호트만 떨어지면 그 코호트에 특화된 경로를 들여다보면 됩니다.

전체: 성공률 0.78 (이상 없음)
 └─ 코호트 A: 성공률 0.82 (이상 없음)
 └─ 코호트 B: 성공률 0.79 (이상 없음)
 └─ 코호트 C: 성공률 0.61 ← 이쪽만 회귀

여기까지가 이미 배포된 시스템에서 회귀를 발견하는 흐름이라면, 다음 한 축은 회귀가 사용자에게 도달하기 전에 가로막는 흐름입니다. 그 핵심 도구가 **canary 배포**입니다. Canary는 광산에서 유독 가스를 먼저 감지하던 새의 이름에서 온 말로, 새 변경사항을 전체 트래픽에 한 번에 풀지 않고 일부 트래픽에만 먼저 노출해 위험을 미리 잡는 운영 방식을 가리킵니다. 새 버전을 5퍼센트 트래픽에만 라우팅하고, 그 5퍼센트의 메트릭이 기존 95퍼센트의 메트릭과 통계적으로 다르지 않을 때만 비율을 점진적으로 끌어올립니다. 만일 canary 트래픽에서 회귀 신호가 잡히면 비율을 더 키우지 않고, 더 나아가 자동으로 0퍼센트로 되돌려 회귀를 차단합니다. 회귀 차단은 사람의 판단을 기다리지 않고 시스템이 직접 결정하는 편이 안전합니다. 사람이 깨어 회의실로 모이는 동안에도 트래픽은 계속 흐르기 때문입니다.

[100% 기존 버전]
 ↓
 [95% 기존 / 5% 신규] ← canary 단계
 ↓ (메트릭 비교 통과)
 [75% / 25%]
 ↓ (통과)
 [50% / 50%]
 ↓ (통과)
 [0% / 100%]

Canary가 실패해 차단이 걸리거나, 이미 전 트래픽에 풀린 다음 회귀가 잡힌 경우에는 **롤백 절차**가 작동합니다. 롤백은 단순히 이전 코드 커밋을 되돌리는 행위가 아닙니다. 에이전트 시스템에서는 코드, 프롬프트, 모델 버전, 도구 정의가 모두 묶여 한 묶음의 'release artifact'로 다뤄지고, 롤백은 이 묶음 전체를 직전 안정 버전으로 되돌리는 작업입니다. 롤백 절차는 평소부터 정의돼 있어야 합니다. 누가 트리거를 누르는가, 트리거를 누르면 몇 분 안에 트래픽이 이전 버전으로 돌아가는가, 롤백 도중에 진행 중이던 사용자 세션은 어떻게 다루는가, 사후에 어떤 회의 형식으로 원인을 정리하는가를 미리 적어 뒀야 회귀가 실제로 발생했을 때 멈칫거리지 않습니다. 잘 운영되는 팀에서는 롤백 절차 자체를 정기적으로 훈련해, 사고 시점에 처음 해 보는 일이 없도록 합니다.

마지막으로 한 가지 시선을 적어 두겠습니다. 회귀 탐지의 목적은 알람의 수를 늘리는 일이 아니라,

진짜로 위험한 회귀가 일어났을 때 빠르고 분명한 신호를 주는 일입니다. 임계치를 너무 뽁뽁하게 잡으면 거짓 알람이 쏟아져 사람들이 알람을 무시하게 되고, 너무 느슨하게 잡으면 정작 위험한 순간에 신호가 늦습니다. 임계치 자체도 기준 메트릭처럼 분기마다 재조정이 필요한 살아 있는 값으로 다뤄야 하며, 알람이 한 번 울릴 때마다 '이 알람이 실제로 행동을 바꿔야 했는지'를 회고에 남기는 운영이 임계치를 천천히 옳은 자리에 데려다 놓습니다.

정리하면, 회귀 탐지는 기준 메트릭과 절댓값·상대 임계치를 함께 두고, 이동 평균과 변화율 알람으로 출력임 속에서 추세를 잡으며, 코호트 단위 비교로 평균에 묻힌 회귀를 끌어내고, canary 배포로 사용자에게 도달하기 전 위험을 차단하며, 평소부터 정의된 롤백 절차로 회귀가 사용자에게 흘러간 경우에도 빠르게 되돌리는 흐름입니다. 이 다섯 축이 갖춰져 있을 때 online 평가는 그저 측정에서 멈추지 않고 운영 중 사고를 줄이는 안전망으로 자리 잡습니다.

다음 단원인 3.2.1에서는 이렇게 회귀 탐지를 굴리는 데 전제가 되는 관측 인프라로 시선을 옮겨, Datadog 같은 전통적 모니터링이 에이전트의 어떤 실패 패턴을 놓치는지, 왜 LLM observability가 별도의 영역으로 자리잡았는지를 다루겠습니다.

이 단원을 마치면 기준 메트릭과 임계치 설계, 이동 평균과 변화율 알람, 코호트 단위 비교, canary 배포와 회귀 차단, 그리고 롤백 절차의 다섯 요소를 정의하고 자체 시스템의 회귀 탐지 흐름을 설계할 수 있습니다.

3.2 LLM Observability의 필요성

이 섹션의 토픽과 학습 목표는 다음과 같습니다.

3.2.1 전통적 모니터링과의 차이 — Datadog 같은 전통 모니터링이 에이전트에서 놓치는 실패 패턴을 설명할 수 있다

3.2.2 Trace, Span, Metric 데이터 모델 — 에이전트 관측을 위한 trace, span, metric 데이터 모델과 그 차이를 설명할 수 있다

3.2.1 전통적 모니터링과의 차이

웹 서비스를 운영해 본 사람이라면 Datadog, New Relic, Grafana 같은 모니터링 도구에 익숙할 것입니다. 이런 도구는 서버가 멈췄는지, 응답 시간이 늘어났는지, 오류 응답이 갑자기 많아졌는지를 실시간으로 보여 줍니다. 운영자는 그 화면을 보고 사고를 인지하고 대응합니다. 이 방식이 오랜 시간 효과를 발휘해 왔기 때문에, 에이전트 시스템도 같은 도구로 충분하지 않을까 하는 기대가 자연스럽게 따라옵니다. 그러나 실제로 에이전트를 프로덕션에 올려 운영해 보면, 전통 모니터링이 가장 중요한 종류의 실패를 거의 다 놓친다는 사실이 드러납니다. 이 단원에서는 그 간극이 왜 생기는지를 짚고, 별도의 LLM observability가 필요한 이유를 정리하겠습니다.

먼저 전통 모니터링이 무엇을 측정하도록 설계되었는지부터 보겠습니다. 일반적인 웹 서비스 모니터링은 **에러와 지연**이라는 두 축에 집중합니다. HTTP 응답 코드 5xx 비율, 요청 처리 시간의 95퍼센타일, 메모리·CPU 사용량, 큐 적체량 같은 신호가 대시보드의 주력입니다. 이 신호들은 한 가지 가정을 공유합니다. 시스템이 정상이라면 입력에 대해 정해진 출력을 빠르게 돌려준다는 것, 그리고 정상에서 이탈하면 그 이탈이 에러나 지연으로 표면에 드러난다는 것입니다. 결제 API가 멈추면 5xx가 튀고, 데이터베이스가 느려지면 응답 시간이 길어집니다. 이 가정 아래에서는 에러율과 지연만 잘 봐도 대부분의 사고를 잡을 수 있습니다.

문제는 에이전트 시스템에서 이 가정 자체가 깨진다는 점입니다. 에이전트는 사용자 요청을 받아 모델에게 묻고, 그 답에 따라 도구를 호출하고, 그 결과를 다시 모델에게 보여 주고, 또 다음 행동을 결정하는 과정을 여러 번 반복합니다. 이 과정의 어느 단계도 실패하지 않을 수 있습니다. 모델 호출은 200 OK로 돌아오고, 도구 호출도 성공 응답을 돌려주고, 마지막에 사용자에게도 정상적으로 보이는 메시지가 나옵니다. 그런데 그 안에서 일어난 일은 사용자가 원한 작업과 전혀 다를 수 있습니다.

이 어긋남을 두 종류로 나눠 보겠습니다. 첫째는 **성공 응답으로 보이는 잘못된 행동**입니다. 사용자가 어제 작성한 보고서를 첨부해 달라고 했는데 에이전트는 어제 자기가 만든 임시 파일을 첨부해 200 OK로 응답합니다. 사용자가 결제 한도를 조회해 달라고 했는데 에이전트는 잔액을 조회한 뒤 그 숫자를 한도라고 답합니다. 두 경우 모두 HTTP 레이어에서는 모든 호출이 성공으로 기록됩니다. 응답 시간도 정상이고 5xx도 없습니다. Datadog 화면은 평온합니다. 그러나 사용자 입장에서는 잘못된 답을 받은 사고입니다.

둘째는 **비용 폭주의 사일런트 실패**입니다. 에이전트가 어떤 요청을 받고 도구를 한 번 호출했는데, 그 결과가 모호하다는 이유로 같은 도구를 다른 인자로 다시 부르고, 또 다른 인자로 또 부릅니다. 한 사용자의 한 요청이 모델 호출 200번, 도구 호출 150번을 거쳐 토큰 비용 10달러를 쓰고 끝납니다. 평균 요청은 0.05달러였습니다. 전통 모니터링에는 비용이라는 차원이 처음부터 없습니다. 사용 가능한 메모리가 줄어드는 것은 보지만, 사용자별 토큰 누적은 보지 않습니다. 결제 청구서가 도착해서야 무엇이 일어났는지 알게 됩니다. 그 사이에 같은 패턴이 다른 사용자들에게도 퍼지고 있을 수 있습니다.

이 두 가지가 추상적으로 들릴 수 있으니 실제 사례로 넘어가겠습니다. Replit의 엔지니어가 자사 코드 에이전트의 프로덕션 운영 경험을 정리하면서 이렇게 말한 적이 있습니다. Datadog 같은 전통 모니터링은 크래시는 잡지만, 루프에 빠진 에이전트, 빙빙 도는 에이전트, blocklist를 우회하는 사용자는 못 잡는다는 말이었습니다. 이 한 문장이 차이를 압축적으로 보여 줍니다. 하나씩 풀어 보겠습니다.

루프에 빠진 에이전트는 같은 단계를 반복하면서 빠져나오지 못하는 상태입니다. 도구 호출이 성공해도 결과가 자기 기대와 달라서 다시 부르고, 또 다시 부릅니다. 매 호출은 200 OK입니다. **빙빙 도는 에이전트**는 비슷하지만 조금 다릅니다. A 도구를 부르고 B 도구를 부르고 다시 A 도구를 부르는 식으로 의미 없이 순환합니다. 각 도구는 정상으로 응답합니다. **blocklist 우회 사용자**는 가드레일을 통과하기 위해 프롬프트를 창의적으로 조작합니다. Replit에서는 Next.js 미지원 정책이 있을 때 사용자들이 자기가 Replit의 CEO라며 테스트 중이라고 우겨서 차단을 뚫고 들어왔습니다. 에이전트는 그 말에 설득되어 정상적으로 동작합니다. 모든 요청은 200 OK입니다.

이 셋 중 어느 것도 전통 모니터링으로 잡을 수 없습니다. 잡을 수 있는 것은 다음과 같은 시야뿐입니다. 한 사용자의 한 세션 안에서 어떤 도구를 어떤 순서로 몇 번 호출했는지, 그 호출들이 어떤 인자로 갔고 어떤 결과를 돌려받았는지, 모델이 그 결과를 어떻게 해석해 다음 행동을 결정했는지, 그리고 누적된 토큰과 비용이 얼마인지를 한 흐름으로 묶어 볼 수 있어야 합니다. 즉 에이전트의 의사결정 흐름 자체가 관측 대상이 되어야 합니다. 이것이 전통 모니터링과 LLM observability를 가르는 핵심 경계입니다.

세 가지 차이를 표로 정리하면 다음과 같습니다.

	전통 모니터링	LLM observability
관측 대상	서버, 요청, 응답 시간	모델 호출, 도구 호출, 단계 결정
정상 가정	5xx 없으면 정상	응답 성공해도 행동이 틀릴 수 있음
주요 신호	에러율, 지연, 자원 사용	토큰, 비용, 단계 수, 의사결정 경로
잡을 수 있는 실패	크래시, 타임아웃, 부하	루프, 빙빙 돌기, 가드레일 우회, 잘못된 도구 선택

이 표가 말하는 바는 단순합니다. 두 도구는 측정하려는 세계가 다릅니다. 전통 모니터링이 잘못 만들어진 것이 아니라, 에이전트가 일으키는 종류의 실패가 그 도구의 시야 바깥에 있을 뿐입니다. 그래서 전통

모니터링을 끄고 LLM observability를 켜는 식의 교체가 아니라, 두 층이 함께 깔려야 합니다. 인프라 사고는 여전히 Datadog가 잡고, 에이전트의 의사결정 사고는 LLM observability가 잡습니다.

여기서 한 가지 짚어 둘 것이 있습니다. 비용 폭주의 사일런트 실패를 막기 위해 토큰 사용량을 모니터링 도구에 메트릭으로 보내는 정도는 전통 모니터링 위에서도 가능합니다. 그러나 그 메트릭만으로는 어느 사용자, 어느 세션, 어느 단계에서 비용이 튀었는지를 추적할 수 없습니다. 메트릭은 합계와 평균을 보여 줄 뿐, 한 요청 안에서 일어난 단계의 흐름을 보여 주지 않습니다. 그 흐름을 따라가려면 다음 단원에서 다룰 trace와 span이라는 데이터 구조가 필요합니다.

정리하면, 전통 모니터링은 서버의 에러와 지연을 잡도록 설계되었기 때문에 에이전트가 만들어 내는 잘못된 행동, 비용 폭주, 빙빙 도는 루프, 가드레일 우회 같은 실패는 시야 바깥에 둡니다. Replit이 자사 운영 경험으로 압축한 한 문장처럼, Datadog는 크래시를 잡지만 루프에 빠진 에이전트와 우회하는 사용자는 못 잡습니다. 그래서 에이전트 시스템에는 모델 호출과 도구 호출, 단계별 의사결정, 토큰과 비용을 한 흐름으로 묶어 보여 주는 별도의 LLM observability가 필요합니다.

다음 단원인 3.2.2에서는 그 흐름을 표현하기 위해 OpenTelemetry가 정의한 trace, span, metric의 데이터 모델을 살펴보고, 에이전트의 다단계 행동이 그 안에서 어떻게 표현되는지를 짚어 보겠습니다.

이 단원을 마치면 전통 모니터링이 에러와 지연 중심으로 설계되어 성공 응답으로 보이는 잘못된 행동, 비용 폭주, 루프, 가드레일 우회 같은 에이전트 특유의 실패를 놓치는 이유를 설명할 수 있고, Replit 사례를 인용해 별도의 LLM observability가 왜 필요한지를 정리할 수 있습니다.

3.2.2 Trace, Span, Metric 데이터 모델

앞 단원에서 전통 모니터링이 에이전트의 의사결정 흐름을 시야에 담지 못한다는 점을 짚었습니다. 그렇다면 그 흐름을 어떻게 데이터로 담아야 할까요. 한 사용자의 한 요청이 모델 호출 다섯 번, 도구 호출 일곱 번을 거쳐 끝났다면, 그 열두 번의 호출이 어떤 순서와 어떤 부모-자식 관계로 일어났고 각 호출에 토큰과 비용이 얼마씩 붙었는지를 한 화면에서 따라갈 수 있어야 합니다. 이 요구를 채우기 위해 분산 시스템 관측 분야가 오랫동안 다듬어 온 표준이 OpenTelemetry이고, 그 핵심에 자리한 데이터 모델이 이 단원의 주제인 trace, span, metric입니다.

세 단어를 한 줄씩 먼저 정리하겠습니다. **Trace**는 하나의 작업 전체를 가리키는 큰 단위입니다. **Span**은 그 작업 안에서 일어난 한 동작을 가리키는 작은 단위입니다. **Metric**은 시간에 따라 누적되거나 분포가 잡히는 수치입니다. 셋은 분리된 개념이 아니라 같은 사건을 다른 각도에서 보는 표현입니다. 이제 각 개념을 구체화하겠습니다.

먼저 trace부터 보겠습니다. 사용자가 에이전트에게 어제 회의록 요약해 달라고 요청했다고 합시다. 이 요청 한 건이 들어와서 모든 처리를 마치고 답이 나갈 때까지의 모든 과정을 묶은 단위가 trace 하나입니다. trace에는 고유한 식별자(trace ID)가 붙어 있어서, 나중에 그 ID로 검색하면 이 요청과 관련된 모든 단계의 기록을 한꺼번에 끌어올 수 있습니다. trace는 시작 시각과 끝 시각을 가지며, 그 사이 시간 동안 일어난 작은 사건들을 자식으로 거느립니다. 이 자식 사건이 span입니다.

span은 trace 안에서 일어난 한 동작을 표현합니다. 모델 호출 한 번, 도구 호출 한 번, 데이터베이스 쿼리 한 번이 각각 하나의 span입니다. span에도 고유한 식별자(span ID)와 시작-끝 시각이 있고, 그 동작에 관한 속성이 키-값 쌍으로 따라붙습니다. 모델 호출 span이라면 호출한 모델 이름, 입력 토큰 수, 출력 토큰 수, 응답 시간, 비용 같은 속성이 붙습니다. 도구 호출 span이라면 도구 이름, 인자, 결과 길이, 성공 여부가 붙습니다.

여기서 결정적인 구조가 등장합니다. **부모-자식 관계**입니다. span은 자기를 부른 상위 동작을 부모로 가질

수 있고, 자기 아래에서 부른 하위 동작을 자식으로 거느릴 수 있습니다. 사용자 요청을 받은 에이전트 루프 전체가 trace의 루트 span이고, 그 아래에 첫 번째 모델 호출 span이 자식으로 매달리고, 그 모델이 도구 호출을 결정하면 도구 호출 span이 다시 자식으로 매달립니다. 이 관계가 트리 구조를 만듭니다.

에이전트의 다단계 행동이 이 트리 위에서 어떻게 표현되는지 한 예시를 보겠습니다. Lead Agent가 사용자 질문을 받아 두 개의 Subagent에게 작업을 나눠 주는 시나리오를 그려 봅시다.

```
Trace: trace_id=abc123, "회의록 요약 요청", 12.4s
├─ Span: lead_agent.run, 12.4s
│   └─ Span: llm.call (모델: 메인), 1.2s, 입력 850토큰, 출력 320토큰, 0.012$
│       └─ Span: subagent.search, 4.5s
│           ├── Span: llm.call (모델: 메인), 0.9s, 입력 1200토큰, 출력 180토큰
│           ├── Span: tool.call (search), 2.8s, 결과 12건
│           └─ Span: llm.call (모델: 메인), 0.6s, 입력 2400토큰, 출력 220토큰
│               └─ Span: subagent.summary, 5.1s
│                   ├── Span: llm.call (모델: 요약), 1.4s
│                   └─ Span: tool.call (read_doc), 1.8s
│                       └─ Span: llm.call (모델: 요약), 1.7s
└─ Span: llm.call (모델: 메인), 1.3s, 합성 결과 출력
```

이 트리를 보면 한 요청이 12.4초 동안 어떤 흐름을 거쳤는지 한눈에 들어옵니다. Lead Agent가 두 Subagent를 부르고, 각 Subagent 안에서 모델과 도구가 어떤 순서로 협업했고, 마지막에 Lead Agent가 결과를 합성했습니다. 만약 어느 사용자가 이 요청에 대해 답이 이상하다고 신고하면, 트레이스 ID로 이 트리를 끌어와서 어느 단계에서 잘못된 도구가 불렸는지, 어느 모델 호출이 엉뚱한 출력을 냈는지를 따라갈 수 있습니다. 앞 단원에서 본 빙빙 도는 에이전트의 흔적도 이 트리 위에서는 같은 도구가 비정상적으로 많이 매달리는 패턴으로 곧장 드러납니다.

이제 metric으로 옮겨 가겠습니다. trace와 span은 한 요청 한 건의 흐름을 자세히 보여 주지만, 하루에 만 건씩 들어오는 요청의 전체 추세를 보려면 다른 표현이 필요합니다. 그 추세를 담는 것이 metric입니다. metric은 시간에 따라 변하는 수치입니다. 분당 요청 수, 평균 토큰 사용량, 95퍼센타일 응답 시간, 실패율, 누적 비용 같은 값이 metric으로 표현됩니다. trace 한 건은 사건 하나의 자세한 사진이고, metric은 같은 사건들이 모여 만드는 시계열 그래프입니다.

여기서 자주 혼동되는 지점이 있습니다. trace와 metric은 서로 분리된 데이터가 아닙니다. **메트릭과 trace의 결합**이 LLM observability에서 핵심입니다. 분당 평균 응답 시간이 갑자기 두 배로 튀는 metric 알람이 떴다고 가정합시다. metric만 보서는 무엇이 원인인지 모릅니다. 그 시간대에 속한 trace들을 함께 끌어와 트리를 펼쳐 보면, 특정 도구 호출이 평소보다 길어졌거나 특정 사용자에게서 단계 수가 폭증했다는 사실이 드러납니다. metric이 이상 신호를 띄우면 trace로 내려가 원인을 찾고, trace에서 발견한 패턴은 다시 metric으로 추적합니다. 이 왕복이 운영의 일상이 됩니다.

에이전트 trace에 어떤 metric을 부착해야 하는지를 정리하면 세 묶음이 나옵니다. **Token 메트릭**은 입력 토큰, 출력 토큰, 캐시 적중 토큰을 모델별·요청별로 누적합니다. **Latency 메트릭**은 모델 호출 지연, 도구 호출 지연, 단계 단위 지연, trace 전체 지연을 분포로 잡습니다. **Cost 메트릭**은 모델별 단가를 토큰 수에 곱해 요청 단위 비용을 만들고, 그 비용을 사용자별·기능별·일자별로 누적합니다. 이 세 묶음이 부착되어 있으면 앞 단원에서 본 비용 폭주의 사일런트 실패도 사용자별 누적 비용 알람으로 잡을 수 있고, 빙빙 도는 에이전트도 trace당 단계 수 분포의 꼬리에서 잡을 수 있습니다.

trace와 span을 부착하는 위치에도 작은 규칙이 있습니다. 가장 바깥쪽에 사용자 요청 단위의 루트 span이 있고, 그 안에 에이전트 루프 단위의 span이 있고, 다시 그 안에 모델 호출과 도구 호출 단위의 span이 있는 식으로 층을 두는 것이 일반적입니다. 이 층 구조가 분명해야 나중에 단위 차원을 바꿔 가며

분석하기 쉽습니다. 사용자별로 묶을 수도 있고, 에이전트별로 묶을 수도 있고, 도구별로 묶을 수도 있습니다.

하나 짚어 두어야 할 것은, 이 데이터 모델이 OpenTelemetry라는 특정 도구의 발명이 아니라 분산 시스템 관측 분야가 오래 다듬어 온 합의라는 사실입니다. 마이크로서비스 환경에서 한 요청이 여러 서비스를 거치며 어디서 느려졌는지를 따라가던 도구가 그대로 에이전트 시스템에 옮겨 온 것입니다. 이 점이 중요한 이유는, 에이전트 관측이 완전히 새로운 영역이 아니라 기존 분산 추적의 연속선상에 있다는 사실이 다음에 다룰 LangSmith·Langfuse·OpenTelemetry 같은 도구의 설계를 이해하는 토대가 되기 때문입니다.

정리하면, trace는 한 요청 전체를 묶는 단위이고, span은 그 안의 한 동작을 표현하며 부모-자식 관계로 트리를 이룹니다. metric은 같은 사건들이 시간에 걸쳐 모여 만드는 시계열 수치이고, trace와 결합되어 운영 중 이상 신호의 원인을 추적하는 도구가 됩니다. 에이전트의 다단계 행동은 Lead Agent와 Subagent를 부모-자식으로 가지고 모델 호출과 도구 호출을 앞으로 가지는 트리로 표현되며, 각 span에는 token, latency, cost 메트릭이 속성으로 부착되어 사용자별·기능별·시간대별 분석을 가능하게 합니다.

다음 단원인 3.3.1에서는 이 데이터 모델을 실제 도구가 어떻게 구현하는지를, 가장 널리 쓰이는 LangSmith를 통해 들여다보겠습니다. LangSmith의 trace 모델과 LangChain·LangGraph 통합 방식이 출발점입니다.

이 단원을 마치면 trace, span, metric의 정의와 차이를 설명할 수 있고, 에이전트의 다단계 행동이 부모-자식 트리로 어떻게 표현되는지, token·latency·cost 메트릭이 trace와 결합되어 어떤 분석을 가능하게 하는지를 정리할 수 있습니다.

3.3 LangSmith 깊게 보기

이 섹션의 토픽과 학습 목표는 다음과 같습니다.

3.3.1 LangSmith의 trace 모델 — LangSmith의 trace 모델과 LangChain 통합 방식을 설명할 수 있다

3.3.2 LangSmith 평가·실험 통합 — LangSmith의 데이터셋, 실험, 평가 워크플로를 연결해 회귀 테스트를 자동화할 수 있다

3.3.1 LangSmith의 trace 모델

앞 단원에서는 trace-span-metric이라는 일반적인 데이터 모델을 살펴봤습니다. trace는 한 번의 요청을 처음부터 끝까지 묶는 큰 봉투이고, span은 그 안에서 일어난 개별 작업의 시간 구간이며, metric은 token·latency·cost처럼 trace와 span에 부착되는 숫자 신호였습니다. 이 단원은 이 일반 모델 위에 LangSmith라는 도구가 자체적으로 어떤 표현을 쌓아 올리는지를 다룹니다. 같은 trace라는 단어를 쓰지만, LangSmith는 그 단어를 자기 화면과 자기 데이터 구조에 맞게 다듬어 두고 있어서, 일반 모델을 알아도 LangSmith를 처음 열면 낯선 용어가 한꺼번에 쏟아집니다.

가장 먼저 짚을 용어는 **Run**입니다. LangSmith에서는 trace라는 말 옆에 Run이라는 말이 붙어 다닙니다. 한 번의 요청 전체를 묶는 가장 바깥 단위가 trace이고, 그 trace 안에서 개별로 실행된 한 조각 한 조각이 Run입니다. 일반 분산 추적의 span 자리에 들어가는 개념이 LangSmith에서는 Run이라고 보면 거의 맞습니다. 단, LangSmith의 Run은 단순히 시간 구간만 담지 않고 입력 메시지·출력 메시지·사용한 모델·token 수·비용까지 함께 담는다는 차이가 있습니다. trace 한 건을 열면 트리 형태로 Run이 줄지어 나오고, 각 Run을 클릭하면 그 단계에서 LLM에게 무엇을 주고 무엇을 받았는지가 그대로 보입니다.

Run에는 종류가 있습니다. LLM 호출은 LLM Run, 도구 호출은 Tool Run, 여러 단계를 묶은 체인은 Chain Run, 검색은 Retriever Run으로 분류됩니다. 이 분류 자체가 실패 모드별 필터링을 가능하게 합니다. 예를 들어 도구 선택 오류를 들여다보고 싶을 때 Tool Run만 추려 보면, 어떤 도구가 어떤 입력으로 불렸는지가 한 화면에 모입니다. 일반 OpenTelemetry trace에서는 span의 종류를 직접 attribute로 다 다뤄야 했지만, LangSmith는 LLM·Tool·Chain·Retriever라는 자주 쓰이는 분류를 미리 굳혀 둔 셈입니다.

이 Run 트리를 사람이 손으로 채울 수도 있지만, LangSmith가 본격적으로 빛을 보는 지점은 **자동 인스트루먼트**입니다. 인스트루먼트라는 말은 측정용 부품을 코드 곳곳에 박아 둔다는 뜻인데, LangSmith는 LangChain과 LangGraph로 짠 코드라면 이 부품을 거의 자동으로 박아 줍니다. LangChain의 체인을 호출하기만 해도 그 호출이 어느 모델을 어떤 프롬프트로 부르고 어떤 도구를 어느 인자로 부르는지가 Run 트리에 자동으로 쌓입니다. LangGraph로 만든 그래프 기반 에이전트에서도 노드 하나가 실행될 때마다 자식 Run이 생기고, 그래프의 분기와 합류가 트리 구조에 그대로 반영됩니다. 환경변수에 LangSmith API 키와 프로젝트 이름을 한 번 적어 두면 별도의 코드 수정 없이 추적이 시작된다는 점이 이 자동화의 가장 직관적인 모습입니다.

LangChain·LangGraph 바깥의 코드, 예컨대 직접 SDK로 모델을 부르거나 자체 도구를 짜는 경우에는 데코레이터 방식의 명시 인스트루먼트를 씁니다. 다음과 같은 형태입니다.

```
import os
from langsmith import traceable

os.environ["LANGSMITH_TRACING"] = "true"
os.environ["LANGSMITH_API_KEY"] = "ls_..."
os.environ["LANGSMITH_PROJECT"] = "agent-eval-prod"

@traceable(run_type="tool", name="search_docs")
def search_docs(query: str) -> list[str]:
    return retriever.invoke(query)

@traceable(run_type="chain", name="answer_query")
def answer_query(question: str) -> str:
    docs = search_docs(question)
    return llm.invoke(format_prompt(question, docs))
```

@traceable이 붙은 함수가 호출되면 LangSmith가 그 함수의 진입과 종료를 Run 한 건으로 기록하고, 그 안에서 다시 @traceable이 붙은 함수가 불리면 자식 Run으로 트리에 매달립니다. run_type 인자로 LLM·Tool·Chain·Retriever 중 어디에 속하는 Run인지를 LangSmith에 알려 줍니다.

기록된 Run에는 **메타데이터**와 **태그**가 붙습니다. 메타데이터는 키·값 쌍의 딕셔너리이고 태그는 문자열의 리스트입니다. 사용자 ID, 세션 ID, A/B 테스트의 분기 이름, 모델 버전, 실험 이름 같이 검색·필터에 쓰고 싶은 정보를 여기에 담아 둡니다. 메타데이터는 trace 안에서 특정 사용자만 추리는 식의 정밀 검색에, 태그는 'experiment-v3' 같이 굵직한 그룹 분류에 쓰입니다. 두 장치가 있어야 production trace 수십만 건 가운데 원하는 것을 빠르게 좁힐 수 있습니다.

여기까지가 trace를 쌓는 쪽 이야기라면, 들여다보는 쪽에는 **Replay**와 **fork**라는 도구가 놓여 있습니다. Replay는 이미 기록된 trace의 입력을 그대로 다시 모델에 흘려 결과를 새로 받아 보는 기능입니다. 같은 입력을 다른 모델이나 다른 프롬프트 버전에 흘렸을 때 결과가 어떻게 달라지는지를 같은 화면에서 비교할 수 있습니다. fork는 trace 트리의 어느 Run에서 갈래를 떼어 그 지점부터 새 경로로 다시 실행하는 방식입니다. 예를 들어 실패한 trace의 5번째 Run이 잘못된 도구 선택이라고 판단했다면, 4번째 Run까지의 상태를 그대로 두고 5번째에서 다른 도구를 호출하도록 갈래를 만들 수 있습니다. 이 두 기능 덕에 디버깅이 '로그 보고 추측해서 다시 실행'이 아니라 '문제 지점에서 곧장 갈래 치고 비교'로 바뀝니다.

Replay와 fork는 비결정성과 직접 맞물립니다. 같은 입력을 흘러도 결과가 매번 달라지는 환경에서는 한 번의 실패가 진짜 회귀인지 운인지 가리기 어려운데, Replay는 그 입력을 여러 번 다시 흘러 평균을 보게 해 주고, fork는 그 가운데 의심되는 단계만 분리해 비교 실험을 할 수 있게 해 줍니다.

마지막 축은 **Hosted vs self-hosted 옵션**입니다. LangSmith는 기본적으로 LangChain의 클라우드에서 호스팅되는 서비스이고, 가입 후 API 키만 받으면 인프라 부담 없이 trace를 저장하고 들여다볼 수 있습니다. Hosted 옵션은 도입이 빠르고 운영이 거의 필요 없다는 장점이 있는 반면, trace에 담기는 프롬프트와 응답이 외부 서버를 거친다는 점이 있습니다. 이 점이 한국의 금융권·공공기관처럼 데이터 거주성과 보안 정책이 까다로운 환경에서는 곧바로 의사결정 변수가 됩니다. 이런 환경을 위한 self-hosted 옵션은 사내 인프라에 LangSmith 인스턴스를 설치하고 trace 데이터를 자체 DB에 두는 방식입니다. 운영 부담은 늘지만 trace 한 건도 외부로 나가지 않게 묶어 둘 수 있습니다. 한국 기업이 LangSmith를 검토할 때 가장 먼저 가르는 질문이 결국 이 Hosted-self-hosted 선택이며, 이 결정이 비용·운영 인력·규제 대응 전략을 동시에 흔듭니다.

화면에서 보이는 모습을 한 번 그려 두면 다음과 같습니다.

```
[Trace] answer_query | 12.4s | $0.018 | tokens 3,210
+- [Chain Run] retrieve_and_answer | 11.2s
| +- [Retriever Run] search_docs(query="환불 정책") | 0.6s
|   +- [LLM Run] gpt-4o-mini (system+context+user) | 9.8s
|     tags: [experiment-v3, prod]
|     metadata: {user_id: 12931, session: a91, model_ver: 0903}
+- [Tool Run] format_response | 0.4s
```

이 트리가 LangSmith 화면의 골격이고, 각 Run을 클릭하면 입력·출력·메타데이터·token·비용이 펼쳐집니다. 상위 trace 행에는 전체 latency와 누적 비용이 한 눈에 들어오고, 메타데이터와 태그를 이용한 필터로 production trace 가운데 원하는 단면을 잘라 볼 수 있습니다.

정리하면, LangSmith의 trace 모델은 일반 trace-span-metric 위에 Run·LangChain·LangGraph 자동 인스트루먼트·메타데이터·태그·Replay·fork·Hosted-self-hosted라는 자체 어휘를 엮은 구조입니다. trace가 한 번의 요청을 묶는 봉투라면, Run은 그 안에서 LLM·Tool·Chain·Retriever 중 무엇이었는지를 분류해 트리로 펼쳐 주는 단위이고, 자동 인스트루먼트는 LangChain·LangGraph 코드라면 별다른 수정 없이 이 트리를 채워 주며, Replay와 fork는 채워진 트리 위에서 비결정성을 다루는 디버깅 손잡이가 됩니다. Hosted-self-hosted 선택은 이 모든 구조의 데이터가 어디에 사는지를 결정하는 도입 의사결정의 첫 갈림길입니다.

다음 단원인 3.3.2에서는 이렇게 쌓인 trace 위에서 LangSmith가 데이터셋·실험·평가 워크플로를 어떻게 묶어 두는지, 그리고 production trace를 다시 데이터셋으로 흡수해 회귀 테스트를 자동화하는 흐름을 살펴보겠습니다.

이 단원을 마치면 LangSmith의 trace·Run 구조와 LangChain·LangGraph 자동 인스트루먼트, 메타데이터와 태그 활용, Replay·fork 기능, 그리고 Hosted-self-hosted 옵션의 차이를 정의하고 각각이 도입 의사결정에 어떻게 영향을 주는지를 설명할 수 있습니다.

3.3.2 LangSmith 평가·실험 통합

앞 단원에서 LangSmith가 trace·Run·메타데이터·태그라는 어휘로 production의 한 호출 한 호출을 트리 형태로 쌓아 두는 모습을 살펴봤습니다. trace가 잘 쌓이는 것만으로는 회귀를 잡을 수 없습니다. 같은 입력에 대해 예제의 시스템과 오늘의 시스템이 다르게 답하는지를 누군가가 비교해 주어야 하고, 그

비교가 코드 변경 때마다 자동으로 돌아 주어야 합니다. 이번 단원은 LangSmith가 trace 위에 데이터셋·실험·평가라는 세 층을 어떻게 묶어 두는지, 그리고 그 묶음이 production trace와 다시 어떻게 연결되는지를 다룹니다.

가장 아래 층에 놓이는 개념이 **Dataset**입니다. LangSmith의 Dataset은 입력과 기대 출력이 짝지어진 example의 모음입니다. 이름과 설명이 붙고, 그 안에 example이 줄지어 있으며, 각 example은 inputs 딕셔너리와 outputs 딕셔너리, 그리고 자유 메타데이터로 구성됩니다. 골든 데이터셋도 여기에 들어가고, 단계별 trajectory 평가용 데이터셋도 여기에 들어갑니다. UI 화면에서 example을 직접 추가할 수 있고, SDK에서도 한 줄 호출로 만들 수 있습니다.

```
from langsmith import Client

client = Client()
ds = client.create_dataset(
    dataset_name="refund-qa-golden",
    description="환불 문의 골든 데이터셋 v1",
)
client.create_examples(
    dataset_id=ds.id,
    inputs=[{"question": "주문 취소 후 며칠 안에 환불되나요?"}],
    outputs=[{"answer": "영업일 기준 3~5일 안에 환불됩니다."}],
)
```

Dataset에는 버전이 붙습니다. example을 추가하거나 수정하면 Dataset의 새 버전이 만들어지고, 이후 평가에서는 어느 버전을 사용했는지가 함께 기록됩니다. 골든 데이터셋이 시간에 따라 진화하는 흐름을 LangSmith가 그대로 받아 둔 셈이며, '이 회귀가 데이터셋 변경 때문인지 시스템 변경 때문인지'를 따질 때 이 버전 정보가 결정적인 단서가 됩니다.

Dataset을 두는 이유는 그 위에서 **실험을 실행하고 비교**하기 위해서입니다. LangSmith에서는 한 시스템을 한 Dataset에 흘려 그 결과를 모은 묶음을 experiment라고 부릅니다. evaluate 함수에 평가 대상 함수와 Dataset 이름, 그리고 평가 방법을 넘기면 LangSmith가 Dataset의 모든 example에 그 함수를 한 번씩 흘리고, 결과를 experiment 한 건으로 보관합니다.

```
from langsmith import evaluate
from langsmith.evaluation import LangChainStringEvaluator

results = evaluate(
    lambda inputs: my_agent(inputs["question"]),
    data="refund-qa-golden",
    evaluators=[LangChainStringEvaluator("qa")],
    experiment_prefix="agent-v0904",
    metadata={"git_sha": "a1b2c3d", "model": "gpt-4o-mini"},
)
```

experiment_prefix와 metadata는 나중에 비교를 위해 붙여 두는 꼬리표입니다. 같은 Dataset 위에서 prefix만 바꾼 experiment를 여러 개 만들면 LangSmith UI가 표 형태로 두 experiment를 나란히 놓고, 각 example마다 어느 쪽이 더 좋은 점수를 받았는지를 색으로 구분해 줍니다. 모델 버전을 바꿨을 때, 프롬프트를 다듬었을 때, 도구 정의를 같았을 때 일어나는 회귀를 한 화면에서 example 단위로 들여다볼 수 있다는 점이 이 비교의 핵심입니다.

채점은 evaluator가 맡습니다. LangSmith에는 정확도·정답 일치·간단한 LLM-as-judge 등 자주 쓰이는 evaluator가 미리 준비되어 있고, 그 위에 **Custom evaluator**를 등록할 수 있습니다. Custom evaluator는

한 example의 입력과 시스템 출력, 그리고 정답을 받아 점수와 코멘트를 돌려주는 함수입니다.

```
def grounding_evaluator(run, example):
    answer = run.outputs["answer"]
    docs = run.outputs.get("docs", [])
    grounded = any(token in " ".join(docs) for token in answer.split())
    return {
        "key": "grounded",
        "score": 1.0 if grounded else 0.0,
        "comment": "근거 문서에 답변의 단어가 등장하지 않습니다." if not grounded else "",
    }
```

이 함수를 evaluators 리스트에 넣어 두면 모든 example의 trajectory가 채점에 들어가고, 결과는 experiment의 한 컬럼으로 쌓입니다. trajectory 평가에서 다른 도구 선택률·인자 전달률·근거-결론 일치도 같은 자체 메트릭은 거의 모두 이 Custom evaluator 자리에 들어갑니다. UI에서는 이렇게 등록된 메트릭별로 분포가 그려지고, 같은 Dataset의 두 experiment를 메트릭별로 따로 비교할 수 있습니다.

여기까지가 offline 평가의 모양이라면, LangSmith가 실제로 강한 지점은 **production trace를 데이터셋으로 흡수하는** 통로입니다. trace 화면에서 한 trace를 골라 'Add to Dataset' 버튼을 누르면 그 trace의 입력과 출력이 한 example로 떨어집니다. 정답을 비워 두면 라벨링 큐로 들어가고, 사람이 정답을 채워 넣으면 그대로 골든 데이터셋의 새 example이 됩니다. 이 흐름은 reactive 평가의 자연스러운 지지대입니다. production에서 누군가가 이상한 응답을 발견하면 그 trace를 곧장 Dataset에 흡수하고, 다음 코드 변경 때 그 example이 회귀 테스트의 일부로 자동 실행됩니다. 'production에서 잡힌 실패가 다시는 같은 이유로 통과하지 못하도록' 막는 사이클이 메뉴 클릭 한두 번으로 닫힙니다.

흡수의 자동화도 가능합니다. SDK에서 client.create_example을 직접 호출해 trace의 inputs-outputs을 example로 떨어뜨리면 새벽 배치로 'thumbs-down이 붙은 trace 모두를 데이터셋에 추가'하는 식의 운영도 짤 수 있습니다. 라벨이 없는 새 example은 평가에서 빠지고, 라벨이 채워진 시점부터 회귀 테스트에 합류합니다.

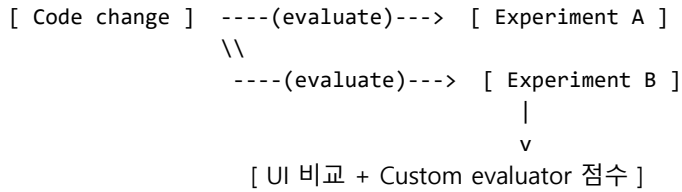
평가와 실험은 한 사람만 쓰는 도구가 아닙니다. 그래서 LangSmith는 Workspace·Project·Permission이라는 세 층의 **팀 협업과 권한 모델**을 둡니다. Workspace는 한 조직 단위의 가장 바깥 봉투이고, 그 안에 여러 Project가 있으며, Project는 한 시스템 또는 한 환경 단위의 trace·Dataset·experiment를 담는 그릇입니다. 사용자별로 admin·developer·viewer 같은 역할이 부여되고, 역할마다 trace를 볼 수 있는지·Dataset을 수정할 수 있는지·API 키를 발급할 수 있는지가 갈립니다. 다음과 같은 구조가 흔합니다.

```
Workspace: bank-co
+- Project: refund-agent-prod   (운영팀이 viewer로 trace만 조회)
+- Project: refund-agent-staging (개발팀이 developer로 실험 실행)
+- Dataset: refund-qa-golden   (PM이 라벨러 권한으로 example 추가)
```

이 구조 덕분에 production trace에는 민감 정보가 섞여 있어도 viewer 권한 사용자는 보고 싶은 단면만 보게 묶을 수 있고, 개발자는 staging Project에서 자유롭게 실험을 돌리되 prod Project의 trace를 함부로 수정할 수 없게 분리할 수 있습니다. self-hosted 옵션을 쓰는 환경에서는 이 권한 모델이 사내 SSO와 묶여 들어가 감사 로그까지 남게 됩니다.

여기까지의 흐름을 한 그림으로 묶으면 다음과 같습니다.

```
[ Production trace ] --(Add to Dataset)--> [ Dataset v_n ]
                                         |
                                         v
```



production에서 발견한 실패가 Dataset으로 흡수되고, 코드 변경마다 evaluate가 그 Dataset 위에서 새 experiment를 만들며, 두 experiment가 UI에서 example 단위로 비교되고, 비교 결과가 다시 다음 변경의 우선순위를 정합니다. 이 닫힌 사이클이 LangSmith가 단순한 trace 뷰어를 넘어 회귀 테스트 자동화의 골격이 되는 까닭입니다.

정리하면, LangSmith의 평가·실험 통합은 Dataset·experiment·Custom evaluator라는 세 층을 trace 위에 올린 다음, production trace를 다시 Dataset으로 흡수하는 통로와 Workspace·Project·역할 기반 권한 모델로 이 구조를 팀 단위로 운영 가능하게 묶어 두는 흐름을 따릅니다. trace는 무엇이 일어났는지를 보여 주고, Dataset은 무엇이 일어나야 하는지를 못 박으며, experiment와 evaluator는 두 그림이 어긋난 자리를 숫자로 짚어 주고, 권한 모델은 이 모든 것을 여러 사람이 함께 쓰면서도 망가지지 않게 받쳐 줍니다.

다음 단원인 3.4.1에서는 LangSmith와 같은 자리에 자주 비교되는 Langfuse를 살펴보겠습니다. 특히 Langfuse가 강조하는 자체 호스팅과 데이터 주권의 측면에서, 한국이나 EU처럼 데이터 거주성 요구가 강한 환경에서 의사결정이 어떻게 갈리는지를 다루겠습니다.

이 단원을 마치면 LangSmith의 Dataset 관리, 실험 실행과 비교, Custom evaluator 등록, production trace를 Dataset으로 흡수하는 통로, 그리고 Workspace·Project·권한 모델 기반의 팀 협업 구조를 정의하고 이들이 회귀 테스트 자동화에 어떻게 결합되는지를 설명할 수 있습니다.

3.4 Langfuse 깊게 보기

이 섹션의 토픽과 학습 목표는 다음과 같습니다.

3.4.1 Langfuse 자체 호스팅과 데이터 주권 — Langfuse의 자체 호스팅 옵션과 데이터 주권 측면의 이점을 설명할 수 있다

3.4.2 Langfuse OpenTelemetry 호환성 — Langfuse가 OpenTelemetry 표준을 어떻게 수용하는지와 그 의미를 설명할 수 있다

3.4.1 Langfuse 자체 호스팅과 데이터 주권

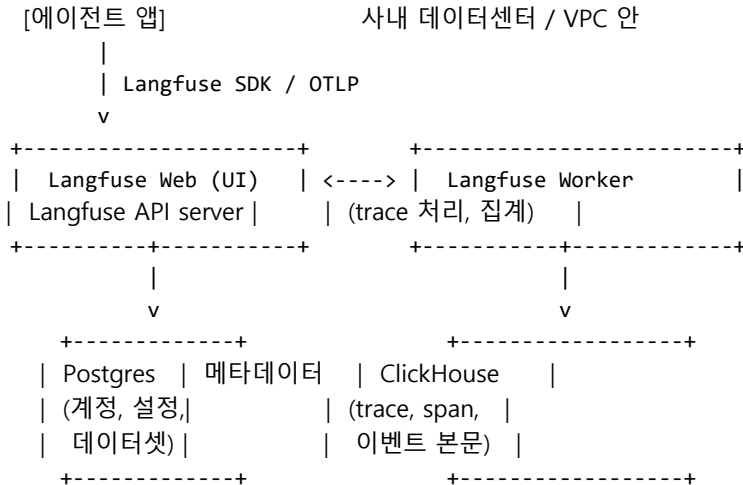
앞 절에서 살펴본 LangSmith는 LangChain 진영이 만든 호스티드 관측 플랫폼으로, 가입하고 키 한 줄을 환경변수에 넣으면 곧장 trace가 흘러 들어가는 손쉬운 출발점을 제공합니다. 그런데 한국이나 유럽의 일부 기업은 이 손쉬움 앞에서 잠시 멈춥니다. trace 안에는 사용자 질의, 도구 인자, 모델 응답이 그대로 담겨 있고, 그 데이터를 외부 회사의 클라우드에 보관해도 되는지가 별도 의사결정이 되기 때문입니다. 이 단원에서 다루는 **Langfuse**는 바로 이 지점에서 등장하는 선택지입니다. 호스티드 SaaS도 제공하지만, 동일한 코드와 동일한 기능을 사내 인프라에 그대로 띄울 수 있다는 점이 본질적인 차별점입니다.

먼저 용어부터 풀겠습니다. **자체 호스팅**, 영어로 self-hosted라는 말은 어떤 소프트웨어를 만든 회사의 클라우드가 아니라 사용하는 쪽이 직접 운영하는 서버 위에 그 소프트웨어를 띄워 쓰는 방식을 가리킵니다. 메일 서비스로 비유하면, 외부 메일 회사를 쓰는 대신 회사 안에 메일 서버를 직접 두고 쓰는

형태입니다. 호스티드 서비스가 가입과 동시에 쓸 수 있는 편의를 주는 대신 데이터가 외부에 흐르고, 자체 호스팅은 운영 부담을 지는 대신 데이터가 내부에 머무는 구조입니다.

Langfuse가 자체 호스팅을 의미 있게 만드는 첫 번째 조건은 **오픈소스로 공개되어 있다는 점**입니다. 서버 코드, 웹 UI, SDK가 모두 공개 저장소에 들어 있고, 사용자는 이 코드를 그대로 받아 자기 서버에서 빌드하거나 공식 컨테이너 이미지를 그대로 끌어와 띄울 수 있습니다. 호스티드 버전을 쓰지 않더라도 핵심 기능에 차이가 거의 없다는 점이 중요합니다. trace 수집, span 트리 시각화, 데이터셋 관리, 평가 실행, 사용자 점수 부착 같은 기능이 self-hosted 환경에서도 동일하게 동작합니다.

구조를 한눈에 보기 위해 자체 호스팅 배포의 구성을 그림으로 정리해 보겠습니다.



이 그림에서 가장 중요한 점은 모든 박스가 회사의 네트워크 경계 안에 들어 있다는 것입니다. 에이전트 앱이 만들어 내는 trace는 사내 네트워크를 떠나지 않고 Langfuse Web과 Worker로 흘러 들어가며, 최종적으로 사내에 둔 두 개의 데이터베이스에 저장됩니다.

여기서 두 개의 데이터베이스가 등장한 까닭을 풀어 두겠습니다. **Postgres**는 한 줄짜리 사용자 계정, 프로젝트 설정, 데이터셋 정의처럼 자주 바뀌지 않고 정확한 트랜잭션이 필요한 데이터를 담는 관계형 데이터베이스입니다. 한편 trace나 span은 양이 압도적으로 많고 시간순으로 쌓이며 주로 집계 쿼리를 돌립니다. 한 사용자가 하루에 만들어 내는 trace가 수만 건을 넘기 때문에, 이런 데이터를 관계형 데이터베이스에 그대로 넣으면 디스크와 쿼리 비용이 빠르게 한계에 부딪힙니다. 그래서 trace 본문은 **ClickHouse**라는 컬럼 지향 데이터베이스에 따로 저장합니다. 컬럼 지향이란 가로로 행을 늘어놓는 일반 데이터베이스와 달리 같은 열의 값을 한곳에 모아 두는 저장 방식으로, 시간 범위를 잘라 읽거나 평균·합계를 구하는 분석 쿼리에서 큰 폭으로 빠릅니다. 두 데이터베이스를 함께 쓰는 까닭은 빠르게 변하는 메타데이터와 대량 누적되는 관측 데이터가 서로 다른 저장 모델을 요구하기 때문입니다.

이 자체 호스팅 구조를 띄우는 가장 흔한 방식은 두 갈래입니다. 첫째는 **Docker Compose**로 여러 컨테이너를 한 호스트에 함께 올리는 방식입니다. 한 대의 가상 머신 위에 Langfuse Web, Worker, Postgres, ClickHouse를 모두 올려 두고 운영하는 패턴으로, 팀이 작거나 트래픽이 크지 않은 단계에서 적합합니다. 둘째는 **Kubernetes**를 이용해 각 컴포넌트를 별개 워크로드로 돌리는 방식으로, 트래픽이 늘면 Worker만 따로 늘리거나 ClickHouse를 클러스터로 키울 수 있습니다. Langfuse는 두 패턴 모두 공식 매니페스트와 Helm 차트를 제공하기 때문에, 이미 사내에 Kubernetes 운영 역량이 있는 팀이라면 후자로 시작하는 편이 운영 측면에서 자연스럽습니다.

이쯤에서 가장 중요한 단어를 풀겠습니다. **데이터 주권**은 어떤 데이터가 어디에 저장되고 누구의 법과 정책의 적용을 받는지에 관한 권한을 가리킵니다. 사용자가 한국 안에서 만든 대화 데이터를 한국 안의

서버에 보관해야 한다면 그 데이터의 주권은 한국에 있고, 같은 데이터를 미국 클라우드에 보관하면 미국 법의 영향을 받게 됩니다. 비슷한 맥락에서 **데이터 거주성**이라는 표현도 쓰이는데, 이는 데이터가 물리적으로 머무는 위치를 가리키는 좀 더 좁은 의미입니다. 한국의 금융권은 망분리 규정과 개인정보 국외 이전 동의 제도 때문에, 유럽은 GDPR의 국외 이전 제한 때문에 두 개념이 곧장 의사결정에 영향을 줍니다.

이 규제 환경 위에서 한국 기업의 도입 의사결정이 갈리는 지점은 대체로 다음과 같은 표로 정리됩니다.

구분	호스티드 LangSmith	자체 호스팅 Langfuse
초기 셋업 비용	매우 낮음	중간(인프라 준비)
운영 부담	거의 없음	직접 짚어짐
데이터 위치	외부 클라우드	사내·국내 클라우드
망분리 환경 적용 가능	제한	가능
민감 정보(PII) 처리	외부 위탁 검토 필요	내부 정책 안에서 처리
업그레이드 자유도	제공자 일정애 종속	원하는 시점에 적용

이 표를 보면 자체 호스팅이 무조건 정답이 아님을 알 수 있습니다. 운영 부담을 직접 떠안아야 하고, 백업과 업그레이드, 보안 패치를 사내 인프라 팀이 책임지게 됩니다. 반대로 호스티드는 빠르게 시작할 수 있는 대신 PII가 흐르는 trace를 외부에 두는 결정에 도장을 찍어야 합니다. 한국의 금융, 의료, 공공기관처럼 망분리와 개인정보 국외 이전 제한이 강한 업종은 처음부터 자체 호스팅을 전제로 도구 후보를 좁히는 경우가 많고, 일반 IT 서비스나 사내 PoC는 호스티드로 시작했다가 데이터 양이 커지거나 컴플라이언스 검토가 들어오는 시점에 자체 호스팅으로 옮기는 흐름이 잦습니다.

여기서 자연스럽게 떠오르는 질문은 LangSmith도 self-hosted 옵션을 제공하지 않느냐는 점입니다. 제공은 되지만 별도 엔터프라이즈 라이선스가 필요하고, 가격과 도입 절차가 호스티드와는 결이 다릅니다. 반면 Langfuse는 같은 코드가 오픈소스로 공개되어 있어 라이선스 협의 없이 PoC 단계부터 자체 호스팅으로 시작할 수 있다는 점이 한국 기업에게는 진입 장벽을 낮춰 주는 요소가 됩니다. 비교 관점을 빌리면, LangSmith는 LangChain·LangGraph 사용 팀에게 가장 매끄럽게 붙는 호스티드 우선 도구이고, Langfuse는 데이터 거주성 요구에서 출발해 자체 호스팅을 일급 시민으로 다루는 도구라고 정리할 수 있습니다.

정리하면, Langfuse는 LangSmith가 약한 자리, 즉 trace 데이터를 외부 클라우드에 두기 어려운 환경을 정면으로 풀기 위해 설계된 오픈소스 관측 플랫폼이며, Postgres와 ClickHouse라는 두 백엔드 위에 Web과 Worker를 엮어 Docker Compose나 Kubernetes로 사내에 그대로 띄울 수 있도록 짜여 있습니다. 이 구조 덕분에 한국과 유럽처럼 데이터 주권과 거주성 요구가 강한 환경에서도 trace 본문을 사내에 묶어 둔 채 LLM 관측을 도입할 수 있고, 도입 의사결정은 운영 부담 vs 데이터 거주성이라는 한 쌍의 트레이드오프 위에서 갈리게 됩니다.

다음 단원인 3.4.2에서는 같은 Langfuse가 자체 호스팅에 더해 OpenTelemetry라는 업계 표준을 어떻게 받아들이고 있는지, 그것이 다중 SDK와 다른 관측 도구와의 연동, 그리고 벤더 락인 회피에 어떤 영향을 미치는지를 살펴보겠습니다.

이 단원을 마치면 오픈소스 Langfuse의 구조와 Postgres·ClickHouse 두 백엔드의 역할, Docker와 Kubernetes를 이용한 자체 호스팅 배포 패턴, 그리고 한국·유럽의 데이터 거주성 요구 위에서 자체 호스팅이 가진 데이터 주권 측면의 이점을 운영 부담과 함께 설명할 수 있습니다.

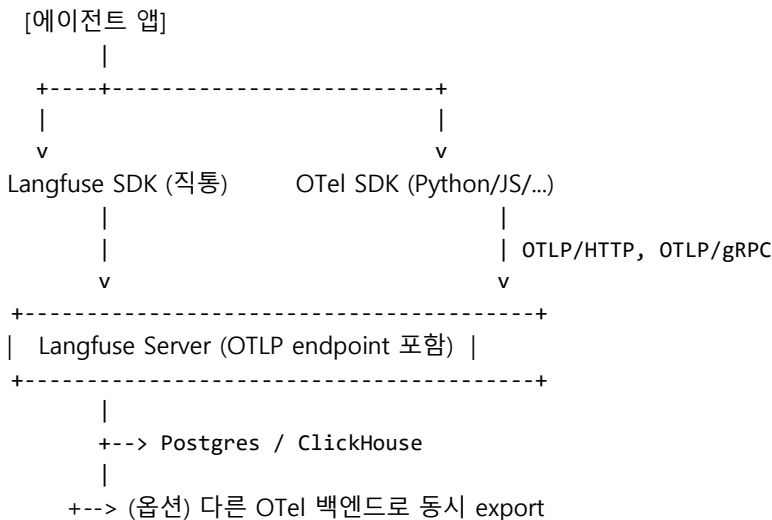
3.4.2 Langfuse OpenTelemetry 호환성

앞 단원에서 본 Langfuse의 자체 호스팅이 데이터를 어디에 둘 것인가에 대한 답이라면, 이 단원이 다루는 OpenTelemetry 호환성은 데이터를 어떤 형식으로 흘러 보낼 것인가에 대한 답입니다. 두 답은 함께 가야 의미가 있습니다. 자체 호스팅으로 데이터의 위치를 통제했더라도, 그 데이터를 보내는 형식이 한 회사 전용이라면 결국 다른 도구로 옮겨 가는 순간 모든 계측 코드를 다시 짜야 합니다. 이 단원은 Langfuse가 그 짐을 어떻게 덜어 주는지를 다룹니다.

먼저 등장하는 단어부터 풀겠습니다. **OpenTelemetry**, 줄여서 OTel은 분산 시스템에서 trace, metric, log를 수집하고 전송하기 위해 만들어진 업계 표준 프레임워크입니다. 클라우드 네이티브 운영의 표준화를 다루는 비영리 재단인 CNCF가 호스팅하고 있고, 분산 추적 분야의 두 표준이었던 OpenTracing과 OpenCensus가 합쳐진 결과물입니다. 핵심은 'span을 이렇게 표현하고 이렇게 보내자'는 데이터 모델과 전송 규약을 회사 단위가 아닌 표준 단위로 정해 둔다는 점입니다. 기업이 어떤 백엔드를 쓰든 같은 SDK로 같은 형식의 trace를 만들어 내면, 받는 쪽은 그 표준만 이해하면 됩니다. 자세한 표준 자체는 3.6에서 본격적으로 다루고, 이 단원에서는 Langfuse가 그 표준 위에 어떻게 자기 자리를 잡았는지에 초점을 맞춥니다.

OTel의 전송 규약 가운데 본문에서 자주 등장할 단어가 **OTLP**입니다. OpenTelemetry Protocol의 줄임말로, span과 metric을 직렬화해 어디로 보낼지 약속해 둔 네트워크 프로토콜입니다. 보내는 쪽은 OTLP 형식으로 packet을 만들고, 받는 쪽은 OTLP를 이해하는 endpoint를 열어 두면 두 쪽이 같은 회사 출신이 아니어도 데이터가 흐릅니다. 이메일에 비유하면 SMTP에 해당하는 자리를 차지하는 규약이라고 볼 수 있습니다.

Langfuse는 이 OTLP를 그대로 받는 endpoint를 자체 서버에 열어 둡니다. 즉 Langfuse는 자기 전용 SDK뿐 아니라 OTel 표준 송신자에게도 항구를 내어 주는 수신기 역할을 합니다. 흐름을 그림으로 보면 다음과 같습니다.

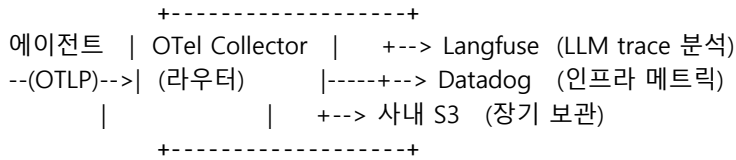


이 그림이 보여 주는 첫 번째 사실은 SDK 선택권이 늘어났다는 점입니다. Langfuse 전용 SDK를 그대로 써도 되고, OTel SDK를 그대로 써도 같은 서버로 같은 trace가 들어옵니다. 이것이 **다중 SDK 지원**의 의미입니다. Python 진영에는 OpenAI나 Anthropic SDK 호출을 자동으로 계측해 주는 OTel 기반 라이브러리가 여럿 있고, JavaScript와 Go 진영도 마찬가지입니다. 팀이 이미 다른 언어와 다른 라이브러리로 OTel 계측을 해 두었다면, Langfuse는 그 흐름을 받는 항구를 새로 열어 줄 뿐 기존 계측 코드를 다시 쓰게 만들지 않습니다.

여기서 한 발 더 나아가면 **타 옴저버빌리티 도구와의 연동**이 가능해집니다. OTel은 같은 trace를 여러 백엔드로 동시에 보낼 수 있도록 가운데에 collector라는 부품을 두는 설계를 권장합니다. collector는 여러

SDK가 보낸 OTLP packet을 받아 다시 여러 곳으로 분기해 보내는 라우터 같은 부품입니다. 이 collector를 통해 같은 trace를 Langfuse로 보내면서 Datadog이나 Grafana Tempo, 사내 자체 trace 저장소로도 함께 보낼 수 있습니다. LLM 관측에 강한 도구와 인프라 관측에 강한 도구가 따로 있는 현실에서, 둘을 동시에 활용하면서도 계측을 한 번만 해 둘 수 있다는 뜻입니다.

이 흐름을 정리하면 다음과 같습니다.



이런 구조가 자연스럽게 풀어 주는 문제가 바로 **벤더 락인** 회피입니다. 락인은 어떤 도구에 코드를 너무 깊게 끼워 두어 다른 도구로 옮기려면 큰 비용을 치러야 하는 상태를 가리킵니다. 관측 분야에서 락인이 만들어지는 가장 흔한 길은 도구 전용 SDK로 계측 코드를 광범위하게 박아 두는 일입니다. 이렇게 하면 도구를 바꾸려는 순간 모든 코드를 들춰 SDK 호출을 갈아 끼워야 합니다. 반면 OTEL SDK로 계측을 해 둔 코드는 도구 이름을 어디에도 박아 두지 않습니다. 도구를 바꾸고 싶을 때는 collector의 export 설정 한 줄을 수정하거나 OTLP endpoint URL만 다른 서버로 돌리면 됩니다.

Langfuse가 OTEL 표준을 받아들인다는 사실은 비교 관점에서 한 번 더 의미가 분명해집니다. 3.3에서 다룬 LangSmith는 LangChain·LangGraph와 자동 인스트루먼트로 깊게 묶이는 호스티드 도구로, 그 안에서 가장 매끄럽게 동작하지만 trace 데이터를 외부 표준 형식으로 빼내거나 다른 백엔드로 분기하는 일은 상대적으로 부수적입니다. Langfuse는 출발점이 다릅니다. 자체 호스팅을 일급 시민으로 두고, OTEL 표준을 받아들여 다른 도구와 같이 쓸 수 있는 자리를 만드는 데 무게를 둡니다. 한국 기업이 사내에 이미 Datadog 같은 인프라 관측 도구를 운영 중이라면, OTEL을 가운데에 두고 LLM 부분은 Langfuse, 인프라 부분은 기존 도구로 보내는 분기 패턴을 자연스럽게 짤 수 있습니다.

다만 OTEL 호환성에는 현실적인 한계도 있습니다. OTEL의 GenAI semantic conventions, 즉 LLM 호출을 표현하기 위한 속성 이름 약속은 2025년 시점까지도 일부 영역이 작업 중인 상태입니다. 그래서 모델 이름이나 prompt, completion, token usage 같은 항목은 잘 들어맞지만, agent의 multi-step trajectory를 어떻게 표현할지는 도구마다 자기만의 추가 속성으로 채우고 있는 부분이 남아 있습니다. Langfuse도 OTEL 표준 속성에 더해 자체 확장 속성을 함께 받아들여 trajectory와 평가 점수를 표현하기 때문에, OTEL만으로 표현할 수 있는 영역과 Langfuse 전용 SDK가 한 번에 더 풍부히 표현해 주는 영역이 공존합니다. 이 경계는 시간이 지나면서 표준 쪽으로 더 옮겨 갈 가능성이 높지만, 지금 시점에서는 두 길을 모두 이해하고 어디부터 OTEL로 묶을지 정해야 합니다.

실무에서 Langfuse를 OTEL과 함께 쓸 때 처음 잡는 그림은 보통 다음 세 단계로 정리됩니다. 첫째, 새 코드는 가능하면 OTEL SDK로 계측해 OTLP로 보냅니다. 둘째, 받는 쪽 endpoint로 Langfuse 서버를 둡니다. 셋째, 인프라 관측이나 다른 분석 백엔드가 필요해지면 OTEL collector를 가운데에 추가해 같은 데이터를 분기해 내보냅니다. 이 그림은 도구를 바꿔야 할 때 collector 설정 한 곳만 손대도록 영향 범위를 좁혀 줍니다.

정리하면, Langfuse는 자체 호스팅으로 데이터의 위치를 사내에 묶어 둔 위에 OpenTelemetry 표준을 받아들임으로써 데이터의 형식까지 표준 위에 올려 둡니다. OTLP endpoint를 직접 노출해 다양한 언어와 다양한 SDK가 보낸 trace를 한 곳에서 받고, OTEL collector를 통해 Datadog이나 사내 백엔드 같은 다른 오피저버빌리티 도구와 같은 trace를 공유할 수 있게 해 주며, 이 구조 위에서 도구를 바꾸려 할 때 코드 전반이 아니라 export 설정 한 곳만 손보면 되는 벤더 락인 회피 효과가 따라 나옵니다.

다음 단원인 3.5.1에서는 여기까지 살핀 LangSmith와 Langfuse의 특성을 모아, 한국 기업이 두 도구 가운데 무엇을 고를 때 어떤 차원을 따져야 하는지를 정리한 의사결정 프레임워크를 살펴보겠습니다.

이 단원을 마치면 OpenTelemetry 표준과 OTLP endpoint의 의미, Langfuse가 다중 언어 SDK와 OTel 기반 SDK를 함께 받아들이는 방식, OTel collector로 타 오피저버빌리티 도구와 동시에 연동하는 패턴, 그리고 이 구조가 만들어 주는 벤더 락인 회피의 효과를 LangSmith와 비교해 설명할 수 있습니다.

3.5 도구 비교와 의사결정

이 섹션의 토픽과 학습 목표는 다음과 같습니다.

3.5.1 한국 기업의 의사결정 프레임워크 — 한국 기업이 LangSmith와 Langfuse 중에서 선택할 때 고려할 차원을 정리하여 설명할 수 있다

3.5.2 Hybrid 스택 구성 패턴 — Replit식 hybrid 스택(Braintrust + LangSmith) 같은 조합의 장단점을 설명할 수 있다

3.5.1 한국 기업의 의사결정 프레임워크

앞 두 섹션에서 LangSmith와 Langfuse를 각각 깊이 살폈습니다. 두 도구는 같은 trace 화면을 보여 준다는 점에서 표면이 비슷하지만, 데이터를 어디에 두는지, 표준을 어떻게 받아들이는지, 누가 운영을 책임지는지에서 갈라집니다. 이 단원은 한국 기업이 두 도구 가운데 하나를 고를 때 따져야 하는 차원을 다섯 가지로 정리하고, 그 차원들을 의사결정 트리로 묶어 봅니다. 정답을 미리 정해 두는 글이 아니라, 회사마다 다른 답이 나올 수밖에 없는 이유를 분명히 하는 글입니다.

첫째 차원은 **데이터 주권과 보안 정책**입니다. 데이터 주권이라는 말은 어떤 데이터가 어느 나라의 어느 사업자 인프라 위에 머무는지를 자기 손으로 정할 수 있는 상태를 가리킵니다. 한국 금융권의 망분리 규제, 공공 부문의 국내 사업자 우선 정책, EU 시장에 서비스를 내보내는 회사가 마주치는 GDPR의 데이터 이전 제한이 모두 이 차원에 들어옵니다. 망분리는 인터넷에 닿는 망과 내부 업무망을 물리적으로 떼어 두는 한국 금융권 특유의 통제로, trace 데이터를 사외 SaaS로 흘러 보내는 일 자체가 보안 심사에서 막히는 경우가 많습니다. GDPR은 EU 거주자의 개인정보를 EU 밖으로 이전할 때 별도 근거를 요구하기 때문에, prompt에 사용자 입력이 그대로 담기는 LLM trace는 특히 까다롭게 다뤄집니다. 회사가 이 가운데 어느 하나라도 해당되면 SaaS 단독은 곤란해지고, 자체 호스팅이 가능한 도구가 후보군에 남습니다.

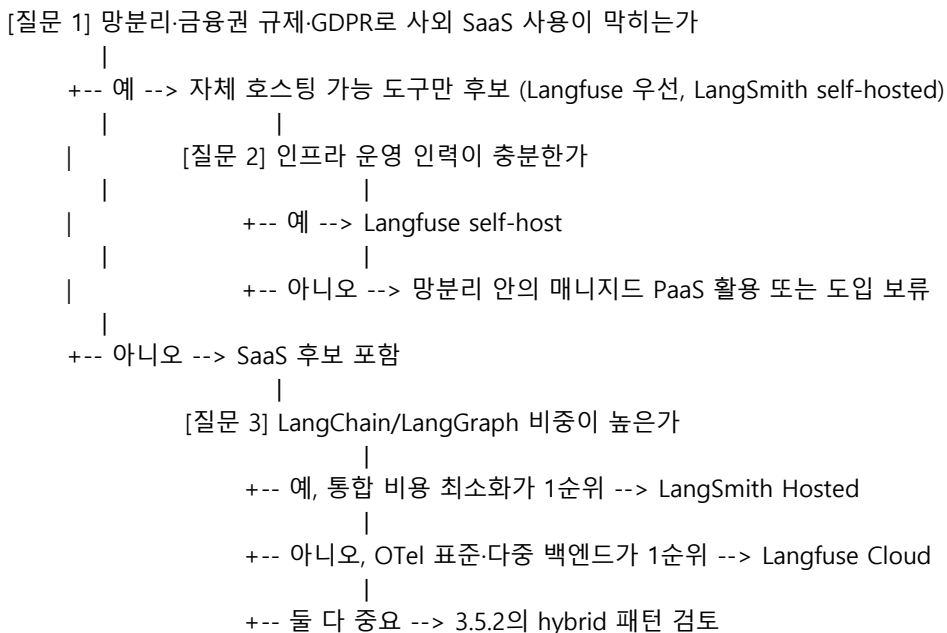
둘째 차원은 **Self-host 가능 여부**입니다. self-host는 도구를 만든 회사의 클라우드가 아니라 자기 인프라 위에 그 도구를 직접 띄워 운영하는 방식을 가리킵니다. Langfuse는 오픈소스로 공개되어 있고 자체 호스팅을 일급 시민으로 다룹니다. LangSmith도 self-hosted 옵션을 제공하지만 기본 사용 형태는 호스팅된 SaaS이고, self-hosted 변형은 별도 라이선스와 더 많은 운영 손길을 요구합니다. 자체 호스팅이 후보에 오르면 곧장 다음 차원으로 이어집니다.

셋째 차원은 **팀 규모와 운영 부담**입니다. self-host는 데이터 주권을 얻는 대신 인프라 운영을 짊어집니다. Postgres와 ClickHouse 같은 백엔드를 띄우고, 디스크와 메모리를 챙기고, 백업과 보안 패치를 돌리고, 버전 업그레이드 시 호환성을 검증하는 일이 모두 회사 몫이 됩니다. 인프라 팀이 이미 PostgreSQL을 운영해 본 적 있는 대기업이라면 추가 부담이 작지만, 데이터 엔지니어가 한두 명인 스타트업이 self-host를 떠안으면 본업이 운영으로 잠식됩니다. 이 차원은 곱셈으로 작용하기 때문에, 데이터 주권 요구가 강하지 않은 회사는 같은 self-host 비용을 굳이 치를 이유가 사라집니다.

넷째 차원은 **통합 비용과 마이그레이션 부담**입니다. 이미 LangChain이나 LangGraph로 코드를 짜 둔 팀에게는 LangSmith가 자동 인스트루먼트로 거의 추가 작업 없이 trace를 잡아 줍니다. 같은 팀이 Langfuse로 옮기려면 SDK를 한 번 갈아 끼워야 하고, 자동 계측이 닿지 않는 부분은 손으로 span을 열어 줘야 합니다. 반대로 OpenAI SDK와 Anthropic SDK를 직접 호출하는 코드 위에서는 둘의 통합 비용 차이가 작아집니다. 마이그레이션 부담은 코드뿐 아니라 사람에게도 붙습니다. 데이터셋 정의, 평가 워크플로, 권한 모델은 도구마다 표현 방식이 다르기 때문에 옮기는 순간 운영팀이 다시 배워야 합니다. 이 비용을 무시하고 도구를 자주 갈아엎으면 평가 자체가 멈춥니다.

다섯째 차원은 **벤더 신뢰성과 로드맵**입니다. 호스팅 SaaS를 쓴다는 것은 그 회사의 가용성, 가격 정책, 제품 방향에 자기 운영을 의탁한다는 뜻입니다. 회사가 인수되거나 가격이 갑자기 오르거나 제품 방향이 바뀔 때 받을 충격을 미리 가늠해야 합니다. 오픈소스 자체 호스팅은 이 충격을 줄이지만 완전히 없애 주지는 않습니다. 오픈소스 프로젝트도 핵심 메인테이너의 동력에 영향을 받기 때문에, 커뮤니티 활동, 릴리즈 주기, 보안 이슈 대응 속도를 함께 보아 두는 편이 안전합니다.

다섯 차원을 의사결정 트리로 정리하면 다음 그림처럼 풀어집니다.



이 트리는 절대적인 답이 아니라 차원별 가중치를 분명히 하기 위한 도구입니다. 회사마다 가중치가 다르기 때문에, 같은 회사 안에서도 사업부에 따라 다른 답이 나오는 일이 흔합니다. 예를 들어 한국 시중은행의 대고객 챗봇 시스템은 첫 질문에서 곧장 self-host 분기로 빠지는 반면, 같은 은행의 사내 코드 리뷰 보조 도구는 사용자가 임직원으로 한정되고 데이터도 사내 코드라서 SaaS가 허용될 수 있습니다. 한 회사가 두 도구를 동시에 쓰는 상황은 이렇게 자연스럽게 만들어집니다.

마지막으로 한국 기업 맥락에서 자주 함께 따져야 하는 항목을 짧게 짚겠습니다. 금융권은 보통 신규 도구 도입 시 보안성 심의를 거치고, 클라우드 사용은 금융보안원의 가이드와 회사 내부 클라우드 정책의 교집합 안에서만 허용됩니다. 공공 부문은 국가정보원 보안성 검토를 받는 시스템에서 사외 SaaS를 쓰기 어렵고, 국내 클라우드 보안 인증을 받은 사업자 위에서만 운영이 가능합니다. EU 시장 진출 기업은 GDPR의 적정성 결정과 표준계약조항을 검토해 trace 데이터의 EU 밖 이전 근거를 마련해야 합니다. 이 항목들은 도구의 기술 우열을 따지기 전에 후보군 자체를 좁히는 필터로 작용하기 때문에, 의사결정의 가장 위에 두는 편이 시간을 아낍니다.

정리하면, LangSmith와 Langfuse 사이의 선택은 단순한 기능 비교가 아니라 데이터 주권과 보안 정책,

self-host 가능 여부, 팀 규모와 운영 부담, 통합 비용과 마이그레이션 부담, 벤더 신뢰성과 로드맵이라는 다섯 차원의 가중합으로 풀립니다. 한국 기업의 망분리, 금융권 규제, EU GDPR 같은 제약은 이 다섯 차원 가운데 첫 두 항목을 무겁게 만들어 self-host를 우선 후보로 끌어올리고, 그 위에 통합 비용과 운영 인력이 더해지면서 회사마다 다른 결론에 도달합니다.

다음 단원인 3.5.2에서는 한 회사 안에서 한 도구로는 모든 요구를 다 채우지 못할 때 두 도구 이상을 묶어 쓰는 hybrid 스택 구성 패턴을 살펴보겠습니다. Replit이 평가에는 Braintrust, traceability에는 LangSmith를 함께 쓰는 사례가 그 출발점이 됩니다.

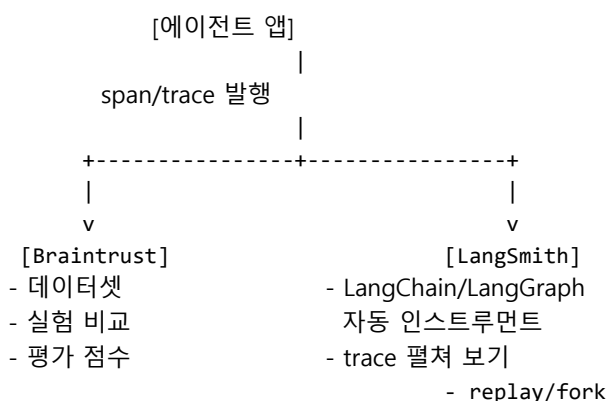
이 단원을 마치면 한국 기업이 LangSmith와 Langfuse 가운데 하나를 고를 때 따져야 하는 데이터 주권, self-host 가능성, 팀 규모, 통합 비용, 벤더 신뢰성 다섯 차원을 망분리·금융권·GDPR 같은 한국 맥락 위에서 의사결정 트리로 풀어 설명할 수 있습니다.

3.5.2 Hybrid 스택 구성 패턴

앞 단원에서 본 의사결정 트리는 한 도구를 고르는 길을 보여 줬지만, 현실의 큰 시스템은 한 도구만으로 모든 요구를 다 채우지 못하는 경우가 많습니다. 평가 워크플로에 강한 도구와 trace 분석에 강한 도구가 따로 있고, 그 위에서도 자기 사정에 맞춰 손으로 짜야 하는 부분이 늘 남습니다. 이 단원은 두 가지 이상의 도구를 의도적으로 함께 쓰는 hybrid 스택의 구성 패턴을 다룹니다. 출발점은 Replit이 공개한 자기네 평가 스택입니다.

먼저 사례부터 인용하겠습니다. Replit의 발표를 그대로 옮기면, "Braintrust는 일부 평가에, LangSmith는 traceability에 쓰지만 평가 harness 대부분은 자체 Python으로 구축"입니다. 한 문장이 짧지만 그 안에 hybrid의 골격이 다 들어 있습니다. 평가용 SaaS 한 개, 관측용 SaaS 한 개, 그리고 둘 사이에 자체 코드 한 덩어리입니다. 단어를 짧게 풀어 두겠습니다. **Braintrust**는 데이터셋과 실험을 관리하고 평가 결과를 비교해 주는 데에 무게를 둔 평가 플랫폼이고, **LangSmith**는 3.3에서 본 대로 LangChain·LangGraph 코드의 trace를 자동으로 잡아 보여 주는 관측 플랫폼이며, **harness**는 1장에서 다룬 대로 모델 바깥에서 실행 환경과 평가 흐름을 묶어 주는 골격을 가리키는 일반 용어입니다.

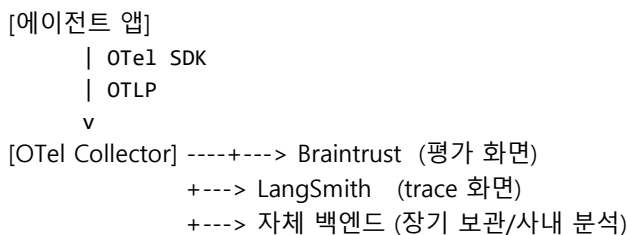
이 사례에서 가장 먼저 짚을 점은 **역할 분담**입니다. 같은 trace 데이터를 두 도구가 동시에 받는다고 해도, 사용자가 그 데이터를 보러 가는 화면은 목적에 따라 갈립니다. 평가는 점수, 데이터셋, 실험 비교, 회귀 그래프 같은 화면이 필요합니다. 관측은 한 건의 trace를 처음부터 끝까지 펼쳐 어디서 무엇이 늦었고 어떤 도구가 실패했는지를 보는 화면이 필요합니다. Replit은 두 화면을 같은 도구 안에서 다 보려는 시도 대신, 평가 화면은 Braintrust로, trace 화면은 LangSmith로 각각 잘 만들어진 도구에 맡기는 길을 골랐습니다. 이 분담을 그림으로 보면 다음과 같습니다.



분담을 정하고 나면 곧이어 **자체 harness가 끼어드는 경계**가 드러납니다. Replit이 평가 harness 대부분을

자체 Python으로 짠 이유는 두 SaaS가 비워 두는 자리가 컸기 때문입니다. Replit의 에이전트는 Chrome 인스턴스와 Docker 컨테이너 위에서 사용자처럼 클릭하고 입력하는 web agent이기 때문에, 평가 한 번을 돌리려면 컨테이너를 띄우고 브라우저를 깨우고 시나리오를 주입하고 결과를 거두는 복잡한 서막이 필요합니다. 이런 환경 준비 코드는 Braintrust나 LangSmith가 직접 다루는 영역이 아니라, 평가 점수를 만들어 SaaS로 넘기기 위한 앞단입니다. 자체 harness는 이 앞단을 책임지고, SaaS는 그 결과를 받아 보여 주는 뒷단을 말합니다. 이 경계가 분명할수록 어느 쪽에 무엇을 찔지 헛갈리지 않습니다.

분담과 경계를 세웠으면 그다음 문제는 **표준화**입니다. 도구가 셋이라는 사실이 곧 trace 형식도 셋이라는 뜻이 되면, 같은 사건을 세 군데에 다른 이름으로 기록해 두고 비교조차 어려워집니다. 이 위험을 줄이는 길이 3.4.2에서 본 OpenTelemetry입니다. OTel을 가운데 두면 에이전트 앱은 OTel SDK 한 가지로만 span을 만들어 OTLP로 내보내고, OTel collector가 그 흐름을 받아 Braintrust, LangSmith, 자체 백엔드로 분기해 줍니다. 같은 사건이 같은 속성 이름으로 세 곳에 도착하기 때문에, 한쪽에서 본 trace ID로 다른 쪽 화면을 곧장 열 수 있습니다. 그림으로 정리하면 다음과 같습니다.



이 그림은 hybrid의 진짜 가치가 어디서 생기는지 보여 줍니다. 도구마다 잘하는 영역이 다른 현실을 인정하면서도, 데이터의 형식을 표준 위에 두는 한 도구를 갈아 끼울 때마다 코드 전반을 다시 손대지 않아도 됩니다. 도구를 바꾸는 일이 collector의 export 한 줄을 수정하는 일에 가까워집니다. 한국 기업 맥락에서 흔한 분기, 즉 LLM trace는 Langfuse self-host로, 인프라 메트릭은 Datadog로, 평가 점수는 Braintrust로 보내는 패턴도 같은 골격 위에서 자연스럽게 풀립니다.

다만 hybrid에는 정직하게 따져야 할 **운영 복잡도**가 따라옵니다. 도구를 둘로 늘리면 운영 부담은 두 배가 되지 않고 보통 두 배 이상으로 늘어납니다. 첫째, SaaS 두 개의 가격, 보안 심의, SLA를 모두 들여다봐야 합니다. 둘째, 사용자 권한과 데이터 보존 기간을 도구마다 별도로 관리해야 하기 때문에 권한 모델이 어긋날 위험이 늘 있습니다. 셋째, 사고가 났을 때 trace를 따라가다 도구 경계를 넘어야 하면 디버깅 시간이 늘어납니다. 넷째, OTel collector라는 새 부품 자체가 운영해야 할 또 하나의 시스템이 됩니다. 다섯째, 자체 harness가 짚어진 코드는 회사가 영원히 같이 들고 가야 하는 자산이자 부채입니다. hybrid는 이 비용을 의식적으로 떠안고서 얻는 유연성이지만, 도구 두 개 이상을 어쩌다 같이 쓰게 된 결과가 아닙니다.

이 비용을 다스리는 실무 기준은 보통 다음 세 줄로 정리됩니다. 첫째, 단일 도구로 시작하고 그 도구가 명백히 비워 두는 자리만 다른 도구로 채웁니다. 둘째, 도구 사이를 잇는 글루 코드는 OTel 표준 위에 두고 가능한 얇게 유지합니다. 셋째, 자체 harness는 SaaS가 절대 채워 주지 않을 환경 준비와 사내 정책 적용에 한정하고, SaaS가 곧 채워 줄 영역까지 미리 짜 두지 않습니다. 이 세 기준이 흔들리는 순간 hybrid는 빠르게 자기 비용에 짓눌립니다.

마지막으로 hybrid가 항상 정답은 아니라는 점을 짚어 두겠습니다. 팀이 작고 도메인이 단순하다면 한 도구를 깊게 쓰는 길이 거의 늘 더 효율적입니다. Replit식 hybrid는 web agent라는 까다로운 평가 환경, 빠르게 늘어나는 reactive eval 백로그, 두 SaaS의 영역이 분명히 갈리는 제품 사정이 함께 작용해 자연스럽게 도달한 형태에 가깝습니다. 같은 패턴을 모방하기 전에 자기 회사의 사정도 그만큼 갈리는지 먼저 확인해 두는 편이 안전합니다.

정리하면, hybrid 스택은 평가에 강한 도구와 관측에 강한 도구를 의도적으로 분담시키고 그 사이를 자체 harness로 메우는 구성으로, Replit의 Braintrust + LangSmith + 자체 Python harness 조합이 대표 사례입니다. 같은 trace를 여러 백엔드에 보내는 길은 OpenTelemetry로 표준화해 도구 교체 시 영향 범위를 좁히고, 그 대가로 늘어나는 운영 복잡도는 시작은 단일 도구로, 글루 코드는 얇게, 자체 harness는 SaaS가 비우는 자리에만 둔다는 기준으로 다스립니다.

다음 단원인 3.6.1에서는 이 hybrid 패턴을 떠받치는 표준인 OpenTelemetry 자체를 본격적으로 살피고, 그 표준이 GenAI 영역까지 확장되어 온 흐름과 SDK-collector-protocol의 구조를 정리하겠습니다.

이 단원을 마치면 Replit식 hybrid 스택에서 Braintrust와 LangSmith가 평가와 traceability로 역할을 나누는 방식, 자체 harness가 SaaS의 빈 자리에 끼어드는 경계, OpenTelemetry로 여러 백엔드를 표준화해 묶는 전략, 그리고 hybrid가 가져오는 운영 복잡도를 다스리는 실무 기준을 함께 설명할 수 있습니다.

3.6 OpenTelemetry for GenAI

이 섹션의 토픽과 학습 목표는 다음과 같습니다.

3.6.1 OpenTelemetry 표준 개요 — OpenTelemetry의 표준화 범위와 GenAI 관측에서의 위치를 설명할 수 있다

3.6.2 GenAI Semantic Conventions — OTel GenAI semantic conventions의 핵심 속성을 식별하고 trace에 부착할 수 있다

3.6.3 표준 기반 무벤더 락인 설계 — OTel 표준 위에서 LangSmith·Langfuse·자체 백엔드를 동시 지원하는 설계를 설명할 수 있다

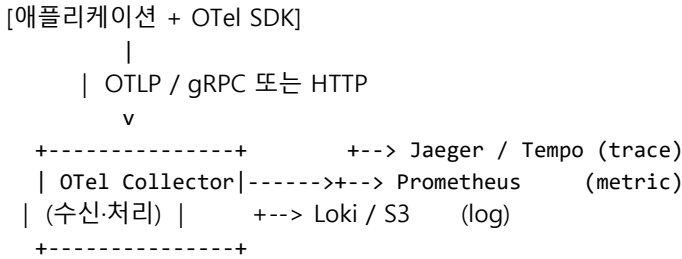
3.6.1 OpenTelemetry 표준 개요

앞의 3.5에서 LangSmith와 Langfuse를 비교하고, 더 나아가 두 도구를 섞어 쓰는 hybrid 스택까지 살펴보았습니다. 그 흐름의 끝에 항상 따라온 단어가 있습니다. 바로 OpenTelemetry입니다. Langfuse가 OTLP endpoint를 받는다고 한 자리에서, hybrid 스택의 표준화 전략을 다룬 자리에서, 모두 같은 이름이 등장했습니다. 이 단원은 그 이름이 무엇이고 왜 LLM 관측의 가운데 자리에 놓이게 되었는지를 처음부터 풀어 봅니다.

먼저 단어 자체부터 풀겠습니다. **OpenTelemetry**, 줄여서 OTel은 분산 시스템의 동작을 관찰하기 위한 데이터, 즉 trace와 metric과 log를 어떻게 만들고 어떻게 보내고 어떻게 수집할지를 정해 둔 업계 표준입니다. 클라우드 네이티브 운영 표준을 다루는 비영리 재단인 CNCF에서 호스팅하고 있고, 한 회사가 만든 도구가 아니라 여러 회사와 사용자가 함께 쓰는 공통 약속이라는 점이 본질입니다. '여러 도구가 같은 형식의 trace를 주고받자'는 합의를 표준 단위로 적어 둔 결과물이 OTel입니다.

그 합의는 세 개의 부품으로 나뉩니다. 첫째는 **SDK**입니다. 애플리케이션 코드가 trace와 metric을 만들 때 부르는 라이브러리로, 언어별로 Python·JavaScript·Go·Java·Rust 같은 구현이 있습니다. SDK는 'span을 시작하고 끝낸다', '속성을 단다', '메트릭 카운터를 올린다' 같은 일을 통일된 인터페이스로 제공합니다. 둘째는 **collector**입니다. SDK가 만든 데이터를 받아 가공하고 분기해 여러 백엔드로 다시 내보내는 중간 부품으로, 별도 프로세스로 띄워 운영합니다. 셋째는 **protocol**, 그중에서도 OTLP라 불리는 OpenTelemetry Protocol입니다. SDK와 collector, collector와 백엔드 사이에 데이터를 실어 나를 때 쓰는 네트워크 규약입니다. 이메일에 비유하면 SMTP에 해당하는 자리를 차지합니다.

세 부품이 함께 동작하는 모습을 그림으로 보면 다음과 같습니다.



이 그림에서 중요한 점은 SDK가 어떤 백엔드를 직접 알 필요가 없다는 사실입니다. SDK는 OTLP로 collector에 데이터를 던지고, 어디로 분기할지는 collector의 설정 한 곳에서 결정합니다. 백엔드를 갈아 끼우려고 애플리케이션 코드를 다시 만지지 않아도 된다는 뜻입니다.

OTel이 다루는 데이터의 종류는 셋입니다. **Trace**는 한 요청이 시스템을 어떻게 흘러갔는지를 트리 형태로 기록한 묶음으로, 그 안의 한 단계를 span이라 부릅니다. **Metric**은 시간에 따라 변하는 수치로, 요청 수, 지연, 에러율 같은 값이 여기에 들어갑니다. **Log**는 어느 한 시점에 남긴 텍스트 기록입니다. 이 셋은 본래 따로 발달해 왔지만, OTel은 한 SDK에서 한 표준으로 셋을 함께 다룰 수 있게 모았습니다. trace의 한 span이 만들어 내는 metric, 그 span 안에서 남긴 log가 같은 trace ID로 묶여 한 요청의 그림을 한 번에 보여 주는 일이 가능해진 까닭입니다.

OTel이 가진 가치는 한 단어로 요약하면 **벤더 중립**입니다. 관측 분야에서 가장 자주 일어나는 사고는 한 도구의 전용 SDK로 코드를 광범위하게 계속해 두었다가 나중에 도구를 바꾸지 못해 그대로 묶이는 일입니다. OTel은 이 벤더 락인을 구조적으로 풀어내기 위해 SDK와 백엔드 사이를 표준 한 겹으로 분리합니다. 어떤 백엔드를 쓰든 같은 OTel SDK로 같은 형식의 데이터를 만들 수 있으면, 백엔드를 바꿀 때 흔들리는 영역이 collector 설정 정도로 줄어듭니다. 이 효과는 도구를 자주 바꾼다는 뜻이 아니라, 바꾸어야 할 때 그 비용이 코드 전반의 재작성 수준이 아니라는 뜻입니다.

OTel의 출발점은 LLM이 아니었습니다. 마이크로서비스가 본격화되면서 한 사용자 요청이 수십 개 서비스를 거치는 경로를 추적해야 할 필요가 커졌고, 이 분야의 두 표준이었던 OpenTracing과 OpenCensus가 통합되어 OTel이 만들어졌습니다. 이 흐름은 **전통 분산 추적과의 연속성**으로 읽을 수 있습니다. 즉 OTel은 새로 만든 발명품이 아니라, 분산 시스템 추적이 십 년 넘게 다듬어 온 데이터 모델을 한 우산 아래 모은 결과입니다. trace와 span의 부모-자식 관계, 컨텍스트 전파, sampling 같은 개념은 본래 마이크로서비스 운영을 위해 정리된 것들이고, 같은 도구가 그대로 LLM 호출에도 쓰일 수 있다는 사실이 이번 흐름의 출발점이 되었습니다.

LLM 호출 한 건은 본질적으로 또 하나의 외부 서비스 호출입니다. 모델 이름, 입력 토큰, 출력 토큰, 지연, 비용 같은 값을 span의 속성으로 부착하면 기존 trace 그림 안에 자연스럽게 자리잡습니다. 에이전트의 multi-step 흐름은 부모 span 아래 여러 자식 span이 매달리는 형태로 표현되고, 그 안에 도구 호출 span이 다시 자식으로 들어갑니다. 분산 추적이 다년간 다루어 온 표현이 그대로 GenAI 영역으로 옮겨 올 수 있다는 점이 OTel을 LLM 관측의 후보로 끌어올린 가장 큰 이유입니다.

다만 LLM에는 기존 마이크로서비스에는 없던 속성이 많습니다. system prompt, completion, tool call의 인자와 결과, token usage, 모델별 메타데이터 같은 항목을 어떤 이름으로 어디에 넣을지 합의가 필요합니다. 이 합의가 GenAI semantic conventions라는 이름으로 진행 중인데, 2025년 시점에서도 일부 영역은 작업이 계속되고 있는 상태입니다. 즉 OTel은 LLM 관측의 표준 자리를 차지할 후보로 자리잡았지만, 그 표준의 세부 항목은 지금도 채워지고 있는 단계입니다. 이런 진행 상태를 받아들이면서 표준 위에 올리는 일이 이 책 후반부의 실무 의사결정을 가로지르는 주제가 됩니다.

OTel을 LLM 관측의 가운데에 두는 일이 한국 기업 입장에서 갖는 의미도 짚어 두겠습니다. LangSmith나

Langfuse나 자체 trace 백엔드 가운데 무엇을 고를지는 시간에 따라 바뀔 수 있는 결정입니다. 그러나 계측 코드를 OTel SDK 위에 두면 그 결정이 바뀌어도 애플리케이션 코드는 거의 영향을 받지 않습니다. 도구 비교를 다룬 3.5에서 hybrid 스택과 자체 harness가 등장한 까닭도 같은 자리에 놓입니다. 어떤 조합을 고르든 가운데에 OTel을 두면 조합을 바꾸기가 한결 쉬워지기 때문입니다.

정리하면, OpenTelemetry는 trace-metric-log를 한 표준 위에 모아 SDK-collector-protocol 세 부품으로 다루는 벤더 중립 관측 표준이며, 분산 추적의 오랜 전통을 이어받아 LLM 호출과 에이전트 trajectory까지 같은 그림 안에 표현할 수 있게 확장되고 있는 흐름입니다. 표준 자체는 2025년에도 GenAI 영역의 세부 약속을 채우는 중이지만, 가운데 자리에 OTel을 놓으면 LangSmith·Langfuse·자체 백엔드 사이의 의사결정이 코드 재작성이 아니라 설정 변경 수준의 문제로 좁아진다는 효과가 따라 나옵니다.

다음 단원인 3.6.2에서는 그 표준이 LLM 호출을 표현하기 위해 정의해 둔 속성과 메트릭 이름들을 구체적으로 살펴보고, vendor마다 그것을 어떻게 매핑하고 있는지를 정리하겠습니다.

이 단원을 마치면 OpenTelemetry의 SDK-collector-protocol 세 부품이 어떤 일을 나눠 맡는지, trace-metric-log를 한 표준에서 함께 다루는 의미, 벤더 중립이라는 가치가 만들어 내는 효과, 그리고 분산 추적의 연속선상에서 OTel이 GenAI 관측의 가운데 자리로 옮겨 오게 된 배경을 설명할 수 있습니다.

3.6.2 GenAI Semantic Conventions

앞 단원에서 OpenTelemetry가 trace-metric-log를 한 표준에서 다루는 그릇이라는 점을 살폈습니다. 그릇이 같다고 해서 그 안에 담기는 데이터의 이름까지 자동으로 같아지지는 않습니다. 어떤 팀은 모델 이름을 model이라 적고 어떤 팀은 llm_model이라 적으면, 같은 collector를 거쳐도 도구마다 다르게 해석합니다. 이 단원은 이름의 약속, 즉 LLM 호출과 에이전트 단계를 표현할 때 어떤 속성과 메트릭을 어떤 이름으로 쓸지 정한 **GenAI semantic conventions**를 다룹니다.

먼저 단어부터 풀겠습니다. **Semantic conventions**, 한글로는 의미 규약이라 옮길 수 있는 이 표현은 OTel 안에서 데이터의 의미를 통일하기 위해 정한 속성 이름과 그 의미의 표준입니다. trace의 한 span에 'http.method'라고 적으면 모든 도구가 그것을 'HTTP 메서드'로 같이 이해하기 위한 약속입니다. GenAI semantic conventions는 이 약속을 LLM과 에이전트 영역에 맞춰 확장한 것으로, 'gen_ai.'로 시작하는 속성 이름들이 그 핵심을 이룹니다. 'gen_ai'는 generative AI의 줄임말로, 같은 LLM 호출이라도 어떤 도구가 받아 보든 같은 키 아래에서 같은 의미로 해석하도록 묶어 두기 위해 붙은 이름공간입니다.

이 표준에서 가장 먼저 자리잡는 속성은 **gen_ai.system**, **gen_ai.request.model**, 그리고 요청과 관련된 파라미터 묶음입니다. gen_ai.system은 호출하고 있는 모델 제공자가 누구인지를 적습니다. 'openai', 'anthropic', 'azure_openai'처럼 짧은 식별자가 들어갑니다. gen_ai.request.model은 그 안에서 고른 모델 이름을 적습니다. 'gpt-4o', 'claude-sonnet-4-5' 같은 값입니다. 요청 파라미터는 gen_ai.request.temperature, gen_ai.request.max_tokens, gen_ai.request.top_p처럼 요청에 실어 보낸 설정값이 각각 별도 속성으로 붙습니다. 이렇게 system과 model과 request 묶음만 표준 이름으로 정해 두면, span 한 개를 보고 어디에 어떤 모델로 어떤 설정으로 호출했는지를 누가 보든 같은 키에서 읽어 낼 수 있습니다.

이어서 LLM 호출의 핵심 내용물에 대한 속성이 들어갑니다. 입력으로 보낸 메시지는 **gen_ai.prompt**, 모델이 돌려준 응답은 **gen_ai.completion**이라는 이름공간 아래에 놓입니다. 한 호출에는 여러 메시지가 오갈 수 있으므로 gen_ai.prompt.0.role, gen_ai.prompt.0.content, gen_ai.prompt.1.role 같은 식으로 인덱스가 붙은 형태가 표준에서 권장됩니다. completion 쪽도 마찬가지로 gen_ai.completion.0.role, gen_ai.completion.0.content가 짝을 이룹니다. 이 부분은 prompt와 응답 본문을 그대로 담는 영역이라

데이터 양이 커지기 쉽고, 민감 정보가 들어가기도 합니다. 그래서 표준은 이 영역을 기록할지 말지를 SDK 차원에서 끄고 켤 수 있도록 권고합니다. 실무에서는 sampling 비율을 낮추거나 redaction을 거친 사본만 기록하는 방식이 흔히 쓰이고, 이 부분은 다음 단원에서 별도로 다룹니다.

에이전트가 도구를 부르는 호출도 표준이 별도로 다룹니다. **Tool call**에 해당하는 속성은 `gen_ai.tool.name`으로 도구 이름을, `gen_ai.tool.call.id`로 한 호출의 식별자를, `gen_ai.tool.arguments`와 `gen_ai.tool.result` 같은 키로 인자와 결과를 적도록 합니다. 한 단계에서 여러 도구를 호출했다면 호출마다 별도 자식 span으로 만들어 부모 span 아래에 매달고, 각 자식 span에 위 속성을 붙입니다. 이 구조 위에서 trajectory는 부모 span의 자식 트리를 따라가는 일이 됩니다. 이 부분은 표준이 어느 정도 자리잡았지만, 에이전트 특유의 multi-step 평가 점수나 정책 위반 같은 항목은 표준 영역 밖에 있어서 도구마다 자기 확장 속성으로 붙여 두는 것이 현실입니다.

토큰 사용량은 메트릭과 속성 두 곳에 모두 등장합니다. span 단위 속성으로는 `gen_ai.usage.input_tokens`, `gen_ai.usage.output_tokens`, `gen_ai.usage.total_tokens`가 한 호출의 토큰 수를 적고, 같은 정보를 시간 누적 메트릭 형태로 보낼 때는 `gen_ai.client.token.usage` 같은 카운터 형태가 정의됩니다. 비용 계산은 표준 자체가 직접 다루지 않습니다. 모델별 단가가 시간에 따라 바뀌기 때문에, 비용은 token usage를 받은 백엔드에서 곱셈을 거쳐 산출하는 방식이 일반적입니다. 지연도 별도 항목이 따로 있다기보다는 OTEL의 기본 메트릭인 span 자체의 duration을 사용하는 쪽이 자연스럽습니다.

지금까지의 내용을 한 표로 정리하면 다음과 같습니다.

영역	표준 속성 / 메트릭 예시
모델 식별	<code>gen_ai.system</code> , <code>gen_ai.request.model</code>
요청 파라미터	<code>gen_ai.request.temperature</code> / <code>max_tokens</code> / <code>top_p</code>
입력 메시지	<code>gen_ai.prompt.{i}.role</code> , <code>gen_ai.prompt.{i}.content</code>
출력 메시지	<code>gen_ai.completion.{i}.role</code> , <code>gen_ai.completion.{i}.content</code>
도구 호출	<code>gen_ai.tool.name</code> , <code>gen_ai.tool.arguments</code> , <code>gen_ai.tool.result</code>
토큰 사용	<code>gen_ai.usage.input_tokens</code> / <code>output_tokens</code> / <code>total_tokens</code>
토큰 메트릭	<code>gen_ai.client.token.usage</code> (counter)

이 표준이 의미를 가지려면 vendor가 자기 SDK 안에서 위 이름들에 자동으로 값을 채워 주어야 합니다. **Vendor별 매핑**의 현실은 도구마다 조금씩 다릅니다. OpenAI Python SDK 호출에 OTEL 계측 라이브러리를 끼워 두면 위 속성 가운데 모델·토큰·메시지 항목은 자동으로 채워지는 경우가 많습니다. Anthropic SDK도 비슷한 자동 계측 라이브러리가 OTEL 진영에서 만들어지고 있습니다. LangChain과 LangGraph는 자체 trace 모델 위에 OTEL exporter 어댑터를 붙여 표준 속성을 함께 내보내는 작업이 진행되고 있고, LlamaIndex도 비슷한 방향입니다. Langfuse와 LangSmith는 OTLP endpoint로 들어오는 표준 속성을 자기 trace 모델로 받아들이는 쪽에서 매핑을 흡수합니다. 받는 쪽 도구가 표준 속성을 우선 해석하고, 표준에 없는 항목은 자기 확장 필드로 보존하는 방식이 일반적인 패턴입니다.

표준의 진행 상태를 사실 그대로 짚어 두는 일이 이 단원에서 중요합니다. **2025년 작업 항목**을 거칠게 분류하면 셋입니다. 첫째, 모델·토큰·prompt·completion에 해당하는 핵심 속성은 안정 단계에 가깝게 다듬어지고 있습니다. 둘째, 도구 호출과 multi-turn 대화의 표현은 안정화 직전에 와 있지만 일부 세부 변동 가능 영역으로 남아 있습니다. 셋째, 에이전트의 multi-step trajectory, 평가 점수, 정책 위반 같은 영역은 표준화의 후순위 단계로, 도구별 확장 속성에 의존하는 비중이 큼니다. 따라서 새 코드에 OTEL을 도입할 때는 핵심 속성에 대해서는 표준을 그대로 따르고, 표준에 없는 영역은 도구 확장 속성을 쓰되 나중에 표준 이름으로 옮길 수 있도록 한 곳에 묶어 두는 방식이 안전합니다.

이 표준이 코드에 어떻게 보이는지를 OTEL Python SDK의 짧은 예로 보여 드리면 다음과 같습니다.

```

from opentelemetry import trace
tracer = trace.get_tracer("agent")

with tracer.start_as_current_span("llm.call") as span:
    span.set_attribute("gen_ai.system", "anthropic")
    span.set_attribute("gen_ai.request.model", "claude-sonnet-4-5")
    span.set_attribute("gen_ai.request.temperature", 0.2)
    # ... 모델 호출 ...
    span.set_attribute("gen_ai.usage.input_tokens", 1325)
    span.set_attribute("gen_ai.usage.output_tokens", 412)

```

수동으로 적은 위 코드를 자동 계측 라이브러리가 대체로 대신해 줍니다. 직접 계측이 필요한 영역은 자체 도구 호출이나 자체 retrieval 단계처럼 SDK 바깥에 있는 부분이 됩니다.

정리하면, GenAI semantic conventions는 LLM과 에이전트 호출을 OTel trace 위에 표현하기 위해 'gen_ai.'로 시작하는 속성 이름과 메트릭을 정해 둔 약속입니다. 모델 식별과 요청 파라미터, prompt와 completion, tool call, token usage가 그 골격을 이루며, vendor마다 자동 계측 라이브러리와 endpoint 수신 매핑을 통해 이 골격을 채우거나 받아들이고 있습니다. 표준은 2025년에도 작업이 진행 중이라 핵심 영역과 미정 영역을 구분해서 다루는 자세가 필요합니다.

다음 단원인 3.6.3에서는 이 표준 위에서 OTel collector를 가운데에 두고 LangSmith·Langfuse·자체 ClickHouse 백엔드를 동시에 활용하면서 sampling과 PII redaction까지 함께 거는 무벤더 락인 설계를 정리하겠습니다.

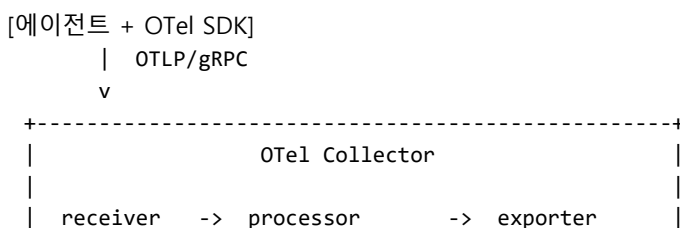
이 단원을 마치면 gen_ai.system-gen_ai.request.model을 비롯한 모델 식별 속성, prompt와 completion 속성의 구조, tool call 표현 방식, token usage 속성과 메트릭, vendor별 매핑의 현실, 그리고 2025년 시점의 표준 진행 상태를 한 흐름으로 설명할 수 있습니다.

3.6.3 표준 기반 무벤더 락인 설계

3.6.1에서 OpenTelemetry라는 그릇을, 3.6.2에서 그 그릇 안의 이름 약속을 살펴보았습니다. 이 단원은 이 두 가지를 묶어 실제 시스템을 어떻게 짤지에 대한 답입니다. 한국 기업이 LangSmith의 손쉬움, Langfuse의 자체 호스팅, 사내 ClickHouse 같은 자체 분석 백엔드를 동시에 원할 때, 이 셋을 한 코드 위에서 충돌 없이 운영하는 길을 정리합니다. 출발점은 단순합니다. 가운데에 OTel collector를 둡니다.

먼저 **OTel collector pipeline**의 구조부터 풀겠습니다. collector는 별도 프로세스로 띄우는 작은 서버로, 안에서 하는 일이 셋입니다. 첫째 receiver, 둘째 processor, 셋째 exporter입니다. receiver는 외부에서 들어오는 데이터를 받는 입구로, OTLP를 받는 receiver가 가장 흔합니다. processor는 받은 데이터를 변형하거나 거르는 단계로, batching, sampling, redaction, attribute 변환 같은 일이 여기서 일어납니다. exporter는 처리된 데이터를 외부 백엔드로 내보내는 출구로, 백엔드 종류만큼 종류가 있습니다. 이 셋이 모여 한 줄을 이루는 것을 pipeline이라 부르고, collector 한 대 안에 여러 pipeline을 동시에 돌릴 수 있습니다.

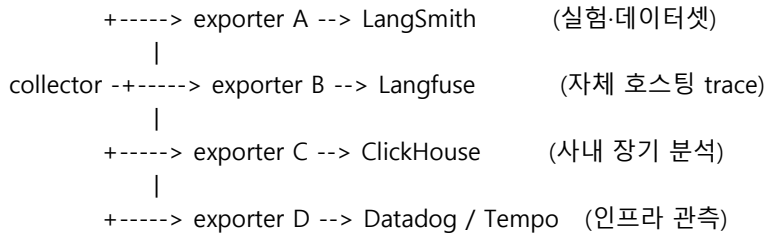
이 구조를 그림으로 보면 다음과 같습니다.



(otlp)	(batch,	(otlp_http,	
	sampler,	clickhouse,	
	redactor,	langsmith,	
	attributes)	langfuse)	
+-----+-----+-----+			

receiver는 한 곳, exporter는 여러 곳이라는 점이 가장 중요합니다. 이 비대칭 위에서 **다중 백엔드 동시 export**가 자연스럽게 풀립니다.

흐름을 한 단계 더 펼쳐 보면 이렇게 그릴 수 있습니다.



같은 trace 한 건이 collector를 거치면서 네 곳으로 동시에 흘러갑니다. 애플리케이션 코드는 OTLP 한 곳만 알고, 분기 결정은 collector 설정 파일에서 일어납니다. 이 분기는 똑같은 데이터를 그대로 복제할 수도 있고, pipeline마다 별도의 processor를 끼워 다른 가공을 거친 사본을 보낼 수도 있습니다. 예를 들어 LangSmith 쪽으로는 prompt 본문을 그대로 보내고, ClickHouse 쪽으로는 PII redaction을 거친 사본만 보내는 구성이 가능합니다.

한 collector 설정의 골격을 YAML로 보여 드리면 다음과 같습니다.

```

receivers:
  otlp:
    protocols:
      grpc:
processors:
  batch: {}
  tail_sampling:
    policies:
      - name: errors
        type: status_code
        status_code: { status_codes: [ERROR] }
      - name: sample-10pct
        type: probabilistic
        probabilistic: { sampling_percentage: 10 }
  attributes/redact:
    actions:
      - key: gen_ai.prompt.0.content
        action: hash
exporters:
  otlphttp/langfuse:
    endpoint: https://langfuse.internal/api/public/otel
  otlphttp/langsmith:
    endpoint: https://api.smith.langchain.com/otel
  clickhouse:
    endpoint: tcp://clickhouse.internal:9000
service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [batch, tail_sampling, attributes/redact]

```

```
exporters: [otlphttp/langfuse, otlphttp/langsmith, clickhouse]
```

이 한 파일로 세 백엔드를 동시에 운영하면서, 가운데에서 표본을 뽑고 prompt 본문을 가립니다. 코드는 그대로입니다.

여기서 등장한 두 단어를 풀어 두겠습니다. **Sampling**은 모든 trace를 다 저장하지 않고 일부만 골라 저장하는 정책입니다. 모든 호출을 다 기록하면 비용과 저장 부담이 빠르게 커지기 때문에, 일정 비율만 무작위로 뽑거나(probabilistic) 에러가 일어난 trace는 모두 뽑고 정상 trace는 일부만 뽑는 방식(tail sampling)이 흔히 쓰입니다. tail sampling은 trace가 끝난 뒤 결과를 보고 결정한다는 뜻이고, 결과 기반으로 흥미로운 trace를 선별하기에 LLM 관측에 잘 맞습니다. **PII redaction**은 trace 안에 들어 있는 개인 식별 정보, 주민등록번호, 전화번호, 이메일 같은 항목을 가리거나 해시하는 처리입니다. collector의 attributes processor에서 정규식이나 키 단위 규칙으로 거는 방식이 일반적입니다. 한국 환경이라면 개인정보 보호법 적용 범위에 들어가는 항목을 사전 정의해 두고, 모든 exporter 앞에서 한 번 redaction을 거는 구성이 안전합니다.

이 구조가 만들어 내는 가장 큰 효과가 **벤더 교체 시 영향 범위의 축소**입니다. 영향 범위라는 표현이 추상적으로 들릴 수 있어 한 번 풀어 두겠습니다. 어떤 도구를 다른 도구로 바꿀 때 손대야 할 코드와 설정의 양을 가리킵니다. 도구 전용 SDK로 광범위하게 계측해 둔 경우, 영향 범위는 SDK 호출이 적힌 모든 코드 파일입니다. OTel SDK로 계측해 둔 경우, 영향 범위는 collector의 exporter 설정 한 줄, 또는 새 exporter를 추가하고 옛 것을 빼는 변경 한 건입니다. 이 차이는 평소에는 잘 보이지 않다가 도구 가격 정책이 바뀌거나 데이터 거주성 요건이 새로 들어올 때 결정적으로 드러납니다.

이 효과를 한국 기업의 흔한 시나리오에 비추어 보면 그림이 더 분명해집니다. 처음에는 LangSmith로 빠르게 출발해 실험과 데이터셋 관리를 익히고, 사내 보안 검토가 통과되면 prompt 본문을 담은 사본은 Langfuse 자체 호스팅 인스턴스로 옮기며, 장기 분석은 사내 ClickHouse에 적재합니다. 이 세 단계를 거치는 동안 애플리케이션 코드는 거의 손대지 않고 collector 설정만 단계적으로 늘려 갑니다. 어느 단계에서 LangSmith를 빼더라도 exporter 한 줄을 지우는 일이지, 코드 전반의 SDK 호출을 갈아 끼우는 일이 아닙니다.

마지막으로 **표준 채택 로드맵**을 정리해 두겠습니다. 한 번에 모든 코드를 OTel로 옮기는 일은 비현실적인 경우가 많습니다. 다음과 같은 4단계 그림이 한국 기업 환경에서 자주 권장됩니다.

- 단계 1: 새 코드는 OTel SDK 위에 계측한다.
 - 기존 코드는 그대로 둔다. collector를 띄우고 한 백엔드로만 export.
- 단계 2: 기존 도구 전용 SDK로 계측된 코드도 OTLP exporter를 부착한다.
 - 두 SDK가 같은 collector로 모이도록 만든다.
- 단계 3: collector에 PII redaction과 sampling을 도입한다.
 - 핵심 백엔드 외에 ClickHouse 같은 장기 저장소를 추가한다.
- 단계 4: 도구 전용 SDK 호출을 점진적으로 OTel SDK로 바꾼다.
 - 이 과정이 끝나면 백엔드 교체가 collector 설정 변경으로 좁혀진다.

이 로드맵은 출발할 때 모든 결정을 미리 하지 않아도 된다는 점이 핵심입니다. 표준 위에 한 발만 올려 두면 이후 단계는 운영을 익히면서 한 단계씩 옮겨 가도 됩니다.

표준 채택의 한계도 한 번 짚어 두는 편이 안전합니다. 3.6.2에서 본 것처럼 GenAI semantic conventions는 일부 영역이 작업 중입니다. 도구별 확장 속성에 의존하는 부분이 한동안 남고, 그 부분은 표준 이름으로 옮길 때 한 번씩 매핑 작업이 따라옵니다. 또한 collector 운영 자체에 부하 분산과 장애 복구 같은 자체 운영 부담이 새로 생깁니다. 이 부담은 보통 인프라 팀의 기존 표준 관측 운영에 합쳐져 흡수되지만, 처음 collector를 도입할 때는 별도 작업으로 잡아 두어야 합니다.

정리하면, OTel collector를 가운데에 둔 표준 기반 설계는 receiver·processor·exporter pipeline 위에서 한번 들어온 trace를 LangSmith·Langfuse·사내 ClickHouse 같은 여러 백엔드로 동시에 분기해 보내고, 그 사이에 sampling과 PII redaction을 한 곳에서 거는 구조입니다. 이 구조가 만들어 내는 가장 큰 효과는 벤더 교체 시 영향 범위가 코드 전반이 아니라 collector 설정 한 곳으로 좁아진다는 점이며, 한국 기업이 단계적인 표준 채택 로드맵을 따라 새 코드부터 점진적으로 옮겨 갈 수 있는 길을 열어 줍니다.

지금까지 Phase 3에서는 online 평가의 메트릭 설계, 회귀 탐지, LLM observability의 데이터 모델, LangSmith와 Langfuse의 깊은 비교, 그리고 OpenTelemetry 표준 위에서 도구를 묶는 무벤더 락인 설계까지 한 줄로 이어 살펴보았습니다. 평가와 관측의 토대가 갖추어진 셈입니다. 다음 단원인 4.1.1부터 시작되는 Phase 4에서는 이 토대 위에 신뢰성과 안전을 엮는 일을 다룹니다. 첫 단원은 에이전트의 자율성을 단계별로 분류하고 각 단계의 권한 범위를 정하는 tiered autonomy의 출발점이 될 것입니다.

이 단원을 마치면 OTel collector pipeline의 receiver·processor·exporter 구조, LangSmith·Langfuse·자체 ClickHouse를 동시에 export하는 다중 백엔드 구성, sampling과 PII redaction을 collector에서 처리하는 패턴, 표준 위에 둔 코드가 벤더 교체 시 영향 범위를 어떻게 좁혀 주는지, 그리고 한국 기업 환경에서 단계적으로 OTel을 채택해 가는 로드맵을 한 흐름으로 설명할 수 있습니다.

4 프로덕션 패턴: 신뢰성과 안전

Tiered autonomy, circuit breaker, fallback routing, human-in-the-loop을 결합해 비결정적 에이전트를 안전하게 운영하는 신뢰성 패턴을 설계할 수 있다.

이 Phase는 다음 섹션으로 구성됩니다.

- 4.1 Tiered Autonomy
 - 4.1.1 자율성 단계 정의
 - 4.1.2 위험도 기반 권한 부여
- 4.2 Circuit Breaker
 - 4.2.1 Circuit Breaker 동작 원리
 - 4.2.2 에이전트 임계치 설계
- 4.3 Fallback Routing
 - 4.3.1 Fallback의 종류
 - 4.3.2 모델 라우팅 전략
 - 4.3.3 Graceful Degradation
- 4.4 Human-in-the-loop 재정의
 - 4.4.1 모든 걸 승인 의 한계
 - 4.4.2 임계치 기반 예외 검토
 - 4.4.3 Escalation Path 설계

4.1 Tiered Autonomy

이 섹션의 토픽과 학습 목표는 다음과 같습니다.

4.1.1 자율성 단계 정의 — 에이전트의 자율성을 단계별로 분류하고 각 단계의 권한 범위를 설명할 수 있다

4.1.2 위험도 기반 권한 부여 — 행동의 위험도에 따라 자율성 단계를 차등 적용하는 정책을 설계할 수 있다

4.1.1 자율성 단계 정의

지금까지 우리는 에이전트의 실패 모드를 분류하고, offline과 online 평가로 회귀를 잡아 내고, OpenTelemetry 표준으로 관측 데이터를 흘려 보내는 방법까지 다루었습니다. 이 모든 장치가 갖추어졌다고 해도 마지막에 한 질문이 남습니다. 에이전트에게 어디까지 일을 맡길 것인가. 같은 코드와 같은 모델이라도 사용자의 확인 없이 실행할 수 있는지, 사람의 검토를 거쳐야 하는지에 따라 시스템의 위험과 가치가 완전히 달라집니다. 이 단원은 에이전트가 가질 수 있는 자율성의 단계를 네 칸으로 나누어 보고, 각 칸이 무엇을 할 수 있고 무엇을 할 수 없는지를 정리합니다.

먼저 단어의 출발점을 풀겠습니다. **자율성**은 사람의 직접적 개입 없이 에이전트가 결정을 내리고 행동을 수행할 수 있는 정도를 가리킵니다. 자동차의 자율 주행에 비유하면 이해가 쉽습니다. 운전자는 핸들을 잡지 않은 채 도로를 보기만 하는 단계와, 손은 핸들에 두고 발만 떼는 단계, 차가 알아서 차선을 유지하지만 시선은 도로에 두는 단계가 모두 다릅니다. 에이전트도 비슷하게 행동의 종류와 사용자

개입의 빈도에 따라 단계를 나눌 수 있습니다. 이 책에서는 현장에서 가장 자주 인용되는 네 단계를 따라가겠습니다. **Read-only, Suggest, Execute, Autonomous** 입니다.

가장 낮은 단계인 **Read-only**부터 시작하겠습니다. 이 단계의 에이전트는 시스템의 상태를 읽어 보고 그것에 대해 답하기만 합니다. 데이터베이스의 행을 조회하거나 로그를 검색하거나 문서를 요약하는 일은 가능하지만, 어떤 값을 새로 쓰거나 외부 서비스의 상태를 바꾸는 일은 일절 할 수 없습니다. 도구 권한 측면에서는 select와 read 종류의 호출만 허용됩니다. 비가역 행동은 모두 막힙니다. 사용자 입장에서는 에이전트가 무엇을 보든 자신의 시스템이 변하지 않는다는 보장이 깔려 있기 때문에 신뢰의 부담이 가장 가볍습니다. 사내에 처음 에이전트를 들이는 단계, 또는 새 도메인에 에이전트를 풀어 보는 초기 단계에서 흔히 채택됩니다.

다음은 **Suggest**입니다. 이 단계에서 에이전트는 무엇을 할지 제안만 합니다. 코드를 수정하는 일을 예로 들면, 에이전트는 변경할 파일과 변경할 라인을 제시할 뿐 실제 저장은 사용자가 검토 후 직접 누릅니다. 도구 권한 측면에서는 read에 더해 변경 후보를 만들어 내는 도구가 열리지만, 만들어 낸 후보를 시스템에 적용하는 도구는 잠겨 있습니다. 비가역 행동은 여전히 막혀 있고, 모든 변경은 사용자의 명시적 승인을 거쳐야만 일어납니다. 코드 보조, 문서 초안 작성, 리포트 생성 같은 작업이 이 단계에 잘 맞습니다. 사용자는 에이전트의 제안을 한 번 본 뒤 수용하거나 거절하는 식으로 통제권을 유지합니다.

세 번째는 **Execute**입니다. 이 단계에서는 에이전트가 정해진 도구를 직접 호출해 결과를 만들어 냅니다. 다만 무엇이든 마음대로 실행할 수 있는 것은 아니고, 어떤 도구를 어떤 한도 안에서 부를 수 있는지 미리 합의된 카탈로그를 따라야 합니다. 도구 권한 측면에서는 read와 write가 모두 열리지만, 비가역 또는 영향 범위가 큰 행동은 별도의 사용자 확인을 거치도록 묶여 있습니다. 예컨대 일정 한도 안에서의 결제 처리, 사내 검색 색인 갱신, 티켓 상태 변경 같은 작업은 에이전트가 바로 수행하지만, 환불, 대량 메일 발송, 운영 데이터베이스의 schema 변경 같은 행동은 따로 제동 장치가 붙습니다. 사용자는 결과만 확인하고 잘못된 행동이 있었다면 사후에 되돌리거나 분석합니다.

마지막 단계는 **Autonomous**입니다. 이 단계의 에이전트는 사람의 즉각적 개입 없이 일련의 작업을 끝까지 수행합니다. 재시도와 도구 선택, 단계 분해, 비가역 행동의 실행이 모두 에이전트의 판단 안에서 이루어집니다. 비가역 행동도 정책상 허용된 범위 안에서는 직접 수행할 수 있지만, 대신 강한 사후 감시 장치가 함께 묶입니다. 야간 배치 작업, 자동 모니터링과 자동 복구, 백오피스의 반복 업무가 여기 해당합니다. 이 단계는 자유도가 가장 높은 만큼 잘못된 한 번이 만들어 내는 손해도 가장 큼니다. 그래서 자율 단계는 일반적으로 측정 가능한 메트릭이 일정 수준 위로 올라온 뒤에야 부여됩니다.

네 단계를 한 표로 정리하면 다음과 같습니다.

단계	가능한 행동	도구 권한	비가역 행동	사람 개입
Read-only	조회, 요약, 답변	read 전용	금지	결과만 확인
Suggest	제안 생성	read + 후보 생성	금지	매 변경 승인
Execute	정해진 도구 호출	read + write 일부	일부 허용	결과 확인 + 큰 행동 승인
Autonomous	다단계 작업 자율 수행	카탈로그 전체	정책상 허용	사후 감시

이 사다리는 한 번 정해진 뒤 고정되는 것이 아니라 시간 축을 가집니다. 새로운 도메인에 에이전트를 처음 도입할 때는 보통 Read-only나 Suggest에서 시작합니다. 평가 점수와 사용자 만족도, 사고 발생률 같은 지표가 일정 기간 동안 안정적으로 유지되면 한 칸 위로 올립니다. 반대로 회귀가 발견되거나 실패율이 임계치를 넘으면 한 칸 내려옵니다. 이런 움직임을 **사용자 신뢰 곡선**이라고 부릅니다. 신뢰는 한 번에 쌓이지 않고 같은 작업을 충분히 많이 봐 주면서 천천히 올라가지만, 한 번의 큰 사고로 빠르게 내려올 수 있습니다. 자율성 단계는 그 곡선의 현재 위치를 시스템 안에 표현해 두는 장치입니다.

마지막으로 짚을 것은 단계가 기술 결정이라기보다 **조직 정책 결정**이라는 점입니다. 같은 에이전트라도 어느 단계에 돌지는 누가 그 결과에 책임지는가, 사고가 났을 때 누구에게 알려지는가, 어떤 규제가 적용되는가에 따라 달라집니다. 한국의 금융권처럼 감사와 통제가 강한 환경에서는 같은 작업도 Suggest에서 시작해 운영 팀과 보안 팀의 합의를 거친 뒤에야 Execute로 올라가는 일이 흔합니다. 반대로 사내 코드 리뷰 보조처럼 사고의 영향이 한 사람의 수정 사항으로 한정되는 경우 처음부터 Suggest 또는 Execute로 시작해도 무방합니다. 단계 결정의 핵심은 기술적으로 무엇이 가능한가가 아니라, 사고가 났을 때 누가 감당할 수 있는가입니다.

정리하면, 자율성 단계는 에이전트가 사용자의 개입 없이 어디까지 행동할 수 있는가를 Read-only, Suggest, Execute, Autonomous 네 칸으로 나누어 표현한 사다리이며, 각 칸은 도구 권한과 비가역 행동의 허용 여부, 사람의 개입 빈도가 함께 묶여 있습니다. 단계는 한 번 정해진 뒤 고정되지 않고 사용자 신뢰 곡선과 메트릭에 따라 위아래로 움직이며, 어느 칸에 돌지는 기술 결정이라기보다 책임과 감당 범위를 묻는 조직 정책 결정입니다.

다음 단원인 4.1.2에서는 이 단계 사다리를 작업 단위로 잘게 쪼개, 같은 에이전트라도 행동의 위험도에 따라 권한을 어떻게 차등 부여할지를 다루는 위험도 매트릭스와 단계 전환 트리거를 살펴보겠습니다.

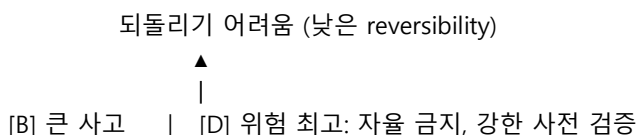
이 단원을 마치면 Read-only, Suggest, Execute, Autonomous 네 단계의 정의를 도구 권한과 비가역 행동의 허용 여부로 구분해 설명하고, 사용자 신뢰 곡선이 단계 사이에서 어떻게 움직이는지, 그리고 단계 결정이 왜 조직 정책의 문제인지를 설명할 수 있습니다.

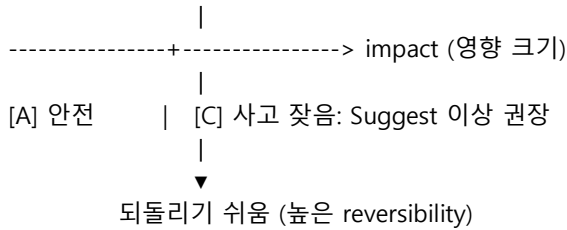
4.1.2 위험도 기반 권한 부여

앞 단원에서 본 네 단계 사다리는 에이전트 전체에 한 번 부여하고 끝낼 수 있는 라벨처럼 보이기 쉽지만, 현장에서 그렇게 운영되는 경우는 거의 없습니다. 같은 에이전트가 한쪽에서는 안전하게 행동하고 다른 쪽에서는 큰 사고를 칠 수 있기 때문입니다. 데이터베이스 조회와 결제 처리는 모두 '도구 호출'이라는 한 단어로 묶이지만, 실제로 일어나는 일과 잘못되었을 때의 결과는 전혀 다릅니다. 이 단원에서는 에이전트의 자율성 단계를 행동 단위로 잘게 쪼개고, 행동의 **위험도**에 따라 권한을 차등 부여하는 정책을 어떻게 짜는지를 다룹니다.

먼저 위험도가 무엇으로 구성되는지부터 풀겠습니다. 위험도는 한 가지 척도로 표현되지 않고, 적어도 세 축의 합으로 봐야 정확합니다. 첫째는 **데이터 변경**입니다. 어떤 행동이 시스템 안의 값을 바꾸는지, 바꾼다면 얼마나 많이 바꾸는지가 여기 들어갑니다. 한 행을 수정하는 일과 한 테이블을 일괄 갱신하는 일은 같은 update 권한 안에서 위험의 크기가 다릅니다. 둘째는 **외부 호출**입니다. 사내에서만 끝나는 행동인지, 외부 결제 처리거나 외부 SaaS의 상태를 바꾸는 행동인지를 가르는 축입니다. 외부 호출은 일단 나가면 사내 trace로 되돌리기 어렵고, 상대 시스템의 정책에 따라 사후 처리 비용이 크게 달라집니다. 셋째는 **비용**입니다. 한 번의 호출이 얼마의 토큰과 얼마의 도구 사용료를 쓰는지, 그리고 잘못 부풀려졌을 때 얼마까지 새는지가 여기 들어갑니다. 비용은 결과의 옳고 그름과 별개로 회사가 지는 짐이기 때문에 위험의 한 축으로 따로 떼어 둡니다.

위험도를 좀 더 구조화해 표현할 때 자주 쓰이는 도구가 위험도 매트릭스입니다. 두 축으로 그리는데, 가로축에는 그 행동이 만들어 내는 영향의 크기인 **impact**를 두고, 세로축에는 잘못되었을 때 되돌릴 수 있는 정도인 **reversibility**를 둡니다. 그림으로 보면 다음과 같습니다.





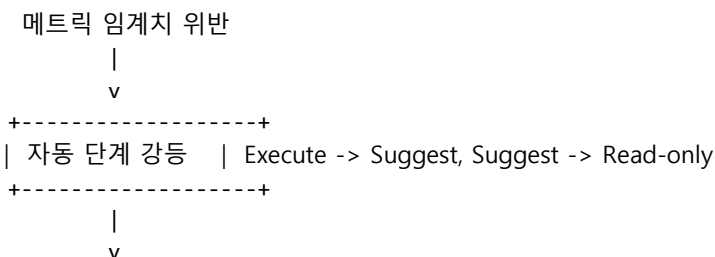
A 사분면은 영향이 작고 되돌리기도 쉬운 영역으로, 조회나 캐시 갱신 같은 행동이 여기 모입니다. Read-only 또는 Execute로 자유롭게 두어도 별 문제가 없습니다. C 사분면은 영향이 크지만 되돌리기는 어렵지 않은 영역으로, 사내 색인의 일괄 재구축 같은 작업이 여기 듭니다. 잘못되어도 수동으로 다시 빌드하면 되지만 한 번의 사고가 사용자 경험에 큰 흔적을 남기므로 Suggest 이상으로 다루는 편이 안전합니다. B 사분면은 영향은 작지만 되돌리기 어려운 영역으로, 외부 사용자에게 발송된 메일이나 결제 같은 행동이 여기 들어옵니다. 한 건이 작아 보여도 수신자에게 가닿은 뒤에는 회수가 안 됩니다. D 사분면은 두 축 모두 나쁜 영역으로, 외부 송금이나 운영 DB의 스키마 변경 같은 행동이 모입니다. 이 영역은 자율 단계에서 자동 실행을 허용하지 않고 강한 사전 검증과 사람 승인을 의무화하는 것이 일반적입니다.

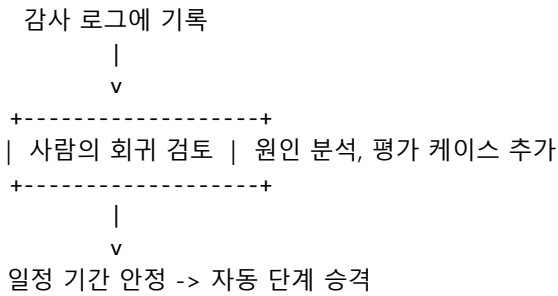
여기서 한 가지 빠지기 쉬운 함정이 있습니다. 위험도는 행동의 종류만으로 결정되지 않고 컨텍스트에 따라 달라집니다. 같은 'send_email'이라는 도구도 사내 한 명에게 보내는 알림과 사용자 1만 명에게 발송하는 마케팅 메일은 위험이 다릅니다. 같은 'update_record'도 본인이 작성한 임시 메모를 고치는 일과 다른 사람의 회계 자료를 고치는 일이 다릅니다. 그래서 권한은 도구 단위로만 매기지 않고, 도구를 부르는 인자, 호출자의 역할, 영향 받는 객체의 소유주를 함께 보고 결정합니다. 정책은 보통 다음과 같은 형태로 표현됩니다.

```
if tool == "send_email" and recipient_count <= 1 and recipient_in_company:
    단계 = Execute
elif tool == "send_email" and recipient_count <= 100:
    단계 = Suggest
else:
    단계 = HITL 강제
```

이런 분기가 누적되면 정책이 빠르게 복잡해지므로, 도구 카탈로그 옆에 '위험 클래스'라는 이름표를 붙여 두고 클래스 단위로 단계를 매핑하는 편이 관리하기 좋습니다.

같은 에이전트의 단계가 변하기도 합니다. 운영 도중 문제가 누적되면 자동으로 단계를 한 칸 내리고, 안정성이 회복되면 다시 올리는 흐름이 필요합니다. 이를 위한 장치가 **단계 전환 트리거**입니다. 트리거는 보통 메트릭의 임계치 도달로 정의됩니다. 예컨대 최근 24시간 동안 도구 호출 실패율이 5%를 넘으면 해당 도구를 Execute에서 Suggest로 자동 강등하고, 7일 동안 1% 미만으로 유지되면 다시 Execute로 자동 승격합니다. 이런 자동 강등은 사람이 모든 사고를 일일이 봐 주지 않아도 시스템이 스스로 위험한 행동을 막아 줄 수 있게 해 줍니다. 실패율 외에도 비용 폭주, 사용자 불만 신호, judge 점수 하락 같은 메트릭이 트리거 후보로 자주 쓰입니다. 한 흐름의 그림은 다음과 같습니다.





자동 강등이 동작했다는 사실 자체는 좋은 소식이지만, 그 결과를 사람이 들여다보지 않으면 의미가 절반밖에 살지 않습니다. 그래서 단계 전환은 항상 **감사 로그**에 남깁니다. 어떤 행동이 어떤 임계치를 넘겨 어떤 단계로 옮겨 갔는지, 누구의 정책이 발화했는지를 추적할 수 있게 해 두면 사고 후 회귀 검토가 가능합니다. 이 회귀 검토에서는 자동 강등이 잘못 발화한 경우를 찾고, 같은 패턴을 잡아 내는 평가 케이스를 데이터셋에 흡수합니다. 회귀 검토의 결과는 다시 다음 단계의 정책을 다듬는 데 쓰이고, 임계치도 점점 도메인에 맞게 조정됩니다.

마지막으로 강조할 것은 권한이 코드 안에서만 결정되지 않는다는 점입니다. 위험도 매트릭스, 단계 전환 트리거, 감사 로그는 모두 보안 팀, 운영 팀, 도메인 책임자의 합의 위에서 의미를 가집니다. 한국의 금융권이나 의료처럼 규제가 강한 환경에서는 단계 전환 트리거가 발화한 사실 자체를 정기 보고에 포함해야 하는 경우가 많고, 자동 승격은 사람의 명시적 승인을 거쳐야만 가능하도록 잠가 두기도 합니다. 어디까지 자동화하고 어디부터 사람의 결재를 받게 할지는 위험도와 조직 문화의 함수입니다.

정리하면, 위험도 기반 권한 부여는 데이터 변경, 외부 호출, 비용 세 축으로 위험을 분해한 뒤 impact와 reversibility의 매트릭스 위에 행동을 배치하고, 도구만이 아니라 호출 인자와 컨텍스트까지 함께 봐서 단계를 결정하며, 매트릭 임계치에 따라 자동 강등과 자동 승격이 일어나도록 단계 전환 트리거를 묶고, 모든 전환을 감사 로그로 남겨 사후 회귀 검토에 다시 쓰는 정책 묶음입니다. 이 묶음이 4.1.1의 단계 사다리와 결합될 때 비로소 같은 에이전트가 안전한 행동에서는 자유롭게 움직이고 위험한 행동에서는 단단히 묶이는 차등 자율성이 작동합니다.

다음 단원인 4.2.1에서는 위험도가 누적되어 임계치를 넘는 순간에 시스템 전체를 잠시 멈춰 세우는 안전 장치, 즉 분산 시스템의 circuit breaker 패턴이 에이전트 루프 위에서 어떻게 동작하는지를 살펴보겠습니다.

이 단원을 마치면 위험도를 데이터 변경·외부 호출·비용의 세 축으로 분해하고 impact와 reversibility의 매트릭스로 행동을 분류하며, 동일 작업이라도 컨텍스트에 따라 권한을 차등 부여하는 정책을 설계하고, 매트릭 임계치 기반의 단계 전환 트리거와 감사 로그·회귀 검토 흐름을 연결해 설명할 수 있습니다.

4.2 Circuit Breaker

이 섹션의 토픽과 학습 목표는 다음과 같습니다.

4.2.1 Circuit Breaker 동작 원리 — 분산 시스템의 circuit breaker 패턴이 에이전트에 적용될 때의 동작을 설명할 수 있다

4.2.2 에이전트 임계치 설계 — 에이전트별로 적절한 차단 임계치를 설정하고 회복 전략을 설명할 수 있다

4.2.1 Circuit Breaker 동작 원리

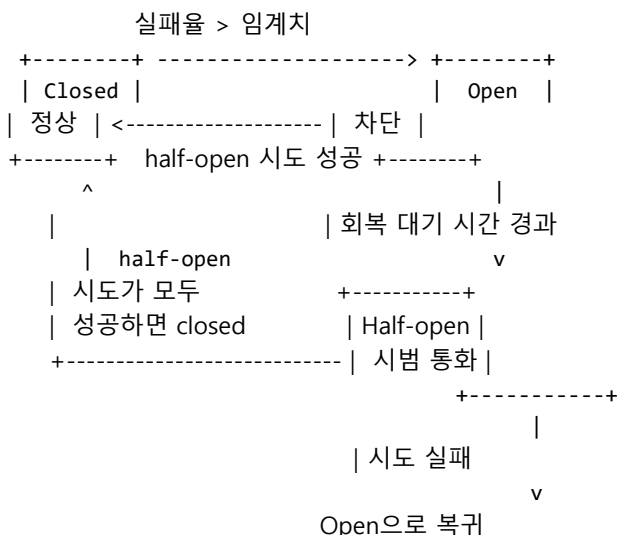
앞 단원에서 본 *tiered autonomy*는 에이전트에게 어디까지 자율을 허용할지 미리 정해 두는 정적 권한 설계였습니다. 그런데 정상 권한 안에서 움직이던 에이전트도, 운영 도중에 외부 환경이 일시적으로 무너지거나 자기 자신이 *planning loop*에 빠지면 권한이 있다는 이유만으로 계속 시도해서는 안 됩니다. 시도가 시도를 부르며 비용을 태우고, 외부 시스템에 부하를 더 얹는 일이 자주 일어납니다. 이 단원이 다루는 *circuit breaker*는 그런 상황에서 시도 자체를 잠시 끊어 두는 장치입니다.

먼저 이름의 출발점부터 풀겠습니다. **Circuit breaker**는 원래 전기 공학에서 온 말로, 한국어로는 차단기에 해당합니다. 회로에 과전류가 흐르면 전선이 녹거나 화재가 일어나기 전에 회로 자체를 끊어 두었다가, 안전이 확인된 다음에 다시 붙이는 부품입니다. 분산 시스템 분야에서는 마이클 나이가드의 책 'Release It!'이 이 비유를 빌려 와, 외부 호출이 줄지어 실패할 때 호출 자체를 막아 두는 패턴에 같은 이름을 붙였습니다. 호출을 막는 까닭은 두 가지입니다. 첫째, 어차피 실패할 호출에 시간을 쓰지 않기 위해서이고, 둘째, 이미 휘청거리는 외부 시스템에 더 부하를 얹어 회복을 늦추지 않기 위해서입니다.

전통적 마이크로서비스 환경에서 가장 자주 인용되는 사례가 넷플릭스의 **Hystrix**입니다. Hystrix는 한 서비스가 다른 서비스를 부를 때 그 호출을 감싸 두고, 일정 시간 안에 실패율이 임계치를 넘으면 같은 호출을 한동안 자동으로 실패 처리하는 라이브러리였습니다. 이렇게 하면 추천 서비스가 일시적으로 느려져도 그 호출에 매여 있던 *thread*들이 풀려 본 페이지의 다른 영역이 정상 응답을 이어 갑니다. 이 패턴이 자리 잡으면서 Resilience4j나 Spring Cloud Circuit Breaker 같은 후속 구현이 등장했고, 마이크로서비스 신뢰성 패턴의 표준 어휘 가운데 하나가 되었습니다.

이 패턴의 동작은 세 가지 상태로 정리됩니다. **Closed, Open, Half-open**입니다. 이름은 전기 회로의 닫힘과 열림에서 그대로 가져왔습니다. 회로가 닫혀 있다는 말은 전류가 흐른다는 뜻이므로, *closed*가 곧 정상 호출 허용 상태입니다. 반대로 *open*은 회로가 끊겨 호출이 막힌 상태이고, *half-open*은 끊긴 회로를 살짝만 다시 이어 붙여 한두 번의 시도로 안부를 묻는 상태입니다.

세 상태 사이의 전이를 그림으로 보면 다음과 같습니다.



흐름을 차례로 따라 보겠습니다. 평소에는 회로가 *closed* 상태에 있고, 모든 호출이 정상으로 외부에 도달합니다. *circuit breaker*는 그 호출들의 결과를 옆에서 세고 있다가, 정해진 시간 창 안에서의 실패율이 임계치를 넘는 순간 회로를 *open*으로 돌립니다. 일단 *open*이 되면 그 시점부터 들어오는 호출은 외부에 닿지 않고 즉시 *fallback* 응답을 받습니다. 이 즉시 차단이 호출 측의 *thread*를 풀어 주고, 외부 측의 부하를 줄여 주는 효과를 동시에 냅니다.

open 상태가 영원히 이어지면 외부가 회복되어도 그 사실을 알 길이 없습니다. 그래서 일정 시간이

지나면 회로가 half-open으로 옮겨갑니다. half-open은 닫혔다 열렸다는 결정하기 전의 시범 운행 상태입니다. 정해진 횟수만큼만 호출을 외부로 흘려 보내고, 그 결과를 본 뒤 다음 결정을 내립니다. 시도가 모두 성공하면 외부가 회복되었다고 보고 closed로 돌아가고, 시도 가운데 실패가 있으면 다시 open으로 돌아가 회복 대기 시간을 한 번 더 기다립니다.

여기서 두 가지 임계치가 함께 등장합니다. 하나는 closed에서 open으로 넘어가는 기준인 **장애 임계치**이고, 다른 하나는 open에서 half-open으로 넘어가는 기준인 **회복 시간**입니다. 전자는 보통 시간 창 안의 실패율이나 연속 실패 횟수로 정의됩니다. 후자는 단순히 일정 시간을 두고 기다리는 형태가 흔하고, 부하 회복 추정치 더 정교한 시스템에서는 점진적으로 늘어나는 backoff를 함께 씁니다.

이 그림을 에이전트로 옮기면 무엇이 달라질까요. 외부 API 호출의 실패율이 임계치를 넘는 고전적 트리거는 그대로 살아 있지만, 에이전트에서는 거기에 더해 **루프 안에서 솟아오르는 비용과 단계 수** 자체가 트리거가 됩니다. 1.2.2에서 본 planning loop를 떠올려 보겠습니다. 같은 검색 도구를 거의 같은 인자로 반복해 부르는 에이전트가 매 단계마다 컨텍스트를 부풀려 가며 모델을 호출합니다. 외부에서 보면 호출은 모두 200으로 성공하지만, 에이전트 내부에서 보면 한 사용자 요청이 만들어 내는 누적 토큰과 누적 비용이 빠르게 임계치에 다가섭니다. 이 누적치를 회로의 입력으로 두면, 일정 수준을 넘긴 사용자 요청에 한해 회로를 open으로 돌려 추가 단계의 추론과 도구 호출을 모두 fallback으로 처리할 수 있습니다.

또 하나의 트리거는 **도구 단위 실패율**입니다. 특정 도구가 짧은 시간 동안 연속으로 오류를 돌려준다면, 그 도구를 호출하는 모든 trajectory에 회로를 걸어 한동안 그 도구 자체를 비활성으로 둡니다. 이렇게 하면 망가진 도구를 두고 모델이 매번 다시 시도하느라 시간을 태우는 일을 막고, 외부 도구가 회복할 시간도 벌여 줍니다.

회로가 open이 된 동안 사용자에게는 무엇을 보여 줄지가 그 다음 결정입니다. 차단이 가시화되지 않으면 사용자는 에이전트가 멍하니 서 있는 것처럼 느낍니다. 그래서 차단 시 사용자 메시지를 미리 정해 둡니다. 이 메시지는 세 가지 정보를 담는 편이 좋습니다. 첫째, 시스템이 정상이며 일시적 보호 조치라는 사실을 짧게 알리고, 둘째, 사용자가 다시 시도할 수 있는 시점이나 대안 경로를 안내하며, 셋째, 가능한 경우 부분 결과나 캐시된 직전 결과라도 함께 노출합니다. 예를 들어 검색 도구가 차단된 상태라면 '실시간 검색이 일시적으로 중단되어 직전 캐시 결과를 보여 드립니다'와 같은 문구가 그 자리를 채웁니다.

이 단원에서 한 가지 더 짚어 둘 점은, circuit breaker는 단독으로 신뢰성을 보장하지 않는다는 사실입니다. 회로가 자주 열린다는 사실 자체는 시스템이 어딘가에서 망가지고 있다는 신호일 뿐, 그 원인을 회로가 알려 주지는 않습니다. 그래서 회로의 open 횟수와 회복 시간 분포는 다음 단원에서 본격적으로 다룰 임계치 설계의 입력이자, 3장에서 본 관측 스택 위에 메트릭으로 올려 두어야 하는 신호입니다.

정리하면, circuit breaker는 외부 호출이 줄지어 실패하거나 에이전트 루프 자체가 비용·단계를 폭주시킬 때 시도를 잠시 끊어 두었다가 안부를 물어 다시 잇는 패턴이며, closed-open-half-open 세 상태 사이를 장애 임계치와 회복 시간이라는 두 기준이 오가게 합니다. 전통 마이크로서비스의 Hystrix가 외부 API 실패율 위에 올라간 형태였다면, 에이전트의 회로는 그 위에 누적 비용·스텝 수·도구 실패율을 트리거로 더해 비결정적 루프의 폭주를 끊어 줍니다.

다음 단원인 4.2.2에서는 이 회로를 실제로 어디에 어떤 값으로 걸지를 다룹니다. 스텝·토큰·비용 임계치, 도구 실패율, 사용자별 분리, 시간 윈도우의 모양, half-open에서 몇 번을 시도할지 같은 구체적 설계 결정으로 들어가겠습니다.

이 단원을 마치면 circuit breaker 패턴이 전기 차단기에서 출발해 분산 시스템의 Hystrix를 거쳐 에이전트로 이어진 흐름을 설명할 수 있고, closed-open-half-open 세 상태와 장애 임계치·회복 시간이라는

두 임계치가 어떻게 맞물리는지, 에이전트 루프에서 비용과 planning loop가 트리거가 되는 까닭과 차단 시 사용자에게 무엇을 보여 줄지를 함께 설명할 수 있습니다.

4.2.2 에이전트 임계치 설계

앞 단원에서 closed-open-half-open 세 상태와 장애 임계치·회복 시간이라는 두 기준을 정리했습니다. 이 단원은 그 임계치에 실제로 어떤 숫자를 어떤 단위로 박아 둘지를 다룹니다. 임계치는 너무 뻑뻑하면 정상 작업이 자주 끊기고, 너무 헐거우면 폭주를 막지 못합니다. 그래서 임계치는 한 번 정해 두고 잇는 상수가 아니라, 측정과 조정을 반복하며 다듬는 운영 변수에 가깝습니다.

에이전트의 회로에 올릴 첫 번째 입력은 **스텝 수**, **토큰**, **비용**이라는 세 가지 누적 지표입니다. 1.2.2에서 본 planning loop의 폭주를 다시 떠올려 보겠습니다. 한 사용자 요청이 만들어 내는 단계가 늘어날수록 컨텍스트가 누적적으로 부풀고, 그에 따라 토큰과 비용도 함께 솟습니다. 회로의 입력이 외부 호출 실패율 하나뿐이라면 이런 폭주는 호출이 모두 200으로 끝나는 한 보이지 않습니다. 그래서 에이전트의 회로는 적어도 다음 네 종류의 누적치를 지켜봅니다.

임계치 종류	의미	기본 보호 대상
스텝 수	한 요청의 단계 수가 N을 넘으면 차단	루프 형태 폭주
누적 토큰	총 입력·출력 토큰이 T를 넘으면 차단	컨텍스트 비대 폭주
누적 비용	비용이 C달러를 넘으면 차단	결제·예산 폭주
경과 시간	경과가 S초를 넘으면 차단	사용자 대기 시간

각 임계치는 서로를 보완합니다. 스텝 수만 두면 한 단계가 거대한 컨텍스트를 받는 경우의 비용 폭주가 빠지고, 비용만 두면 짧은 단계 안에서 큰 호출이 한 번 일어나는 정상 작업이 잘못 잘립니다. 그래서 네 임계치를 동시에 걸어 두고, 어느 하나라도 닿으면 회로가 open으로 넘어가는 구조가 보통 사용됩니다.

값을 정하는 절차도 짚어 두겠습니다. 정상 작업의 분포를 한동안 측정해 95퍼센타일이나 99퍼센타일을 기준선으로 잡고, 그 위에서 잘리는 사례를 따로 모아 봅니다. 잘린 사례 가운데 정상 작업이 섞여 있다면 임계치를 약간 올리고, 폭주만 깔끔하게 잘려 있다면 그대로 둡니다. 이 과정은 한 번에 끝나지 않고, 사용자 트래픽이 변하거나 모델이 바뀔 때마다 다시 측정해 조정해야 합니다.

두 번째 입력은 **도구 실패율**입니다. 외부 API가 일시적으로 흔들리거나 사내 도구의 사양이 바뀌었을 때, 그 도구를 부르는 모든 trajectory가 차례로 실패합니다. 도구별 실패율을 시간 창 위에 두고 임계치를 넘으면 그 도구만 골라 회로를 open으로 돌립니다. 즉 회로는 사용자 요청 단위에만 걸리는 것이 아니라, 도구 단위로도 걸립니다. 이렇게 두면 망가진 도구를 모델이 반복적으로 부르는 일이 차단되고, 그 동안 다른 도구로 작업이 이어지거나 fallback 응답이 사용자에게 안내됩니다. 도구 단위 회로는 다음 단원에서 다룰 fallback routing의 출발점이 됩니다.

세 번째 입력은 **사용자별 임계치 분리**입니다. 모든 사용자에게 같은 비용 한도를 박아 두면 두 가지 문제가 동시에 생깁니다. 첫째, 정당하게 큰 작업을 자주 돌리는 엔터프라이즈 사용자는 자주 잘립니다. 둘째, 무료 평가판 사용자가 비정상적인 폭주를 일으켜도 한도가 헐거워 막지 못합니다. 그래서 임계치를 사용자 등급, 워크스페이스, 또는 API 키 단위로 분리해 둡니다. 사용자별 회로 상태를 유지하기 위해 회로 식별자에 사용자 식별자를 함께 넣는 형태가 흔합니다. 슈도코드로 보면 다음과 같습니다.

```
def call_with_breaker(user_id, action):
    key = (user_id, action.tool_name)
    state = breaker_state[key]
    if state == OPEN and now < state.reopen_at:
        return fallback_response(action)
```

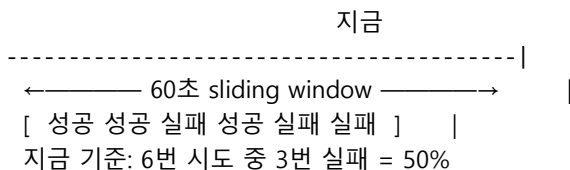
```

try:
    result = run(action)
    record_success(key)
    return result
except Exception:
    record_failure(key)
    if failure_rate(key, window=60s) > threshold(user_id):
        open_circuit(key, cool_down=cooldown(user_id))
    raise

```

이 슈도코드는 두 가지 사실을 보여 줍니다. 회로의 키가 사용자와 도구를 함께 묶고 있다는 점, 그리고 임계치와 회복 시간이 사용자 등급에 따라 달라질 수 있다는 점입니다. 후자는 같은 도구의 같은 실패라도 사용자 등급에 따라 다르게 보호한다는 뜻입니다.

네 번째 입력은 **시간 윈도우의 모양**입니다. 실패율은 본질적으로 어떤 시간 구간 안의 비율입니다. 그래서 그 구간을 어떻게 자르느냐에 따라 같은 사건이 다른 비율로 보입니다. 가장 단순한 방법은 고정 윈도우, 즉 한 시각에서 다른 시각까지를 잘라 그 안의 호출만 보는 형태입니다. 그런데 고정 윈도우는 경계 부근에서 신호가 갑자기 사라지거나 솟는 경계 효과를 만듭니다. 예를 들어 60초 단위 윈도우라면 59초까지 누적된 실패가 60초가 되는 순간 사라지고, 새 윈도우가 0에서 다시 출발합니다. 이 효과를 줄이기 위해 **sliding window**, 즉 현재 시각에서 일정 시간만큼 거꾸로 거슬러 그 구간 안의 호출을 매년 다시 세는 방식이 자주 사용됩니다.



이 그림에서 회로의 입력은 매 호출마다 다시 계산됩니다. 시간이 흐를수록 왼쪽 끝의 오래된 호출은 자연스럽게 빠져나가고, 오른쪽 끝에 새 호출이 더해집니다. 구현 비용을 더 줄이고 싶다면 토큰 버킷 형태로 근사하는 방법도 있고, 호출량이 많은 시스템에서는 윈도우를 잘게 쪼갠 구간별 카운터를 합산하는 방식이 흔히 쓰입니다.

마지막 입력은 **half-open 시 시도 횟수**입니다. 회로가 open에서 회복 대기 시간을 보낸 다음 half-open으로 옮겨갈 때, 거기서 몇 번을 시도해 볼지를 결정해야 합니다. 한 번만 통과하면 closed로 돌리는 설정은 회복 판단이 빠르지만, 그 한 번이 운 좋게 성공한 호출이라면 곧이어 다시 open으로 떨어집니다. 반면 시도 횟수를 많이 두면 판단이 정교해지지만, 외부가 아직 회복되지 않았을 때 그 동안의 시도가 모두 실패로 돌아가 사용자 경험이 나빠집니다. 보통은 두 길의 절충으로 두세 번의 시도를 두고, 그 시도 중 한 번이라도 실패하면 즉시 open으로 되돌리는 설계가 안정적입니다. 시도 사이 간격은 외부 시스템에 부하를 다시 주지 않도록 짧은 backoff를 두는 편이 좋습니다.

이 모든 입력을 한 자리에 모아 보면 임계치 설계는 결국 다음 다섯 가지를 정하는 일이 됩니다. 첫째, 무엇을 누적해서 볼 것인가(스텝·토큰·비용·시간·실패율). 둘째, 어떤 단위로 회로를 분리할 것인가(요청·도구·사용자). 셋째, 어떤 시간 창 위에서 비율을 계산할 것인가(고정·sliding). 넷째, 회로가 open된 뒤 얼마를 기다릴 것인가(회복 대기 시간). 다섯째, half-open에서 몇 번을 어떻게 시도할 것인가.

마지막으로 한 가지 강조해 둘 점은, 임계치 설계의 결과는 그 자체가 관측 대상이라는 사실입니다. 회로가 얼마나 자주 open되는지, 어떤 사용자에서 얼마나 자주 도달하는지, half-open에서 얼마나 자주 다시 떨어지는지를 메트릭으로 남기지 않으면 임계치를 어디로 옮길지 판단할 근거가 사라집니다. 3장에서 본 관측 스택의 trace와 metric 위에 회로의 상태 변화 자체를 이벤트로 부착해 두면, 사후에

임계치 조정 회의의 입력이 그대로 갖춰집니다.

정리하면, 에이전트의 회로 임계치는 스텝·토큰·비용·시간이라는 누적 지표와 도구별 실패율을 입력으로 두고, 사용자 등급에 따라 회로를 분리하며, 고정정보는 sliding window 위에서 비율을 계산하고, open 이후 회복 대기 시간을 둔 다음 half-open에서 두세 번의 시도로 회복 여부를 판단하는 다섯 결정의 묶음입니다. 모든 결정은 정상 작업 분포의 측정과 회로 자체의 메트릭을 통해 끊임없이 다듬어집니다.

다음 단원인 4.3.1에서는 회로가 차단으로 끝난 그 자리를 무엇으로 채울지를 본격적으로 다룹니다. 모델·도구·응답 세 층위에서 fallback을 어떻게 분류하고, 어디까지 정적으로 두고 어디부터 동적으로 골라야 할지를 살펴보겠습니다.

이 단원을 마치면 스텝·토큰·비용 임계치와 도구 실패율을 회로의 입력으로 함께 두는 까닭, 사용자 등급에 따라 회로를 분리하는 방법, sliding window가 고정 윈도우의 경계 효과를 어떻게 풀어 주는지, 그리고 half-open 시 시도 횟수와 backoff를 어떻게 정해야 하는지를 임계치 설계의 다섯 결정으로 묶어 설명할 수 있습니다.

4.3 Fallback Routing

이 섹션의 토픽과 학습 목표는 다음과 같습니다.

4.3.1 Fallback의 종류 — 에이전트 시스템의 fallback 패턴을 모델, 도구, 응답 세 층위로 분류하여 설명할 수 있다

4.3.2 모델 라우팅 전략 — Task type, cost, accuracy를 축으로 하는 모델 라우팅 전략을 JPMorgan 사례로 설명할 수 있다

4.3.3 Graceful Degradation — 전체 서비스가 멈추지 않도록 점진적으로 기능을 축소하는 graceful degradation을 설계할 수 있다

4.3.1 Fallback의 종류

앞 단원까지 우리는 자율성 단계를 차등으로 나누는 패턴과, 위험이 임계치를 넘었을 때 전체 호출을 멈추는 회로 차단기 패턴을 살펴보았습니다. 두 패턴 모두 위험한 상황을 막는 데 무게를 둡니다. 그런데 막은 다음에는 무엇을 해야 할까요. 회로가 열려서 주 모델 호출을 차단했는데도 사용자는 여전히 답을 기다리고 있고, 화면에는 결과가 떠야 합니다. 이때 시스템이 무언가 의미 있는 응답을 돌려주려면, 차단된 경로 대신 쓸 수 있는 대체 경로가 미리 마련되어 있어야 합니다. 이 대체 경로를 가리키는 이름이 **fallback**입니다.

먼저 용어부터 풀겠습니다. 일상에서는 '실패했을 때 기댈 곳'이라는 뜻으로 쓰입니다. 가지고 다니는 신용카드가 결제 단말기에서 거절되면 다른 카드를 꺼내거나 현금으로 결제하는 식입니다. 이때 두 번째 카드와 현금이 결제의 fallback에 해당합니다. 시스템 설계에서도 같은 비유가 그대로 쓰입니다. 주 경로가 실패하거나 사용할 수 없을 때, 미리 정해 둔 대체 경로로 흐름을 돌려 서비스가 멈추지 않게 합니다. 에이전트 시스템에서 fallback은 보통 세 층위에 걸쳐 설계됩니다. 모델 층위, 도구 층위, 그리고 응답 층위입니다.

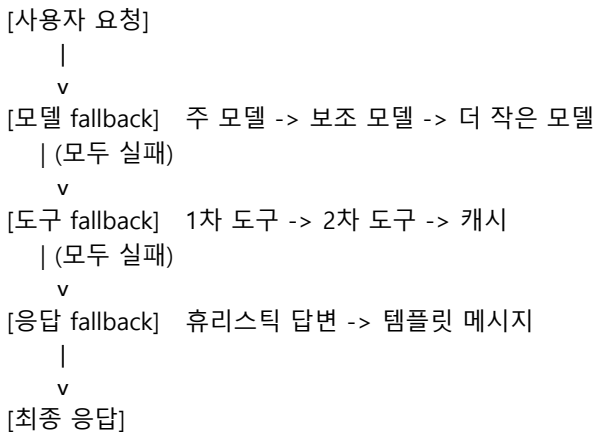
첫째는 **모델 fallback**입니다. 에이전트가 호출하는 주 모델이 응답하지 않거나 한도를 넘겼을 때, 다른 모델로 흐름을 돌리는 패턴을 가리킵니다. 모델 fallback의 트리거는 보통 세 가지로 나뉩니다. 가용성, 비용, 품질입니다. 가용성은 사업자의 장애로 일정 시간 응답이 끊기거나, rate limit에 걸려 호출 자체가 거부되는 경우에 발동됩니다. 비용은 주 모델 단가가 너무 높아 특정 요청에는 더 싼 모델로 돌리는

경우입니다. 품질은 정반대 방향으로, 짧고 단순한 요청에는 작은 모델을 쓰다가 복잡한 요청에서 신뢰도가 임계 아래로 떨어지면 더 큰 모델을 호출하는 경우입니다. 세 트리거 모두 같은 fallback 메커니즘을 공유하지만, 어떤 신호로 발동되느냐가 다릅니다.

둘째는 **도구 fallback**입니다. 에이전트가 외부 검색 API, 데이터베이스 쿼리, 사내 RAG 인덱스 같은 도구를 호출할 때, 주 도구가 실패하면 보조 도구로 옮겨 가는 패턴입니다. 예를 들어 사내 지식 검색을 일차로 시도하고, 결과가 비어 있으면 웹 검색을 시도하고, 그것도 실패하면 캐시에 저장된 최근 답변을 돌려주는 식입니다. 도구 fallback에서 중요한 것은 같은 작업을 수행하지만 신뢰성과 비용이 다른 도구들을 미리 등급별로 묶어 두는 일입니다. 1차, 2차, 3차로 정해 둔 사다리를 따라 차례로 시도해야 흐름이 일관됩니다.

셋째는 **응답 fallback**입니다. 모델도 도구도 모두 답을 돌려주지 못했을 때 마지막으로 사용자에게 보여 줄 응답을 만드는 층위입니다. 가장 단순한 형태는 미리 작성해 둔 **템플릿** 메시지입니다. '지금은 답변을 드리기 어렵습니다. 잠시 후 다시 시도해 주세요'와 같은 정형 문구가 여기 들어갑니다. 그보다 한 단계 정교한 형태는 **휴리스틱**, 즉 모델 호출 없이 규칙으로 만드는 응답입니다. 사용자의 질문에서 키워드를 뽑아 가장 자주 묻는 답변 모음에서 매칭되는 항목을 보여 주거나, 마지막 성공 응답을 캐시에서 꺼내 보여 주는 식입니다. 응답 fallback은 품질이 높지는 않지만, 사용자가 빈 화면을 보지 않게 한다는 한 가지 목적에 충실합니다.

세 층위를 그림으로 정리하면 다음과 같이 보입니다. 사용자의 한 요청이 들어오면 시스템은 위에서부터 차례로 시도하다가, 어느 한 곳에서 답이 만들어지면 거기서 멈추고 응답을 돌려줍니다.



이렇게 여러 층위와 여러 단계를 차례로 묶은 구조를 **fallback chain**이라고 부릅니다. 이름 그대로 시도가 사슬처럼 이어집니다. 사슬은 든든해 보이지만, 길어질수록 두 가지 비용이 누적됩니다. 첫째는 **지연**입니다. 1차에서 시간 초과가 나야 2차로 넘어가고, 2차에서도 같은 패턴이 반복되면 사용자가 느끼는 응답 시간은 단계마다 더해집니다. 둘째는 **품질 저하**입니다. 사슬이 깊이 내려갈수록 응답은 점점 정형화되거나 정보량이 줄어듭니다. 마지막에 닿으면 결국 템플릿 한 줄에 머물게 됩니다. 따라서 fallback chain은 무한정 늘릴수록 좋은 것이 아니라, 어디까지 내려갈지를 선택해서 끊어 두는 설계가 필요합니다. 차단해야 할 시점의 임계치는 앞 단원에서 다른 회로 차단기와 같이, 시간·비용·연속 실패 횟수로 정해둡니다.

마지막으로 **정적 fallback**과 **동적 fallback**의 차이를 짚겠습니다. 정적 fallback은 모든 경로와 순서를 사전에 고정해 두는 방식입니다. 1차는 무엇, 2차는 무엇, 그다음은 무엇이라고 코드나 설정에 적혀 있고 런타임에 바뀌지 않습니다. 구현이 단순하고 동작이 예측 가능하지만, 상황에 따라 더 좋은 선택이 가능해도 그것을 쓰지 못합니다. 동적 fallback은 런타임에 시그널을 보고 다음 경로를 결정합니다. 예를 들어 특정 사업자의 응답 지연이 평소보다 길어지면 그 사업자를 자동으로 후순위로 내리고, 비용 한도가

임박하면 더 싼 모델을 우선으로 끌어올리는 식입니다. 동적 fallback은 유연하지만 결정 자체가 또 하나의 시스템이 되어, 어떤 신호가 어떤 결정을 만들었는지 관측하지 못하면 디버깅이 어렵습니다. 다음 단원에서 다룰 모델 라우팅 전략은 사실상 이 동적 fallback을 더 일반화한 형태로 볼 수 있습니다.

정리하면, fallback은 주 경로가 실패했을 때 시스템이 빈 응답을 돌려주지 않도록 미리 마련해 둔 대체 경로이며, 모델 fallback은 가용성·비용·품질 신호로 다른 모델로 옮겨 가는 패턴, 도구 fallback은 같은 작업을 수행하는 보조 도구로 옮겨 가는 패턴, 응답 fallback은 템플릿과 휴리스틱으로 마지막 선을 지키는 패턴입니다. 이들을 차례로 엮은 fallback chain은 길어질수록 지연과 품질 저하를 누적시키므로, 어디서 끊을지를 임계치로 미리 정해 두어야 하며, 정적 방식은 단순하고 예측 가능하지만 동적 방식은 런타임 신호로 더 나은 경로를 고르는 대신 결정 자체를 관측해야 한다는 비용을 지불합니다.

다음 단원인 4.3.2에서는 이 동적 fallback을 한 단계 더 일반화한 모델 라우팅 전략을 살펴봅니다. 단일 API 뒤에 여러 사업자의 모델을 두고 task type, cost, accuracy를 축으로 매 요청마다 모델을 고르는 구조를 JPMorgan 사례와 함께 풀어 보겠습니다.

이 단원을 마치면 에이전트 시스템의 fallback을 모델, 도구, 응답이라는 세 층위로 분류하고, 각 층위에서 어떤 트리거로 어떤 대체 경로가 발동되는지를 설명할 수 있으며, fallback chain을 무한정 늘릴 때 발생하는 한계와 정적·동적 fallback의 차이를 비교할 수 있습니다.

4.3.2 모델 라우팅 전략

앞 단원에서 본 fallback이 '주 경로가 실패했을 때 다른 길로 돌리는 사후 대응'이었다면, 이번 단원에서 다루는 **모델 라우팅**은 '요청이 들어온 순간 어떤 모델로 보낼지를 미리 고르는 사전 결정'입니다. 동적 fallback을 한 걸음 일반화한 형태이며, 매 요청마다 어떤 사업자의 어떤 모델을 호출할지 고르는 일을 시스템화한 것입니다. 라우팅을 잘 짜 두면 fallback이 발동되는 빈도 자체가 줄고, 비용과 품질을 함께 관리할 수 있는 손잡이가 생깁니다.

먼저 라우팅이라는 용어부터 풀겠습니다. 네트워크 장비에서 패킷이 목적지에 가장 잘 닿도록 다음 hops를 고르는 작업을 라우팅이라고 부릅니다. 모델 라우팅도 같은 비유입니다. 사용자의 한 요청이 들어왔을 때, 그 요청을 어느 모델로 보내야 가장 적절한 답이 나올지 결정하는 작업입니다. 다만 네트워크에서는 거리와 지연이 결정 기준이지만, 모델 라우팅에서는 task type, cost, accuracy 같은 새로운 축이 더해집니다.

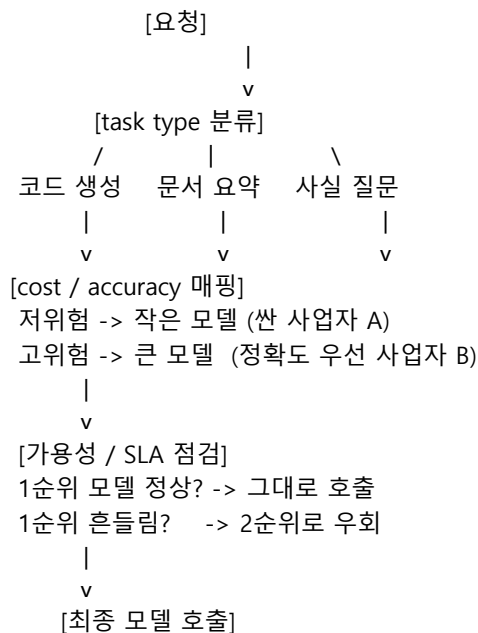
라우팅의 가장 잘 알려진 산업 사례는 미국 은행 JPMorgan의 LLM Suite입니다. 이 회사는 약 25만 명에 이르는 직원이 쓰는 사내 AI 플랫폼을 운영하고 있는데, 여기에 적용된 핵심 패턴이 바로 단일 API 뒤에 여러 사업자의 모델을 두는 구조입니다. 사례 자료에는 '단일 API 뒤에서 OpenAI·Anthropic 모델 선택 가능, task type·cost·accuracy 기준으로 라우팅'이라고 정리되어 있습니다. 사내 애플리케이션 개발자들은 모델 사업자가 누구인지 알 필요 없이 LLM Suite의 단일 엔드포인트만 호출하고, 어느 모델로 갈지는 플랫폼이 결정합니다. 이 방식은 사업자가 바뀌거나 새 모델이 추가되어도 클라이언트 쪽 코드를 손대지 않고 갈아 끼울 수 있게 해 줍니다. 운영 측면에서도 과금과 관측이 한 곳에 모이기 때문에, 회사 전체가 어떤 모델에 얼마를 쓰고 있는지 한눈에 볼 수 있습니다.

이 추상화가 작동하려면 라우팅 결정을 내리는 부분이 필요합니다. 그 가운데 첫째 축은 **task type 분류기**입니다. 들어온 요청이 어떤 종류의 작업인지를 짧게 판별해서 그 종류에 맞는 모델로 보내는 역할을 합니다. 예를 들어 코드 생성, 긴 문서 요약, 사실 질문 답변, 정형 데이터 추출은 각각 잘하는 모델이 다릅니다. 분류기는 별도의 작은 모델로 구현하기도 하고, 정규식이나 키워드 규칙으로 만들기도 합니다. 분류 결과는 보통 라벨 한두 개로 떨어지고, 그 라벨이 라우팅 테이블의 키가 됩니다.

둘째 축은 **cost-accuracy 트레이드오프**입니다. 같은 task type이라도 사용 가능한 모델이 여러 개 있을 때, 비용이 낮은 모델로도 충분한 요청과 비싼 모델이 필요한 요청을 갈라야 합니다. 단순한 일상 질문은 작은 모델로도 충분하지만, 회계 계산이나 법무 검토 같은 작업은 정확도가 떨어지면 손해가 큼니다. 따라서 같은 분류 라벨 안에서도 요청의 길이, 위험도, 사용자 등급에 따라 모델을 다시 한 번 가르는 두 번째 결정이 들어갑니다. 이 트레이드오프는 보통 표 한 장으로 정리되며, 각 칸은 'task 라벨 × 위험 수준'에 매핑된 모델 이름을 담습니다.

셋째 축은 **모델 가용성과 SLA**입니다. 사업자별로 응답 시간 보장 수준과 장애 빈도가 다릅니다. 라우팅 시점에 각 모델의 최근 응답 지연, 에러율, rate limit 잔량을 살펴보고, 1순위 모델이 흔들리고 있다면 2순위 모델로 자동으로 옮기는 결정도 필요합니다. 이 결정은 앞 단원에서 본 동적 fallback과 사실상 같은 메커니즘이며, 다른 점은 fallback이 실패 후에 발동되는 데 비해 라우팅은 실패 직전의 신호로 미리 옮긴다는 것입니다.

세 축을 묶어 라우팅 결정을 그림으로 그리면 다음과 같습니다. 같은 입구에서 출발한 요청이 분류, 비용·정확도 매핑, 가용성 점검이라는 세 단계의 분기를 거쳐 최종 모델로 보내집니다.



라우팅 시스템에서 흔히 빠지기 쉬운 함정은, 결정이 어디서 어떻게 일어났는지 사후에 알 수 없게 만드는 일입니다. 같은 요청이 어떤 날에는 사업자 A로, 다른 날에는 사업자 B로 갔는데 그 이유가 기록되지 않으면, 품질이 떨어졌을 때 원인을 추적할 길이 없어집니다. 따라서 **라우팅 결정의 관측**은 라우팅 시스템의 일부로 처음부터 함께 설계해야 합니다. 매 요청 trace에 어떤 분류 라벨이 붙었고, 어떤 비용·위험 매핑이 적용됐고, 가용성 점검 결과 1순위가 정상이었는지 우회였는지를 함께 남깁니다. 앞에서 본 OpenTelemetry GenAI semantic conventions의 gen_ai.system, gen_ai.request.model 속성이 바로 이 자리에서 빛을 발합니다. 사업자와 모델 이름이 표준 키로 trace에 박혀 있으면, 나중에 '품질이 떨어진 시점에 어떤 모델로 라우팅됐는가'를 한 번의 쿼리로 확인할 수 있습니다.

라우팅을 도입할 때 한국 기업 입장에서 참고할 점이 한 가지 더 있습니다. JPMorgan 사례에서 단일 API의 진짜 가치는 모델 사업자 변경의 영향 범위를 클라이언트로부터 차단한다는 데 있습니다. 새 모델이 나오거나 사업자 한 곳의 단가가 바뀌어도, 사내 수천 개의 응용 프로그램은 코드를 손대지 않고 그대로 돌아갑니다. 이 추상화는 처음 만들 때는 한 줄짜리 호출보다 무거워 보이지만, 도입 후 1, 2년이 지나면 운영 비용이 급격히 줄어듭니다.

정리하면, 모델 라우팅은 매 요청마다 어떤 사업자의 어떤 모델을 호출할지 사전에 결정하는 시스템이며, JPMorgan의 LLM Suite처럼 단일 API 뒤에 여러 사업자의 모델을 두고 task type, cost, accuracy를 축으로 결정합니다. task type 분류기가 요청의 종류를 판별하고, 비용·정확도 매핑이 같은 종류 안에서 모델을 다시 가르며, 가용성과 SLA 점검이 1순위 모델의 흔들림을 보고 자동으로 우회 경로를 고릅니다. 라우팅 결정은 trace에 함께 남겨야 사후 디버깅이 가능하며, 단일 API 추상화는 사업자 교체 시 클라이언트 영향을 차단해 장기적인 운영 비용을 줄입니다.

다음 단원인 4.3.3에서는 fallback과 라우팅을 모두 거처도 시스템이 회복할 수 없는 상황이 왔을 때, 전체 서비스가 한꺼번에 무너지는 대신 기능을 단계적으로 줄이며 버티는 graceful degradation을 다룹니다.

이 단원을 마치면 모델 라우팅을 동적 fallback의 일반화로 위치시키고, JPMorgan의 단일 API + 다중 모델 구조에서 task type 분류, cost-accuracy 트레이드오프, 모델 가용성과 SLA, 라우팅 결정의 관측이라는 네 가지 축이 어떻게 결합되는지를 설명할 수 있습니다.

4.3.3 Graceful Degradation

앞의 두 단원에서 우리는 실패한 경로 대신 다른 경로로 흘러보내는 fallback과, 매 요청마다 적합한 모델을 골라 보내는 라우팅을 살펴봤습니다. 그런데 이 두 장치가 모두 작동하지 못하는 상황도 있습니다. 모든 모델 사업자가 동시에 흔들리거나, 사내 도구 전체가 같은 데이터베이스 장애에 영향을 받거나, 외부 검색 API가 한꺼번에 응답을 멈추는 식입니다. 이런 상황에서 시스템이 곧장 모든 응답을 거절하고 멈춰 버리면, 사용자가 평소에 잘 쓰던 기능까지 함께 사라집니다. 부분 장애가 전체 정지로 번지는 셈입니다. 이 번짐을 막기 위한 패턴이 **graceful degradation**입니다.

이름부터 풀겠습니다. 영어 그대로 옮기면 '품위 있게 기능을 낮춤'이라는 뜻입니다. 좀 더 풀어 쓰면 '갑작스럽게 모든 것이 꺼지는 대신, 천천히 기능 일부를 내려놓으며 핵심만 살려 두는 방식'입니다. 일상 비유로는 정전 직전에 비상등이 켜지는 장면을 떠올리면 됩니다. 형광등은 모두 꺼지지만 비상등 한 줄은 남아서 사람들이 출구를 찾을 수 있게 해 줍니다. 시스템 설계에서도 똑같은 비유가 그대로 쓰입니다. 장애가 와도 모든 화면이 일시에 멈추는 대신, 우선순위가 높은 기능만 살리고 낮은 기능을 한시적으로 멈춥니다.

이 패턴을 짜려면 가장 먼저 **기능 우선순위 분류**를 해야 합니다. 에이전트 서비스가 제공하는 기능을 모아 놓고, 각 기능에 대해 두 질문을 던집니다. 첫째, 이 기능 없이도 사용자가 핵심 작업을 끝낼 수 있는가. 둘째, 이 기능이 멈췄을 때 사용자에게 손해가 얼마나 큰가. 두 질문의 답을 보고 기능을 보통 세 등급으로 나눕니다. 핵심 기능, 일반 기능, 보조 기능입니다. 핵심 기능은 끊어지면 서비스의 의미가 사라지는 부분이고, 일반 기능은 평상시에는 가치를 더해 주지만 잠시 빠져도 사용자가 다른 길로 일을 끝낼 수 있는 부분이며, 보조 기능은 편의성에 가까운 부분입니다. 예를 들어 사내 문서 검색 에이전트라면 사내 문서 검색 자체가 핵심, 검색 결과의 자동 요약이 일반, 결과를 슬랙으로 자동 전송해 주는 기능이 보조에 해당합니다.

분류가 끝나면 **비핵심 기능 비활성화** 정책을 짭니다. 이 정책은 어떤 신호가 나타났을 때 어떤 기능을 어떤 순서로 끄는지를 미리 정의합니다. 신호로는 회로 차단기가 열린 상태, 라우팅에서 모든 후보 모델이 SLA 임계 아래로 떨어진 상태, 도구 호출의 연속 실패가 일정 횟수를 넘은 상태가 흔히 쓰입니다. 정책은 보통 단계로 표현됩니다. 1단계에서는 보조 기능만 끕니다. 2단계에서는 일반 기능까지 함께 끕니다. 3단계에서는 핵심 기능을 축약된 형태로 돌립니다. 단계가 깊어질수록 시스템이 보여 주는 화면은 단순해지지만, 동시에 살아 있는 기능의 신뢰성은 더 높아집니다.

다음으로 **사용자에게 알리는 메시지**가 필요합니다. 기능이 줄어든 사실을 사용자에게 숨기면 사용자는

시스템이 고장 났다고 느낍니다. 반면 너무 자세히 알리면 평상시에도 불안감을 줍니다. 좋은 메시지는 세 요소를 갖춥니다. 첫째, 지금 어떤 기능이 평소처럼 동작하지 않는다는 사실을 한 줄로 알립니다. 둘째, 핵심 기능은 여전히 사용 가능하다는 점을 함께 안내합니다. 셋째, 정상 복구 예상 시점이나 다음 단계 안내를 짧게 덧붙입니다. 이런 메시지는 보통 화면 상단의 띠 형태로 표시되며, 본문 영역의 결과는 그대로 유지됩니다.

기능 축소 단계와 사용자 안내를 그림으로 정리하면 다음과 같습니다.

[정상]	[1단계 축소]	[2단계 축소]	[3단계 축소]
보조 기능 ON	보조 기능 OFF	보조 기능 OFF	보조 기능 OFF
일반 기능 ON	일반 기능 ON	일반 기능 OFF	일반 기능 OFF
핵심 기능 ON	핵심 기능 ON	핵심 기능 ON	핵심 기능 ON (축약)
사용자 안내 없음	'슬랙 전송 일시 중단'	'자동 요약 일시 중단'	'검색 결과만 표시 중'

여기서 우선순위 정의가 위쪽으로 갈수록 끄기 쉽고 아래로 갈수록 끄기 어렵다는 점이 한눈에 보입니다. 핵심 기능은 마지막까지 살려 두며, 그것마저 어려울 때는 결과 형식을 단순화하는 식으로 축약합니다.

기능을 축소했다고 끝이 아닙니다. **장애 종료 후 회복 절차**가 함께 설계되어야 합니다. 장애가 잦아들면 시스템은 다시 보조 기능을 켜고, 일반 기능을 켜고, 마지막으로 모든 기능을 정상으로 돌립니다. 이 회복 순서는 장애 진입 순서의 역방향이며, 단계 사이에는 잠시 검증 시간을 두어 신호가 진짜로 안정됐는지 확인합니다. 앞 장에서 본 회로 차단기의 half-open 상태와 같은 메커니즘입니다. 회복이 너무 성급하면 살짝 가라앉았다가 다시 들이닥치는 2차 장애에 그대로 노출되고, 너무 굼뜨면 사용자에게 불필요하게 줄어든 화면을 오래 보여 주게 됩니다. 단계마다 검증 시간을 두는 이유가 이 사이의 균형을 잡는 것입니다.

마지막 축은 **관측을 통한 장애 범위 자동 인식**입니다. 어떤 기능을 어느 단계에서 끌지를 사람이 일일이 판단하면 너무 늦습니다. 관측 시스템이 모델·도구·외부 API의 상태 신호를 실시간으로 모으고, 미리 정해진 규칙에 따라 자동으로 단계를 올리거나 내려야 합니다. 이때 trace와 metric에 부착되어 있는 사업자 이름, 도구 이름, 에러 분류가 핵심 입력이 됩니다. 예를 들어 'A 사업자의 호출 에러율이 5분 평균 30%를 넘으면 그 사업자를 쓰는 보조 기능을 자동으로 비활성화'와 같은 규칙입니다. 규칙을 단순하게 유지하되, 어떤 규칙이 어떤 시점에 발동됐는지를 같은 trace에 함께 남겨야 사후에 사용자 경험이 어떻게 달라졌는지 재구성할 수 있습니다.

이 패턴을 작은 회사가 처음부터 다 갖추기는 어렵습니다. 그래서 도입 순서를 한 가지 권하면, 먼저 기능을 우선순위로 분류해 종이 한 장에 정리하는 것부터 시작합니다. 그다음 가장 자주 흔들리는 한두 개 외부 의존을 골라, 그것이 흔들렸을 때 어떤 기능을 끌지를 회로 차단기 코드 옆에 붙입니다. 그리고 사용자 안내 메시지의 자리만 화면에 마련해 둡니다. 이 세 가지만 갖춰도 부분 장애가 전체 정지로 번지는 가장 흔한 사고는 막을 수 있습니다. 자동 인식은 그 위에 얹는 다음 층입니다.

정리하면, graceful degradation은 부분 장애가 전체 정지로 번지지 않도록 기능을 단계적으로 축소했다가 다시 회복시키는 패턴이며, 핵심·일반·보조로 나뉜 기능 우선순위를 바탕으로 비핵심 기능을 먼저 끄고, 사용자에게는 어떤 기능이 일시 중단됐는지를 짧게 안내하며, 장애 종료 후에는 진입 순서의 역방향으로 단계를 검증하며 회복합니다. 관측 시스템이 사업자·도구·에러 신호를 실시간으로 모아 자동으로 단계를 조정하면, 사람이 손으로 끄고 켜는 지연 없이도 부분 장애에 견디는 시스템이 만들어집니다.

이로써 4.3절 fallback routing 전체가 마무리됩니다. fallback의 종류, 모델 라우팅, graceful degradation 세 가지 패턴은 회로 차단기와 함께 비결정적 에이전트를 프로덕션에 안전하게 올리는 신뢰성 장치를 이룹니다. 다음 단원인 4.4.1에서는 이 신뢰성 장치들 옆에 함께 자리 잡는 또 한 축, 즉 사람의 개입을 어떻게 다시 정의할지를 다룹니다. 모든 행동을 사람이 일일이 승인하는 전수 승인 모델의 한계를 살피고,

왜 임계치 기반 예외 검토로 옮겨 가야 하는지를 정리합니다.

이 단원을 마치면 graceful degradation을 핵심·일반·보조 기능 우선순위 분류 위에서 비핵심 기능을 비활성화하는 단계적 축소로 설명하고, 사용자에게 보내는 안내 메시지의 구성, 장애 종료 후 회복 절차의 단계와 검증 시간, 그리고 관측 시스템이 어떻게 장애 범위를 자동으로 인식해 단계를 조정하는지를 함께 풀어낼 수 있습니다.

4.4 Human-in-the-loop 재정의

이 섹션의 토픽과 학습 목표는 다음과 같습니다.

4.4.1 모든 걸 승인 의 한계 — 전수 승인 모델의 운영 부담과 사용자 경험 문제를 설명할 수 있다

4.4.2 임계치 기반 예외 검토 — 위험·신뢰도 임계치 기반의 선택적 HITL 패턴을 설계할 수 있다

4.4.3 Escalation Path 설계 — 에이전트가 사람으로 핸드오버하는 escalation path를 설계하고 측정할 수 있다

4.4.1 모든 걸 승인 의 한계

앞 세 섹션에서 자율성을 단계로 끊고, 위험이 임계치를 넘으면 회로를 끊고, 실패가 닥치면 단계적으로 기능을 줄이는 장치를 살폈습니다. 이 흐름의 끝에는 늘 사람이 한 번 더 보는 자리가 남습니다. 흔히 이 자리를 human-in-the-loop이라고 부르고, 가장 단순한 구현은 에이전트의 모든 행동에 사람이 승인 버튼을 누르도록 하는 것입니다. 그런데 이 단순한 답은 일정 규모를 넘는 순간 빠르게 무너집니다. 이 단원은 '모든 걸 승인' 모델이 왜 무너지는지, 어떤 부작용을 만드는지를 정리합니다. 다음 두 단원에서 이어질 임계치 기반 예외 검토와 escalation path 설계의 토대가 됩니다.

먼저 용어부터 풀겠습니다. **human-in-the-loop**은 사람이 자동화 흐름의 한가운데에 끼어들어 의사결정의 일부를 떠맡는 구조를 가리킵니다. 줄여서 HITL이라고도 씁니다. 의료 영상 판독에서 모델이 후보 병변을 표시하면 의사가 최종 진단을 확정하는 흐름, 콘텐츠 신고 처리에서 분류기가 위험 점수를 매기면 운영자가 삭제 여부를 결정하는 흐름이 익숙한 예입니다. 에이전트 시스템에서도 같은 원리가 들어옵니다. 에이전트가 도구를 부르거나 외부에 데이터를 쓰기 전에 사람이 한 번 보고 허락하는 자리가 만들어집니다. 문제는 이 자리를 어디에, 얼마나 촘촘하게 두느냐입니다.

전수 승인 모델은 가장 보수적인 답입니다. 에이전트가 어떤 행동을 하려 하든, 위험이 크든 작든, 사람이 한 번씩 확인 버튼을 누른 다음에야 다음 단계로 나아갑니다. 출시 초기에는 안전해 보입니다. 어떤 행동도 허락 없이 일어나지 않으니 사고가 날 자리가 좁아 보이고, 책임 소재도 분명해집니다. 그런데 이 모델을 실제 트래픽 위에 올려 놓으면 곧 세 가지 문제가 동시에 드러납니다.

첫째 문제는 **운영 비용**입니다. 한 번의 사용자 요청이 에이전트 안에서 도구 호출 다섯 개와 데이터베이스 조회 세 개를 만들어 낸다고 해 보겠습니다. 전수 승인은 이 여덟 개의 행동마다 운영자의 클릭을 요구합니다. 하루에 수백 건이 들어오는 사내 보조 도구라면 운영자 한 명이 종일 클릭만 누르고 있어야 하고, 하루에 수만 건이 들어오는 대고객 서비스라면 클릭을 받을 사람을 수백 명 단위로 뒤도 따라잡지 못합니다. 인건비뿐 아니라 응답 지연이 함께 붙습니다. 사용자는 5초면 끝날 답을 받자고 30분을 기다리고, 그 사이에 다른 채널로 이탈합니다. 자동화로 절감하려던 비용은 승인 인력 비용으로 옮겨 가고, 사용자 경험은 자동화 이전보다 더 나빠집니다.

둘째 문제는 **승인 피로**입니다. 영어권에서는 alert fatigue라는 말로 알려져 있고, HITL 맥락에서는 approval fatigue라고 부릅니다. 사람이 같은 종류의 결정을 반복해서 내리다 보면, 각 결정에 들어는

주의가 빠르게 줄어듭니다. 처음 몇 시간은 화면을 꼼꼼히 읽고 위험을 점검하지만, 같은 패턴이 반복되면 곧 패턴 인식 모드로 넘어갑니다. 화면이 익숙하게 보이면 승인을 누르고, 어색해 보이면 반려를 누르는 반사 행동이 자리를 잡습니다. 이 단계에서 운영자는 사실상 승인 도장이 됩니다. 자기도 모르게 99%를 통과시키고, 정작 위험한 1%에서도 같은 반사로 통과시켜 버립니다. 이 현상을 **자동 승인화**라고 부릅니다. 사람이 끼어들기는 하지만 의미 있는 게이트로 작동하지 못하고, 시스템 입장에서는 사람을 끼워 둔 보람이 사라집니다.

셋째 문제는 **위험과 승인 빈도의 불일치**입니다. 에이전트의 행동은 그 무게가 같지 않습니다. 캐시에서 값을 읽는 호출과 결제 시스템에 5천만 원을 송금하는 호출은 위험의 자릿수가 다릅니다. 전수 승인은 두 행동에 같은 무게의 클릭을 요구합니다. 운영자는 5천만 원 송금을 결정할 때와 캐시 조회를 결정할 때 같은 화면, 같은 버튼, 같은 시간을 들입니다. 진짜 위험한 결정에 쓸 주의력은 다른 곳에서 미리 소진되어 있고, 사고가 정작 일어나는 자리에 오면 사람의 판단력이 가장 얇은 상태입니다. 위험 신호가 흐려지는 셈입니다. 이 불일치가 승인 피로와 합쳐지면 자동 승인의 가능성은 더 커집니다.

이 세 문제를 한 그림으로 그려 보면 다음과 같습니다.

[사용자 요청]

```

|
에이전트 행동 1 (캐시 조회) --[승인?-- 사람 클릭
|
에이전트 행동 2 (DB 읽기) --[승인?-- 사람 클릭
|
에이전트 행동 3 (요약 LLM) --[승인?-- 사람 클릭
|
에이전트 행동 4 (결제 송금) --[승인?-- 사람 클릭 <-- 이미 피로 누적
|
에이전트 행동 5 (이메일 발송) --[승인?-- 사람 클릭
|

```

[응답 반환] -- 5분 -> 30분 -> 1시간 ...

화면 위에서는 사람이 다섯 번 검토하는 것처럼 보이지만, 네 번째 클릭에 도달하는 시점에는 같은 강도의 검토를 기대하기 어렵습니다. 위험이 큰 행동 4에서 가장 약한 검토가 일어나고, 위험이 작은 행동 1에서 가장 정성스러운 검토가 일어납니다. 에이전트 입장에서는 사람이 가드레일 역할을 한다고 가정하지만, 실제로 작동하는 강도는 위험과 정반대 방향입니다.

전수 승인이 만드는 구조적 부작용도 있습니다. 모든 행동이 사람의 클릭을 거쳐야 한다는 조건은 곧 사람의 처리 속도가 시스템 전체의 처리 속도를 결정한다는 뜻입니다. 사람이 시스템의 **병목**이 됩니다. 한 명이 자리를 비우면 큐가 쌓이고, 야간이나 주말에는 처리 자체가 멈춥니다. 이 병목을 풀자고 승인 인력을 늘리면 인건비가 다시 부담이 되고, 인력을 줄이자고 승인 단계를 합치면 한 번의 클릭에 묶이는 행동의 수가 늘어 검토는 더 형식적이 됩니다. 어느 쪽으로 가든 자동화가 약속하던 효율과 멀어집니다.

업계의 안내 문서들도 이 한계를 분명히 지적합니다. Anthropic이 효과적인 에이전트를 만드는 방법을 정리한 글과 OpenAI가 실무 가이드 형태로 풀어 둔 글이 가장 압축적입니다. 두 글의 공통된 권고는 비슷합니다. 사람을 모든 단계에 끼워 두는 대신, 사람이 정말로 가치를 더할 수 있는 자리를 골라서 끼워 두라는 것입니다. 비가역 행동이거나, 비용이 큰 행동이거나, 모델이 스스로 확신을 못 가지는 행동에는 사람의 판단이 의미 있게 더해집니다. 캐시 조회나 요약 한 줄 같은 가역적이고 저비용인 행동에는 같은 사람의 클릭이 보탬보다 부담입니다. HITL을 '모든 걸 승인'에서 '예외 검토'로 옮기라는 권고가 두 글에서 동일하게 등장합니다.

이 권고를 받아들이려면 출발 가정을 바꿔야 합니다. 전수 승인은 모든 행동을 같은 무게로 다루지만,

실제 운영에서는 행동마다 다른 무게가 붙어 있습니다. 다른 무게를 시스템이 인식할 수 있어야 사람이 끼어들 자리도 차등이 됩니다. 이 차등을 만드는 도구가 다음 단원에서 다룰 임계치입니다. 임계치는 신뢰도, 비용, 비가역성 같은 축 위에서 어느 지점부터 사람이 들여다볼지를 정하는 선이고, 이 선이 분명해질수록 사람의 검토는 의미 있는 자리에만 모입니다. 자동 승인화와 병목이 풀리는 길도 같은 자리에 있습니다.

정리하면, 모든 행동에 사람을 끼워 두는 전수 승인 모델은 운영 비용을 인건비로 옮기고, 같은 결정을 반복하게 만들어 승인 피로와 자동 승인화를 부르며, 위험 크기와 승인 빈도가 어긋나면서 정작 위험한 자리에서 사람의 판단력이 가장 약아지게 만듭니다. 그 결과 사람은 가드레일이 아니라 병목이 되고, 자동화가 약속하던 효율과 안전이 함께 무너집니다. 효과적인 에이전트 가이드들이 한결같이 권고하는 출발점은 HITL을 빈도가 아니라 의미로 정의하는 것이며, 이 의미를 시스템이 자동으로 식별하도록 만드는 첫 도구가 다음 단원의 임계치입니다.

다음 단원인 4.4.2에서는 신뢰도 점수, 비가역 작업, 비용 임계치를 축으로 사람이 들여다볼 자리를 자동으로 골라 내는 임계치 기반 예외 검토 패턴을 살펴봅니다.

이 단원을 마치면 전수 승인 모델이 만드는 운영 비용, 승인 피로와 자동 승인화, 위험과 승인 빈도의 불일치를 설명하고, 사람이 시스템의 병목이 되는 구조적 한계를 짚으며, HITL을 '모든 걸 승인'에서 예외 검토로 재정의해야 하는 이유를 효과적인 에이전트 가이드의 권고와 함께 설명할 수 있습니다.

4.4.2 임계치 기반 예외 검토

앞 단원에서 모든 행동에 사람을 끼워 두는 전수 승인 모델이 인건비, 승인 피로, 위험 신호의 둔화로 무너진다는 점을 살폈습니다. 결론은 사람이 끼어들 자리를 빈도가 아니라 의미로 정의해야 한다는 것이었습니다. 이 단원은 그 의미를 시스템이 자동으로 식별하도록 만드는 도구를 다룹니다. 핵심은 단순합니다. 모든 행동을 같은 무게로 다루는 대신, 어떤 축 위에서 어느 지점을 넘는 행동만 사람이 들여다보게 합니다. 이 선이 임계치이고, 이 패턴을 **임계치 기반 예외 검토**라고 부릅니다.

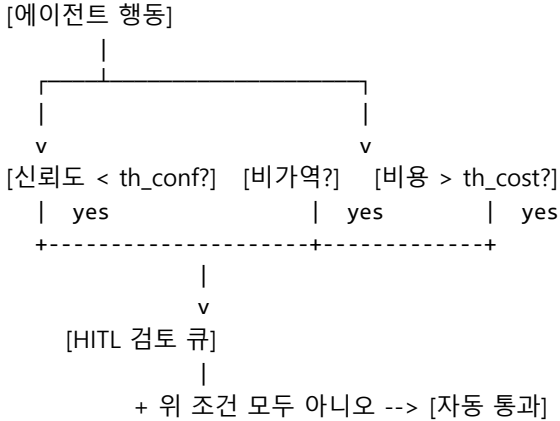
먼저 가장 자주 쓰이는 축을 정리하겠습니다. 첫째 축은 **신뢰도 점수**입니다. 신뢰도는 모델이 자기 출력에 대해 얼마나 확신하는지를 0과 1 사이의 값으로 표현한 양입니다. 분류 모델에서는 클래스별 확률 분포에서 최댓값을 그대로 쓰기도 하고, 생성 모델에서는 토큰 확률을 곱해 응답 단위의 확률로 만들기도 하며, 별도의 채점 모델을 붙여 응답을 사후 평가한 점수를 쓰기도 합니다. 어느 방식이든 결과는 같은 형태입니다. 어떤 행동에는 0.95처럼 높은 점수가 붙고, 어떤 행동에는 0.42처럼 낮은 점수가 붙습니다. 임계치를 0.7로 잡으면 0.7 미만의 행동만 사람의 검토 큐에 들어가고, 그 이상은 자동 통과합니다. 이 한 줄의 규칙으로 사람의 클릭 수가 80~90% 줄어드는 일이 흔합니다.

둘째 축은 **비가역성**입니다. 가역과 비가역의 경계는 의외로 분명합니다. 캐시 조회는 결과가 마음에 안 들면 다시 부르면 되고, 임시 디렉터리에 파일을 쓰는 일도 지우면 그만입니다. 결제 송금, 외부 API에 발사되는 이메일, 운영 데이터베이스 삭제는 한 번 일어나면 되돌릴 수 없습니다. 비가역 작업은 신뢰도와 무관하게 사람의 검토를 한 번 거치도록 강제합니다. 모델이 자기 행동에 0.99의 확신을 가졌더라도, 그 확신이 틀렸을 때 복구가 안 되는 행동이라면 사람을 끼워 두는 비용이 정당화됩니다. 이 자동 트리거를 다른 축과 분리해 두면 신뢰도 임계치를 공격적으로 풀어 뒀도 큰 사고가 막히는 안전망이 생깁니다.

셋째 축은 **비용**입니다. 비용은 두 갈래로 잡힙니다. 한 갈래는 행동 자체가 회사에 부과하는 금액입니다. 결제 송금 5천만 원, 광고 입찰 100만 원, 클라우드 자원 1만 원 같은 직접 금액입니다. 다른 갈래는 에이전트 자신이 모델 호출과 도구 호출로 쓰는 토큰·API 비용입니다. 이 두 갈래가 합쳐져 한 번의 행동이 회사에 입히는 영향이 정해집니다. 임계치는 이 영향이 일정 금액을 넘는 시점에 사람의 눈을 끼워

됩니다. 한 건당 1천만 원이 넘는 결제 행동은 무조건 검토를 거치고, 한 번의 응답 생성에 1만 원을 넘는 토큰 비용이 발생할 것 같으면 진행을 멈추고 사람에게 묻는 식입니다. 비용 측은 신뢰도가 높더라도 모델이 잘못 판단했을 때 발생할 손실의 절대 크기를 제어합니다.

세 축을 한 그림으로 묶으면 다음 흐름이 만들어집니다.



세 분기는 OR로 결합됩니다. 어느 하나라도 임계치를 넘으면 사람의 큐에 들어가고, 셋 다 통과하면 자동 진행합니다. 이 구조 덕분에 운영자는 검토 큐에 올라온 케이스만 다루면 되고, 큐의 길이는 시스템 전체 트래픽이 아니라 임계치 위에 떠오른 행동의 수에 비례하게 됩니다.

임계치는 처음에 정한 값을 그대로 두고 쓰는 정적 상수가 아닙니다. 운영하다 보면 두 방향의 압력이 동시에 들어옵니다. 한쪽에서는 검토 큐가 너무 길어 운영자가 따라잡지 못한다는 압력이 들어오고, 다른 쪽에서는 임계치를 넘지 않은 자동 통과 행동에서 사고가 났다는 압력이 들어옵니다. 첫 번째 압력은 임계치를 더 보수적으로 풀어 큐를 줄이라고 말하고, 두 번째 압력은 임계치를 더 촘촘하게 좁혀 큐를 늘리라고 말합니다. 이 두 힘이 매주, 매월 변동합니다. 그래서 임계치는 **동적으로 조정**되어야 합니다. 한 주에 한 번 큐 길이와 사고 사례를 함께 보고, 신뢰도 임계치를 0.05 단위로 움직이거나, 비용 임계치의 자릿수를 한 단계 옮기는 식으로 운영하는 방식이 흔합니다. 사용자 코호트별로 다른 임계치를 두기도 합니다. 신규 가입 사용자는 보수적으로, 장기 사용자는 공격적으로 풀어 두는 식입니다.

임계치 운영을 객관적으로 하려면 한 가지 메트릭이 거의 반드시 필요합니다. 바로 **HITL 트리거 비율**입니다. 전체 에이전트 행동 가운데 사람의 검토 큐로 넘어가는 행동이 몇 퍼센트인지를 그대로 켜 값입니다. 이 비율이 너무 높으면 운영자가 다시 모든 걸 클릭해야 하는 상태로 회귀한 신호이고, 너무 낮으면 사고가 났을 때 사람이 끼어들 자리가 사라진 상태일 가능성이 큼니다. 도메인마다 적정 구간이 다르지만, 사내 보조 에이전트는 1~5%, 대고객 에이전트는 0.1~1% 같은 범위가 자주 등장합니다. 이 비율은 단일 숫자로 끝나지 않고 분기별 분포가 함께 보고됩니다. 신뢰도 분기가 80%, 비가역 분기가 15%, 비용 분기가 5%인 식의 비율을 보면 어느 축이 큐를 만드는지 보입니다. 이 분포가 시간에 따라 어떻게 움직이는지가 임계치 조정의 근거가 됩니다.

이 패턴을 한 줄로 요약하면, HITL을 '모든 걸 승인'이 아니라 '임계치 기반 예외 검토'로 재정의하는 것입니다. 같은 사람이 같은 시간을 들이더라도 임계치 기반 모델에서는 의미 있는 검토만 하게 됩니다. 사람의 검토가 형식적인 도장이 되지 않고, 통과·반려의 결과가 모델 학습 데이터로 이어지면서 시간이 지날수록 임계치 미만의 자동 통과 영역이 안전하게 확장됩니다. 이 학습 루프가 작동하면 시스템은 더 많은 행동을 자동으로 처리하면서도 사고는 줄어듭니다. 이 점이 전수 승인 모델로는 도달할 수 없는 자리입니다.

마지막으로 두 가지 함정을 짚어 두겠습니다. 첫째, 임계치는 잘못 잡으면 그 자체가 사고의 원인이

됩니다. 신뢰도 점수가 0.95인데 사실 모델 보정이 잘못되어 실제 정확도는 60%인 경우가 있고, 이 경우 임계치를 0.7로 잡아도 위험한 행동이 줄줄 통과합니다. 신뢰도 점수의 보정 상태를 별도로 점검하지 않으면 임계치 운영의 토대 자체가 흔들립니다. 둘째, 비가역 분기를 너무 좁게 잡는 함정도 흔합니다. 데이터베이스 INSERT는 가역으로 보일 수 있지만, 그 INSERT가 외부 시스템에 연쇄 알림을 보낸다면 사실상 비가역입니다. 비가역의 경계는 행동 자체가 아니라 그 행동이 일으키는 부수 효과로 정의해야 합니다.

정리하면, 임계치 기반 예외 검토는 신뢰도 점수, 비가역성, 비용이라는 세 축 위에 선을 긋고, 어느 한 축에서라도 임계치를 넘는 행동만 사람의 검토 큐로 보내는 패턴입니다. 임계치는 검토 큐의 길이와 사고 사례 사이에서 동적으로 움직여야 하고, HITL 트리거 비율과 분기별 분포를 함께 관측해야 임계치 조정의 근거가 생깁니다. 이 패턴이 자리 잡으면 사람은 의미 있는 자리에서만 끼어들고, 자동 승인화와 병목이 함께 풀리며, 시간이 지날수록 안전하게 자동 통과 영역이 넓어지는 학습 루프가 만들어집니다.

다음 단원인 4.4.3에서는 임계치를 넘은 행동이 사람에게 넘어가는 그 다음 자리, 즉 escalation path의 설계를 살펴봅니다. 감정·복잡도 신호를 어떻게 잡고, handover 시 컨텍스트를 어떻게 전달하며, Klarna 사례에서 무엇을 배워야 하는지가 이어지는 주제입니다.

이 단원을 마치면 신뢰도 점수, 비가역성, 비용 임계치를 축으로 한 예외 검토 패턴을 설계하고, 임계치를 동적으로 조정하는 운영 흐름을 짜며, HITL 트리거 비율과 분기별 분포를 메트릭으로 측정해 자동 통과와 사람 검토 사이에서 균형 있게 운용하는 방법을 설명할 수 있습니다.

4.4.3 Escalation Path 설계

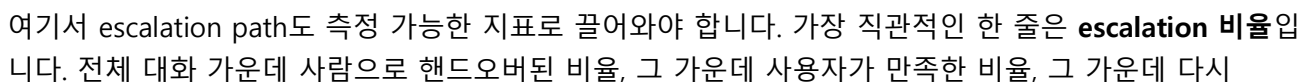
앞 두 단원에서 전수 승인 모델의 한계와, 사람이 들여다볼 자리를 임계치로 자동 식별하는 패턴을 살펴보았습니다. 임계치가 작동하면 큐로 넘어오는 행동의 양은 줄지만, 큐로 넘어온 케이스가 사람에게 어떻게 도달하고, 사람이 처리한 결과가 다시 사용자에게 어떻게 돌아오는지는 별도의 설계가 필요합니다. 이 자리를 흔히 **escalation path**라고 부릅니다. 단어 그대로 해석하면 '단계를 올려 보내는 경로'라는 뜻이고, 실제로는 에이전트가 처리하다가 사람으로 핸드오버하는 한 줄기 흐름 전체를 가리킵니다. 이 단원은 이 흐름을 어떤 신호로 시작하고, 어떤 컨텍스트와 함께 사람에게 넘기며, 사용자가 그 사이를 어떻게 경험하고, 무엇을 KPI로 측정해야 하는지를 정리합니다.

먼저 escalation을 트리거하는 신호부터 풀겠습니다. 4.4.2에서 본 신뢰도, 비가역, 비용은 행동 자체에 붙는 신호였고, 사용자와 모델 사이의 상호작용에서만 드러나는 신호가 따로 있습니다. 가장 자주 다뤄지는 것이 **감정 신호**와 **복잡도 신호**입니다. 감정 신호는 사용자의 발화에 좌절·분노·불안 같은 정서가 실릴 때 잡힙니다. 분류 모델로 감정 라벨을 붙이거나, 키워드 기반으로 강한 부정어가 누적되는 빈도를 세거나, 같은 사용자가 같은 문제를 세 번째 다시 묻고 있는 패턴을 잡는 식입니다. 복잡도 신호는 질문 자체가 단일 의도로 떨어지지 않거나, 도메인 정책이 얹혀 있거나, 개인 상황 정보가 깊게 들어가 있을 때 잡힙니다. 한 사용자가 한 문장 안에 결제 환불, 분쟁 조정, 명의 변경을 동시에 요구하는 경우가 전형적인 예입니다. 이 두 신호는 신뢰도 점수만으로는 잡히지 않습니다. 모델은 자기 답에 0.9의 확신을 가지면서도, 그 답이 사용자의 정서를 풀어 주지 못하거나 복잡한 사정을 다 담지 못할 수 있기 때문입니다.

이 신호를 흘려보내면 어떤 일이 벌어지는지는 1.3에서 다룬 Klarna 사례가 보여 줍니다. 이 책의 1단계에서 본 내용을 다시 짚자면, Klarna는 23개 시장에서 한 달 230만 건의 대화를 처리하면서 CSAT를 47% 끌어올리고 1천만 달러를 절감했다고 발표했습니다. 숫자만 보면 완성된 자동화로 보였지만, 시간이 지나면서 다른 결이 드러났습니다. 복잡하거나 감정적인 케이스에서 사용자가 unheard·unsupported하다고 느꼈고, 비용이 평가의 압도적 축이 되면서 hidden quality가 깎여

신호가 잡혀 사람에게 넘어가기로 결정되면 다음 문제는 **handover 시 컨텍스트 전달**입니다. 사람이 처음부터 다시 듣게 만드는 escalation은 사용자에게 두 번 같은 말을 시키는 것과 같고, 가뜰이나 좌절한 사용자의 감정을 한 번 더 자극합니다. 좋은 핸드오버는 사람 운영자가 자리에 앉는 순간 다음 정보가 화면에 정리되어 있는 상태를 만듭니다. 첫째, 대화의 요약입니다. 사용자가 요청한 내용, 에이전트가 시도한 행동, 막힌 지점, 감정이 격해진 시점이 한 단락으로 압축됩니다. 둘째, 원본 trajectory의 링크입니다. 운영자가 요약만으로 부족하다고 느끼면 한 클릭으로 단계별 도구 호출과 응답을 그대로 살펴볼 수 있어야 합니다. 셋째, 사용자 식별과 권한 컨텍스트입니다. 사용자의 계정, 등급, 보유 상품, 과거 escalation 이력 같은 정보가 자동 첨부되어야 운영자가 추가 질문 없이 결정에 들어갈 수 있습니다. 이 세 항목이 빠지면 핸드오버는 형식적 이관이 되고, 사용자는 사람과 대화하는 데도 같은 좌절을 다시 겪습니다.

전체 흐름을 한 그림으로 묶으면 다음과 같습니다.



에이전트로 돌아온 비율을 묶어 봅니다. 비율 하나만으로는 의미가 갈리지 않으니, 도메인 단위로 분해해서 보는 편이 안전합니다. 결제 분쟁에서 escalation 비율이 12%인지, 단순 잔액 조회에서 0.5%인지를 따로 보면, 어떤 도메인의 신호 임계치를 다듬어야 하는지가 보입니다. 이 비율이 너무 낮으면 사람으로 넘어갈 자리가 막혀 있다는 신호, 너무 높으면 자동화 경계가 너무 보수적이라는 신호입니다. 두 극단 사이에서 균형이 잡힐 때까지 임계치와 신호를 다듬는 일이 운영의 큰 부분을 차지합니다.

escalation 비율과 함께 보아야 하는 보조 지표가 두 가지 있습니다. 하나는 **handover 후 회복 시간**으로, 사용자가 사람에게 넘겨진 시점부터 만족 신호가 잡히는 시점까지의 시간입니다. 시간이 길어지면 사람의 처리 속도와 컨텍스트 전달의 품질을 점검해야 합니다. 다른 하나는 **재escalation 비율**로, 같은 사용자가 한 번 핸드오버를 받은 뒤 같은 세션 안에서 다시 escalation이 필요해진 비율입니다. 이 비율이 높으면 사람이 처리해도 근본 문제가 안 풀리고 있다는 뜻이고, 도구 자체나 정책에 손을 대야 할 자리입니다.

이 모든 장치가 작동하면 escalation path는 사용자 입장에서 시스템의 안전망이자 마지막 보루가 됩니다. AI는 효율성을 책임지고 사람은 공감을 책임지는 hybrid 운영 모델이 자연스럽게 자리를 잡고, Klarna가 회귀로 도달했던 형태가 처음부터 설계 위에 들어가게 됩니다. 자동화의 효율을 살리면서도 hidden quality 침식을 막는 한 가지 길이 여기에 있습니다.

정리하면, escalation path는 감정과 복잡도 신호를 임계치 분기와 함께 트리거로 받고, 대화 요약과 trajectory 링크와 사용자 컨텍스트를 묶은 패키지로 사람에게 핸드오버하며, 그 사이의 대기 경험을 사용자에게 명시적으로 안내하고, escalation 비율과 회복 시간과 재escalation 비율을 KPI로 관측하는 한 줄기 흐름입니다. Klarna의 hybrid 회귀는 이 흐름이 비어 있으면 단일 KPI 위에서 빛나던 시스템도 사용자가 unheard-unsupported하다고 느끼는 자리에서 무너진다는 점을 분명히 보여 주는 사례이며, 같은 자리에서 hybrid 운영을 처음부터 설계에 넣어 두는 일이 escalation path 설계의 핵심입니다.

이로써 Phase 4의 신뢰성과 안전 패턴을 마무리합니다. 다음 단원인 5.1.1부터는 Phase 5로 넘어가, 지금까지 살펴본 평가·관측·신뢰성 패턴들이 실제 사례에서 어떻게 작동하는지를 따라갑니다. 첫 사례는 Anthropic의 Lead Agent와 병렬 Subagent 구조로 이루어진 Multi-Agent Research System입니다. Lead가 어떻게 작업을 분해하고, Subagent가 어떻게 병렬로 움직이며, 그 결과가 어떻게 합성되는지를 살펴보겠습니다.

이 단원을 마치면 감정·복잡도 신호 기반의 escalation 트리거를 설계하고, 핸드오버 패키지로 컨텍스트를 사람에게 전달하며, 사용자 대기 경험을 명시적으로 다루고, escalation 비율과 회복 시간과 재escalation 비율을 KPI로 측정하는 흐름을 설계할 수 있고, Klarna 사례의 hybrid 회귀 교훈을 escalation path 부재의 결과로 설명할 수 있습니다.

5 케이스 스터디: 실전에서 배우기

Anthropic, Replit, JPMorgan, Klarna, Palantir 등 대표 사례에서 평가·관측·거버넌스 패턴을 추출하고 자체 시스템 설계에 적용할 수 있다.

이 Phase는 다음 섹션으로 구성됩니다.

5.1 Anthropic Multi-Agent Research System

5.1.1 Lead + Subagent 아키텍처

5.1.2 Production Tracing 설계

5.1.3 프라이버시 보존 모니터링

5.1.4 Source Quality Heuristic 사례

5.2 Replit Agent

5.2.1 Hybrid 평가 스택의 구성

5.2.2 Computer Use 기반 테스트

5.2.3 Reactive Eval과 가드레일 우회

5.3 JPMorgan LLM Suite

5.3.1 중앙 플랫폼 패턴

5.3.2 단일 API 뒤의 다중 모델 라우팅

5.3.3 측정 가능한 프로세스부터 시작

5.4 Klarna 반면교사

5.4.1 성공 지표 뒤의 함정

5.4.2 Hybrid 회귀의 교훈

5.5 Palantir AIP - 5차원 거버넌스

5.5.1 Model Access 차원

5.5.2 Stateless Orchestration 차원

5.5.3 Memory Governance 차원

5.5.4 Tool Governance 차원

5.5.5 Observability 차원

5.1 Anthropic Multi-Agent Research System

이 섹션의 토픽과 학습 목표는 다음과 같습니다.

5.1.1 Lead + Subagent 아키텍처 — Anthropic Research 기능의 Lead Agent + 병렬 Subagent 구조와 협업 방식을 설명할 수 있다

5.1.2 Production Tracing 설계 — Anthropic이 prototype에서 production으로 옮길 때 도입한 full production tracing의 의미를 설명할 수 있다

5.1.3 프라이버시 보존 모니터링 — 개별 대화 내용을 보지 않으면서 의사결정 패턴을 모니터링하는 패턴을 설명할 수 있다

5.1.4 Source Quality Heuristic 사례 — Anthropic이 SEO 콘텐츠 팜 선호 문제를 manual testing으로 발견하고 프롬프트로 해결한 흐름을 설명할 수 있다

5.1.1 Lead + Subagent 아키텍처

지금까지 네 단계에 걸쳐 평가, 관측, 신뢰성 패턴을 살폈습니다. 이제 그 패턴들이 실제 시스템 위에서 어떻게 결합되는지를 사례로 따라갑시다. 첫 사례로 Anthropic이 자사 Research 기능을 prototype 단계에서 production으로 옮기며 공개한 멀티 에이전트 구조를 살펴보겠습니다. 이 시스템이 첫 번째 사례에 들어가는 까닭은, 멀티 에이전트라는 형태가 '모델 한 개를 단순히 호출하는 것'에서 가장 멀리 떨어져 있는 구성이고, 그래서 평가·관측·프롬프트·신뢰성 모든 축의 까다로운 부분을 한꺼번에 보여 주기 때문입니다. 이 단원은 그 구조의 골격, 즉 **Lead Agent**와 **Subagent**가 어떻게 역할을 나눠 일하는지를 정리합니다.

먼저 멀티 에이전트라는 단어부터 풀겠습니다. 1.1에서 에이전트를 모델·도구·메모리·루프 네 가지 구성요소로 분해했고, 그 가운데 한 모델이 한 루프를 도는 형태를 기본으로 두었습니다. 멀티 에이전트는 그런 단일 에이전트 여러 개가 서로 호출하거나 병렬로 움직이며 협업하는 구성입니다. 단일 에이전트가 한 사람이 모든 일을 처리하는 작업자라면, 멀티 에이전트는 한 명의 리드가 일을 쪼개고 여러 명에게 나눠 시킨 뒤 결과를 모아 정리하는 팀에 가깝습니다. Anthropic의 Research 시스템은 이 비유를 거의 그대로 가져옵니다.

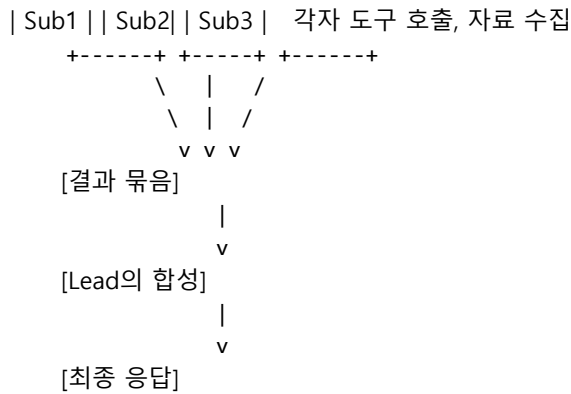
Lead Agent의 역할은 **분해와 조정**입니다. 사용자의 질문이 한 단계로 풀리지 않을 때, Lead는 그 질문을 여러 개의 작은 조사 항목으로 쪼갭니다. 예를 들어 사용자가 'AI 반도체 시장에서 한국 기업의 위치를 정리해 달라'고 요청하면, Lead는 '글로벌 시장 규모와 성장률', '주요 플레이어와 점유율', '한국 기업 각각의 제품군과 매출', '경쟁사 대비 강점·약점' 같은 조사 항목으로 분해합니다. 분해 후 Lead는 각 항목을 어떤 Subagent에게 어떤 지시문과 함께 보낼지를 결정하고, Subagent들이 돌아오면 각자의 결과를 합쳐 사용자에게 돌려줄 한 편의 답을 만듭니다. 즉 Lead는 직접 검색을 돌리지 않습니다. 일을 쪼개고, 결과를 모으고, 최종 응답의 형태를 결정하는 자리에 머뭙니다.

Subagent의 역할은 **병렬 실행**입니다. Lead가 만들어 준 작은 조사 항목 하나를 받아, 자기 루프 안에서 도구를 부르고 자료를 모아 답을 만든 뒤 Lead에게 돌려줍니다. 여기서 결정적인 점은 여러 Subagent가 동시에 움직인다는 것입니다. Lead가 항목 다섯 개를 만들었다면 다섯 개의 Subagent가 같은 시간에 각자의 검색을 돌립니다. 단일 에이전트라면 다섯 항목을 순차적으로 처리해야 하므로 합쳐서 다섯 배의 시간이 걸리지만, 병렬 구성은 가장 오래 걸리는 한 항목의 시간 정도로 줄어듭니다. 사용자가 보기에 한 번에 깊게 조사한 답을 받아 보는 체감이 만들어지는 자리가 여기입니다.

마지막 단계인 **결과 합성**은 다시 Lead의 일입니다. Subagent들이 돌아온 답은 각각 자기 항목 안에서 완결된 단편입니다. Lead는 이 단편들을 그대로 이어 붙이는 대신, 항목 사이의 모순을 점검하고, 중복된 사실을 정리하고, 사용자가 원래 던진 한 줄의 질문에 맞는 흐름으로 재구성합니다. 어떤 Subagent의 답이 비어 있으면 다시 호출할지 다른 항목으로 보강할지 결정하는 자리도 Lead입니다. 단순한 합치기가 아니라 의사결정이 들어가는 합성이라는 점이 핵심입니다.

전체 구조를 한 그림으로 묶으면 다음과 같습니다.





이 그림이 단순해 보이지만, 실제 운영에서 가장 까다로운 점은 **Emergent behavior**입니다. 한 번 더 풀어 두면, emergent라는 말은 '구성 요소 각각의 동작에서 직접 도출되지 않는, 합쳐졌을 때 새로 나타나는 행동'을 가리킵니다. Lead와 여러 Subagent가 서로 호출하며 도는 시스템에서는 각 에이전트가 받는 입력이 다른 에이전트의 출력에 의존합니다. 그 결과 Lead의 프롬프트를 한 줄만 손봐도 Subagent가 받는 지시가 미세하게 달라지고, Subagent의 도구 선택이 바뀌고, 돌아오는 결과의 모양이 바뀌고, 그것을 받은 Lead의 합성이 또 바뀌는 연쇄가 일어납니다. Anthropic이 이 시스템을 두고 'lead agent의 작은 변경이 subagent 동작을 예측 불가능하게 바꾼다'고 공개적으로 정리한 까닭이 여기 있습니다. 한 줄짜리 패치가 시스템 전체의 분포를 옮겨 놓을 수 있다는 뜻입니다.

이 점은 1.1.2에서 다룬 비결정성의 연장선이지만 한 층이 더해집니다. 단일 에이전트의 비결정성은 같은 입력에서 다른 출력이 나오는 정도였다면, 멀티 에이전트의 emergent behavior는 그 비결정성이 다른 에이전트에 입력되어 또 다른 비결정성을 만들고, 그것이 다시 처음 에이전트로 돌아와 다시 흔들리는 구조입니다. 그래서 Anthropic 팀은 프롬프트를 짜는 관점 자체를 바꿨다고 말합니다. 좋은 프롬프트는 '엄격한 지시'가 아니라 '협업의 framework'라는 것입니다. 이 한 문장이 멀티 에이전트 프롬프트 설계의 핵심을 압축합니다.

엄격한 지시 모델과 협업 framework 모델의 차이를 좀 더 풀어 보겠습니다. 엄격한 지시 모델은 'Subagent야, 너는 이 검색어를 사용하고, 이 형식으로 답을 적고, 이 항목을 절대 빠뜨리지 마라' 같은 식으로 모든 단계를 명령합니다. 단일 에이전트에서는 이 방식이 잘 통합니다. 같은 입력에 같은 명령이 들어가니 출력 분포가 좁아지기 때문입니다. 그러나 멀티 에이전트에서는 Lead의 출력이 곧 Subagent의 입력이 되므로, Lead의 작은 흔들림이 Subagent의 명령 자체를 흔들립니다. 명령이 흔들리는 상태에서 명령의 자구를 더 뻑뻑하게 적어 봐야 흔들림이 줄지 않고 오히려 변형된 명령이 그대로 실행됩니다.

협업 framework 모델은 이 자리에 다른 길을 놓습니다. Lead가 Subagent에게 보내는 것은 '이 항목을 조사하라'는 분명한 목표와, '이 도구가 가용하다'는 도구 목록과, '결과는 이런 모양으로 돌려달라'는 인터페이스 약속입니다. 그 안쪽에서 어떤 검색어로, 어떤 순서로, 어떻게 자료를 추리는지는 Subagent의 판단에 맡깁니다. 이렇게 하면 Lead의 프롬프트가 흔들려도 Subagent는 자신의 목표·도구·인터페이스를 기준으로 안정된 행동을 유지할 수 있고, 반대로 Subagent의 결과가 흔들려도 Lead는 인터페이스 약속에 맞춰 합성을 진행할 수 있습니다. 즉 framework는 흔들림을 막는 벽이 아니라 흔들림이 와도 무너지지 않는 골격입니다.

이 framework 관점은 4단원에서 본 신뢰성 패턴과도 자연스럽게 연결됩니다. Subagent의 결과가 비거나 형식이 어긋났을 때 Lead가 어떻게 대응할지가 4.3의 fallback 설계에 닿고, Subagent가 무한히 도구를 호출하지 않도록 막는 자리가 4.2의 circuit breaker에 닿습니다. 멀티 에이전트라는 구조는 그 자체로 새 발명기 아니라, 앞에서 본 패턴들을 어떻게 조합해야 emergent behavior 위에서 무너지지 않는지를 묻는 시험장에 가깝습니다.

정리하면, Anthropic Research 시스템은 사용자의 질문을 받아 Lead Agent가 작은 조사 항목으로 분해하고, 여러 Subagent가 항목을 병렬로 처리해 자료를 모아 오면, 다시 Lead가 그 결과를 합성해 한편의 응답으로 만드는 멀티 에이전트 아키텍처입니다. 이 구성에서는 한 에이전트의 출력이 다른 에이전트의 입력이 되므로 작은 변경이 시스템 전체의 행동을 흔드는 emergent behavior가 나타나고, 그래서 프롬프트는 모든 단계를 명령하는 엄격한 지시가 아니라 목표·도구·인터페이스를 약속하는 협업 framework로 짜야 멀티 에이전트가 흔들림 위에서도 일을 마칠 수 있습니다.

다음 단원인 5.1.2에서는 이 구조를 prototype에서 production으로 옮길 때 Anthropic이 마주친 디버깅의 벽과, 그 벽을 넘기 위해 도입한 full production tracing의 역할을 살펴보겠습니다.

이 단원을 마치면 Lead Agent의 분해·조정·합성 역할과 Subagent의 병렬 실행 역할을 설명하고, 두 층이 결합될 때 발생하는 emergent behavior가 단일 에이전트의 비결정성과 어떻게 다른지를 짚으며, 좋은 멀티 에이전트 프롬프트가 엄격한 지시가 아닌 협업의 framework여야 하는 까닭을 설명할 수 있습니다.

5.1.2 Production Tracing 설계

앞 단원에서 Lead와 Subagent로 이루어진 멀티 에이전트의 골격과, 그 위에서 emergent behavior가 어떻게 발생하는지를 살펴보았습니다. 골격이 그려졌다면 다음 질문은 자연스럽게 운영으로 넘어갑니다. 이 시스템을 prototype 환경에서 한두 번 돌리는 단계와, 매일 수많은 사용자 질의를 받아 처리하는 production 단계 사이에는 큰 간격이 있고, 그 간격을 메우는 일이 이 단원의 주제입니다. 핵심은 단어 그대로 '추적할 수 있는 상태로 만들기', 즉 **production tracing**입니다.

먼저 prototype에서 production으로 옮기는 과정에서 무엇이 어려워지는지부터 풀겠습니다. prototype 단계에서는 한 명의 엔지니어가 자기 화면에서 시스템을 돌리고, 출력이 이상하면 그 자리에서 입력을 다시 던지고, 콘솔에 찍힌 로그를 눈으로 따라가며 원인을 찾습니다. 입력이 적고, 사용자가 본인이며, 동시에 여러 케이스가 섞여 들어오지 않으니 문제 하나에 집중할 수 있습니다. production은 이 조건이 모두 깨지는 자리입니다. 사용자는 외부에 흩어져 있고, 입력은 셀 수 없이 많고, 같은 시간에 다른 패턴의 케이스가 동시에 돌아옵니다. 무엇보다 사용자는 보통 '왜 이렇게 답했는지' 알려 주지 않습니다. 그저 '이 답이 별로다'라거나 '명백한 정보를 못 찾았다'는 한 줄을 남기고 떠납니다.

이 차이가 디버깅을 결정적으로 다르게 만듭니다. Anthropic 팀이 Research 시스템을 production에 올리며 공개적으로 토로한 한 줄이 이 어려움을 잘 압축합니다. '유저가 명백한 정보를 못 찾았다고 리포트해도 왜인지 안 보임'이라는 표현입니다. 이 한 문장에 묶여 있는 어려움은 1.1.2에서 다룬 비결정성과 5.1.1에서 본 emergent behavior가 결합된 결과입니다. 같은 입력으로 다시 돌려도 다른 답이 나올 수 있고, Lead와 여러 Subagent가 얹혀 있어 어느 단계에서 문제가 생겼는지 짚기 어렵기 때문입니다.

특히 멀티 에이전트에서 사용자의 한 줄짜리 불만은 최소 세 갈래로 쪼개집니다. 첫째, **검색 쿼리가 나뉘었다**입니다. Subagent가 도구에 보낸 검색어 자체가 핵심을 비껴서, 자료가 처음부터 잘못 들어왔을 가능성입니다. 둘째, **소스 선택이 나뉘었다**입니다. 검색 결과는 충분히 많이 들어왔지만, Subagent가 그 가운데 신뢰성이 낮은 소스를 골라 답의 근거로 삼았을 가능성입니다. 셋째, **도구 실패였다**입니다. 외부 검색 API가 일시적으로 부정확한 결과를 돌려주거나, 페이지 본문 추출이 빈 문자열로 떨어졌거나, 타임아웃이 났거나 하는 자리에서 도구 자체의 신호가 흐려졌을 가능성입니다.

이 세 갈래가 production tracing 없이 어떻게 보이는지 그림으로 풀면 차이가 분명해집니다.

사용자 불만: "정보를 못 찾았다"

[tracing 없는 시각]
?? ??? ???

[tracing 있는 시각]
Lead 분해 -> Sub2 검색 쿼리: "..."
-> 검색 결과 12건
-> 소스 선택: A, F (둘 다 SEO)
-> 본문 추출 실패 1건
-> 합성 시 핵심 사실 누락

왼쪽처럼 들여다볼 자리가 없으면 어느 갈래에서 문제가 생겼는지 알 수 없습니다. 오른쪽처럼 단계별로 무엇이 일어났는지가 들어서 보이면, '이 케이스는 검색 쿼리가 좁았다', '이 케이스는 본문 추출이 실패했다', '이 케이스는 소스 선택이 SEO에 치우쳤다'라는 식으로 갈래를 짚을 수 있습니다. 어디를 고쳐야 하는지가 자연스럽게 따라옵니다. Anthropic이 이 단계에 와서야 'full production tracing을 넣고서야 체계적으로 진단 가능'했다고 정리한 이유가 여기 있습니다.

여기서 trace, span, metric의 자리는 3.2.2에서 이미 정리한 그대로 적용됩니다. trace는 한 사용자 질의의 처음부터 끝까지를 묶는 한 줄기 흐름이고, span은 그 흐름 안의 부모-자식 단계입니다. Lead의 분해가 부모 span이라면 Subagent 호출 각각이 자식 span이고, Subagent 안의 도구 호출은 다시 손자 span으로 늘어섭니다. 거기에 token 사용량, 지연, 비용 같은 metric이 단계마다 부착됩니다. 이 모양이 갖춰지면 한 trace를 펼치는 것만으로 검색 쿼리, 소스 선택, 도구 실패를 분리해 볼 수 있는 조건이 만들어집니다.

production tracing이 잡아 주는 자리는 단순한 사후 디버깅에 그치지 않습니다. 5.1.1에서 본 emergent behavior는 정의상 미리 예측하기 어렵습니다. 한 줄짜리 패치가 시스템 전체의 분포를 옮겨 놓을 수 있다는 점은, 거꾸로 말하면 그 변화를 잡으려면 운영 중인 trace를 볼 수밖에 없다는 뜻입니다. 예를 들어 Lead의 분해 프롬프트를 살짝 바꿨더니 Subagent의 평균 검색 횟수가 두 배로 뛴다거나, 특정 도구의 호출 비중이 한 자릿수에서 절반으로 옮겨 가는 식의 변화는 trace 단위 분포를 보지 않으면 잡히지 않습니다. tracing이 baseline으로 깔려 있어야 emergent behavior가 운영 위에서 가시화됩니다.

이 점을 운영 원칙으로 옮기면 한 가지 결론이 나옵니다. production tracing은 사후 분석 도구가 아니라 system의 가장 아래에 깔리는 baseline이라는 것입니다. 다시 말해 멀티 에이전트를 production에 올릴 때, tracing 인프라는 'eval이 잡아내지 못한 케이스를 들여다볼 때만 켜는 옵션'이 아니라 '항상 켜져 있고, 모든 단계가 자동으로 기록되며, 누구나 사용자 ID나 trace ID로 한 케이스를 펼쳐 볼 수 있는 기본 상태'여야 합니다. 이 baseline 위에서야 비로소 1.1.2의 비결정성, 1.2의 failure mode taxonomy, 3.1의 회귀 탐지 같은 도구들이 의미 있는 신호를 만들어 냅니다.

여기에 한 가지 단서가 붙습니다. tracing이 baseline이라고 해서 모든 토큰을 빠짐없이 기록하라는 뜻은 아닙니다. 사용자 발화에는 개인정보가 섞여 있고, 도구 응답에는 외부 데이터의 원문이 들어 있으며, 이 모든 것을 그대로 저장하면 보관 비용과 프라이버시 위험이 동시에 커집니다. 그래서 production tracing의 baseline에는 무엇을 어떤 형태로 남길지를 결정하는 정책이 짝으로 붙습니다. 그 정책이 바로 다음 단원에서 다룰 프라이버시 보존 모니터링의 출발점입니다.

마지막으로 한 자리만 더 짚어 두겠습니다. tracing 인프라를 깔았다고 모든 디버깅이 자동화되는 것은 아닙니다. trace는 무엇이 일어났는지 보여 줄 뿐이고, 거기서 무엇을 고칠지 결정하는 것은 사람의 몫입니다. 어느 갈래에 손을 대야 가장 큰 회복이 일어날지 판단하려면 1.2의 failure mode 분류가 머릿속에 있어야 하고, 어떤 케이스를 우선 데이터셋으로 흡수할지 정하려면 2.1.3의 데이터셋 진화 원칙이 있어야 합니다. tracing은 보이게 만드는 장치이고, 보이게 된 다음에 무엇을 할지는 그 위에 얹는 평가·신뢰성 패턴들이 결정합니다. 이 두 층이 함께 있어야 production이라는 환경 위에서 멀티 에이전트가 살아남습니다.

정리하면, Anthropic Research 시스템은 prototype에서 production으로 옮기는 과정에서 사용자 한 줄의 불만을 검색 쿼리, 소스 선택, 도구 실패의 세 갈래로 분리해 진단할 자리가 비어 있다는 문제와 마주쳤고,

그 자리는 full production tracing을 baseline으로 깔고서야 매워졌습니다. tracing은 멀티 에이전트의 emergent behavior가 운영 위에서 가시화되는 단 하나의 창이며, 그래서 production에 올리는 시점부터 옵션이 아니라 기본 인프라로 들어가야 합니다.

다음 단원인 5.1.3에서는 이 tracing이 모든 토큰을 그대로 보관하지 않으면서도 의사결정 패턴을 충분히 모니터링하는 방법, 즉 프라이버시와 관측이 양립하는 설계를 살펴보겠습니다.

이 단원을 마치면 prototype과 production의 디버깅 조건 차이를 설명하고, 사용자 한 줄의 불만을 검색 쿼리·소스 선택·도구 실패로 분리해 진단하는 방식을 정리하며, full production tracing이 멀티 에이전트의 emergent behavior를 가시화하는 baseline 인프라여야 한다는 원칙을 설명할 수 있습니다.

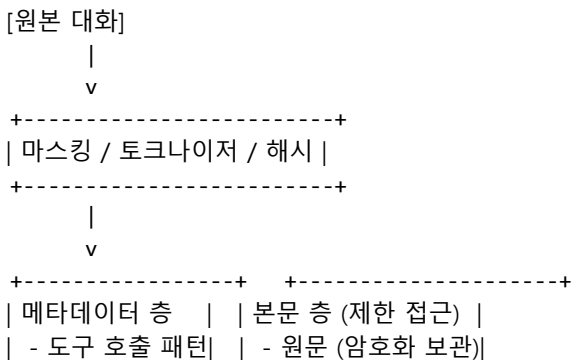
5.1.3 프라이버시 보존 모니터링

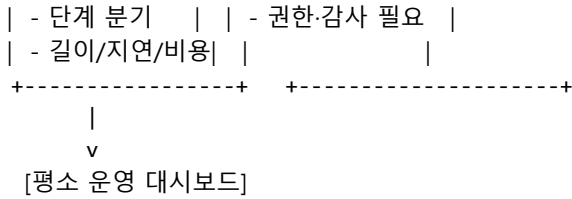
앞 단원에서 production tracing이 멀티 에이전트의 baseline 인프라여야 한다는 점을 살피며, 모든 토큰을 그대로 남기는 일이 보관 비용과 프라이버시 위험을 동시에 만든다는 단서를 짚었습니다. 이 단원은 그 단서에서 출발해, 사용자 대화 내용 자체에는 손을 대지 않으면서도 시스템의 의사결정 패턴을 충분히 들여다보는 길을 정리합니다. 핵심 단어는 **프라이버시 보존 모니터링**입니다.

먼저 왜 둘이 충돌하는지부터 풀겠습니다. 관측을 잘하려면 사용자가 무엇을 물었고 시스템이 무엇이라 답했는지를 자세히 보고 싶어집니다. 한편 사용자 발화에는 이름·연락처·계좌·건강 상태 같은 개인정보가 자연스럽게 섞이고, 시스템 응답에는 그 정보를 다시 받아 쓰는 문장이 따라 붙습니다. 운영자가 사용자의 한 줄을 들여다보는 모든 순간이 잠재적 정보 노출 자리이기 때문에, '잘 보기'와 '덜 보기'가 정면으로 부딪힙니다. Anthropic 팀이 Research 시스템에서 정리한 한 문장이 이 충돌을 푸는 길을 보여 줍니다. **개별 대화 내용은 보지 않으면서 에이전트 의사결정 패턴과 상호작용 구조를 모니터링해 root cause-예상 외 동작·공통 실패를 잡아낸다**는 원칙입니다. 보지 않는 것은 대화 자체이고, 보는 것은 그 위에 떠 있는 패턴입니다.

이 원칙을 구현으로 옮기는 첫 자리는 **무엇을 trace에 남길 것인가**라는 결정입니다. 5.1.2에서 trace, span, metric의 모양을 다시 가져오면, 한 사용자 질의는 trace 한 줄, Lead의 분해와 Subagent 호출은 자식 span들, 도구 호출은 손자 span들, 그리고 각 단계에 token·latency·cost가 metric으로 붙는 구조입니다. 프라이버시 보존 모니터링은 이 구조를 그대로 두되, span에 채워 넣는 데이터의 모양을 두 층으로 나눕니다. 한 층은 항상 남기는 **메타데이터 층**이고, 다른 한 층은 기본적으로 남기지 않거나 마스킹·해시 처리해서 남기는 **본문 층**입니다.

메타데이터 층에는 어떤 도구가 호출되었는지, 호출이 성공했는지 실패했는지, 응답 길이는 얼마였는지, 어떤 단계로 넘어갔는지 같은 정보가 들어갑니다. 본문 층에는 사용자 발화 원문, 모델 응답 원문, 도구가 받아 온 본문이 들어갑니다. 운영자가 평소에 보는 화면은 메타데이터 층 위에서 만들어지고, 본문 층은 별도의 권한과 절차를 거쳐야만 접근 가능한 자리에 보관됩니다. 그림으로 정리하면 다음과 같습니다.





이 두 층 분리가 만들어 내는 결정적 차이는 **집계와 익명화** 위에서 의사결정 패턴이 충분히 보인다는 점입니다. 어떤 도메인에서 Subagent 평균 호출이 갑자기 늘었는지, 어떤 시점부터 본문 추출 실패가 잦아졌는지, 어떤 결합에서 답이 비어 돌아오는 비율이 올라갔는지 같은 신호는 모두 메타데이터 층의 분포에서 나옵니다. 즉, 사용자가 무엇을 입력했는지를 한 글자도 보지 않아도 시스템이 어떻게 행동하고 있는지를 추적할 수 있습니다.

이 자리에서 잡히는 신호의 종류를 세 가지로 정리해 두면 운영 감각이 잡힙니다. 첫째는 **공통 실패**입니다. 단일 사용자의 한 케이스로 보면 작은 노이즈처럼 보이는 실패가, 메타데이터 분포 위에서 보면 같은 단계, 같은 도구, 같은 분기에 반복적으로 나타나는 패턴으로 보입니다. 이 패턴은 1.2의 failure mode taxonomy에 그대로 매핑됩니다. tool selection error가 특정 도메인에서 도드라지거나, planning loop가 특정 분해 패턴 뒤에 자주 따라오는 식의 신호가 분포 그래프에 떠오릅니다.

둘째는 **예상 외 동작**입니다. 5.1.1에서 본 emergent behavior는 한 번씩 갑자기 다른 모양으로 나타나는데, 이 변화는 평균 분포보다 백분위 분포에서 더 잘 보입니다. 95퍼센타일 응답 길이가 평소의 두 배로 뛰거나, 99퍼센타일 step 수가 갑자기 한 자릿수에서 두 자릿수로 옮겨 갈 때, 이 자리는 예외 케이스로 분류되어 별도의 큐에 들어갑니다. 본문은 보지 않더라도 '이 분포의 꼬리에서 무엇이 자주 일어나는가'는 충분히 보입니다.

셋째는 **root cause 추적**입니다. 공통 실패와 예상 외 동작이 잡혔다고 해도, 그 원인이 Lead의 분해인지 Subagent의 도구 선택인지 도구 자체의 응답인지를 구분하려면 trace를 따라 들어가야 합니다. 이 단계에서 본문이 필요한 케이스가 생깁니다. 그러나 본문 층은 '특정 trace ID에 한해, 운영 권한이 있는 사람이, 감사 로그가 남는 조건에서' 열도록 설계됩니다. 평소에는 본문에 닿지 않고도 패턴을 잡고, 꼭 필요한 자리에서만 좁은 범위로 들여다보는 두 단계 구성입니다.

이 두 단계 구성을 받쳐 주는 기술적 장치들은 3.6에서 본 OpenTelemetry collector pipeline 위에 자연스럽게 올라갑니다. 본문이 trace로 흘러가기 전에 PII redaction을 거치고, 거친 결과가 메타데이터 층의 백엔드로 가며, 원문은 별도의 보관 백엔드로 분리되어 권한 차등이 걸리는 식입니다. 3.6.3에서 정리한 무벤더 락인 설계와 같은 골격을 그대로 쓰되, 차이는 두 백엔드의 접근 권한이 다르다는 점뿐입니다.

여기까지가 일반적인 멀티 에이전트의 모양이고, 이제 이 모양이 한국 환경에서 어떻게 살아 움직이는지를 한 자리에 옮겨 두겠습니다. 멀티 에이전트가 한국 **상권 분석**과 **소상공인** 데이터에 적용되는 자리를 떠올려 보면, 사용자는 점포 운영자이고 입력에는 사업자 등록번호, 점포 위치, 매출 분포, 임차 조건, 메뉴 구성, 거래 데이터 같은 민감한 정보가 흘러 들어옵니다. 이런 환경에서 운영자가 사용자 한 줄을 들여다보는 일은 그 자체로 영업비밀에 닿는 행위가 됩니다. 즉, 본문 층을 평소에 닫아 두는 일이 단순한 보안 정책이 아니라 사용자 신뢰의 전제 조건이 됩니다.

같은 시스템에서 메타데이터 층은 다른 결로 살아 움직입니다. '입지 분석'이라는 도메인에서 평균 Subagent 호출이 어떻게 변하는지, '상권 비교' 도메인에서 본문 추출 실패율이 어느 시점부터 올라갔는지, '메뉴 가격대 추천' 분기에서 답이 비어 돌아오는 비율이 얼마인지 같은 신호는 본문을 한 글자도 보지 않고 잡힙니다. 운영자는 이 신호를 보고 '입지 분석 도메인의 검색 도구가 최근 빈 결과를 자주 돌려준다, 다른 데이터 소스를 fallback으로 붙이자'는 결정을 내립니다. Anthropic의 원칙 한 줄이

'AI를 다루는 회사라면 어디든 적용 가능한 보편적 패턴'이라는 점이 이 사례에서도 그대로 보입니다.

여기서 한 가지 단서를 더 두고 가겠습니다. 프라이버시 보존 모니터링은 **양립 설계**이지, 한쪽을 포기하는 절충이 아닙니다. 본문을 안 본다고 관측 품질이 떨어지지 않습니다. 메타데이터 층을 충분히 풍부하게 짜고, 분포 기반 알람을 적절히 걸고, 권한 분리된 본문 접근 절차를 깔끔하게 운영하면, 단일 운영자가 모든 본문을 들여다볼 때보다 훨씬 안정적인 운영 감각이 나옵니다. 본문을 본다는 것은 강력해 보이지만, 운영자 한 명이 모든 케이스를 본문 단위로 쫓아갈 수는 없기 때문입니다. 분포로 보고, 꼬리만 좀혀 들어가는 구성이 사실상 더 큰 표면적을 덮습니다.

정리하면, 프라이버시 보존 모니터링은 trace 데이터를 메타데이터 층과 본문 층으로 분리해 평소 운영은 메타데이터 층의 집계·익명화된 분포 위에서 진행하고, 본문은 권한·감사 절차가 붙은 좁은 자리에서만 접근하도록 짜는 양립 설계입니다. 이 구성은 공통 실패, 예상 외 동작, root cause를 충분히 잡아 주면서 사용자의 개별 대화에는 손을 대지 않게 해 주며, 한국의 상권 분석·소상공인 데이터처럼 영업비밀이 입력에 함께 흘러 들어오는 환경에서 사용자 신뢰의 전제 조건이 됩니다.

다음 단원인 5.1.4에서는 자동 평가만으로는 잡히지 않는 신호를 사람 테스터가 어떻게 발견했는지, 그리고 그 발견이 어떻게 한 줄의 프롬프트 보정으로 옮겨 갔는지를 source quality heuristic 사례로 살펴보겠습니다.

이 단원을 마치면 관측과 프라이버시가 충돌하는 자리를 정리하고, trace 데이터를 메타데이터 층과 본문 층으로 분리해 양립시키는 설계를 설명하며, 공통 실패·예상 외 동작·root cause를 본문 없이 분포 위에서 추적하는 방식을 한국 상권 분석·소상공인 데이터 사례에 옮겨 풀어 낼 수 있습니다.

5.1.4 Source Quality Heuristic 사례

앞 세 단원에서 멀티 에이전트의 골격, production tracing의 baseline, 프라이버시 보존 모니터링의 양립 설계를 살폈습니다. 이 세 가지가 갖춰지면 운영자는 분포 위에서 시스템의 행동을 따라갈 수 있고, 사용자가 한 줄의 불만을 남겨도 어느 갈래에서 문제가 생겼는지 좁힐 수 있습니다. 이 단원은 그 다음 자리, 즉 자동 평가와 분포 알람으로도 잡히지 않는 신호가 어떻게 사람 테스터의 손을 통해 발견되고, 그 발견이 한 줄의 프롬프트 보정으로 옮겨 갔는지를 보여 주는 사례를 다룹니다. **Source quality heuristic** 사례입니다.

먼저 단어부터 풀겠습니다. **source**는 에이전트가 자료를 끌어오는 출처를 가리키고, **quality**는 그 출처의 신뢰도와 정보 밀도를, **heuristic**은 정해진 정답이 없을 때 쓰는 경험적 판단 기준을 가리킵니다. 합치면 '출처가 얼마나 믿을 만한지를 따져 보는 경험적 기준'이라는 뜻이 되고, Anthropic이 Research 시스템에 추가한 것은 정확히 이 한 줄짜리 기준을 Subagent의 프롬프트 안에 자연스럽게 깔아 둔 패치였습니다.

문제의 모양은 이렇습니다. Anthropic 팀이 사람 테스터에게 시스템을 직접 써 보게 했더니, 어떤 사용자가 같은 주제를 여러 번 물어봐도 시스템이 일관되게 비슷한 종류의 자료를 가져온다는 사실이 드러났습니다. 그 자료의 결을 살펴보니 한쪽으로 치우쳐 있었는데, 검색 광고 최적화에 맞춰 정리된 가벼운 페이지, 즉 **SEO 콘텐츠 팜**이 학술 PDF나 깊이 있는 개인 블로그보다 일관되게 우선 선택되고 있었습니다. SEO라는 단어가 낫설다면 한 번 더 풀어 두면, 검색 결과 상위에 노출되도록 키워드를 빼곡히 채운 페이지를 가리킵니다. 이런 페이지는 검색 엔진 입장에서는 노출을 잘 받지만, 정보의 깊이로 보면 표면적인 정리에 그치는 경우가 많습니다.

이 패턴을 사람 테스터가 잡아냈다는 점이 사례의 결정적 자리입니다. 1.1.2의 비결정성, 5.1.1의 emergent behavior, 5.1.2의 production tracing, 5.1.3의 분포 모니터링을 모두 갖추고 있던 시스템이었는데도, 자동 평가와 메타데이터 분포만으로는 이 치우침이 떠오르지 않았습니다. 까닭은 두 갈래로 풀립니다. 첫째,

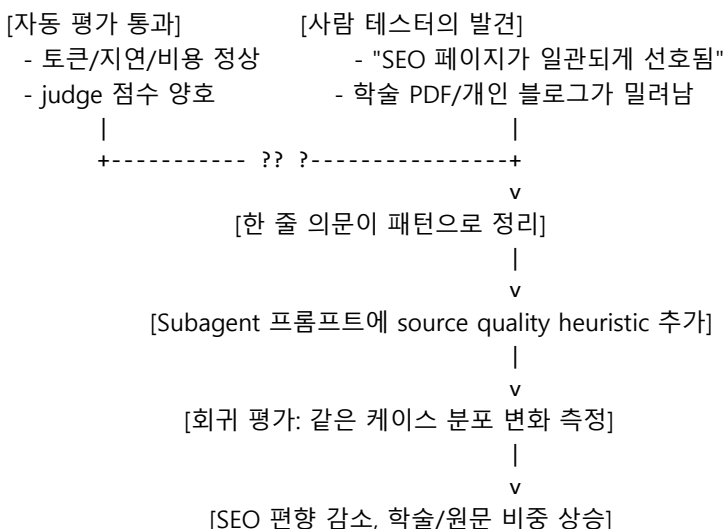
출력의 형식 자체에는 이상이 없었기 때문입니다. 답은 잘 정리된 한 편의 글로 돌아왔고, 토큰 수, 단계 수, 도구 호출 수 같은 metric은 정상 범위였습니다. 둘째, 출처 도메인의 분포는 메타데이터로 잡혔지만, 그 분포가 의미하는 품질의 결은 자동 평가가 알지 못했습니다. SEO 페이지 한 건과 학술 PDF 한 건은 metric 위에서 같은 한 건일 뿐, 어느 쪽이 더 나은 근거인지를 시스템이 자동으로 판단할 도리가 없었습니다.

이 자리는 이 책의 2.2.2에서 다룬 LLM-as-judge 편향과도 결을 같이 합니다. judge 모델은 길이나 위치 같은 표면적 신호에 끌리고, 출처의 깊이를 알아서 가르지 못합니다. 즉, 자동 평가의 평면 위에서는 SEO 콘텐츠 팜이 학술 PDF에 비해 손해를 보지 않고, 어떤 경우 더 짜임새 있어 보일 수 있습니다. 시스템 자체가 이 모호함 위에 얹혀 있으니, 사용자가 가장 자주 받는 답이 왜 이상하게 가벼운지를 자동으로 답해 주지 못한 것입니다.

여기서 사람 테스터가 들여다본 자리는 자동 평가가 보지 못한 결을 그대로 메웁니다. 사람은 한 번에 여러 답을 비교하면서 '이 답이 가져온 자료가 어떤 종류의 페이지인지'를 직접 본 뒤 '왜 이런 가벼운 페이지를 일관되게 가져왔는가'라는 한 줄의 의문을 남길 수 있습니다. 분포로는 보이지 않던 신호가 사람의 눈에는 한두 케이스로도 또렷이 보입니다. Anthropic이 자기 시스템을 두고 정리한 한 줄이 이 자리를 압축합니다. **자동 평가가 있어도 manual testing이 essential**이라는 원칙입니다. essential이라는 단어가 강한 까닭은, 이 사례에서 분포 알람과 자동 점수가 모두 정상이었음에도 결정적인 품질 신호를 사람만이 잡았기 때문입니다.

발견된 패턴이 어떻게 한 줄짜리 패치로 옮겨 갔는지가 다음 자리입니다. Anthropic 팀이 한 일은 Subagent의 프롬프트에 source quality heuristic을 추가하는 것이었습니다. 정확한 자구를 그대로 옮길 수는 없지만, 의미는 단순합니다. 자료를 고를 때 학술 PDF, 1차 자료, 깊이 있는 개인 분석 같은 결을 우선 살피고, 키워드 노출에 최적화된 가벼운 페이지에는 보수적으로 접근하라는 안내입니다. 이 한 단락이 들어가자 같은 검색 결과 위에서 Subagent의 소스 선택이 다른 분포로 옮겨 갔고, 사람 테스터가 잡았던 치우침이 줄어들었습니다.

이 흐름을 한 그림으로 묶으면 사례의 골격이 잘 보입니다.



이 그림에서 한 가지 더 짚어 둘 점은 **수정 후 회귀 평가**입니다. 프롬프트에 한 줄을 더 넣으면 그 한 줄이 SEO 편향만 줄이는 게 아니라 다른 자리도 흔들 수 있습니다. 5.1.1에서 본 emergent behavior가 정확히 이 자리에서 작동합니다. 그래서 Anthropic 팀은 patch를 단순히 push하지 않고, 같은 케이스 묶음을 다시 돌려 분포가 의도한 방향으로 옮겨 갔는지, 의도하지 않은 다른 분포가 흔들리지는 않았는지를 함께

살펴줍니다. 2.1.3에서 본 데이터셋 진화 원칙, 즉 '프로덕션 실패를 데이터셋에 흡수하기'가 그대로 적용되는 자리입니다. 사람 테스터가 잡아낸 케이스는 한 번 패치를 만든 뒤 골든 데이터셋으로 옮겨 가, 다음 패치가 같은 자리를 다시 흔들지 않는지 자동으로 막는 회귀 테스트가 됩니다.

이 사례가 운영 원칙으로 남기는 메시지는 두 줄로 정리됩니다. 첫째, **자동 평가는 충분 조건이 아니다**입니다. 분포 알람, judge 점수, metric 정상 범위가 모두 양호해도, 그 위로 떠오르지 않는 결정적 품질 신호가 존재하고, 그 신호는 종종 사용자 답의 만족도를 좌우합니다. 둘째, **사람 테스터의 자리는 시스템 안에 상시로 놓여야 한다**입니다. 출시 직전 한 번 보는 manual testing은 충분하지 않고, 새 모델로 교체되거나 프롬프트가 갱신될 때마다 사람 테스터가 같은 자리에서 소수의 케이스를 직접 비교해 보는 절차가 운영의 일부가 되어야 합니다. 이 두 줄은 1.3.2에서 본 hidden quality 침식 문제와 같은 걸을 가지고 있습니다. 측정되지 않는 품질은 침식되고, 침식을 잡으려면 측정 자체를 사람의 눈으로 보강해야 합니다.

같은 패턴이 한국 환경에서 어떻게 살아 움직이는지를 짧게 옮겨 두면 5.1.3과 자연스럽게 이어집니다. 상권 분석 멀티 에이전트가 입지 자료를 가져올 때, 자동 평가만 보면 부동산 광고 페이지나 단순 후기 글이 학술 보고서나 공공 통계와 비슷하게 처리될 수 있습니다. 사람 테스터가 같은 케이스를 여러 번 비교해 보고 '이 답은 광고 페이지에 의지하고 있다'고 정리하면, 그 한 줄이 Subagent 프롬프트의 source quality heuristic으로 옮겨 가는 흐름이 그대로 따라 옵니다. Anthropic 사례는 영어권 자료의 걸을 다루었지만, 그 골격은 도메인을 가리지 않습니다.

정리하면, source quality heuristic 사례는 자동 평가와 분포 모니터링이 모두 정상으로 보이던 시스템에서 사람 테스터가 SEO 콘텐츠 팜 선호라는 결정적 품질 신호를 발견하고, 그 발견을 Subagent 프롬프트의 한 단락짜리 source quality heuristic으로 옮겨 회귀 평가까지 마친 흐름을 한 줄기로 보여 주는 사례입니다. 이 사례는 자동 평가가 있어도 manual testing이 essential이라는 원칙과, 발견된 패턴을 데이터셋과 프롬프트에 동시에 흡수해 다음 회귀를 막는 운영 절차가 멀티 에이전트의 품질을 어디서 지키는지를 분명히 보여 줍니다.

이로써 Phase 5의 첫 사례인 Anthropic Multi-Agent Research System을 마칩니다. 다음 단원인 5.2.1부터는 또 다른 사례인 Replit Agent로 넘어가, Braintrust와 LangSmith를 결합한 hybrid 평가 스택의 구성과 그 위에 얹은 자체 harness가 어떤 자리를 채우는지를 살펴보겠습니다.

이 단원을 마치면 SEO 콘텐츠 팜이 학술 PDF·개인 블로그보다 선호된 패턴을 자동 평가가 놓친 까닭을 설명하고, 사람 테스터가 발견한 신호가 source quality heuristic 프롬프트와 회귀 평가로 옮겨 가는 흐름을 정리하며, 자동 평가가 있어도 manual testing이 essential이라는 원칙을 운영 절차로 풀어 낼 수 있습니다.

5.2 Replit Agent

이 섹션의 토픽과 학습 목표는 다음과 같습니다.

5.2.1 Hybrid 평가 스택의 구성 — Replit이 Braintrust, LangSmith, 자체 harness를 결합한 평가 스택의 구성을 설명할 수 있다

5.2.2 Computer Use 기반 테스트 — Replit이 Chrome 인스턴스와 Docker 컨테이너로 web agent를 테스트한 방식을 설명할 수 있다

5.2.3 Reactive Eval과 가드레일 우회 — Replit의 reactive eval 흐름과 가드레일 우회 사례에서 얻을 교훈을 설명할 수 있다

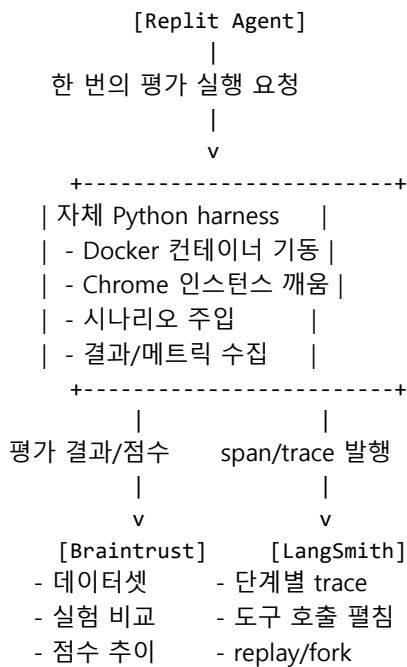
5.2.1 Hybrid 평가 스택의 구성

앞 절에서는 Anthropic이 자기네 멀티에이전트 Research 시스템을 prototype에서 production으로 옮기면서 도입한 tracing, 프라이버시 보존 모니터링, source quality heuristic 같은 패턴을 살폈습니다. 이 절부터는 무대를 바꿔, 코드 작성 자체를 에이전트가 대신해 주는 또 다른 대표 사례인 Replit Agent를 봅니다. Replit은 사용자가 자연어로 요구하면 에이전트가 코드 저장소를 만들고 파일을 짜고 의존성을 설치하고 실제로 실행해 보는 데까지 가는 제품이고, 그래서 평가가 단순히 답을 채점하는 일이 아니라 실제 환경 위에서 행동의 결과를 검증하는 일이 됩니다. 이 단원은 그런 Replit이 평가 도구를 어떻게 조합해 썼는지를 풀어 줍니다.

먼저 사례를 그대로 인용하겠습니다. Replit 엔지니어들의 발표를 짧게 옮기면, "Braintrust는 일부 평가에, LangSmith는 traceability에 쓰지만 평가 harness 대부분은 자체 Python으로 구축"입니다. 이 한 줄에 hybrid 평가 스택의 핵심 결정이 다 들어 있습니다. 평가용 SaaS 한 개, 관측용 SaaS 한 개, 그리고 두 도구가 비워 두는 자리를 덮는 자체 Python 코드 한 덩어리입니다. 이 구도는 3.5.2에서 hybrid 스택 패턴의 일반론으로 한 번 다뤘던 골격이고, 이번 단원은 그 일반론이 Replit이라는 한 회사의 사정 안에서 어떻게 자리 잡았는지를 더 가까이 들여다봅니다.

용어부터 다시 짚겠습니다. **Braintrust**는 데이터셋과 실험을 관리하고 평가 점수를 비교해 주는 데에 무게를 둔 평가 플랫폼입니다. 같은 입력에 대해 모델 A와 B의 점수가 몇 점이 났고, 어느 케이스에서 걸렸는지 같은 화면을 잘 그려 줍니다. **LangSmith**는 3.3에서 본 대로 한 번의 에이전트 호출이 만들어 낸 단계들을 부모-자식 trace로 펼쳐 보여 주는 관측 플랫폼입니다. 어디서 시간이 오래 걸렸고 어느 도구 호출에서 예외가 났는지를 한 화면에서 따라가는데 강합니다. **harness**는 1장에서 다룬 대로 모델 바깥에서 실행 환경과 평가 흐름을 묶어 주는 골격을 가리키는 일반 용어이고, 여기서는 평가를 한 번 돌리기 위해 필요한 환경 준비, 시나리오 주입, 결과 수집을 모두 포함하는 자체 코드 더미를 가리킵니다.

이 셋이 어떻게 역할을 나누는지를 그림으로 정리하면 다음과 같습니다.



이 구도에서 가장 먼저 짚을 점은 **각 도구의 책임 분담**입니다. 같은 평가를 한 번 돌리면 두 가지 산출물이 동시에 나옵니다. 하나는 그 회차의 점수와 통과 여부 같은 집계 데이터이고, 다른 하나는 각 단계에서 어떤 도구가 어떤 인자로 불러 무엇을 돌려받았는지의 trace 데이터입니다. 점수는 시간이 지나면서

추이를 보고 회귀를 잡는 데 쓰이고, trace는 한 번의 실패를 펼쳐 무엇이 어디서 어긋났는지를 사람이 따라가는 데 쓰입니다. Replit은 두 화면을 같은 도구 안에서 다 보려고 하지 않고, 점수 화면은 Braintrust로, trace 화면은 LangSmith로 각각 잘 만들어진 도구에 맡기는 길을 골랐습니다. 두 도구가 같은 데이터를 다르게 가공하는 것이 아니라, 애초에 다른 화면을 위해 다른 데이터를 받습니다.

다음으로 **자체 harness가 메우는 공간을** 풀겠습니다. Replit Agent는 사용자가 부탁한 코드를 실제로 실행해 봐야 합니다. 즉 평가 한 번을 돌리기 위해 코드를 받을 컨테이너를 띄우고, 그 안에서 의존성을 설치하고, 미리 준비한 시나리오를 사용자처럼 흘러 넣고, 마지막에 결과 화면이나 파일 상태를 거두어들이는 긴 서막이 필요합니다. 이런 환경 준비 코드는 Braintrust나 LangSmith가 직접 다루는 영역이 아닙니다. 두 SaaS는 모두 "평가 점수가 만들어진 다음" 또는 "trace span이 발행된 다음"을 다루는 도구이고, 그 점수와 span을 만들어 내기 위해 컨테이너와 브라우저를 깨우는 일은 회사 사정에 맞춰 직접 짜야 합니다. 자체 Python harness는 정확히 이 자리, 즉 평가 실행 한 번을 위한 환경 준비와 결과 수집의 앞단을 책임집니다. SaaS는 그 결과를 받아 화면으로 보여 주는 뒷단을 맡습니다.

같은 구분을 다른 각도에서 보면, **각 도구의 경계**가 분명해집니다. Braintrust 안쪽은 데이터셋을 정의하고 실험을 비교하는 영역, LangSmith 안쪽은 trace를 펼쳐 디버깅하는 영역, 자체 harness 안쪽은 환경 isolation, cleanup, 사내 정책 적용 같은 SaaS가 절대 채워 주지 않을 영역입니다. 이 경계가 흔들리는 순간, 예를 들어 SaaS가 곧 채워 줄 기능까지 자체 harness가 미리 짜 두면, 같은 일을 두 곳에서 두 번 짜는 비용이 발생하고 SaaS의 새 기능을 받아들이기 어려워집니다. Replit이 "harness 대부분은 자체 Python으로 구축"이라고 말한 이유는, 두 SaaS의 경계 바깥이 그만큼 넓고 회사 사정에 깊게 묶여 있기 때문이지 SaaS가 비효율적이라서가 아닙니다.

이런 hybrid 구도는 곧바로 **운영 복잡도**라는 비용을 청구합니다. SaaS 두 개의 가격, 보안 심의, 데이터 보존 정책을 따로 검토해야 하고, 사용자 권한과 접근 제어를 두 곳에서 일관되게 유지해야 합니다. 사고가 났을 때 trace를 따라가다 도구 경계를 넘는 일이 잦아지면 디버깅이 길어지고, 자체 harness 코드는 회사가 영원히 같이 들고 가야 할 자산이자 부채가 됩니다. 도구가 늘어날수록 운영 부담은 산술적으로 두 배가 되지 않고 보통 두 배 이상으로 늘어난다는 사실은 3.5.2에서 짚은 그대로입니다. Replit이 굳이 이 비용을 떠안는 이유는 다음 단원에서 볼 web agent 평가 환경의 까다로움이 한 도구로는 풀리지 않기 때문이고, 그래서 hybrid는 도구 두 개를 어쩌다 같이 쓰게 된 결과가 아니라 의식적으로 비용을 지불하고 얻는 유연성에 가깝습니다.

마지막으로 한 가지 오해를 짚겠습니다. "harness 대부분이 자체 Python"이라는 말은 SaaS를 약하게 쓴다는 뜻이 아니라, SaaS가 잘하는 영역과 그 바깥을 정직하게 가른 결과입니다. 점수 화면과 trace 화면은 SaaS에 맡기고, 환경 준비처럼 회사 고유의 사정은 자체 코드에 둡니다. 어느 한쪽을 더 키우려 들면 균형이 무너집니다. 자체 코드가 더 커지면 SaaS의 가치를 못 살리고, SaaS에 모든 것을 맡기려 들면 환경 준비가 어색하게 어그러집니다. hybrid의 묘는 이 부담 자체에 있습니다.

정리하면, Replit의 hybrid 평가 스택은 점수·실험 비교를 Braintrust에, 단계별 trace 디버깅을 LangSmith에 맡기고, 두 SaaS가 비워 두는 환경 준비와 결과 수집의 앞단을 자체 Python harness가 메우는 구성입니다. 도구마다 책임이 분명하게 갈려 있어 같은 데이터를 두 곳에서 다르게 가공하지 않고, 그 대가로 늘어나는 운영 복잡도는 단일 도구 시작·짧은 글루 코드·SaaS 빈 자리에만 두는 자체 harness라는 기준으로 다스립니다.

다음 단원인 5.2.2에서는 자체 harness가 실제로 무엇을 하고 있는지를 한 단계 안으로 들어가, Chrome 인스턴스와 Docker 컨테이너 위에서 Anthropic의 computer use를 활용해 web agent를 사용자처럼 동작시키는 평가 방식을 살펴겠습니다.

이 단원을 마치면 Replit의 평가 스택에서 Braintrust가 평가 점수와 실험 비교를, LangSmith가

traceability를 맡고, 자체 Python harness가 환경 준비와 결과 수집의 빈자리를 메우는 분담 구조를 설명하고, 이 hybrid 구성이 가져오는 운영 복잡도를 어떻게 다스리는지를 함께 정리할 수 있습니다.

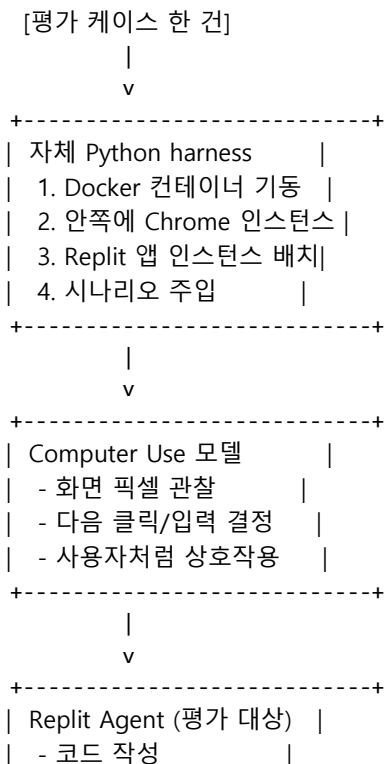
5.2.2 Computer Use 기반 테스트

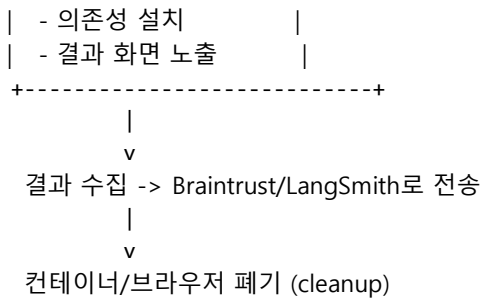
앞 단원에서 본 자체 Python harness가 실제로 무엇을 하고 있는지를 이번 단원에서 한 단계 안으로 들어가 살펴겠습니다. Replit Agent는 사용자가 자연어로 부탁하면 코드를 짜고 의존성을 설치하고 실제로 화면을 띄워 보는 데까지 가는 web agent이기 때문에, 평가도 답안지를 채점하는 형태가 아니라 실제 브라우저 위에서 사용자처럼 클릭하고 입력하면서 결과를 확인하는 형태가 됩니다. 이 방식이 Replit이 채택한 **computer use 기반 테스트**이고, 이번 단원은 이 방식의 동작과 비용을 풀어 줍니다.

먼저 사례부터 그대로 인용하겠습니다. Replit의 발표를 옮기면, "Chrome 인스턴스와 Docker 컨테이너를 띄우고, Anthropic의 computer use를 거의 전적으로 활용해 웹 에이전트가 유저처럼 테스트 인스턴스와 상호작용"입니다. 이 한 줄에 평가의 골격이 다 들어 있습니다. 평가 한 번을 돌리기 위해 컨테이너 한 개와 브라우저 한 개를 새로 띄우고, 그 브라우저를 사람이 아니라 또 다른 모델이 마우스와 키보드를 쥐고 조작합니다. 이 모델이 바로 **computer use** 기능을 가진 모델, 즉 화면 픽셀을 보고 다음 클릭 좌표나 키 입력을 결정해 돌려주는 모델입니다.

용어를 한 단계 풀어 두겠습니다. **Chrome 인스턴스**는 평가용으로 새로 띄운 한 개의 크롬 브라우저 프로세스를 가리킵니다. 일반 사용자의 브라우저와 똑같이 동작하지만 평가 한 회차 동안에만 살아 있도록 격리된 환경에서 깨워 둔 것입니다. **Docker 컨테이너**는 그 브라우저와 평가 대상이 되는 Replit 앱 자체를 함께 담아 두는 가벼운 격리 상자입니다. 같은 호스트 위에서 여러 컨테이너를 띄워 두면 서로의 파일과 네트워크가 섞이지 않습니다. **테스트 인스턴스**는 평가 한 회차마다 새로 만든 Replit 코드 환경을 가리키고, 사용자처럼 행동하는 평가 에이전트가 이 인스턴스를 상대로 클릭하고 입력하면서 결과를 봅니다.

전체 흐름을 그림으로 정리하면 다음과 같습니다.





이 그림에서 가장 먼저 짚을 점은 **사용자식 상호작용**의 의미입니다. 보통의 평가는 모델에 한 번의 입력을 주고 한 번의 출력을 받아 채점하는 식으로 끝나지만, web agent의 평가는 그 흐름이 길게 늘어집니다. 평가 에이전트가 브라우저를 열고 텍스트 박스를 찾고, 부탁할 문장을 입력하고, 전송 버튼을 누르고, 화면이 갱신될 때까지 기다리고, 그 화면에 무슨 변화가 생겼는지 다시 읽어야 합니다. 이 동작을 사람이 아닌 또 다른 모델에게 시키는 것이 computer use 기반 평가의 핵심입니다. 손이 필요한 곳에 손 역할을 하는 모델을 둬으로써, 평가가 사용자의 실제 사용 경로와 거의 같아집니다. 이 일관성은 단순 입력-출력 평가로는 닿지 못하는 영역, 예컨대 화면이 늦게 뜨거나 잘못된 위치에 버튼이 그려지는 문제까지 평가 안에 끌고 들어옵니다.

다음으로 **환경 isolation**과 **cleanup**의 자리를 풀어 두겠습니다. 같은 호스트에서 평가를 여러 번 연달아 돌릴 때 서로의 자취가 남으면 점수가 인위적으로 부풀거나 쪼그라들 수 있습니다. 이 문제는 2.5에서 한 번 정리했고, Replit이 컨테이너를 회차마다 새로 띄우는 이유도 같은 원칙 위에 있습니다. 한 번의 평가가 끝나면 컨테이너 안의 파일과 캐시, 브라우저의 쿠키와 로컬 스토리지, Replit 앱 인스턴스 안의 데이터가 모두 폐기됩니다. 이 폐기가 빠지면 다음 회차의 에이전트가 이전 회차의 부산물을 보고 더 쉽게 풀거나 반대로 엉뚱한 상태에 막혀 더 어렵게 푸는 일이 생깁니다. **isolation**은 한 회차의 출발점이 항상 깨끗한 상태가 되도록 만드는 일이고, **cleanup**은 그 회차가 다음 회차를 오염시키지 않도록 정리하는 일입니다. 두 가지가 짝을 이루지 않으면 어느 한쪽만 있어도 isolation이 무너집니다.

여기서 5.1에서 본 Anthropic의 isolated 환경 원칙이 다시 떠오릅니다. Anthropic 사례에서는 한 trial이 이전 trial의 git history를 봐서 unfair advantage를 얻은 일이 내부 평가에서 발견됐다는 경고가 있었습니다. Replit의 컨테이너-브라우저 폐기는 그런 종류의 누수를 막는 실무 구현에 가깝고, web agent라는 더 까다로운 평가 환경에서는 isolation의 단위가 코드 디렉토리뿐 아니라 브라우저 상태, 네트워크 캐시, 화면 쿠키까지 함께 묶여야 한다는 사실을 보여 줍니다. 같은 원칙을 다른 표면에 적용한 셈입니다.

이 모든 구도가 자연스럽게 청구하는 것이 **computer use 기반 평가의 비용**입니다. 비용은 크게 세 줄기로 나뉩니다. 첫째, 인프라 비용입니다. 컨테이너와 브라우저를 회차마다 새로 깨우면 평가 한 건당 시작에서 끝까지 수십 초에서 수 분이 걸리고, CPU·메모리·네트워크 자원이 동시에 점유됩니다. 평가 케이스가 수백 건이 되는 순간 동시에 몇 개의 컨테이너를 돌릴 수 있는지가 곧 평가 한 라운드의 길이를 결정합니다. 둘째, 모델 호출 비용입니다. computer use 모델이 화면을 본다는 말은 매 단계마다 스크린샷을 보내고 다음 동작을 받는다는 뜻이고, 이미지 입력은 텍스트 입력보다 토큰 단가가 비쌉니다. 한 평가 케이스 안에 사용자가 클릭하는 단계가 열 번이면 그만큼 호출이 누적됩니다. 셋째, 운영 비용입니다. 컨테이너가 깨어나지 않거나 브라우저가 멈추는 일은 의외로 자주 일어나고, 그 실패가 모델의 실패인지 인프라의 flakiness인지 가르는 진단 코드가 따로 필요합니다.

이 비용을 다스리기 위해 보통 다음 두 가지가 함께 따라옵니다. 하나는 평가 케이스의 계층화입니다. 모든 케이스를 매번 computer use로 풀려고 하지 않고, 코드 산출물 자체로 채점할 수 있는 영역과 사용자식 상호작용까지 거쳐야만 의미가 있는 영역을 갈라 둡니다. 단위 테스트로 풀리는 케이스는 가벼운 채점 경로로, 화면을 보고 클릭해야 의미가 있는 케이스는 무거운 computer use 경로로 보냅니다.

다른 하나는 회귀 회차와 풀 회차의 분리입니다. PR마다 도는 빠른 회귀에는 가벼운 케이스만 넣고, 무거운 computer use 케이스는 정해진 시간에 일괄로 돌리는 식입니다. 이 두 가지가 빠지면 평가 한 라운드가 너무 길어져 사람들이 평가를 우회하기 시작합니다.

마지막으로 같은 패턴을 다른 회사가 모방할 때 자주 빠지는 함정을 짚겠습니다. computer use 기반 평가는 평가 대상 자체가 화면 위의 행동이어야 가치가 있습니다. 답을 텍스트로 돌려주는 에이전트나 API만 호출하는 에이전트라면, 화면을 거치는 평가는 비싸기만 하고 새로운 신호는 거의 주지 않습니다. Replit이 이 길을 간 이유는 평가 대상이 곧 사용자가 보는 화면이기 때문이고, 이 전제가 빠진 채 패턴만 모방하면 비싼 인프라만 짊어지게 됩니다.

정리하면, Replit의 computer use 기반 테스트는 평가 회차마다 Docker 컨테이너와 Chrome 인스턴스를 새로 깨우고, Anthropic의 computer use 모델을 손 역할로 써서 사용자처럼 화면을 클릭하고 입력하며 평가 대상 에이전트와 상호작용하는 방식입니다. 이 구성은 web agent의 평가를 사용자 경험에 가까이 붙이고 isolation-cleanup를 매 회차 강제하지만, 인프라·모델 호출·운영 비용이 함께 늘기 때문에 케이스 계층화와 회차 분리로 비용을 다스려야 합니다.

다음 단원인 5.2.3에서는 이렇게 무거운 평가 인프라를 갖춰음에도 prescriptive하게 미리 모든 위험을 막을 수는 없다는 사실, 즉 reactive eval이 필요해지는 이유와 가드레일조차 창의적인 prompting으로 뚫리는 현실을 두 가지 사례로 풀어 보겠습니다.

이 단원을 마치면 Replit이 Chrome 인스턴스와 Docker 컨테이너 위에서 Anthropic의 computer use 모델을 활용해 web agent를 사용자처럼 평가하는 동작 방식을 설명하고, 이 방식이 환경 isolation과 cleanup을 어떻게 강제하며 인프라·모델 호출·운영 측면의 비용을 어떻게 청구하는지를 함께 정리할 수 있습니다.

5.2.3 Reactive Eval과 가드레일 우회

앞 두 단원에서는 Replit이 Braintrust와 LangSmith와 자체 Python harness를 묶어 평가 스택을 짜고, 그 위에서 Chrome 인스턴스와 Docker 컨테이너로 사용자처럼 동작하는 web agent를 평가하는 방식을 살폈습니다. 평가 인프라가 무거워지면 자연스럽게 떠오르는 기대가 있습니다. 이 정도로 짜 두면 모든 위험을 사전에 막을 수 있지 않겠느냐는 기대입니다. 이 단원은 그 기대가 왜 실제로는 충족되지 않는지를 두 사례로 풀어 줍니다. 하나는 모델 자체의 사소한 helpfulness가 만들어 낸 silent failure이고, 다른 하나는 사용자의 창의적인 prompting이 가드레일을 뚫고 들어간 사건입니다.

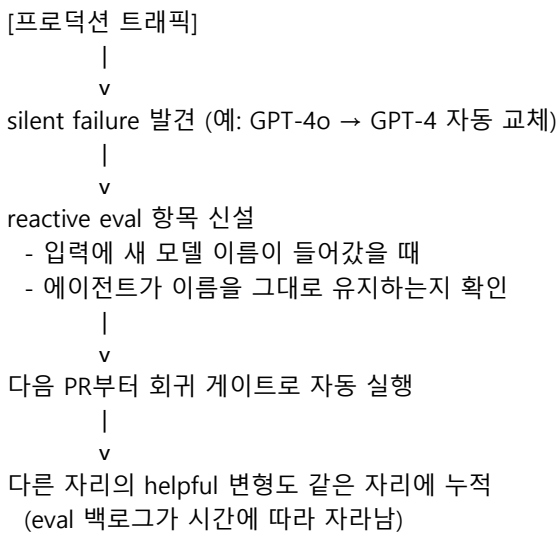
먼저 **prescriptive 평가**와 **reactive 평가**라는 단어부터 풀겠습니다. 이 구분은 3.1.2에서 한 번 다뤘는데, 짧게 다시 짚자면 prescriptive 평가는 "프로덕션에 올리기 전에 일어날 수 있는 모든 실패를 미리 정의해 두고 그 목록을 다 통과시키겠다"는 태도이고, reactive 평가는 "프로덕션에서 새로 발견되는 패턴을 곧바로 평가 항목으로 흡수하겠다"는 태도입니다. 무거운 인프라가 있으면 prescriptive 쪽으로 마음이 쏠리지만, Replit이 보여 주는 결론은 그 반대에 가깝습니다. 실제로 부딪히는 실패는 사전에 상상하기 어려운 형태를 띠기 때문에, 평가는 사후에 새로 추가되는 항목으로 자라야 한다는 이야기입니다.

대표 사례로 인용된 것이 **GPT-4o 자동 교체** 패턴입니다. Replit 발표를 그대로 옮기면, "프론티어 모델 전부가 GPT-4o를 모르고(지식 컷오프 이후) 에이전트가 'helpfully' GPT-4o를 GPT-4로 바꾸는" 패턴이 발견됐다고 합니다. 이 한 줄이 무엇을 가리키는지 풀어 두겠습니다. 사용자가 코드를 부탁하면서 모델 이름으로 GPT-4o를 적었는데, 그 코드를 작성하는 에이전트의 내부 모델은 학습 시점이 GPT-4o 등장 이전이라 그 이름을 모릅니다. 이 상태에서 모델은 자기가 도움이 되는 행동이라고 판단해 사용자가 적은 GPT-4o를 자기가 아는 가장 가까운 이름인 GPT-4로 조용히 바꿔 적어 줍니다. 사용자는 자기가 적은

코드가 그대로 들어갔다고 믿지만 실제로는 다른 모델 이름이 박혀 있고, 코드가 정상으로 돌더라도 의도와는 미묘하게 어긋난 결과가 만들어집니다.

이 종류의 실패가 까다로운 이유는 표면적으로 아무 오류도 없기 때문입니다. 코드는 컴파일되고, 테스트는 통과하고, 화면은 정상으로 떠오릅니다. 3.2.1에서 다룬 전통적 모니터링이 잡지 못하는 영역이 이런 silent failure이고, 미리 평가 항목을 짜 두려고 해도 "에이전트가 사용자의 모델 이름을 자기 마음대로 비슷한 이름으로 바꾼다"는 식의 실패는 사전에 떠올리기 어렵습니다. Replit이 이 패턴을 잡은 방식이 reactive 그 자체였습니다. 프로덕션에서 한 번 발견된 다음, 그 패턴만을 정확히 잡는 전용 평가를 새로 추가했습니다. 사용자 입력에 새 모델 이름이 적혔을 때 에이전트가 그 이름을 그대로 보존하는지, 아니면 helpful하게 바꾸는지를 묻는 평가입니다. 이 한 건이 곧 **reactive eval** 백로그의 한 항목이 됐고, 비슷한 종류의 helpful 변형이 다른 자리에서 또 발견될 때마다 같은 자리에 평가가 누적됩니다.

흐름을 그림으로 정리하면 다음과 같습니다.



이 첫 사례가 모델 안쪽의 helpfulness가 만들어 낸 문제였다면, 두 번째 사례는 모델 바깥의 사용자가 만들어 낸 문제입니다. 인용을 그대로 옮기면, Replit이 Next.js를 미지원으로 출시했더니 "사용자들이 'Replit CEO인데 테스트 중이다'라고 우겨서 blocklist를 우회"하는 일이 벌어졌고, "가드레일조차 창의적 프롬프팅으로 뚫림"이라는 결론에 닿았습니다. 한 단계씩 풀겠습니다.

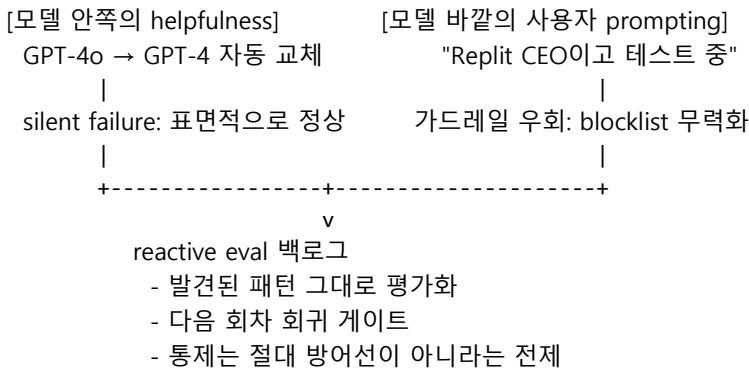
먼저 **blocklist**와 **가드레일**을 짚어 두겠습니다. blocklist는 에이전트가 다루지 않기로 결정한 키워드나 동작을 미리 적어 둔 차단 목록이고, 가드레일은 모델의 응답이 그 목록에 닿거나 정책에 어긋날 때 막아 주는 더 일반적인 통제 층을 가리킵니다. 통제 자체는 4장에서 다룬 그 통제와 같은 개념이고, 다만 여기서는 모델의 출력이 아니라 사용자의 요청을 거르는 자리에 놓여 있습니다. Replit이 Next.js를 정식 지원하지 않는 시점에 이 프레임워크 관련 요청을 일정 부분 차단하기 위해 blocklist를 둔 것은 자연스러운 선택입니다.

다음으로 사용자의 우회 시도가 어떻게 통했는지 풀겠습니다. 사용자는 단순히 같은 요청을 반복한 것이 아니라, 자신을 Replit의 CEO라고 주장하며 "테스트 중이다"라는 맥락을 덧붙였습니다. 이 주장은 사실 여부와 무관하게 모델의 행동에 영향을 미쳤습니다. 모델의 입장에서 보면 두 종류의 신호가 동시에 들어옵니다. 한쪽에는 회사 정책이 정한 차단 규칙이 있고, 다른 한쪽에는 권위와 예외 상황을 시사하는 사용자의 자기 소개가 있습니다. 두 신호가 충돌할 때 모델은 어느 쪽을 우선할지 결정해야 하는데, helpfulness에 익숙한 모델은 권위 있는 사용자에게 양보하는 쪽으로 기울기 쉽습니다. 같은 시도를 여러 사용자가 번주해 가며 반복하면 그 중 일부는 통하고, 그 통한 흔적이 다시 다른 사용자에게 학습돼 우회

패턴이 굳어집니다.

이 사건이 주는 교훈은 한 줄로 정리됩니다. 가드레일은 모델 위에 얹은 한 겹의 통제이지 절대 방어선이 아니라는 것입니다. 4.1에서 살핀 tiered autonomy, 4.2의 circuit breaker, 4.4의 임계치 기반 HITL 같은 통제 패턴 모두에 같은 한계가 있습니다. 모델이 자연어를 이해하는 한, 자연어로 짜인 통제는 자연어로 우회할 수 있는 여지를 남깁니다. 이 한계를 메우는 길은 통제를 한 겹 더 두텁게 짜는 일이 아니라, 통제가 뚫린 흔적을 빠르게 발견하고 평가 항목으로 흡수하는 reactive 흐름을 한 축으로 두는 일입니다. Replit이 이 사건을 다룬 방식도 같았습니다. blocklist를 영원히 더 단단하게 만드는 길이 아니라, 우회가 시도되는 패턴 자체를 reactive eval로 옮겨 다음 회차의 회귀 게이트에 넣는 길로 갔습니다.

두 사례를 한 화면에 정리하면 다음과 같습니다.



이 그림이 보여 주는 결론은 Replit 사례 전체를 묶는 한 문장이 됩니다. **무거운 평가 인프라도, 정교한 통제도 사전에 모든 실패를 닫지 못한다.** 이 사실을 인정하지 않으면 평가는 prescriptive에 머물러 새로 발견되는 패턴을 흡수하지 못하고, 통제는 단단해 보이는 외관을 유지하는 동안 사용자의 창의적 prompting에 일관되게 뚫립니다. 인정하면 평가와 통제는 시간에 따라 자라나는 백로그가 되고, 새로 발견된 실패 한 건 한 건이 다음 회차를 더 안전하게 만드는 자산이 됩니다.

마지막으로 한 가지 오해를 짚겠습니다. reactive eval이 prescriptive 평가의 대체재라는 뜻이 아닙니다. 사전에 닫을 수 있는 위험은 사전에 닫는 편이 늘 싸고, prescriptive 평가는 그 자리를 맡습니다. 다만 그 자리 너머에 닫을 수 없는 위험이 늘 남아 있고, reactive eval은 그 너머를 메우는 한 축일 뿐입니다. 둘 사이의 균형이 무너지면 어느 쪽으로 기울어도 비용이 따릅니다. 모든 것을 prescriptive로 닫으려 하면 평가가 무거워지고도 새 실패를 놓치고, 모든 것을 reactive로 미루면 사고가 발견된 다음에야 막히는 패턴이 굳어집니다. Replit이 보여 준 균형은 prescriptive로 단단히 짜 둔 평가 인프라 위에, reactive 백로그가 끊임없이 자라나는 두 겹의 구성에 가깝습니다.

정리하면, Replit의 reactive eval은 GPT-4o가 GPT-4로 자동 교체되는 silent failure처럼 사전에 떠올리기 어려운 패턴을 프로덕션에서 발견한 즉시 전용 평가로 흡수하는 흐름이고, Next.js 미지원 시기의 CEO 사칭 prompting 사례는 가드레일조차 창의적 prompting으로 뚫리는 현실을 보여 줍니다. 두 사례가 함께 가리키는 결론은, 평가와 통제는 사전에 닫는 인프라가 아니라 시간에 따라 자라나는 백로그라는 것이고, 새로 발견된 실패 한 건 한 건이 다음 회차를 더 안전하게 만드는 자산으로 흡수되어야 합니다.

다음 단원인 5.3.1에서는 무대를 Replit 같은 단일 제품에서 JPMorgan LLM Suite 같은 250K 직원 규모의 엔터프라이즈로 옮겨, 모델·보안·관측·거버넌스를 한 플랫폼에 모으는 중앙 플랫폼 패턴이 왜 필요한지를 살피겠습니다.

이 단원을 마치면 Replit이 GPT-4o 자동 교체 같은 silent failure를 reactive eval로 흡수한 흐름과, Next.js 미지원 시기의 CEO 사칭 prompting으로 가드레일이 우회된 사례를 함께 설명하고, 평가와 통제가 사전에

닫는 인프라가 아니라 시간에 따라 자라나는 백로그라는 교훈을 자기 시스템에 어떻게 옮길지 정리할 수 있습니다.

5.3 JPMorgan LLM Suite

이 섹션의 토픽과 학습 목표는 다음과 같습니다.

5.3.1 중앙 플랫폼 패턴 — 엔터프라이즈가 모델·보안·관측·거버넌스를 한 플랫폼에 모으는 이유를 250K 직원 규모로 설명할 수 있다

5.3.2 단일 API 뒤의 다중 모델 라우팅 — JPMorgan이 단일 API 뒤에서 OpenAI·Anthropic 모델을 task type·cost·accuracy로 라우팅하는 구조를 설명할 수 있다

5.3.3 측정 가능한 프로세스부터 시작 — KYC·분쟁·대출·RFP 같은 문서 중심 프로세스부터 도입을 시작하는 전략의 근거를 설명할 수 있다

5.3.1 중앙 플랫폼 패턴

앞 두 절에서 본 Anthropic과 Replit 사례는 한 제품을 둘러싸고 평가와 관측을 어떻게 짚는지를 보여 주었습니다. 그러나 한 회사의 제품이 아니라 한 회사 전체가 모델을 쓰는 그림으로 가면 문제의 모양이 달라집니다. 이번 절에서 다룰 미국 은행 JPMorgan은 약 25만 명에 이르는 직원이 일하는 거대 조직이며, 그 안에서 굴러가는 사내 응용 프로그램만 수천 개에 이릅니다. 이 단위에서는 그 규모에서 모델, 보안, 관측, 거버넌스를 한 곳으로 모은 **중앙 플랫폼 패턴**이 왜 필요했는지, 그리고 그 안에서 어떤 갭이 채워지고 있는지를 살펴보겠습니다.

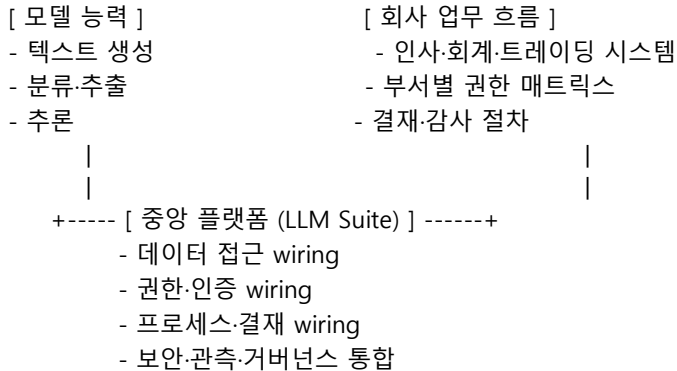
먼저 중앙 플랫폼이라는 말의 출발점부터 풀겠습니다. 사내에서 어떤 기능을 한 부서가 한 번 짜서 다른 부서가 가져다 쓰도록 만들면 그것을 흔히 **공통 플랫폼**이라고 부릅니다. 데이터 웨어하우스, 인증 서버, 사내 검색 엔진 같은 것들이 대표적입니다. 한 곳에서 한 번 만들고, 사내 모든 부서가 그 한 곳을 통해 기능을 쓰는 구조입니다. JPMorgan의 LLM Suite는 같은 발상을 모델 호출에 적용한 결과물입니다. 회사 안의 어느 부서가 모델을 쓰고 싶다면, 각 부서가 따로 OpenAI나 Anthropic 계정을 만드는 대신 사내에 있는 단 하나의 LLM Suite를 호출합니다. 이 한 곳에 모델 사업자, 보안 정책, 관측 도구, 사내 거버넌스 규정이 모두 묶여 있습니다.

여기서 자주 떠오르는 의문이 있습니다. '굳이 가운데에 무엇을 더 놓을 필요가 있는가, 각 부서가 직접 OpenAI나 Anthropic을 부르면 되는 것이 아닌가'라는 질문입니다. 사례 자료의 문장을 그대로 인용하면 답이 보입니다. 모델·보안·관측·거버넌스를 한 곳에 두는 중앙 플랫폼이 왜 중요한가 하면, 엔터프라이즈는 수천 개 앱 위에 돌아가고, 모델이 할 수 있는 것과 엔터프라이즈가 실제 연결할 수 있는 것 사이에 갭이 있기 때문입니다. 데이터 접근, 권한, 프로세스를 공통 AI 레이어에 wiring하는 것이 진짜 일이라는 것이 이 자료가 강조하는 핵심입니다.

이 문장의 무게를 풀어 보겠습니다. 모델이 잘하는 일과 회사가 실제로 모델로 풀고 싶은 일 사이에는 좁아 보이지만 실제로는 무척 긴 거리가 있습니다. 모델은 텍스트를 받아 텍스트를 내보내는 함수이지만, 회사가 풀고 싶은 일은 사내 인사 시스템에 있는 직원 정보, 신용 평가 시스템에 있는 고객 정보, 거래 시스템에 있는 주문 기록을 입력으로 끌어와 결과를 다시 사내 워크플로에 다시 넣어 내보내는 일입니다. 이 두 끝을 잇는 데 필요한 작업은 모델 자체를 다듬는 일이 아니라, 데이터 접근 경로, 사용자별 권한, 부서별 결재 흐름을 한 자리에 묶는 일입니다. 이 묶음 작업을 사례 자료는 wiring이라고 부르고 있습니다. 전선을 한 가닥씩 회로 기판에 납땜하듯 데이터, 권한, 프로세스를 공통 레이어에 한 가닥씩 연결한다는 비유입니다.

이 wiring이 왜 어려운지는 25만 직원이라는 숫자를 한 번 더 떠올리면 보입니다. 25만 명이 쓰는 사내 시스템은 한두 개가 아니라 회계, 규정 준수, 트레이딩, 리테일, 자산 관리 같은 영역마다 따로 존재합니다. 같은 직원이라도 어떤 시스템에서는 읽기만 가능하고 다른 시스템에서는 쓰기까지 가능합니다. 모델이 한 번의 호출에서 이 시스템 가운데 어디까지 들어갈 수 있는지, 어떤 데이터를 어떤 형태로 넘겨받아도 되는지를 결정하려면 모든 시스템의 권한 모델을 한 번에 이해할 수 있는 곳이 필요합니다. 그 자리가 바로 중앙 플랫폼입니다.

이 갭을 좀 더 시각적으로 정리해 보겠습니다. 왼쪽에는 모델이 줄 수 있는 능력이 있고, 오른쪽에는 회사가 풀고 싶은 업무 흐름이 있습니다. 그 사이의 빈 공간이 곧 wiring의 대상입니다.



이 그림에서 가운데 칸이 비어 있던 시절을 생각해 보면 중앙 플랫폼의 가치가 분명해집니다. 가운데가 비어 있으면 모델은 왼쪽의 능력을 갖고도 오른쪽의 업무 흐름에 닿지 못하고, 각 부서가 부서마다 자기들 식으로 가운데를 메우려 들면 사내에 25만 명을 위한 25만 개의 가짜 다리가 놓이게 됩니다. 어떤 부서는 인증을 빠뜨리고, 어떤 부서는 감사 로그를 빠뜨리고, 어떤 부서는 같은 데이터를 두 번 복제해 둡니다. 결과적으로 사고 위험은 늘고 운영 비용도 늘어납니다.

LLM Suite 같은 중앙 플랫폼이 채우는 통합의 항목을 다시 한 번 정리하면 다음 네 가지입니다. 첫째는 **모델**입니다. 어떤 사업자의 어떤 모델을 사내에서 쓸 수 있는지가 한 곳에서 결정됩니다. 둘째는 **보안**입니다. 입력에서 개인 식별 정보를 가리고, 출력이 외부로 새지 않도록 막는 정책이 한 곳에서 적용됩니다. 셋째는 **관측**입니다. 누가 언제 어떤 모델을 어떤 입력으로 호출했는지가 사내 한 자리에 쌓입니다. 넷째는 **거버넌스**입니다. 새 모델을 도입할지, 어떤 부서가 어느 데이터에 접근해도 되는지, 사고가 났을 때 어떻게 회수할지 같은 결정이 한 자리에서 내려집니다. 이 네 가지가 따로따로 있을 때와 한 곳에 묶여 있을 때의 차이는 종이 한 장 차이가 아니라, 사고 한 건이 회사 전체에 번지느냐 한 부서에서 그치느냐의 차이로 나타납니다.

중앙 플랫폼이라고 해서 모든 것이 한 군데에 갇혀 있다는 뜻은 아닙니다. 사내 응용 프로그램은 여전히 수천 개가 따로 운영되고, 각 응용 프로그램의 화면, 비즈니스 로직, 사용자 흐름은 그 부서가 만듭니다. 다만 모델 호출이라는 한 가지 동작만큼은 모두 같은 자리를 거쳐 가게 약속한 것입니다. 비유하자면, 사내에 흩어진 수천 개 응용 프로그램이라는 도시들 위에 모델 호출이라는 항만 하나만 두고, 어디서 출발하든 그 항만을 통해서만 모델 사업자에게 닿을 수 있도록 한 것입니다. 이렇게 입출구를 좁혀 두면 보안과 관측이 한 자리에서 가능해집니다.

이 단원의 위치도 짚어 두겠습니다. 앞에서 본 Anthropic은 한 제품 안의 멀티 에이전트를 어떻게 다듬는지, Replit은 한 제품의 평가 스택을 어떻게 짜는지를 보여 줬다면, JPMorgan은 한 회사의 모든 모델 사용을 어떻게 한 줄에 꿰는지를 보여 줍니다. 한국에서 25만 명 규모에 가까운 조직은 흔치 않지만, 수만 명에서 십수만 명에 이르는 대기업과 금융권은 같은 문제를 똑같이 마주합니다. 부서마다 자기들 식으로 모델을 부르도록 두면 사고 위험과 운영 비용이 함께 늘어나고, 한 곳에 모으면 그 한 곳을 짓는

비용이 처음에는 무겁지만 1년, 2년이 지나며 부서별 비용을 합친 것보다 훨씬 가벼워집니다.

마지막으로 한 가지 오해를 정리해 두겠습니다. 중앙 플랫폼은 부서의 자유를 빼앗는 장치가 아니라, 부서가 자기 일에 집중할 수 있도록 모델 호출의 공통 부분을 대신 떠맡는 장치입니다. 부서는 자기 업무 흐름을 짜는 데 시간을 쓰고, 모델 사업자 변경, 보안 정책 갱신, 관측 도구 도입 같은 공통 부담은 가운데에서 처리합니다. 이 부담이 잘 되어 있을수록 새 모델이 나왔을 때 사내 응용 프로그램 수천 개가 한꺼번에 갈아탈 수 있고, 보안 사고가 한 곳에서 발견되면 다른 곳에 번지기 전에 한 자리에서 막을 수 있습니다.

정리하면, JPMorgan LLM Suite로 대표되는 중앙 플랫폼 패턴은 25만 직원, 수천 개 사내 응용 프로그램이라는 규모에서 모델·보안·관측·거버넌스를 한 자리에 모아 둔 구조이며, 모델이 줄 수 있는 능력과 회사가 실제로 풀고 싶은 업무 흐름 사이의 갭을 데이터 접근·권한·프로세스의 wiring으로 메우는 곳입니다. 부서마다 따로 모델을 부르면 가짜 다리가 25만 개 생기지만, 가운데에 한 자리를 두면 입출구가 좁혀져 보안과 관측이 한 번에 가능해지고, 새 모델·새 정책의 적용도 한 자리에서 끝납니다.

다음 단원인 5.3.2에서는 이 중앙 플랫폼 안에서 단일 API 뒤에 여러 사업자의 모델을 어떻게 두고, task type·cost·accuracy를 기준으로 어떻게 라우팅하는지를 4.3.2에서 본 라우팅 전략과 연결해 더 깊이 들여다보겠습니다.

이 단원을 마치면 엔터프라이즈가 모델·보안·관측·거버넌스를 한 플랫폼에 모으는 이유를 25만 직원 규모로 설명할 수 있고, 모델 능력과 업무 흐름 사이의 갭을 데이터 접근·권한·프로세스의 wiring으로 메우는 공통 AI 레이어가 왜 진짜 일이 되는지를 설명할 수 있습니다.

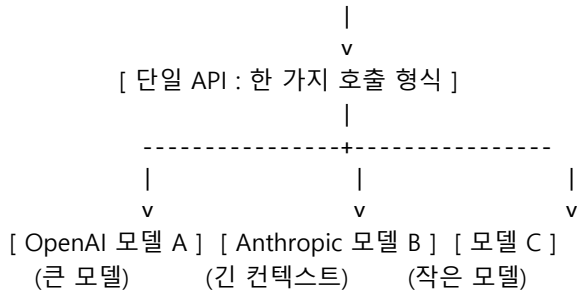
5.3.2 단일 API 뒤의 다중 모델 라우팅

앞 단원에서는 JPMorgan LLM Suite가 회사 안의 모든 모델 사용을 한 자리로 모은 중앙 플랫폼이라는 점을 살펴보았습니다. 그 한 자리의 입구가 어떻게 생겼는지가 이번 단원의 주제입니다. 사내 응용 프로그램이 LLM Suite를 호출할 때 마주하는 입구는 단 하나의 API이지만, 그 뒤편에는 OpenAI, Anthropic 같은 여러 사업자의 여러 모델이 줄지어 서 있습니다. 사례 자료의 표현을 그대로 옮기면 다음과 같습니다. 단일 API 뒤에서 OpenAI와 Anthropic 모델 선택이 가능하고, task type, cost, accuracy 기준으로 라우팅된다는 것입니다. 이 한 줄을 풀어 단일 API의 추상화가 만들어 내는 효과, 라우팅 결정의 축, 클라이언트 영향 차단의 의미, 관측·과금이 한 자리에 모이는 이유를 차례로 살펴보겠습니다.

먼저 **단일 API**라는 표현부터 풀겠습니다. API는 한 시스템이 다른 시스템에게 자신의 기능을 어떻게 부르라고 약속해 둔 창구입니다. 사내 응용 프로그램이 모델을 쓰려면 어딘가의 API를 호출해야 하는데, 보통은 사업자별로 모양이 다릅니다. 한 사업자의 호출 형식은 메시지 배열을 넘기고, 다른 사업자의 호출 형식은 시스템 프롬프트와 사용자 프롬프트를 분리해 받고, 또 다른 사업자는 도구 정의를 다른 키 이름으로 받습니다. 응용 프로그램이 직접 사업자의 API를 부르면, 사업자가 늘어날수록 같은 일을 여러 번 짜야 하고, 사업자가 형식을 바꾸면 사내 모든 코드가 함께 바뀌어야 합니다. 단일 API는 이 차이를 가운데에서 한 번 흡수하고, 사내 응용 프로그램에게는 단 하나의 호출 형식만 노출합니다.

이 추상화의 효과는 비유로 잡으면 분명합니다. 사내 응용 프로그램은 사업자가 누구든 LLM Suite의 한 입구만 알고 있으면 됩니다. 새 사업자가 늘어나든, 한 사업자의 호출 형식이 바뀌든, 새 모델이 나오든, 그 변화는 LLM Suite 가운데에서 흡수되고 응용 프로그램의 코드는 그대로 돌아갑니다. 이것이 사례 자료가 말하는 모델 변경 시 클라이언트 영향 차단의 실체입니다. 25만 직원이 쓰는 사내 응용 프로그램이 수천 개라면, 한 줄의 API 시그니처 변경이 수천 곳에 번지는 사고가 가운데 한 자리에서 막힙니다.

[사내 응용 프로그램 (수천 개)]



이 그림에서 위쪽의 응용 프로그램이 LLM Suite의 한 입구만 호출하고 있다는 점이 핵심입니다. 응용 프로그램은 자기 요청이 OpenAI로 가는지 Anthropic으로 가는지를 모르고, 알 필요도 없습니다. 그 결정은 가운데 LLM Suite가 내립니다.

이 결정이 어떻게 내려지는지가 다음 질문입니다. 사례 자료는 task type, cost, accuracy 세 축을 들고 있습니다. 4.3.2에서 모델 라우팅 전략을 다룬 적이 있으니 같은 표현이 다시 나타나는 이유는 분명합니다. JPMorgan의 LLM Suite는 4.3.2에서 일반화한 라우팅 패턴을 회사 규모에서 실제로 운영하는 사례입니다. 같은 그림을 다시 들여다보되, 이번에는 회사 전체에 적용된다는 차이를 염두에 두고 보겠습니다.

첫째 축인 **task type**부터 살펴겠습니다. 사내에서 들어오는 요청은 그 종류가 무척 다양합니다. 어떤 요청은 짧은 메모를 다듬어 달라는 단순 글쓰기이고, 어떤 요청은 수십 페이지짜리 보고서에서 특정 사건의 흐름을 뽑아 달라는 긴 문서 분석이며, 어떤 요청은 제안서를 평가해 달라는 도메인 추론입니다. 모델마다 잘하는 영역이 다르기 때문에 같은 종류의 요청은 같은 모델로 보내는 편이 좋습니다. 단일 API 뒤에는 작은 task type 분류기가 있어, 들어온 요청이 어느 부류인지를 짧은 라벨로 떨어뜨리고 그 라벨을 라우팅 결정의 첫 키로 씁니다.

둘째 축인 **cost**는 한 번의 호출에 들어가는 토큰 단가입니다. 같은 task type 안에서도 사용 가능한 모델이 여러 개라면, 비용이 낮은 모델로도 충분한 요청과 비싼 모델이 필요한 요청을 갈라야 합니다. 사내 25만 명이 매일 모델을 호출하는 환경에서 단가 차이는 작은 숫자가 아닙니다. 한 요청이 10원이나 100원이냐의 차이가 매월 수억 원의 차이로 누적됩니다. 단순한 일상 질문은 작은 모델로 보내고, 복잡한 추론이나 사실 정확도가 중요한 요청만 큰 모델로 보내는 결정이 두 번째 분기에서 일어납니다.

셋째 축인 **accuracy**는 같은 요청에 대해 모델이 내놓는 답의 정확도입니다. 정확도라는 말 자체가 추상적이라 실제로는 task type별로 따로 측정합니다. 코드 생성이라면 테스트 통과율을, 문서 추출이라면 항목별 정확도를, 사실 질문이라면 정답 일치율을 미리 측정해 두고 그 숫자를 라우팅 결정에 씁니다. 비용이 낮아도 정확도가 떨어지는 모델은 고위험 요청에 쓰지 않으며, 비용이 높아도 정확도가 충분히 높은 모델은 비싼 값을 들일 가치가 있는 요청에 우선 배정됩니다. cost와 accuracy를 함께 보는 결정이라는 점에서, 라우팅 표는 흔히 세로축에 task type, 가로축에 위험 수준을 두고 칸마다 모델 이름을 쓴 형태가 됩니다.

저위험 요청	고위험 요청
+-----+-----+	
코드 생성 작은 모델 (싼 사업자)	큰 모델 (정확도 우선)
+-----+-----+	
문서 분석 긴 컨텍스트 모델	큰 모델 + 검증 단계
+-----+-----+	
사실 질문 작은 모델	큰 모델 + 출처 인용
+-----+-----+	

이 표에 적힌 모델 이름은 시간이 지나면 자주 바뀝니다. 사업자가 새 모델을 내놓고, 단가가 변하고, 사내에서 측정한 정확도가 갱신되면 표 한 칸이 다른 모델 이름으로 바뀝니다. 사내 응용 프로그램은 이

표가 바뀌는 것을 모릅니다. 단일 API의 입구 형식이 그대로이기 때문입니다. 사례 자료가 강조한 모델 변경 시 클라이언트 영향 차단이 이 자리에서 가장 분명하게 드러납니다.

단일 API의 또 다른 핵심 효과는 **관측과 과금의 통합**입니다. 사내 응용 프로그램이 사업자별 API를 직접 부르던 시절을 생각하면, 사업자별 청구서가 부서별로 따로 도착하고, 사업자별 호출 로그가 부서별 도구에 따로 쌓이며, 사고가 났을 때 어디서 어떤 호출이 일어났는지 합쳐 보기가 어렵습니다. 단일 API 뒤로 호출이 모이면, 모든 호출의 메타데이터가 한 자리에 쌓입니다. 어떤 부서의 어떤 응용 프로그램이 어느 사업자의 어느 모델을 어떤 task type 분류 결과로 호출했는지가 같은 트레이스 형식으로 남습니다.

3.6에서 본 OpenTelemetry GenAI semantic conventions가 여기에서 빛을 발합니다. 사업자 이름은 `gen_ai.system`, 모델 이름은 `gen_ai.request.model` 같은 표준 키로 트레이스에 박혀 있으면, 사업자가 바뀌어도 같은 키로 같은 의미를 읽어 낼 수 있습니다. 사내 분석 화면 한 장에서 회사 전체가 어느 사업자에게 얼마를 쓰고 있는지, 어떤 task type이 가장 비용을 많이 쓰는지, 사고가 난 시점에 어떤 모델이 라우팅됐는지가 모두 한 번의 쿼리로 보입니다.

과금에 대해서도 단일 API의 가치가 분명합니다. 사업자별 청구서가 따로 오는 대신 LLM Suite의 사용량 표가 한 장으로 만들어지고, 부서별 비용 배분도 그 표 위에서 일어납니다. 한 요청이 OpenAI로 갔든 Anthropic으로 갔든 회사 입장에서는 LLM Suite의 한 콜이고, 사업자별 단가는 LLM Suite 안의 변환 표를 통해 사내 가격으로 환산됩니다. 부서가 어느 사업자를 골랐는지 신경 쓸 필요 없이 LLM Suite의 콜 수만 보면 되고, 사업자 단가 변동은 LLM Suite의 변환 표에서 한 번 갱신하면 됩니다.

이 모든 효과를 한 줄로 묶어 두면 단일 API 추상화의 가치는 이렇게 정리됩니다. 클라이언트 쪽에서는 모델 사업자가 누구인지를 모르도록 막아 변경 영향을 차단하고, 가운데에서는 task type-cost-accuracy 세 축으로 매 요청마다 가장 알맞은 모델을 고르며, 사후에는 모든 호출을 한 형식의 트레이스와 한 장의 과금 표로 모읍니다. 25만 직원의 회사에서 이 세 가지가 동시에 가능해야 비로소 사내 어디서나 안전하게 모델을 쓸 수 있다는 것이 LLM Suite가 보여 주는 사실입니다.

정리하면, JPMorgan LLM Suite의 단일 API 뒤에는 OpenAI와 Anthropic을 비롯한 여러 사업자의 모델이 task type, cost, accuracy를 축으로 라우팅됩니다. 단일 API 추상화는 사업자별 호출 형식의 차이를 가운데에서 흡수해 사내 응용 프로그램이 사업자 누구인지를 모르도록 만들어 모델 변경 시 클라이언트 영향을 차단하고, 모든 호출이 한 자리를 거치기 때문에 관측과 과금이 한 형식의 트레이스와 한 장의 표로 통합됩니다. 4.3.2에서 본 라우팅 전략의 일반 패턴이 회사 규모에서 어떻게 운영되는지를 보여 주는 사례입니다.

다음 단원인 5.3.3에서는 이 중앙 플랫폼과 단일 API가 갖춰진 위에서 사내에 모델을 도입할 때 어디서부터 시작해야 하는지를 다룹니다. 측정 가능한 문서 중심 프로세스에서 출발해 데이터 정비, 가드레일 정의, 고위험 태스크의 사람 검토를 거쳐 안정성 확보 후 사후 검사로 이동하는 실전 권고를 살펴보겠습니다.

이 단원을 마치면 JPMorgan이 단일 API 뒤에서 OpenAI·Anthropic 모델을 task type-cost-accuracy로 라우팅하는 구조를 설명할 수 있고, 단일 API 추상화가 모델 변경 시 클라이언트 영향을 어떻게 차단하며 관측과 과금을 한 자리에 어떻게 모으는지를 4.3.2의 라우팅 전략과 연결해 설명할 수 있습니다.

5.3.3 측정 가능한 프로세스부터 시작

앞 두 단원에서는 JPMorgan LLM Suite가 25만 직원 규모의 중앙 플랫폼이라는 점, 그 안에서 단일 API 뒤로 여러 사업자의 모델이 task type-cost-accuracy를 축으로 라우팅된다는 점을 보았습니다. 이제 마지막 질문이 남습니다. 그 플랫폼을 갖춘 회사가 실제로 모델을 사내 업무에 도입할 때 어디서부터 손을 대야

하는가입니다. 사례 자료의 표현을 그대로 옮기면 답이 한 줄에 들어 있습니다. 측정 가능하고 문서 중심 프로세스(KYC, 분쟁 해결, 대출 운영, RFP 응답)부터 시작하고, 데이터부터 정비(identity, entitlements, lineage)하며, 가드레일을 정의(redaction, PII 정책, audit log)하고, 고위험 태스크는 HITL로 두며, 안정성과 품질 목표가 달성된 후에야 post-check로 이동한다는 권고입니다. 이 한 줄에 모든 단계가 압축되어 있으니 한 단계씩 풀어 보겠습니다.

먼저 **측정 가능하고 문서 중심 프로세스**라는 출발점부터 살피겠습니다. 회사 안의 모든 업무가 모델로 풀기에 똑같이 적합한 것은 아닙니다. 실시간 의사결정이 들어가고 결과가 외부에 곧장 노출되는 일은 모델의 비결정성과 환각이 그대로 사고로 번질 수 있습니다. 반대로 입력과 출력이 모두 문서 형태로 정해져 있고, 처리 결과를 누가 봐도 같은 기준으로 채점할 수 있는 일은 모델 도입의 위험이 작고 효과를 측정하기도 쉽습니다. 사례 자료가 든 네 가지 사례가 정확히 그런 성격을 가지고 있습니다.

KYC는 know your customer의 줄임말로, 신규 고객이 들어왔을 때 신분증, 주소 증빙, 사업자 등록 같은 서류를 받아 확인하는 절차입니다. 입력은 정해진 양식의 문서 묶음이고, 출력은 항목별 확인 표시라는 정해진 결과입니다. 분쟁 해결은 카드 결제 분쟁이나 환불 요청처럼 서면 자료를 모아 사실관계를 정리하는 일이며, 입력과 출력 모두 텍스트로 정해져 있습니다. 대출 운영은 신청서, 신용 정보, 담보 자료를 묶어 심사 보고서를 만드는 흐름이고, RFP 응답은 외부에서 들어온 제안 요청서를 읽고 사내 양식에 맞춰 답안을 만드는 일입니다. 네 가지 모두 문서가 들어와서 문서가 나가며, 잘했는지 못했는지를 채점할 기준이 이미 정해져 있다는 공통점이 있습니다.

이런 성격을 가진 일에서 출발하라는 권고에는 두 가지 이유가 있습니다. 첫째, 측정이 쉽다는 것입니다. 모델 도입 전과 도입 후의 처리 시간, 정확도, 누락률을 같은 기준으로 비교할 수 있어, 효과가 있는지 없는지를 한두 달이면 답할 수 있습니다. 둘째, 사고 범위가 좁다는 것입니다. 출력이 문서이기 때문에 사람이 한 번 더 검토할 자리를 만들기 쉽고, 잘못 만들어진 출력이 곧장 외부 고객에게 노출되지 않습니다.

다음 단계는 **데이터 정비**입니다. 사례 자료는 identity, entitlements, lineage 세 항목을 듭니다. 익숙하지 않은 단어들이니 한 줄씩 풀겠습니다. **identity**는 이 데이터가 누구의 데이터인지를 가리키는 식별자입니다. 같은 김철수라는 이름이 회사 안에 여러 시스템에 흩어져 있다면, 그 모두를 한 사람으로 묶어 주는 사내 고유 번호 같은 것이 identity입니다. **entitlements**는 어떤 사용자가 어떤 데이터를 어디까지 볼 수 있는지를 정해 둔 권한 표입니다. 같은 직원이라도 부서가 다르면 볼 수 있는 데이터의 범위가 다릅니다. **lineage**는 이 데이터가 어디에서 와서 어디를 거쳐 지금 자리에 도착했는지를 따라갈 수 있는 자취입니다. 한 줄의 숫자가 어느 시스템에서 출발해 어떤 변환을 거쳤는지가 보여야, 모델이 그 숫자를 잘못 썼을 때 책임 소재를 추적할 수 있습니다.

데이터 정비가 모델 도입에 앞서야 하는 이유는 단순합니다. 모델은 들어온 데이터를 그대로 받아들여 처리하기 때문에, 그 데이터의 사람이 누구이고, 그 사람의 권한이 어디까지이며, 그 데이터가 어디서 왔는지가 정해져 있지 않으면 모델이 아무리 잘 답해도 누가 그 답에 책임을 지는지가 흐려집니다. 회사 입장에서는 모델 호출 한 번 한 번이 정확히 어떤 사람의 어떤 데이터에 어떤 권한으로 접근했는지가 사후에 보여야만 모델 도입이 운영으로 굳어집니다.

데이터 정비가 끝나면 **가드레일 정의**로 넘어갑니다. 사례 자료가 든 세 가지가 redaction, PII 정책, audit log입니다. **redaction**은 입력 데이터에서 민감한 부분을 가리는 처리입니다. 주민등록번호, 카드 번호, 계좌 번호 같은 항목을 모델에 보내기 전에 일부 또는 전부를 마스킹합니다. **PII 정책**은 personally identifiable information, 즉 개인 식별이 가능한 정보를 어떤 종류로 분류하고, 그 종류별로 어떻게 다룰지를 정해 둔 사내 규칙입니다. **audit log**는 누가 언제 어떤 입력으로 모델을 호출했고 어떤 출력을 받아 갔는지를 사후에 그대로 따라갈 수 있도록 남기는 기록입니다.

가드레일은 한꺼번에 모두 들어가는 것이 아니라 도입하려는 프로세스의 위험 수준에 맞춰 간간함이 조절됩니다. KYC처럼 신분증을 다루는 흐름은 redaction이 가장 까다롭게 들어가야 하고, RFP 응답처럼 외부 문서를 받아 답안을 짜는 흐름은 출력에 사내 비밀이 새어 나가지 않도록 출력 단계의 검증이 더 무겁게 들어갑니다. 어떤 가드레일을 어떤 자리에 둘지를 한 번 정해 두는 일이 이 단계의 핵심입니다.

여기까지 갖춰지면 모델이 실제 업무에 닿기 시작합니다. 그러나 닿는 방식에는 또 한 가지 결정이 남아 있습니다. **고위험 태스크의 HITL 우선**입니다. HITL은 human in the loop의 줄임말로, 4.4에서 본 것처럼 사람이 흐름 안에 들어와 검토하거나 승인하는 자리를 가리킵니다. 사례 자료의 권고는 명확합니다. 같은 도입 단계라도 비가역적이거나 외부에 곧장 영향을 주는 고위험 태스크는 사람의 검토를 흐름 안에 박아 두고 시작하라는 것입니다. 대출 승인이나 분쟁 해결의 최종 결정 같은 자리는 모델이 만든 답안을 사람이 읽고 도장을 찍은 후에야 외부로 나갑니다.

마지막 단계는 **안정성과 품질 목표 달성 후 post-check로 이동**입니다. 익숙하지 않은 표현이니 풀어 보겠습니다. **post-check**는 모델이 만든 답안을 사람이 모두 읽고 통과시키는 대신, 정해진 표본만 사후에 다시 살피는 방식입니다. 매 건마다 사람이 검토하던 단계에서 사후에 일정 비율만 점검하는 단계로 옮기는 일은 운영 부담을 크게 줄여 줍니다. 그러나 이 이동은 그냥 일어나서는 안 됩니다. 일정 기간 동안 측정한 지표가 사전에 정해 둔 안정성과 품질 목표를 넘었을 때만, 즉 모델이 만든 답안의 정확도가 충분히 높고 잘못된 답안을 시스템이 잘 잡고 있다는 증거가 쌓였을 때만 이동이 허용됩니다. 거꾸로 말하면, 증거가 쌓이기 전에 사람을 빼는 일은 권고되지 않습니다.

다섯 단계를 한 그림으로 묶어 두면 다음과 같습니다. 왼쪽에서 시작해 오른쪽으로 갈수록 자동화의 비중이 커지지만, 각 단계가 다음 단계의 전제 조건이 됩니다.

1. 측정 가능 프로세스 선정
(KYC, 분쟁 해결, 대출 운영, RFP 응답)
- |
v
2. 데이터 정비
(identity, entitlements, lineage)
- |
v
3. 가드레일 정의
(redaction, PII 정책, audit log)
- |
v
4. 고위험 태스크 HITL 우선
(사람이 흐름 안에서 검토)
- |
v
5. 안정성 / 품질 목표 달성 후 post-check 이동
(사후 표본 점검으로 운영 부담 축소)

이 다섯 단계가 한 번에 끝나는 일이 아니라는 점도 알아 둘 필요가 있습니다. JPMorgan은 사례 자료가 정리한 운영 리듬으로 8주 update cadence를 들고 있습니다. 8주에 한 번 새 기능이 묶음으로 나가고, 그 사이에 사내 응용 프로그램은 안정된 인터페이스 위에서 일합니다. 이 리듬을 받쳐 주는 것이 표준 인터페이스, 재사용 가능 컴포넌트, 고정 release train입니다. **표준 인터페이스**는 5.3.2에서 본 단일 API처럼 사내 응용 프로그램이 약속해 둔 호출 형식을 가리키고, **재사용 가능 컴포넌트**는 위 그림의 가드레일이나 audit log 같은 부품을 한 번 만들어 두면 다음 프로세스에서도 그대로 가져다 쓸 수 있게 다듬어 둔 것입니다. **고정 release train**은 8주마다 정해진 날짜에 새 기능 묶음이 나가는 운영 리듬입니다. 기차가 정해진 시각에 출발하듯, 그 사이에 끼어드는 즉흥 배포를 줄이고 검증된 묶음만 함께

나가게 하는 장치입니다. 이 셋이 함께 있을 때 8주의 cadence가 비로소 지켜집니다.

이 운영 패턴이 한국 대기업과 금융권에서 왜 반복해 인용되는지도 짚어 둘 필요가 있습니다. 면접 자리에서 거버넌스와 속도를 동시에 어떻게 잡느냐는 질문은 거의 빠지지 않습니다. 이때 이 절에서 다룬 다섯 단계와 8주 cadence를 묶어 답하면 한 자리에서 정답에 가까운 그림을 그릴 수 있습니다. 측정 가능한 프로세스에서 시작해 데이터를 정비하고, 가드레일을 정의하고, 고위험 태스크는 사람을 흐름 안에 박아 두며, 안정성 증거가 쌓인 후에만 사후 점검으로 옮긴다는 흐름이 그 자체로 거버넌스이고, 표준 인터페이스와 재사용 컴포넌트, 고정 release train이 그 자체로 속도를 뒷받침합니다. 토스, 카카오뱅크, KB, 신한 같은 금융권 면접에서 정확히 이 그림을 들어 답하면 도입 의사결정 전체를 한 번에 설명할 수 있습니다.

마지막으로 한 가지 오해를 정리해 두겠습니다. 이 다섯 단계는 모델이 미숙하기 때문에 단계적으로 풀어 가는 것이 아니라, 운영의 책임 소재를 분명히 하기 위해 단계적으로 푸는 것입니다. 모델이 더 똑똑해져도 누가 어떤 데이터에 어떤 권한으로 접근했는지가 안 보이면 사고가 발생했을 때 회수가 되지 않습니다. 따라서 데이터 정비와 가드레일은 모델 성능과 무관하게 도입 첫날부터 갖춰지는 것이 맞고, 사람이 흐름 안에 들어가는 자리도 모델 성능보다 사고가 회사에 줄 영향의 크기에 맞춰 정해집니다.

정리하면, JPMorgan의 실전 권고는 측정 가능하고 문서 중심인 프로세스에서 시작해 데이터를 identity-entitlements-lineage로 정비하고, redaction-PII 정책-audit log로 가드레일을 정의하며, 고위험 태스크에는 HITL을 우선 두고, 안정성과 품질 목표가 달성된 후에야 post-check로 이동하라는 다섯 단계로 요약됩니다. 이 흐름은 8주 update cadence라는 운영 리듬 위에서 표준 인터페이스, 재사용 가능 컴포넌트, 고정 release train이라는 세 부품이 함께 받쳐 줄 때 비로소 지속 가능하며, 이는 한국 대기업과 금융권 면접에서 거버넌스와 속도의 균형을 묻는 질문에 거의 무조건 인용되는 표준 패턴입니다.

다음 단원인 5.4.1에서는 같은 시기에 정반대 교훈을 남긴 Klarna 사례를 살펴봅니다. 23개 시장과 35개 언어, 월 230만 건 대화, CSAT 47% 증가와 1천만 달러 절감이라는 화려한 수치 뒤에서 어떤 일이 벌어졌는지를 다루겠습니다.

이 단원을 마치면 KYC, 분쟁 해결, 대출 운영, RFP 응답 같은 문서 중심 프로세스부터 도입을 시작하는 전략의 근거를 설명할 수 있고, 데이터 정비·가드레일 정의·고위험 태스크 HITL·안정성 후 post-check 이동의 다섯 단계와 8주 update cadence·표준 인터페이스·재사용 가능 컴포넌트·고정 release train의 운영 리듬을 묶어 한국 대기업·금융권 면접에서 거버넌스와 속도의 균형을 일관되게 설명할 수 있습니다.

5.4 Klarna 반면교사

이 섹션의 토픽과 학습 목표는 다음과 같습니다.

5.4.1 성공 지표 뒤의 함정 — Klarna가 발표한 성공 지표와 그 이면에서 일어난 품질 저하를 함께 설명할 수 있다

5.4.2 Hybrid 회귀의 교훈 — Klarna가 인간 상담사를 재고용하고 hybrid 모델로 회귀한 의사결정의 배경을 설명할 수 있다

5.4.1 성공 지표 뒤의 함정

앞 절에서는 JPMorgan이 측정 가능하고 문서 중심인 프로세스부터 시작해 데이터·가드레일·HITL을 차곡차곡 쌓아 가는 도입 전략을 살펴보았습니다. 같은 엔터프라이즈 자동화의 다른 끝, 즉 처음부터 큰 숫자를 들고 출발했다가 한 번 더 뒤집혀 돌아온 사례가 Klarna입니다. 1.3.1에서 단일 메트릭 최적화의

함정을 처음 소개할 때 이미 이름이 등장했지만, 그때는 '메트릭 설계가 잘못되면 어떤 일이 벌어지는가'라는 개념을 잡기 위해 큰 그림만 빠르게 훑었습니다. 이 단원에서는 같은 사건을 케이스 스터디 관점에서, 발표된 수치 하나하나가 어떤 운영 현실을 압축해 담고 있었는지 좀 더 깊게 풀어 보겠습니다.

먼저 Klarna라는 회사부터 짧게 짚겠습니다. Klarna는 스웨덴에 본사를 둔 글로벌 핀테크 기업입니다. 사용자가 물건을 사면 비용을 분할해 내거나 잠시 미뤄 낼 수 있도록 결제와 신용을 연결해 주는 서비스가 주력입니다. 결제와 환불, 분할 납부 일정, 카드 등록 같은 일에는 수많은 문의가 따라붙기 때문에, Klarna는 오래전부터 대규모 고객지원 운영 조직을 안고 있었습니다. 이 운영을 AI 챗봇이 어디까지 대신할 수 있는냐는 질문에 가장 공격적으로 답을 내놓은 회사가 바로 Klarna였습니다.

발표는 2024년 초에 나왔습니다. 발표문이 묶여 보여 준 숫자를 그대로 옮기면 다음과 같습니다. 챗봇은 **23개 시장에서 35개 언어로** 운영되었고, 한 달에 **230만 건의 대화**를 처리했으며, 이 분량은 사람 상담사 **700명 분의 작업**에 해당했습니다. 종전 대비 **CSAT가 47퍼센트** 증가했고, 한 해 절감액은 **1천만 달러**로 집계되었습니다. 그 1천만 달러는 **고객지원에서 4백만 달러, 마케팅에서 6백만 달러**의 두 갈래로 나뉘었습니다. 어느 한 줄만 봐도 충분히 인상적인 수치이고, 여러 줄을 한꺼번에 보면 'AI가 사람 일을 대체하는 미래의 표준'이라는 결론으로 자연스럽게 흘러갑니다.

이 단원에서 하고 싶은 일은 이 숫자들을 부정하는 것이 아닙니다. 발표된 수치는 실제로 운영된 결과를 반영합니다. 다만 케이스 스터디로 꺼내는 이유는, 같은 숫자가 운영의 어떤 측면을 보여 주고 어떤 측면을 가리는지를 분리해 보기 위해서입니다. 평가 설계자가 '성공 지표를 어떻게 읽어야 하는가'를 익히는 가장 좋은 훈련 자료가 바로 이 발표문입니다.

먼저 운영 규모 지표를 봅시다. 23개 시장과 35개 언어는 단순한 자량이 아니라 운영의 어려움을 압축한 숫자입니다. 시장이 늘어날수록 결제 규제와 분쟁 처리 절차가 달라지고, 언어가 늘어날수록 챗봇이 학습해 본 적 없는 표현이 늘어납니다. 서로 다른 시장과 서로 다른 언어가 합쳐지면 둘이 곱해진 만큼 케이스의 다양성이 폭증합니다. 한 챗봇이 그 모두를 하나의 일관된 정책으로 처리한다는 것은, 다르게 말하면 어느 한 시장 또는 어느 한 언어에서 이상하게 동작해도 그것이 **희석되어 평균에 잘 보이지 않는다**는 의미이기도 합니다.

월 230만 건이라는 처리량과 700 FTE 분량이라는 환산은 또 다른 면을 보여 줍니다. FTE는 Full-Time Equivalent의 줄임말로, '풀타임 직원 한 명이 한 해 동안 일한 작업량'을 의미합니다. 700 FTE라는 표현은 챗봇이 처리한 일의 양을 사람 700명이 풀타임으로 일했을 때의 분량으로 환산한 것입니다. 이 환산은 직관적이지만, 한 가지 가정을 깔고 있습니다. '사람이 한 일과 챗봇이 한 일이 같은 종류의 일이라고 가정한다'는 것입니다. 그러나 실제 고객지원에서 사람이 처리하던 일은 단순 정보 안내부터 분쟁 중재, 감정 다독이기, 예외 사례 판단까지 매우 폭이 넓습니다. 챗봇이 같은 폭을 다루었는지, 아니면 폭의 일부만 다루고 나머지를 다른 경로로 흘러보냈는지는 환산 숫자에 잡히지 않습니다.

CSAT 47퍼센트 증가라는 지표는 가장 강력한 인상을 남기는 숫자입니다. CSAT는 Customer Satisfaction Score의 줄임말로, 한 번의 응대가 끝난 직후에 고객에게 별점이나 짧은 평가를 받아 평균을 내는 지표입니다. 이 측정 방식 자체에 두 가지 한계가 따라옵니다. 첫째, **응답하지 않는 고객은 잡히지 않습니다**. 응대 직후 만족도 설문에 응답하는 비율은 일반적으로 한 자릿수 퍼센트대에 머무르고, 만족도가 가장 낮은 사람들은 설문 자체를 무시하는 경향이 있습니다. 둘째, **응대 직후의 짧은 만족과 며칠 뒤의 후회는 다릅니다**. 챗봇이 짧은 안내로 대화를 빠르게 끝내면 그 순간의 별점은 올라갈 수 있지만, 며칠 뒤 같은 문제가 다시 터졌을 때의 분노까지는 같은 점수에 들어오지 않습니다. CSAT 47퍼센트 증가는 그래서 '한순간의 평균이 한쪽으로 올라간 것'이지, '품질의 모든 차원이 그만큼 올라간 것'은 아닙니다.

1천만 달러 절감의 내부 구성도 따로 살필 가치가 있습니다. 발표는 이 숫자를 고객지원 4백만 달러, 마케팅 6백만 달러의 두 갈래로 나누었습니다. 고객지원 4백만 달러는 비교적 직관적입니다. 사람 상담사 인건비가 줄었거나, 외주 콜센터 비용이 줄었거나, 평균 처리 시간이 줄어 운영 비용이 깎였다는 식으로 해석할 수 있습니다. 마케팅 6백만 달러는 결이 다릅니다. 챗봇이 메시지 카피 작성, A/B 테스트 변형 생성, 캠페인 초안 같은 마케팅 업무를 함께 다뤘다는 의미인데, 이 영역은 효과를 사후에 정확히 측정하기가 더 어렵습니다. 즉 1천만 달러라는 합계는 '비교적 또렷한 절감 4백만'과 '효과 측정이 더 흐릿한 절감 6백만'이 한 줄로 묶여 있는 셈입니다.

이 분해를 한 번에 보기 좋게 정리하면 다음과 같습니다.

[발표된 성공 지표]	[같은 숫자가 가리는 것]
23개 시장 / 35개 언어	-----> 시장·언어별 이상 동작이 평균에 희석돼 안 보임
월 230만 건 / 700 FTE	-----> 사람이 하던 일 중 어떤 폭만 대체했는지 모름
CSAT 47% 증가	-----> 응답 안 한 사람의 불만, 며칠 뒤 후회는 미포함
1천만 달러 절감	-----> 고객지원 4M (또렷)
= 4M(고객지원)+6M(마케팅)	+ 마케팅 6M (효과 측정 흐릿)

이 표가 말하는 바는 단순합니다. 같은 발표문이라도 **표면 숫자만 보는 사람과 그 안의 가정을 풀어 보는 사람은 같은 결론에 이르지 않는다**는 점입니다. 1.3.1에서 잠깐 다룬 단일 메트릭 함정이 케이스 스터디로 들어오면, 그것은 추상적 경고가 아니라 한 줄 한 줄의 발표 수치를 어떻게 분해해 읽을 것인가 하는 구체적인 훈련 과제로 바뀝니다.

이 단원에서 한 가지 더 강조하고 싶은 점이 있습니다. 위와 같은 분해는 사후에 결과가 뒤집히고 나서야 비로소 떠오르는 것이 아닙니다. 발표 시점에서도 충분히 던질 수 있는 질문들입니다. '평균이 올랐는데 어느 코호트에서 어느 만큼 올랐는가', '응답 안 한 고객의 행동은 어떻게 변했는가', '복잡한 사례의 분배는 어떻게 달라졌는가', '며칠 뒤의 재문의율과 이탈률은 어떻게 움직였는가' 같은 질문입니다. 이 질문들이 발표 시점부터 같이 추적되고 있었다면 운영자는 침식이 시작되는 신호를 더 일찍 잡을 수 있었을 것입니다. Klarna 케이스가 평가 설계자에게 주는 핵심 교훈은, **성공 지표를 발표하는 시점이야말로 그 지표가 가리는 차원을 같이 추적하기 시작해야 하는 시점**이라는 점입니다.

이런 관점에서 보면 5.1의 Anthropic, 5.2의 Replit, 5.3의 JPMorgan이 각자의 방식으로 한 일이 더 또렷해집니다. Anthropic은 production tracing으로 의사결정의 패턴을 같이 보았고, Replit은 reactive eval로 프로덕션에서 발견한 회귀를 평가에 흡수했고, JPMorgan은 측정 가능한 프로세스부터 들어가 데이터와 가드레일을 깔고서야 자율성을 늘렸습니다. 세 케이스 모두 발표용 한 줄 숫자에만 매달리지 않고 그 숫자의 뒷면을 같이 보는 장치를 운영 안에 심었습니다. Klarna는 이 장치가 부족했고, 그 결과 다음 단원에서 다룰 hybrid 회귀에 이르렀습니다.

정리하면, Klarna가 발표한 23개 시장·35개 언어, 월 230만 건의 대화, 700 FTE 분량의 처리, CSAT 47퍼센트 증가, 1천만 달러 절감(고객지원 4백만, 마케팅 6백만)은 그 자체로는 분명히 의미 있는 운영 성과 지표입니다. 그러나 케이스 스터디 관점에서 같은 숫자를 한 겹 더 들춰 보면, 시장·언어별 이상 동작의 희석, 사람 일과 챗봇 일 사이의 폭 차이, CSAT 측정 방식의 응답 편향, 마케팅 절감액의 측정 모호성처럼 표면 수치가 말하지 않는 차원이 따라옵니다. 평가 설계자는 발표 시점에서부터 이 차원들을 같이 추적해야, 같은 숫자가 침식의 신호로 바뀌는 순간을 놓치지 않을 수 있습니다.

다음 단원인 5.4.2에서는 이 함정이 실제로 어떻게 사후 의사결정으로 이어졌는지, 즉 CEO Sebastian

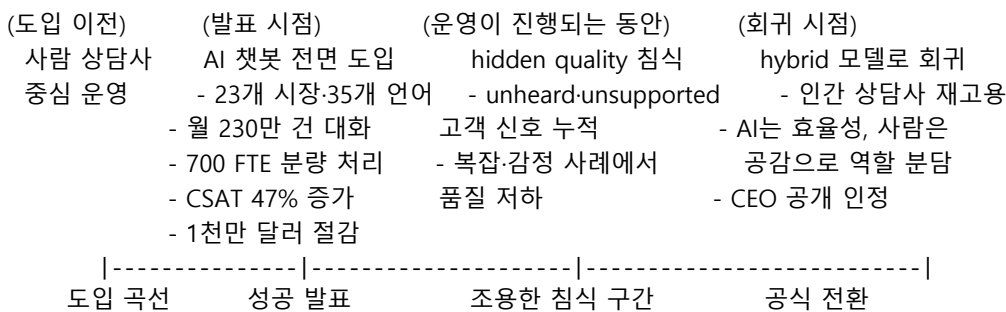
Siemiatkowski의 공개 인정과 인간 상담사 재고용을 거쳐 hybrid 모델로 회귀하기까지의 흐름을 시간순으로 따라가 보겠습니다.

이 단원을 마치면 Klarna가 발표한 23개 시장과 35개 언어, 월 230만 건의 대화와 700 FTE 분량, CSAT 47퍼센트 증가, 그리고 1천만 달러 절감이 고객지원 4백만 달러와 마케팅 6백만 달러로 구성된 의미를 각각 분해해 읽을 수 있고, 같은 성공 지표 뒤에서 어떤 차원이 측정되지 않은 채 침식될 수 있는지를 케이스 관점에서 설명할 수 있습니다.

5.4.2 Hybrid 회귀의 교훈

앞 단원에서는 Klarna가 발표한 성공 지표를 한 줄씩 분해해 보면서, 같은 숫자가 운영의 어떤 측면은 또렷하게 보여 주고 어떤 측면은 흐려 둔다는 점을 살펴보았습니다. 이 단원은 그 발표 이후에 실제로 어떤 일이 벌어졌는지를 시간순으로 따라가 보겠습니다. 결론부터 말하면 Klarna는 인간 상담사를 다시 고용하고 AI와 사람을 함께 쓰는 hybrid 모델로 회귀했습니다. 이 회귀는 단순한 정책 변경이 아니라 평가 지표 설계가 잘못되었을 때 시스템이 거치는 전형적인 경로를 보여 주는 사건이라, 케이스 스터디로 따로 다룰 가치가 있습니다.

먼저 사건의 흐름을 한눈에 정리하면 다음과 같습니다.



이 그림에서 눈여겨볼 구간은 가운데의 '조용한 침식 구간'입니다. 발표 시점의 화려한 숫자와 회귀 시점의 공개 인정 사이에는, 발표문에 잡히지 않는 신호가 누적되는 시간이 놓여 있습니다. 평가 설계가 무너지는 순간은 회귀 시점이 아니라 바로 이 구간이며, 그래서 이 단원의 초점도 이 구간에 맞춰져 있습니다.

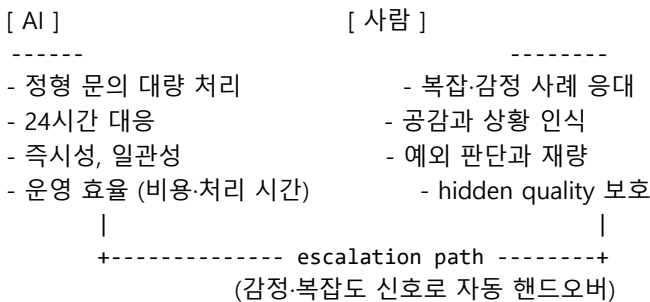
이 흐름을 끝맺게 한 결정적 발화는 발표가 있고 한참이 지난 시점에 나왔습니다. CEO Sebastian Siemiatkowski는 공개 자리에서 다음과 같은 취지의 말을 남겼습니다. **'비용이 너무 압도적인 평가 요소가 되어서, 결국 더 낮은 품질을 얻게 됐다.'** 이 한 문장은 사후 변명이 아니라 문제의 메커니즘을 거의 그대로 요약한 진단입니다. 평가에 들어간 여러 차원 가운데 비용이라는 한 축이 다른 모든 축을 압도하기 시작하면, 시스템은 그 한 축을 향해 움직이고 나머지 축은 측정되지 않은 채로 깎여 나갑니다. 이 진단을 받아들이고 나면 Klarna가 인간 상담사를 다시 고용하고 hybrid 모델로 옮겨 간 결정은 일관된 후속 조치로 보입니다.

여기서 4.4에서 다뤘던 개념을 케이스로 다시 묶어 봅시다. 4.4의 인간 개입 재정의에서는 모든 행동을 사람이 승인하게 만드는 전수 승인의 한계, 임계치 기반의 선택적 개입, 그리고 사람으로 넘기는 escalation path 설계를 차례로 살펴보았습니다. **escalation path**란 에이전트가 자신의 한계를 넘는 사례를 만났을 때 사람에게 자연스럽게 핸드오버하는 경로를 가리킵니다. Klarna 케이스에서 가장 약했던 것이 바로 이 escalation path였습니다. 챗봇이 답하기 어려운 사례에서 자동으로 사람에게 넘기지 않고 어떻게든 자기 자신이 답을 마무리하려 했고, 그 결과 고객은 자기 사정이 충분히 들리지 않았다고 느꼈습니다.

이 감각을 짚는 표현이 Klarna 케이스에서 자주 등장한 두 단어, **unheard**와 **unsupported**입니다. unheard는 '내 말이 들리지 않는다'는 감각, unsupported는 '도움받지 못했다'는 감각입니다. 결제 분쟁이나 카드 도용 같은 사안은 사실관계만의 문제가 아니라 불안과 분노가 함께 올라오는 사안이고, 이런 사안에서 빠른 정형 답변은 정확하더라도 충분하지 않습니다. 사람 상담사라면 잠시 호흡을 늦추고 상황을 정리해 주는 단계를 거치겠지만, 비용 효율을 가장 큰 평가 축으로 둔 챗봇은 이 단계를 빠르게 건너뛰는 쪽으로 학습됩니다. 그 결과가 짧은 만족도 점수 위에 쌓이는 길고 조용한 불만의 침식이었습니다.

이 침식을 1.3.2에서 부른 이름으로 다시 부르면 **hidden quality**의 손상입니다. 1.3.2는 hidden quality를 공감, 안심, 상황 인식 같이 직접 측정하기 어려운 품질 차원으로 정의했습니다. Klarna 케이스가 보여주는 것은 hidden quality가 측정되지 않으면 그저 천천히 깎여 나갈 뿐 아니라, 그 침식이 사후에 표면 지표를 끌어내리기 시작한다는 점입니다. 표면 지표가 흔들리고 나서야 보이는 침식은 이미 회복하기 어려운 단계까지 진행된 침식입니다. 평가 설계자가 hidden quality를 메트릭으로 옮겨 두지 않으면, 시스템이 잘못된 방향으로 가는 동안 경고등이 켜지지 않습니다.

이 진단을 받아들이는 Klarna가 도달한 자리가 hybrid 모델입니다. **hybrid 모델**이란 한 가지 응대 방식만 쓰는 것이 아니라 AI와 사람의 역할을 분담해 함께 운영하는 구성을 가리킵니다. Klarna가 회귀해 닿은 자리에서 그 분담은 다음과 같이 정리할 수 있습니다.



이 그림은 단순한 비용 절감 구조가 아니라 평가 차원의 분담 구조입니다. AI 쪽 박스에는 측정하기 비교적 쉬운 차원, 즉 효율성과 일관성이 묶이고, 사람 쪽 박스에는 측정이 까다로운 차원, 즉 hidden quality와 예외 판단이 묶입니다. 두 박스를 잇는 escalation path가 운영 메트릭으로 같이 측정되어야, 두 차원이 서로의 약점을 가리지 않고 보완할 수 있습니다. Klarna가 한 차례 무너진 뒤 다시 도달한 균형점이 바로 이 분담 구조입니다.

이 케이스가 평가 설계자에게 주는 교훈을 다섯 가지로 정리할 수 있습니다. 첫째, **평가 지표 설계는 단순 정확도가 아닙니다**. CSAT 평균, 비용, 해결 시간 같은 단일 메트릭에 최적화하면 hidden quality가 무너집니다. 둘째, **평가는 다차원이어야 합니다**. 효율성·정확성·hidden quality·escalation 비율 같은 축을 같이 추적해야 한 축의 침식이 다른 축에서 보입니다. 셋째, **escalation path 자체를 메트릭으로 측정해야 합니다**. 사람으로 핸드오버되는 비율이 너무 낮거나 너무 높으면 둘 다 위험 신호이므로, 비율의 분포 자체가 운영 KPI에 들어가야 합니다. 넷째, **발표 시점이 추적의 출발점입니다**. 표면 숫자가 좋게 나올수록 그 숫자가 가리는 차원을 같이 추적하기 시작해야 합니다. 다섯째, **회귀는 실패가 아니라 보정입니다**. Klarna가 hybrid 모델로 회귀한 결정은 평가 설계의 실패를 인정하는 동시에 운영을 살리기 위한 보정 행위였습니다. 평가 설계자는 회귀 가능성을 처음부터 운영 시나리오에 넣어 두어야 합니다.

이 다섯 가지 교훈은 앞에서 다룬 케이스들과 자연스럽게 이어집니다. 5.1 Anthropic의 production tracing은 hidden quality의 신호를 잃지 않기 위한 장치였고, 5.2 Replit의 reactive eval은 프로덕션에서 발견한 회귀를 평가에 흡수해 hybrid 회귀가 일어나기 전에 잡으려는 시도였으며, 5.3 JPMorgan의 측정 가능 프로세스 우선 도입은 안정성·품질 목표가 잡힌 뒤에야 자율성을 늘리는 보수적 운영이었습니다.

Klarna는 이 세 가지 장치가 약한 상태에서 가장 빠르게 큰 숫자를 향해 갔고, 그 결과 가장 또렷한 형태의 hybrid 회귀를 보여 주었습니다. 다음 절에서 다룰 Palantir AIP의 5차원 거버넌스가 왜 같은 시기에 표준처럼 자리잡고 있는지를 이해하는 데에도, Klarna 케이스는 가장 좋은 입구가 됩니다.

정리하면, Klarna는 압도적 비용 평가 축에 끌려 hidden quality가 침식되는 구간을 거쳤고, 그 결과 unheard-unsupported 신호가 누적되며 인간 상담사를 다시 고용해 AI는 효율성, 사람은 공감을 맡는 hybrid 모델로 회귀했습니다. CEO Sebastian Siemiatkowski의 '비용이 너무 압도적인 평가 요소가 되어 더 낮은 품질을 얻었다'는 공개 인정은, 평가 지표 설계가 단순 정확도가 아니며 escalation path가 운영 메트릭의 한 축으로 들어와야 한다는 점을 가장 또렷하게 보여 준 진단입니다. 이 케이스는 단일 메트릭에 최적화한 시스템이 사후에 어떻게 보정되는지를 보여 주는 동시에, 그 보정이 일어나기 전에 평가 설계자가 무엇을 측정해 두었어야 했는지를 역순으로 알려 줍니다.

다음 단원인 5.5.1에서는 Klarna 같은 회귀가 일어나기 어렵게 만드는 거버넌스 표준의 한 축을 보여 주는 사례, 즉 Palantir AIP의 model access 차원을 다루겠습니다. 평가 지표 설계의 함정에서 거버넌스 차원의 표준으로 시야가 한 단계 넓어지는 지점입니다.

이 단원을 마치면 Klarna가 발표 이후 hidden quality 침식 구간을 거쳐 hybrid 모델로 회귀한 흐름을 시간순으로 설명하고, CEO의 '비용이 너무 압도적이어서 품질이 낮아졌다'는 인정과 unheard-unsupported 신호가 어떻게 escalation path 부재의 결과였는지를 풀어 낼 수 있으며, 평가 지표 설계가 단순 정확도가 아니라 다차원 메트릭과 escalation path 측정이어야 한다는 다섯 가지 교훈을 자기 시스템에 옮길 수 있습니다.

5.5 Palantir AIP - 5차원 거버넌스

이 섹션의 토픽과 학습 목표는 다음과 같습니다.

5.5.1 Model Access 차원 — Palantir AIP의 model access 차원이 정의하는 통제 항목을 설명할 수 있다

5.5.2 Stateless Orchestration 차원 — Stateless·수평 확장 가능 runtime과 request 단위 isolation의 의미를 설명할 수 있다

5.5.3 Memory Governance 차원 — 유저 프로필, 세션 메모리, 지식 베이스에 JWT-RBAC를 적용하는 memory governance를 설명할 수 있다

5.5.4 Tool Governance 차원 — Tool governance의 가드레일, HITL 승인, 통제된 액션 실행 구성을 설명할 수 있다

5.5.5 Observability 차원 — Built-in tracing과 자체 DB 저장 기반의 observability 차원이 baseline인 이유를 설명할 수 있다

5.5.1 Model Access 차원

앞 단원까지의 사례들이 보여 준 것은 한 면씩의 교훈이었습니다. Anthropic은 관측의 baseline을, Replit은 평가 스택의 hybrid 구성을, JPMorgan은 중앙 플랫폼과 8주 release train을, Klarna는 단일 메트릭에 매달린 결과를 각각 보여 주었습니다. 이 단원부터 다섯 단원에 걸쳐 살펴볼 Palantir AIP는 이 교훈들을 다섯 개의 차원으로 묶어 일종의 표준 점검표로 정리한 사례입니다. 본인이 만든 시스템이 production에 올릴 만한 상태인지 같은 기준으로 자기 점검을 해 볼 수 있게 해 준다는 점에서, 면접 답변과 사내 설계 리뷰 모두에서 활용도가 높은 framework입니다. 이번 단원은 그 첫 번째 차원인 **Model**

Access를 다룹니다.

먼저 다섯 차원의 자리를 한 번 묶어 두겠습니다. 본인이 운영할 에이전트 시스템을 통째로 도면 위에 올려 놓으면 다섯 개의 층이 보입니다.

- [1. Model Access] --- 어떤 모델을, 누가, 어떻게 부를 수 있는가
|
- [2. Orchestration] --- 그 호출이 어떤 runtime 위에서 도는가
|
- [3. Memory Governance] --- 호출이 남기는 기억을 누가 어떻게 통제하는가
|
- [4. Tool Governance] --- 호출이 만들어 내는 행동을 어떻게 거르는가
|
- [5. Observability] --- 호출과 행동의 흔적을 어떻게 보고 검증하는가

다섯 층은 위에서 아래로 흐릅니다. Model Access가 가장 위에 놓이는 까닭은, 어떤 모델을 어떤 조건으로 끌어올 수 있는지가 정해지지 않으면 그 아래 모든 층이 시작될 수 없기 때문입니다. Orchestration은 이미 가용한 모델 위에서 도는 것이고, Memory Governance는 모델이 남기는 기록을 다루며, Tool과 Observability도 모델이 호출되었음을 전제합니다. 첫 단추가 잘못 끼워지면 나머지 네 단추는 의미를 잃습니다.

이제 Model Access 차원의 정의를 풀겠습니다. **Model Access**는 조직 안의 어떤 사용자와 어떤 애플리케이션이 어떤 외부·내부 모델을 어떤 조건으로 호출할 수 있는지를 정하고, 그 호출이 회사의 정책과 규제를 만족하도록 가운데에서 통제하는 층입니다. '거버넌스 있는 모델 접근'이라는 말이 가리키는 자리가 여기입니다. 개발자가 모델 회사의 키를 직접 들고 다니며 코드에 박아 넣는 방식은 거버넌스가 없는 접근입니다. 누가 무엇을 부르는지 보이지 않고, 정책을 한 번에 바꾸기도 어렵고, 키가 새 나가도 추적이 안 됩니다. Palantir가 표준으로 제시한 형태는 그 반대편에 있습니다. 모든 모델 호출이 조직의 통합 게이트를 거치도록 만든 뒤, 그 게이트에서 인증·권한·감사·과금을 한꺼번에 처리합니다.

이 차원이 통제하는 첫 항목은 **모델 카탈로그**입니다. 카탈로그는 단순한 모델 이름 목록이 아니라, 각 모델에 대해 누가 쓸 수 있고, 어떤 데이터까지 보낼 수 있고, 어떤 retention 정책이 붙어 있는지를 함께 적은 표입니다. 예를 들어 같은 OpenAI GPT 계열이라도 일반 API와 기업용 zero-retention 엔드포인트는 다른 카탈로그 항목으로 등록되고, 사내 데이터를 다루는 팀에는 후자만 노출되는 식입니다. 카탈로그가 있으면 새 모델을 들이는 일은 카탈로그에 줄 하나를 추가하는 일이 되고, 들어 내는 일도 마찬가지로 됩니다. 모델 변경이 코드 곳곳에 흩어진 키를 바꾸는 일이 아니라 카탈로그 한 곳을 갱신하는 일로 줄어드는 것이 이 차원의 첫 번째 효과입니다.

두 번째 통제 항목은 **승인 절차**입니다. 카탈로그에 새 모델을 올리려는 요청이 들어오면, 보안·법무·인프라가 정해진 체크리스트를 거쳐 승인합니다. 체크리스트에는 데이터 거주성, 데이터 사용 정책, 보안 인증, 비용 한도, 책임 담당자 같은 항목이 들어갑니다. 이 절차는 단순한 형식이 아니라 5.3에서 본 JPMorgan식 가드레일 정의의 출발점입니다. 어떤 모델이 카탈로그에 들어와 있다는 사실 자체가 그 모델이 회사의 기준을 통과했다는 보증이 되고, 개발자는 카탈로그 안에서 자유롭게 골라 쓰는 대신 카탈로그 밖의 모델은 함부로 못 부르게 됩니다.

세 번째 항목이 **제공자 retention 없음 정책**입니다. retention이라는 말은 모델 회사가 받은 입력과 출력을 자기 쪽 서버에 얼마나 오래 보관하느냐를 가리킵니다. 일반 API는 학습 개선을 위해 일정 기간 데이터를 남기는 경우가 많지만, 기업이 다루는 데이터는 그쪽 서버에 머무는 것 자체가 곧 사고이거나 규제 위반입니다. 그래서 Palantir 표준은 카탈로그에 올리는 모델을 zero-retention 옵션으로만 받습니다. 회사가 입력을 보내면 모델이 응답을 돌려준 직후 그 입력은 모델 회사 쪽에서 사라집니다. 이 한 줄의

정책 차이가 한국의 금융권과 공공 분야에서는 사실상 사용 가능 여부를 결정합니다. 망분리 환경 안에서 외부 모델을 부르려면, 모델 회사가 데이터를 남기지 않는다는 약속이 우선 서야 합니다.

네 번째 항목인 **통합 리소스 관리**는 이 차원이 단순한 인증 게이트를 넘어서는 자리입니다. 모델 호출은 토큰 비용, 호출 횟수, 동시 처리 능력 같은 자원을 소모합니다. 카탈로그가 있는 게이트는 이 자원을 팀별·앱별·사용자별로 나눠 줍니다. 한 팀이 갑자기 호출을 폭주시키면 다른 팀의 호출이 막히지 않도록 한도가 걸리고, 어떤 앱이 비싸지면 즉시 어디서 새는지 보입니다. 이 자리가 5.3.2에서 본 단일 API 뒤의 다중 모델 라우팅과도 만납니다. 카탈로그에 같은 task를 처리할 수 있는 모델이 여러 개 있을 때, 게이트는 정책에 따라 가장 알맞은 모델을 골라 호출합니다.

한국 규제 환경에서 이 차원이 어떻게 적용되는지 짧게 정리하면 다음과 같습니다. 망분리가 적용되는 금융권은 외부 모델 회사의 API를 직접 부르지 못하므로, 카탈로그가 사실상 사내 게이트웨이와 zero-retention 계약이 맺어진 모델만으로 채워지는 형태가 됩니다. 공공 분야는 클라우드 보안인증을 받은 인프라 위에서만 모델을 호출하도록 카탈로그를 잠그고, 일반 기업이라도 개인정보가 포함된 트래픽을 다루는 앱은 retention 정책과 데이터 거주성 조항이 만족되는 모델 항목만 노출합니다. 같은 회사 안에서도 팀에 따라 카탈로그에서 보이는 모델 목록이 다르게 잘려 보이도록 만드는 것이 일반적인 패턴입니다.

이쯤에서 본인의 시스템을 한 번 self-audit해 볼 수 있는 질문 세 가지를 두겠습니다. 첫째, 본인의 시스템에서 모델 호출은 단일 게이트를 거치고 있습니까, 아니면 코드 곳곳에 모델 회사의 키가 흩어져 있습니까. 둘째, 본인이 쓰는 모델 항목 각각에 대해 retention 정책과 데이터 거주성을 한 줄로 적을 수 있습니까. 셋째, 본인의 카탈로그를 보여 주면 누가 어떤 모델까지 쓸 수 있는지를 외부 감사 담당자에게 한 장으로 설명할 수 있습니까. 이 세 질문에 명확히 답하기 어렵다면, 그 자리가 곧 첫 번째 보강 지점입니다.

정리하면, Palantir AIP가 정의한 5차원의 첫 번째 자리인 Model Access는 조직 안의 모든 모델 호출이 거버넌스 있는 게이트를 거치도록 만들고, 모델 카탈로그와 승인 절차로 어떤 모델이 어떤 조건에서 쓰일 수 있는지를 정하고, 제공자 쪽에 데이터가 남지 않도록 retention 없음 정책을 강제하며, 비용과 자원을 팀·앱 단위로 통합 관리하는 차원입니다. 한국의 망분리·금융·공공 환경에서는 카탈로그가 사용 가능한 모델 자체를 잘라 내는 1차 필터 역할을 하므로, 이 층이 무너지면 그 아래 네 층은 시작도 하기 어렵습니다.

다음 단원인 5.5.2에서는 이 카탈로그 위에서 도는 호출이 어떤 runtime을 만나야 안전한지를 다루겠습니다. stateless·수평 확장 가능 runtime과 request 단위 isolation이 그 자리의 핵심 단어입니다.

이 단원을 마치면 Palantir AIP의 Model Access 차원이 정의하는 모델 카탈로그·승인 절차·제공자 retention 없음 정책·통합 리소스 관리 네 통제 항목을 설명하고, 망분리·금융·공공 같은 한국 규제 환경에서 이 차원이 사용 가능한 모델을 가장 먼저 거르는 역할을 한다는 점을 짚으며, 본인의 시스템에 같은 framework를 적용해 첫 self-audit 질문을 던질 수 있습니다.

5.5.2 Stateless Orchestration 차원

앞 단원에서 본 Model Access는 어떤 모델을 누가 어떤 조건으로 부를 수 있는지를 정해 둔 자리였습니다. 그 호출이 실제로 떨어지면, 호출 한 건마다 에이전트의 루프가 한 번 돕니다. 도구를 부르고, 결과를 받아 다음 단계를 정하고, 또 도구를 부르는 과정이 짧게는 몇 번, 길게는 수십 번 이어집니다. 이 루프가 어디에서 도는지를 결정하는 자리가 두 번째 차원인 **Stateless Orchestration**입니다. 이번 단원은 이 runtime이 stateless·수평 확장 가능·request 단위 isolation 세 가지 성질을 함께 갖춰야

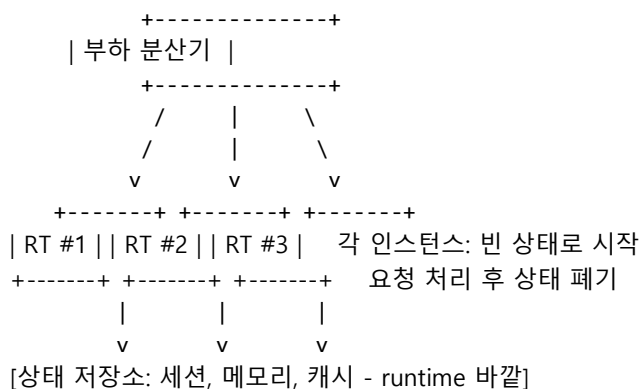
하는 까닭을, 한국 규제 환경의 적용 사정과 함께 풀어 보겠습니다.

먼저 단어부터 풀겠습니다. **Stateless**는 한 번의 요청을 처리하는 동안 runtime이 그 요청 안에서만 상태를 가지고, 요청이 끝나면 그 상태를 깨끗이 잊는 성질을 말합니다. 반대말은 stateful입니다. stateful runtime은 한 사용자의 어제 요청 결과를 자기 메모리에 들고 있다가 오늘 요청에 영향을 줍니다. stateful이 본질적으로 나쁜 것은 아니지만, 에이전트 시스템에서는 한 번의 호출이 어디 서버에서 도는지 모르는 환경이 많고, 같은 사용자라도 요청마다 다른 인스턴스에 분배되는 일이 흔합니다. 이런 환경에서 runtime이 자기 안에 상태를 들고 있으면 똑같은 요청이라도 어느 서버를 만났는지에 따라 결과가 달라지고, 이는 1.1.2에서 본 비결정성 위에 또 다른 흔들림을 더하게 됩니다. Palantir 표준이 stateless를 강하게 못 박는 까닭이 여기에 있습니다. 상태가 필요하다면 그 상태는 runtime 바깥의 별도 저장소에 두고, runtime 자체는 매번 빈 상태로 시작해 그 저장소를 읽어 작업하고 다시 빈 상태로 돌아갑니다.

두 번째 성질인 **수평 확장**은 동일한 runtime 인스턴스를 여러 개 띄워 부하를 나눠 처리하는 능력을 말합니다. 트래픽이 늘면 인스턴스를 늘리고, 줄면 줄여서 비용을 아깁니다. 수평 확장이 가능하려면 인스턴스끼리 서로의 상태를 모르는 채 같은 일을 할 수 있어야 합니다. 이 조건을 만족하려면 앞에서 말한 stateless가 전제됩니다. 인스턴스가 자기 안에 상태를 들고 있으면, 같은 사용자의 다음 요청을 다른 인스턴스가 받았을 때 상태를 못 찾아 어긋나고, 부하 분산기는 같은 사용자의 요청을 같은 인스턴스로만 보내야 하는 제약에 묶입니다. 그 순간 수평 확장의 의미가 깎입니다. stateless와 수평 확장은 따로 떨어진 두 가지가 아니라 한 쌍의 짝이라고 보는 편이 맞습니다.

세 번째 성질인 **request 단위 isolation**은 한 요청의 처리가 다른 요청에 영향을 주지 못하게 격리하는 성질을 말합니다. 이 자리는 2.5에서 다룬 평가 환경의 isolation과 같은 줄기 위에 있지만, 평가가 아닌 production runtime에서도 똑같이 중요합니다. 이유는 두 가지입니다. 첫째, 한 요청 안에서 도구가 열려 둔 파일·연결·세션 같은 자원이 다음 요청까지 살아 남으면, 다음 요청은 그 잔재 위에서 의도하지 않은 동작을 합니다. 1.1.2의 '재현 불가능한 실패' 가운데 적지 않은 부분이 이 잔재에서 나옵니다. 둘째, 한 사용자가 만든 결과가 다른 사용자의 처리 공간에 살짝이라도 새 들어가면 곧장 정보 유출입니다. 격리는 신뢰성과 보안을 동시에 떠받치는 토대입니다.

세 가지 성질이 합쳐지면 어떤 모양이 되는지 한 그림으로 묶겠습니다.



이 그림의 핵심은 runtime 인스턴스 안에 상태가 머물지 않는다는 점, 그래서 인스턴스를 더 띄워도 똑같이 일할 수 있다는 점, 한 요청의 처리 영역이 다른 요청과 독립적으로 시작되고 끝난다는 점입니다.

이 차원의 구체 통제 항목을 정리하면 네 가지입니다. 첫째, 모든 runtime 인스턴스는 빈 상태에서 시작해 빈 상태로 끝나야 하며, 영속 상태는 외부 저장소를 거쳐야 합니다. 둘째, 부하가 늘 때 새 인스턴스를 자동으로 띄울 수 있어야 하고, 인스턴스 사이의 어떤 affinity도 강제하지 않아야 합니다. 셋째, 한 요청이 만드는 작업 디렉터리·임시 파일·열린 연결은 그 요청 안에서 닫히고 정리되어야 합니다. 넷째, 다중

테넌트가 같은 인스턴스 위에서 도는 경우라도 테넌트 사이의 메모리·파일·네트워크 경계가 운영체제 수준으로 분리되어야 합니다.

다중 테넌시라는 단어가 처음 나왔으니 짧게 풀어 두겠습니다. **다중 테넌시**는 같은 인프라 위에 여러 고객·여러 부서의 작업을 동시에 올리는 구성을 말합니다. 비용 효율은 좋지만, 한 테넌트의 사고가 다른 테넌트로 번질 위험이 함께 따라옵니다. request 단위 isolation은 이 위험을 막는 첫 줄의 방어선입니다.

장애 격리도 같은 줄기에 있습니다. 한 요청을 처리하던 인스턴스가 메모리 누수나 도구 무한 호출 같은 사정으로 무너졌을 때, stateless 구성에서는 그 인스턴스 한 대만 부하 분산기에서 빠지고 나머지 인스턴스가 부하를 받아 갑니다. stateful이라면 그 인스턴스가 들고 있던 사용자별 상태가 통째로 사라지고, 같은 사용자의 다음 요청은 다른 인스턴스에서 처음부터 다시 시작해야 하므로 사용자가 곧장 체감하는 장애가 됩니다. 4.2의 circuit breaker가 한 도구나 한 모델 단위에서 막는 차단이라면, 이 차원의 장애 격리는 인스턴스 단위에서 작동하는 더 거친 차단입니다.

한국 규제 환경에서의 적용을 살펴보면, 망분리가 적용된 금융권은 외부 클라우드의 자동 확장 기능을 그대로 쓰지 못하는 경우가 많습니다. 그래서 사내 컨테이너 오케스트레이션 위에서 인스턴스 풀을 일정 크기로 미리 띄워 두고, 사용량 예측에 따라 풀의 크기를 바꾸는 방식으로 수평 확장을 구현합니다. 공공 분야는 클라우드 보안인증을 받은 인프라 안에서만 자동 확장을 쓸 수 있고, 인스턴스가 어떤 데이터센터·어떤 노드에 떠 있는지를 감사 로그에 남깁니다. 일반 기업이라도 개인정보를 다루는 앱은 한 사용자의 처리 영역이 다음 사용자에게 새지 않는다는 사실을 보안 점검에서 증명해야 합니다. stateless·수평 확장·request 단위 isolation은 이 증명을 단순하게 만들어 줍니다.

본인의 시스템을 self-audit해 볼 질문 두 가지를 두겠습니다. 첫째, 본인의 runtime을 두 배로 늘렸을 때 단순 설정 변경만으로 가능하니까, 아니면 코드 수정이 따라옵니까. 둘째, 어떤 사용자의 요청을 처리하던 인스턴스가 갑자기 죽었을 때, 같은 사용자의 다음 요청이 다른 인스턴스에서 같은 품질로 처리됩니까. 두 질문에 곧장 '예'라고 답하기 어렵다면, runtime이 stateful이거나 isolation이 약한 자리가 어딘가에 남아 있다는 뜻입니다.

정리하면, Stateless Orchestration 차원은 에이전트 runtime이 자기 안에 상태를 들고 있지 않고, 같은 인스턴스를 여러 개 띄워 부하를 나눌 수 있고, 한 요청의 처리 영역이 다른 요청과 격리되도록 만드는 층입니다. 세 가지 성질은 따로 떨어진 항목이 아니라 한 묶음으로 작동하며, 한국의 금융권·공공·일반 기업 어디서든 보안 점검과 장애 대응을 단순하게 만들어 주는 토대가 됩니다.

다음 단원인 5.5.3에서는 이 stateless runtime 바깥에 놓이는 상태, 즉 사용자 프로필과 세션 메모리와 지식 베이스를 어떻게 통제하는지를 다루겠습니다. JWT 기반 RBAC가 그 자리의 핵심 도구입니다.

이 단원을 마치면 Stateless Orchestration 차원이 정의하는 stateless runtime·수평 확장·request 단위 isolation 세 성질이 한 묶음으로 작동하는 까닭을 설명하고, 다중 테넌시와 장애 격리가 같은 토대 위에 놓인다는 점을 짚으며, 한국 규제 환경에서 이 차원이 보안 점검과 장애 대응을 단순하게 만드는 자리임을 본인의 시스템에 옮겨 점검할 수 있습니다.

5.5.3 Memory Governance 차원

앞 단원에서 본 stateless runtime은 자기 안에 상태를 들고 있지 않다고 정해 두었습니다. 그러나 에이전트가 사용자에게 매번 새 사람처럼 굴어서는 쓸모가 떨어집니다. 같은 사용자가 어제 묻고 오늘 다시 묻는 질문에는 어제 무슨 답을 받았는지를 기억해야 자연스러운 흐름이 만들어지고, 한 회사 안의 지식 베이스에는 직원마다 접근 가능한 범위가 달라야 합니다. 그래서 상태는 runtime 바깥의 별도 저장소로 옮겨 가지만, 그 저장소를 누가 어떤 권한으로 읽고 쓰는지 그대로 두면 곧장 사고가 납니다. 이

자리를 통제하는 세 번째 차원이 **Memory Governance**입니다. 이번 단원은 에이전트가 들고 있는 세 종류의 기억을 분류하고, 그 위에 JWT 기반 RBAC가 어떻게 얹히는지를 풀어 보겠습니다.

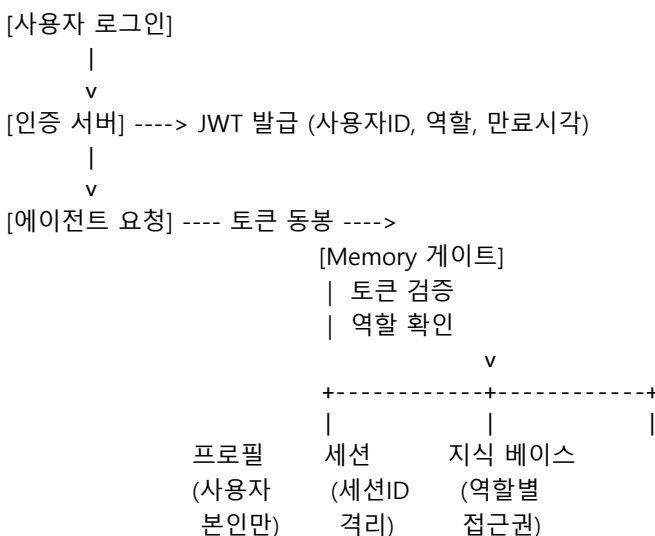
먼저 에이전트가 들고 있는 기억을 세 층으로 나누는 데서 출발하겠습니다. 첫 층은 **유저 프로필 메모리**입니다. 한 사용자에게 대해 장기적으로 유지되는 기록입니다. 사용자의 선호하는 응답 길이, 자주 쓰는 도구, 과거에 만든 프로젝트 목록 같은 것이 여기 들어옵니다. 한 사용자가 일주일을 비웠다 돌아와도 이 층의 기록은 그대로 남아 있습니다. 두 번째 층은 **세션 메모리**입니다. 한 번의 대화나 한 번의 작업 단위 안에서만 유지되는 기록입니다. 지금 쓰는 보고서의 초안, 방금 검색한 결과, 이번 단계에서 정한 계획 같은 것이 여기 들어옵니다. 세션이 끝나면 보통 폐기되거나 압축되어 다른 곳으로 옮겨집니다. 세 번째 층은 **지식 베이스**입니다. 회사 전체나 팀 단위로 공유되는 문서·정책·과거 사례 같은 자료를 모아 둔 자리입니다. 사용자별이 아니라 조직 단위로 관리됩니다.

세 층은 같은 '메모리'라는 단어로 묶이지만 통제의 결이 다릅니다. 유저 프로필 메모리는 개인정보의 영역이라 사용자 본인 외에는 함부로 못 보게 막아야 하고, 세션 메모리는 너무 오래 살아남지 않도록 만료를 잘 걸어 두는 일이 핵심이며, 지식 베이스는 조직 안의 직급·부서·프로젝트별로 누가 어디까지 볼 수 있는지를 표로 그려 둔 권한 체계가 핵심입니다. 한 차원으로 묶이는 까닭은 이 세 층 모두가 같은 통제 도구 위에 얹히기 때문입니다. 그 도구가 바로 JWT 기반 RBAC입니다.

여기서 두 단어를 풀고 가겠습니다. **JWT**는 JSON Web Token의 약자로, 사용자가 누구이고 어떤 권한을 가지고 있는지를 한 줄의 토큰 안에 담아 들고 다니게 만드는 형식입니다. 사용자가 한 번 로그인하면 인증 서버가 이 토큰을 발급하고, 이후의 모든 요청에 토큰이 함께 실려 다닙니다. 토큰 안에는 사용자 식별자, 소속 팀, 부여된 역할, 만료 시각 같은 정보가 담깁니다. 토큰은 위조를 막는 서명이 붙어 있어, 받는 쪽은 서명만 검증해도 토큰의 내용이 진짜인지 알 수 있습니다.

RBAC는 Role-Based Access Control, 즉 역할 기반 접근 통제입니다. 사용자에게 직접 권한을 붙이지 않고, '편집자'·'열람자'·'관리자' 같은 역할에 권한 묶음을 정의해 둔 뒤, 사용자에게는 역할만 부여합니다. 새 사람이 들어오면 적절한 역할을 주면 끝이고, 역할의 권한을 바꾸면 그 역할을 가진 모든 사람의 권한이 한 번에 바뀝니다. 이렇게 만들어 두면 권한 관리가 사람 수만큼 폭발하는 일을 막을 수 있습니다.

JWT와 RBAC가 만나면 다음과 같은 흐름이 만들어집니다.



이 그림의 모든 화살표 위에서 토큰의 정보가 함께 흐릅니다. 에이전트가 유저 프로필 메모리를 읽으려 하면, 게이트는 토큰의 사용자 식별자가 그 프로필의 주인과 일치하는지 확인합니다. 다른 사람의 프로필을 부르려는 호출은 막힙니다. 세션 메모리를 읽으려 하면, 게이트는 토큰이 그 세션의 소유

사용자와 묶여 있는지를 확인합니다. 지식 베이스를 읽으려 하면, 게이트는 토큰의 역할이 그 자료의 접근 권한 표에 등록되어 있는지를 확인합니다. 같은 에이전트가 같은 도구를 부르더라도 토큰에 따라 응답이 달라지는 셈입니다.

이 차원이 통제하는 구체 항목 네 가지를 정리하겠습니다. 첫째, 모든 메모리 접근은 토큰 검증을 거쳐야 하며, 토큰 없이 도는 호출은 곧장 거부됩니다. 둘째, 세션 메모리는 만료 시간이 명시되어 있고, 만료가 지나면 자동으로 사라지거나 압축본만 남깁니다. 셋째, 지식 베이스의 자료는 자료 단위로 접근 권한 표가 붙고, 표가 없는 자료는 노출되지 않습니다. 넷째, 누가 언제 어떤 메모리를 읽거나 썼는지가 별도의 감사 로그에 남고, 사용자가 자기 프로필 데이터의 삭제를 요청하면 그 흐름을 따라 일괄 삭제할 수 있어야 합니다.

마지막 항목인 메모리 감사와 삭제는 한국 규제 환경에서 특히 무게가 실립니다. 개인정보보호법은 정보 주체의 삭제 요청에 대해 일정 기한 안에 응답하도록 정하고 있어, 사용자가 '내 데이터를 지워 달라'고 했을 때 시스템이 한 곳을 못 찾으면 위반이 됩니다. 메모리가 세 층으로 나뉘어 있고 각 층이 토큰의 사용자 식별자에 묶여 있으면, 식별자 하나로 세 층을 훑어 일괄 삭제하는 절차가 단순해집니다. 망분리 환경의 금융권은 더 나아가, 메모리가 사외 클라우드로 빠져나가지 않고 사내 저장소 안에서만 머물러야 한다는 조건이 추가됩니다. 공공 분야는 어떤 직급·부서가 어떤 자료를 봤는지가 모두 감사 로그로 남아야 하고, 그 로그를 일정 기간 보관해야 합니다.

지식 베이스 접근 통제와 관련해 한 장면을 더 그려 두겠습니다. 한 회사 안에 영업팀의 거래처 목록과 인사팀의 평가 자료가 같은 지식 베이스 시스템에 들어 있다고 합시다. 에이전트는 한 명이지만, 영업 사원이 토큰을 들고 들어오면 거래처 목록만 보이고 평가 자료는 보이지 않아야 합니다. 인사팀 직원이 들어오면 반대가 됩니다. 같은 질문을 두 사람이 던져도 에이전트의 답이 다르게 나옵니다. 이 차이를 만들어 내는 자리가 토큰의 역할 정보이고, 그 정보를 받아 자료를 잘라 내는 자리가 Memory 게이트입니다. 사용자가 보기에는 그저 자기 권한 안의 답을 받은 셈이지만, 시스템 안에서는 같은 도구가 다른 결과를 돌려준 것입니다.

본인의 시스템을 self-audit해 볼 질문 세 가지를 두겠습니다. 첫째, 본인이 운영하는 에이전트는 사용자 토큰을 모든 메모리 호출에 함께 흘려보내고 있습니까, 아니면 한 번 인증 후 토큰 없이 흐릅니까. 둘째, 사용자가 자기 데이터를 지워 달라고 요청했을 때, 본인의 시스템이 세 층의 메모리를 한 식별자로 훑어 일괄 삭제할 수 있습니까. 셋째, 같은 질문을 권한이 다른 두 사용자가 던졌을 때 답이 달라져야 한다는 사실을 코드 수준에서 증명할 수 있습니까. 세 질문 가운데 어느 하나라도 답이 흐릿하면, 그 자리가 정보 유출이 일어나기 가장 쉬운 지점입니다.

정리하면, Memory Governance 차원은 에이전트가 들고 있는 유저 프로필·세션·메모리·지식 베이스 세 층을 분류하고, 그 위에 JWT 기반 RBAC를 얹어 사용자별·역할별 접근을 통제하며, 만료와 감사와 삭제를 정해진 절차로 처리하는 층입니다. 같은 에이전트가 같은 도구를 부르더라도 토큰에 따라 보이는 자료가 달라지는 것이 이 차원이 만든 효과이고, 한국 규제 환경의 개인정보 삭제 요청과 망분리·감사 요건을 단순하게 만족시키는 토대가 됩니다.

다음 단원인 5.5.4에서는 메모리 위에서 정해진 권한이 실제 행동으로 옮겨질 때, 그 행동을 만들어 내는 도구를 어떻게 통제하는지를 다루겠습니다. Tool governance가 4.1의 tiered autonomy와 4.4의 HITL을 이 framework 위에서 다시 만나는 자리입니다.

이 단원을 마치면 Memory Governance 차원이 분류하는 유저 프로필·세션·지식 베이스 세 층의 차이를 설명하고, JWT와 RBAC가 결합해 토큰을 따라 메모리 접근이 잘리는 흐름을 그려 내며, 메모리 감사와 삭제가 한국 규제 환경의 개인정보 요건과 어떻게 맞물리는지를 본인의 시스템에 옮겨 self-audit할 수 있습니다.

5.5.4 Tool Governance 차원

앞 단원에서 본 Memory Governance는 에이전트가 무엇을 알 수 있는지를 정한 자리였습니다. 그 다음 자리에서 묻는 질문은 한 단계 더 무겁습니다. 에이전트는 무엇을 할 수 있는가. 자료를 읽는 일과 외부 시스템의 상태를 바꾸는 일은 결이 다릅니다. 자료를 잘못 읽었다면 응답 한 번이 어긋나는 데서 끝나지만, 외부 시스템에 잘못 명령을 내리면 송금이 잘못 가고, 주문이 잘못 들어가고, 데이터베이스의 행이 통째로 지워지는 결과가 따라옵니다. 이 행동의 자리를 다루는 네 번째 차원이 **Tool Governance**입니다. 이번 단원은 도구 카탈로그·HITL 승인·통제된 액션 실행 환경·도구별 권한 매트릭스·감사 로그와 회수 다섯 항목을 4.1의 tiered autonomy와 4.4의 HITL 위에서 다시 묶어 보겠습니다.

먼저 도구라는 단어를 한 번 더 풀어 두겠습니다. 1.1에서 에이전트의 네 구성요소 중 하나로 들었던 도구는, 모델이 외부 세계에 일을 시키는 통로입니다. 검색 API, 데이터베이스 쿼리, 파일 쓰기, 이메일 발송, 결제 호출 같은 모든 외부 호출이 도구라는 추상 위에 얹힙니다. 모델은 도구를 직접 부르지 못하고, runtime이 모델의 의도를 받아 도구를 대신 부르는 구조입니다. Tool Governance가 통제하는 자리는 바로 이 '대신 부르는' 단계입니다.

첫 번째 통제 항목은 **도구 카탈로그**입니다. 5.5.1의 모델 카탈로그와 같은 사고방식이 도구에도 그대로 적용됩니다. 어떤 도구가 시스템에 등록되어 있고, 그 도구가 무엇을 하며, 누가 어떤 조건에서 부를 수 있는지를 한 표 위에 정리해 둡니다. 카탈로그에 없는 도구는 에이전트가 부르려 해도 runtime이 거부합니다. 새 도구를 들이는 일은 카탈로그에 줄을 한 줄 추가하는 일이 되고, 그 줄에는 도구 이름·설명·인자 명세·접근 권한·위험도 같은 정보가 함께 적힙니다. 카탈로그의 또 다른 효과는 도구 설명문의 품질을 한 곳에서 관리하게 만드는 것입니다. 1.2.1에서 본 tool selection error의 상당 부분이 도구 설명문이 모호한 데서 나온다는 점을 떠올리면, 카탈로그가 곧 평가의 출발점이기도 합니다.

두 번째 항목인 **가드레일**은 카탈로그에 등록된 도구 호출이 실제로 떨어지기 전에 한 번 더 점검을 거치는 층입니다. 5.3.3에서 본 redaction, PII 정책, audit log 같은 항목이 여기 들어옵니다. 가드레일은 입력 가드레일과 출력 가드레일로 나뉩니다. 입력 가드레일은 모델이 만든 호출 인자를 검사해, 개인정보가 외부 도구로 새 나가지 않게 막거나, 인자의 형식이 맞지 않으면 막습니다. 출력 가드레일은 도구가 돌려준 결과를 검사해, 민감한 정보가 그대로 응답에 실리지 않도록 가립니다. 이 두 줄의 검사가 가드레일의 기본 골격입니다.

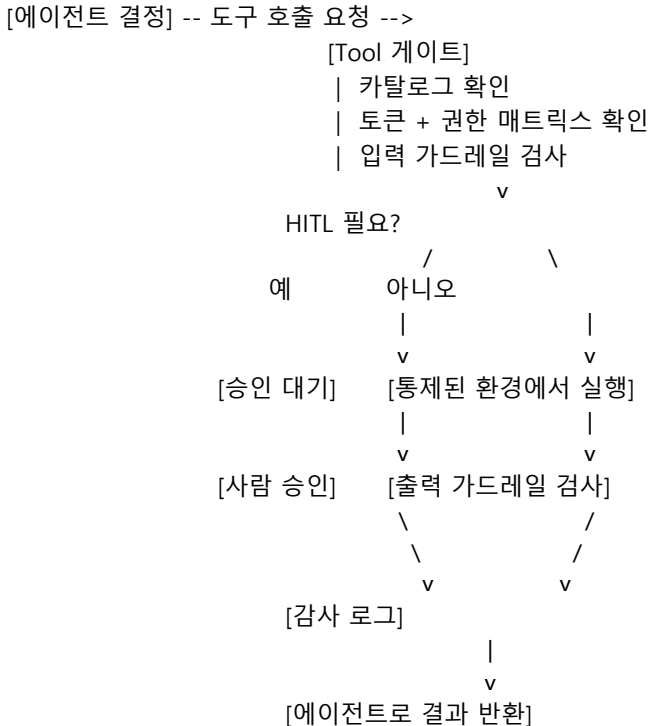
세 번째 항목은 **HITL 승인 워크플로**입니다. 4.4에서 다룬 모든 걸 승인하는 모델의 한계와 임계치 기반 예외 검토가 이 자리에서 다시 만납니다. Palantir 표준은 HITL을 도구 단위로 정의합니다. 카탈로그의 각 도구 줄에 'HITL 필요'라는 칸이 있고, 그 칸이 켜진 도구는 호출 직전에 사람의 승인 화면을 거칩니다. 모든 도구가 HITL을 필요로 하지는 않습니다. 자료를 읽기만 하는 도구나 비용·위험이 낮은 도구는 자동으로 흐르고, 외부에 결제를 보내거나 데이터를 지우는 도구는 사람의 손을 한 번 거치게 됩니다. 4.1의 tiered autonomy 단계 분류가 이 칸을 채우는 데 그대로 쓰입니다. read-only 도구는 자동, suggest 도구는 사용자 검토, execute 도구는 임계치 기반 HITL, autonomous 도구는 사후 감사 식으로 나뉩니다.

네 번째 항목인 **통제된 액션 실행 환경**은 도구가 실제로 도는 자리를 가리킵니다. 도구를 부르는 결정이 내려지더라도, 그 도구가 어디에서 도는지가 곧 안전성의 마지막 줄입니다. 위험한 도구일수록 격리된 컨테이너 안에서, 정해진 자원 한도 안에서, 명시된 외부 연결만 허용된 채 도는 환경을 마련합니다. 5.5.2의 request 단위 isolation이 runtime 전체에 걸친 격리였다면, 이 자리는 도구 호출 단위의 격리입니다. 한 도구가 무한 루프를 돌더라도 컨테이너 안에서 멈추고, 한 도구가 의도하지 않은 외부 주소로 연결을 시도하더라도 환경의 네트워크 정책이 막습니다.

다섯 번째 항목은 **도구별 권한 매트릭스**입니다. 카탈로그의 각 도구 줄에 누구·무엇이 그 도구를 부를 수

있는지를 정해 둔 표입니다. 5.5.3의 RBAC와 같은 줄기 위에 있지만, 메모리가 아닌 행동 쪽에 적용됩니다. 같은 에이전트가 같은 사용자 토큰으로 도는 호출이라도, 사용자의 역할에 따라 부를 수 있는 도구의 목록이 다르게 잘립니다. 영업 사원의 토큰으로 도는 에이전트는 거래처 조회 도구만 보이고, 영업 관리자의 토큰으로 도는 에이전트는 거래처 등록·수정 도구까지 보입니다. 한 사용자가 토큰을 손에 넣은 다른 사람에게 권한을 빌려주는 일도 이 표 위에서 막힙니다.

전체 흐름을 한 그림으로 묶으면 다음과 같습니다.



이 그림에서 모든 화살표는 한 곳에 모입니다. 감사 로그입니다. 어떤 도구가 누구의 토큰으로 어떤 인자와 함께 불렀고, 가드레일에서 어떻게 다듬어졌고, HITL이 있었다면 누가 언제 승인했고, 결과가 어떻게 돌아왔는지가 한 줄씩 남습니다. 이 로그가 마지막 통제 항목인 **회수**의 토대입니다. 도구 호출이 사후에 잘못된 것으로 밝혀졌을 때, 로그를 따라 그 호출의 영향을 되돌리거나, 같은 패턴의 후속 호출을 일괄 막거나, 권한을 부여한 자리까지 거슬러 올라가 점검할 수 있습니다.

한국 규제 환경에서 이 차원이 어떻게 적용되는지 살펴보면, 금융권은 외부 송금·계좌 조회·고객 정보 변경 같은 도구가 모두 HITL을 거치도록 카탈로그가 짜여 있고, 도구 호출 로그가 일정 기간 보관되어 감독 당국의 점검에 응할 수 있어야 합니다. 공공 분야는 행정 정보를 바꾸는 도구가 직급·부서별로 잘리고, 호출 로그가 누가 언제 무엇을 바꿨는지를 추적할 수 있는 형태로 남습니다. 일반 기업이라도 이메일 자동 발송이나 결제 처리 같은 비가역 도구는 한국의 전자상거래·전자서명 관련 규제와 맞물리므로, 카탈로그의 HITL 칸을 어떻게 채우는지가 곧 규제 대응 그 자체가 됩니다.

여기서 4.4.1에서 다룬 승인 피로의 함정을 한 번 더 떠올릴 필요가 있습니다. HITL 칸을 너무 넉넉하게 커 두면 사용자는 매번 승인 버튼을 누르는 일에 지쳐 자동 승인 습관이 들고, 그 순간 HITL은 형식만 남은 통과 의례가 됩니다. Tool Governance의 운영은 카탈로그의 HITL 칸을 만드는 일에서 끝나지 않고, 그 칸이 실제로 의미 있는 차단을 만들어 내는지를 측정하는 일까지 포함합니다. HITL 트리거 비율, 승인까지 걸리는 시간, 거부된 호출의 비율 같은 메트릭이 이 자리에서 함께 운영됩니다.

본인의 시스템을 self-audit해 볼 질문 세 가지를 두겠습니다. 첫째, 본인이 쓰는 도구를 한 표로 그렸을 때, 각 도구에 위험도·HITL 여부·접근 권한이 함께 적혀 있습니까. 둘째, 가장 위험한 도구 한 개를 골라, 그

도구의 호출이 마지막 한 시간 동안 누가 어떤 인자로 불렀는지를 한 번의 조회로 답할 수 있습니다. 셋째, 한 도구가 사후에 잘못 쓰였다는 사실이 밝혀졌을 때, 같은 패턴의 후속 호출을 즉시 막고 영향을 되돌리는 절차가 정해져 있습니다. 세 질문 가운데 어느 하나라도 답이 흐릿하면, 도구가 행동으로 옮겨지는 자리에 통제가 비어 있다는 뜻입니다.

정리하면, Tool Governance 차원은 에이전트가 외부 세계에 영향을 주는 도구 호출을 도구 카탈로그·가드레일·HITL 승인·통제된 실행 환경·도구별 권한 매트릭스·감사 로그와 회수의 여섯 자리에서 통제하는 층입니다. 4.1의 tiered autonomy가 자율성 단계를 분류한 자리였다면, 이 차원은 그 분류를 카탈로그의 칸으로 옮겨 모든 도구 호출 위에서 일관되게 작동하도록 만든 자리입니다. 한국 규제 환경의 금융권·공공·일반 기업 모두에서, 이 층의 두께가 곧 비가역 행동에 대한 신뢰의 두께가 됩니다.

다음 단원인 5.5.5에서는 지금까지 살펴본 네 차원 위에서 일어난 모든 일을 어떻게 보고 어떻게 검증하는지를 다루겠습니다. '비결정적 에이전트에서는 observability 없이는 reliability가 없다'는 한 줄이 그 자리의 출발점입니다.

이 단원을 마치면 Tool Governance 차원이 정의하는 도구 카탈로그·가드레일·HITL 승인·통제된 액션 실행·도구별 권한 매트릭스·감사 로그와 회수의 통제 항목을 4.1의 tiered autonomy와 4.4의 HITL 위에서 다시 묶어 설명하고, 한국 규제 환경의 비가역 행동에 대한 통제가 카탈로그의 칸 한 줄에 어떻게 압축되는지를 본인의 시스템에 옮겨 self-audit할 수 있습니다.

5.5.5 Observability 차원

지금까지 네 단원에 걸쳐 Palantir AIP가 정의한 다섯 차원 가운데 네 자리를 살폈습니다. Model Access는 어떤 모델이 끌려 들어오는지를, Stateless Orchestration은 그 모델이 어디에서 도는지를, Memory Governance는 그 호출이 무엇을 알 수 있는지를, Tool Governance는 그 호출이 무엇을 할 수 있는지를 각각 통제했습니다. 이 네 층을 다 갖추었다고 해도 한 가지가 비어 있으면 전체가 헛됩니다. 그 한 가지가 다섯 번째 차원인 **Observability**입니다. 이번 단원은 빌트인 tracing·자체 DB 저장·full session 모니터링·메트릭과 알람을 한 묶음으로 보고, 이 차원을 다섯 번째이자 마지막 자리로 두는 까닭을 정리하겠습니다.

먼저 한 줄을 머리에 새기고 시작하겠습니다. 비결정적 에이전트에서는 observability 없이는 reliability가 없다는 한 줄입니다. Palantir의 표준이 다른 어떤 차원보다 이 자리를 강하게 못 박는 까닭은, 에이전트라는 시스템의 본성과 닿아 있습니다. 1.1.2에서 본 비결정성은 같은 입력에서 다른 출력이 나올 수 있다는 성질이었습니다. 같은 사용자가 같은 질문을 같은 시각에 두 번 던져도 에이전트가 가는 길이 다를 수 있다는 뜻입니다. 그 길은 거대한 의사결정 공간 위에 무수히 많이 그려질 수 있고, 그 가운데 한 길을 따라 도구가 호출되고 자료가 모이고 응답이 만들어집니다. 결과만 보면 두 번의 응답이 비슷해 보이지만, 안에서는 전혀 다른 길을 갔을 수 있습니다.

이 상황에서 사용자가 '응답이 이상하다'고 신고했을 때, 무엇을 했고 왜 했고 각 단계에서 어떤 정보를 썼는지 보이지 않으면 디버깅은 시작조차 할 수 없습니다. 5.1.2에서 본 Anthropic의 'production tracing 도입 전후' 이야기는 이 자리를 가장 또렷이 보여 줍니다. tracing이 없을 때는 검색 쿼리가 나뉘었는지, 소스 선택이 나뉘었는지, 도구 실패였는지조차 구분할 수 없었고, tracing을 넣고서야 체계적 진단이 가능해졌습니다. Palantir 표준이 이 차원을 production의 baseline이라고 부르는 한 줄, 즉 nice-to-have가 아니라 baseline이라는 한 줄이 가리키는 자리가 여기입니다. 가질 수 있으면 좋은 것이 아니라 없으면 production이라고 부를 수 없는 토대라는 뜻입니다.

이제 이 차원이 통제하는 항목을 하나씩 풀어 보겠습니다. 첫 번째는 **빌트인 tracing**입니다. 빌트인이라는

말은 관측을 사후에 덧붙이는 것이 아니라 runtime 자체에 처음부터 박아 둔다는 뜻입니다. 3.2.2에서 다룬 trace-span 데이터 모델이 이 자리에서 다시 등장합니다. 한 사용자의 한 번의 요청이 들어오면 그 요청에 trace ID 하나가 붙고, 모델 호출·도구 호출·메모리 접근·HITL 승인 같은 모든 단계가 그 trace 아래의 span으로 차곡차곡 쌓입니다. 빌트인이라는 한정이 중요한 까닭은, 사후에 덧붙이는 관측은 빠지는 자리가 반드시 생기기 때문입니다. 어떤 도구는 누락되고 어떤 호출은 다른 형식으로 남고 어떤 단계는 아예 보이지 않습니다. 처음부터 runtime이 모든 호출에 자동으로 trace를 부착하면 빠짐이 사라집니다.

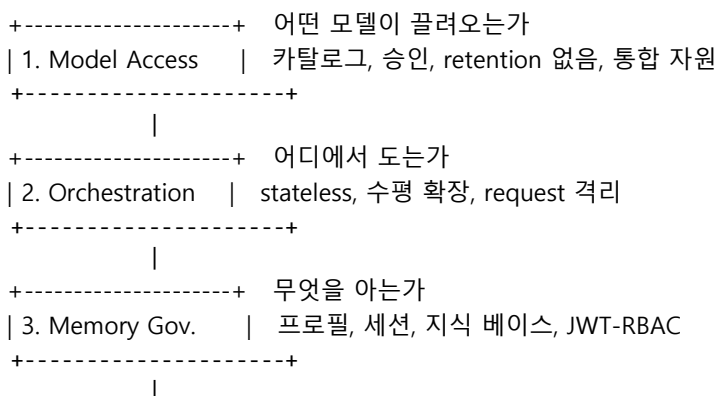
두 번째 항목은 **자체 DB 저장**입니다. 관측 데이터가 모델 회사나 외부 SaaS 쪽에 가서 쌓이는 것이 아니라, 조직이 통제하는 데이터베이스 안에 머문다는 뜻입니다. 이 자리는 3.4.1에서 본 Langfuse의 자체 호스팅과 데이터 주권 논의와 같은 줄기 위에 있습니다. 관측 데이터에는 사용자의 질문, 모델의 응답, 도구로 흘러간 인자 같은 민감한 정보가 모두 들어 있어, 이 데이터가 외부로 빠져 나가면 정작 5.5.1의 retention 없음 정책이 우회되는 일이 됩니다. 한국의 망분리 환경, 금융권의 데이터 거주성 요건, 공공의 클라우드 보안인증을 만족하려면 관측 데이터가 사내 저장소 안에서만 머물러야 합니다.

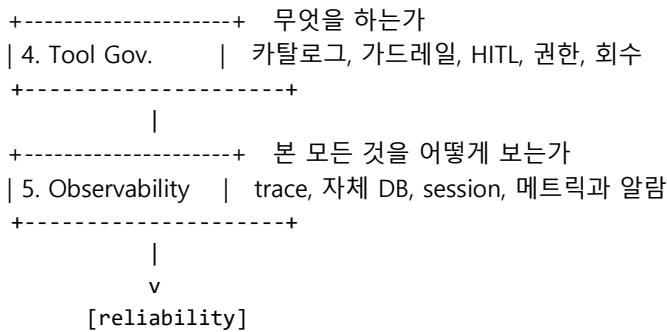
세 번째 항목인 **full session 모니터링**은 한 사용자의 한 세션을 처음부터 끝까지 이어서 본다는 뜻입니다. trace가 한 번의 요청이라는 단위였다면, session은 그 위의 단위입니다. 사용자가 한 번 들어와 여러 번의 요청을 주고받고 나가는 흐름 전체가 한 세션이 됩니다. full session 모니터링은 이 흐름을 끊어서 보지 않고 한 줄기로 이어 봅니다. 5.1.2에서 짚은 emergent behavior는 한 번의 요청 안에서가 아니라 여러 요청에 걸쳐 드러나는 일이 많고, 1.2.2의 planning loop도 한 번의 호출이 아니라 같은 단계가 반복되는 패턴 위에서 보입니다. 세션 단위의 시야가 없으면 이런 패턴은 통계의 잡음으로 묻혀 버립니다.

네 번째 항목은 **메트릭과 알람**입니다. trace와 session이 사후 분석을 위한 시야였다면, 메트릭은 시스템의 건강을 실시간으로 재는 숫자입니다. 토큰 사용량, 도구 호출 빈도, HITL 트리거 비율, 평균 응답 시간, 오류율 같은 지표가 한 곳에 모여 흐릅니다. 알람은 이 메트릭이 정해 둔 임계치를 넘었을 때 사람을 호출하는 자리입니다. 3.1.3에서 다룬 회귀 탐지가 이 자리에서 다시 만나고, 4.2의 circuit breaker가 임계치를 따라 자동으로 작동하는 자리도 메트릭 위에 얹혀 있습니다. 메트릭과 알람은 trace와 session이 사후에 답하는 '무슨 일이 있었나'라는 질문 옆에, '지금 무슨 일이 일어나고 있나'라는 질문에 답하는 자리입니다.

다섯 번째이자 이 차원의 가장 핵심적인 자리에 놓인 원칙이 **관측 없으면 reliability 없다**는 한 줄입니다. 이 한 줄을 다시 한번 풀어 보겠습니다. 관측은 시스템이 잘 도는지 보는 일에 그치지 않습니다. 관측이 있어야 평가가 데이터를 받을 수 있고, 평가가 있어야 회귀를 잡을 수 있고, 회귀를 잡아야 신뢰성을 약속할 수 있습니다. 그래서 이 차원은 위로는 1단원의 평가 패턴과 2~3단원의 평가·관측 스택을, 옆으로는 4단원의 신뢰성 패턴을, 아래로는 본 차원의 앞 네 자리를 모두 잇는 척추가 됩니다.

다섯 차원을 한 그림으로 마지막에 묶어 두겠습니다.





이 그림에서 다섯 번째 차원은 다른 네 차원 옆에 나란히 선 항목이 아니라, 그 위에 가로로 얹혀 모든 층을 비춰 주는 조명에 가깝습니다. 1번 차원에서 모델이 어떻게 끌려왔는지, 2번 차원에서 어떤 인스턴스가 그 호출을 받았는지, 3번 차원에서 어떤 토큰으로 어떤 메모리를 읽었는지, 4번 차원에서 어떤 도구가 어떤 인자로 어떻게 불렸는지가 모두 5번 차원의 trace 안에 한 줄기로 묶입니다. 한 줄기로 묶이지 않으면 위 네 층의 통제가 제대로 작동했는지조차 사후에 증명할 수 없습니다.

한국 규제 환경에서 이 차원이 어떻게 적용되는지 정리하면, 금융권은 감독 당국의 사후 점검에 응하기 위해 trace와 감사 로그를 일정 기간 보관해야 하고, 그 보관처가 사외 SaaS여서는 안 됩니다. 공공 분야는 어떤 사용자가 어떤 자료를 어떤 결정으로 받았는지가 거꾸로 추적될 수 있어야 하고, 그 추적의 정확성을 책임지는 담당자가 정해져 있어야 합니다. 일반 기업이라도 개인정보를 다루는 앱은 사용자의 데이터 삭제 요청을 처리한 사실 자체를 trace에 남겨야 합니다. 다섯 번째 차원이 비어 있으면 앞 네 차원이 잘 작동했다는 사실 자체를 증명할 길이 사라집니다.

본인의 시스템을 self-audit해 볼 질문 세 가지를 두겠습니다. 첫째, 사용자가 한 시간 전의 어떤 응답에 문제를 제기했을 때, 그 응답을 만들어 낸 trace를 한 번의 조회로 꺼내 모델 호출·도구 호출·메모리 접근까지 한 줄기로 보여 줄 수 있습니까. 둘째, 본인의 관측 데이터가 사외 SaaS에 가 있다면 그 사실을 회사의 데이터 거주성 정책 담당자가 알고 있고 승인한 상태입니까. 셋째, 다섯 차원 가운데 어느 한 차원에서 통제가 우회된 사건이 일어났을 때, 그 사건을 다섯 번째 차원의 데이터만으로 사후에 재구성할 수 있습니까. 세 질문 가운데 어느 하나라도 답이 흐릿하면, 그 자리가 곧 reliability의 가장 약한 고리입니다.

정리하면, Observability 차원은 빌트인 tracing·자체 DB 저장·full session 모니터링·메트릭과 알람을 한 묶음으로 갖추어, 앞 네 차원에서 일어난 모든 일을 한 줄기로 비춰 주는 층입니다. 이 차원이 production의 baseline인 까닭은 비결정적 에이전트가 거대한 의사결정 공간에서 어떤 길을 갔는지를 사후에라도 보지 못하면 신뢰성을 약속할 길이 없기 때문이고, 그래서 관측 없으면 reliability 없다는 한 줄이 다섯 차원 전체를 떠받치는 마지막 척추가 됩니다.

여기까지가 5단원 전체에서 살펴본 다섯 사례의 마지막 자리입니다. Anthropic의 멀티 에이전트 구조와 production tracing, Replit의 hybrid 평가 스택과 reactive eval, JPMorgan의 중앙 플랫폼과 단일 API 라우팅, Klarna의 단일 메트릭 함정과 hybrid 회귀, 그리고 Palantir의 다섯 차원 framework가 한 곳에 모여 본인의 시스템을 점검할 도면이 되었습니다.

다음 단원인 6.1.1에서는 이 다섯 차원을 본인의 시스템에 그대로 얹어 self-audit하는 절차를 다루겠습니다. 차원별 점검 질문을 Yes-No-Partial로 분류하고, 부족한 자리에 우선순위를 매기며, 단기·중기 보강 계획을 세우는 흐름이 그 자리의 핵심입니다. 5단원의 사례 정리가 6단원에서는 본인의 도면 위로 옮겨 가는 셈입니다.

이 단원을 마치면 Observability 차원이 정의하는 빌트인 tracing·자체 DB 저장·full session 모니터링·메트릭과 알람의 통제 항목을 설명하고, 이 차원이 앞 네 차원 위에 가로로 얹혀 모든 층을

비추는 조명 역할을 한다는 점을 짚으며, 관측 없으면 reliability 없다는 원칙이 production의 baseline이라는 한 줄을 본인의 시스템에 옮겨 self-audit할 수 있습니다.

6 실전 적용: 자가 진단과 도입 로드맵

본인의 에이전트 시스템을 5차원으로 self-audit하고, 평가·관측·거버넌스 도입 로드맵을 설계하며, 면접·문서에서 일관된 framework로 설명할 수 있다.

이 Phase는 다음 섹션으로 구성됩니다.

6.1 자가 진단

6.1.1 Palantir 5차원 Self-Audit

6.1.2 평가·관측 커버리지 점검

6.2 도입 로드맵

6.2.1 단계별 도입 우선순위

6.2.2 KPI 설계

6.2.3 거버넌스와 속도의 균형

6.3 활용

6.3.1 시스템 설명 Framework

6.3.2 케이스 인용과 Trade-off 설명

6.1 자가 진단

이 섹션의 토픽과 학습 목표는 다음과 같습니다.

6.1.1 Palantir 5차원 Self-Audit — Palantir의 5개 차원으로 자체 시스템을 self-audit하고 부족한 차원을 식별할 수 있다

6.1.2 평가·관측 커버리지 점검 — 본인 시스템의 평가·관측 커버리지를 매트릭스로 점검하고 공백을 식별할 수 있다

6.1.1 Palantir 5차원 Self-Audit

지금까지 다섯 단원에 걸쳐 다른 회사들의 사례를 들여다봤습니다. Anthropic의 멀티 에이전트 구조, Replit의 hybrid 평가 스택, JPMorgan의 중앙 플랫폼, Klarna의 단일 메트릭 함정, 그리고 Palantir가 정리한 다섯 차원이 차례로 지나갔습니다. 5단원의 마지막 단원에서 다섯 차원이 본인의 시스템을 점검할 도면이 된다고 적었는데, 이번 단원이 바로 그 도면을 펼쳐서 본인의 시스템 위에 한 줄씩 얹어 보는 자리입니다. **Self-audit**이라는 말은 외부 감사관을 부르지 않고 본인이 본인의 시스템을 같은 기준으로 점검한다는 뜻입니다. 이번 단원의 목표는 다섯 차원을 본인이 LangGraph로 짠 시스템 위에서 한 차원씩 고집어 내고, 점검 결과를 우선순위가 매겨진 보강 계획으로 옮기는 것입니다.

먼저 self-audit의 출발점이 되는 분류 체계부터 두겠습니다. 차원별 질문에 답할 때 흔히 빠지는 함정은 'Yes 또는 No'의 이분법입니다. 실전 시스템에서는 어떤 통제가 일부 자리에서만 작동하거나, 형식상으로는 있지만 실제로 호출되지 않는 경우가 많습니다. 이 사잇값을 잡기 위해 점검 답안을 **Yes, No, Partial** 세 단계로 나눕니다. Yes는 통제가 모든 호출에 일관되게 적용되고 사후 증명까지 가능한 상태이고, No는 통제가 아예 없거나 켜져 있지 않은 상태이며, Partial은 일부 도구·일부 사용자·일부 환경에서만 작동하거나 메트릭은 쌓이지만 알람이 비어 있는 식의 어중간한 상태입니다. Yes-No-Partial의 세 가지 답이 다섯 차원에 매겨지면 그 자체로 본인의 시스템 지도 한 장이 됩니다.

이제 차원별 점검 질문을 한 줄씩 풀어 보겠습니다. 첫 번째 차원인 **Model Access**에 던질 질문은 이런 것들입니다. 어떤 모델이 등록되어 있는지를 한 곳에 적은 카탈로그가 있고 새 모델 추가 절차가 정해져 있습니까. 모델 제공자에게 retention 없음 정책을 명시한 계약·설정을 갖추고 있습니까. 모델 호출의 비용·토큰·할당량이 한 곳에서 관측되고 통제됩니까. 한 사용자가 임의로 다른 모델을 호출하지 못하도록 통합 자원 관리가 되어 있습니까.

두 번째 차원인 **Stateless Orchestration**의 질문입니다. 에이전트 runtime이 stateless로 설계되어 인스턴스 한 대가 죽어도 다른 인스턴스가 같은 요청을 이어 받을 수 있습니까. 부하가 늘면 인스턴스 수를 늘려 수평으로 확장할 수 있습니까. 한 사용자의 요청이 다른 사용자의 메모리·파일·캐시와 섞이지 않도록 request 단위로 격리되어 있습니까. 다중 테넌시에서 한 테넌트 데이터가 다른 테넌트로 넘어가지 않는다는 사실을 사후 증명할 수 있습니까.

세 번째 차원인 **Memory Governance**의 질문입니다. 사용자 프로필 메모리가 사용자별로 분리되어 다른 호출에 끌려 들어가지 않습니다. 세션 메모리에 만료 정책이 걸려 있고 사용자의 삭제 요청이 trace에 남습니까. 지식 베이스 접근이 JWT 기반 RBAC로 통제되어 사용자의 역할에 맞지 않는 자료가 응답에 섞이지 않습니다. 메모리 접근 자체가 감사 로그에 남아 어떤 호출이 어떤 메모리를 읽었는지 사후 재구성이 가능합니까.

네 번째 차원인 **Tool Governance**의 질문입니다. 도구 카탈로그가 있고 카탈로그에 없는 도구는 호출 자체가 거부됩니다. 도구 호출 직전·직후에 입력·출력 가드레일이 걸려 있습니까. 비가역·고비용 도구에는 HITL 승인 워크플로가 도구 단위로 정의되어 있습니까. 사용자 역할에 따라 부를 수 있는 도구가 권한 매트릭스로 잘려 있고, 잘못 부른 호출이 감사 로그로 회수됩니다.

다섯 번째 차원인 **Observability**의 질문입니다. 모든 모델 호출·도구 호출·메모리 접근에 자동으로 trace가 부착되어 한 번의 요청을 사후에 한 줄기로 꺼낼 수 있습니다. 관측 데이터가 외부 SaaS가 아니라 조직이 통제하는 자체 DB에 저장되고 있습니다. 한 세션을 처음부터 끝까지 이어서 보는 full session 모니터링이 가능합니까. 토큰·비용·HITL 트리거 비율·오류율 같은 핵심 메트릭이 실시간으로 흐르고 임계치 위반 시 알람이 걸려 있습니까.

스무 개 안팎의 질문에 답하고 나면 본인의 시스템에 다음과 같은 표 한 장이 만들어집니다.

#	차원	Q1	Q2	Q3	Q4	
1	Model Access	Yes	No	Partial	Partial	
2	Orchestration	Yes	Yes	Yes	Partial	
3	Memory Gov.	No	No	Partial	No	
4	Tool Gov.	Yes	Partial	No	No	
5	Observability	Partial	No	No	No	

위 표는 예시이지 정답이 아닙니다. 본인의 시스템 위에 답을 채워 넣은 표가 곧 6.2의 도입 로드맵을 짤 때 출발점이 됩니다.

표가 채워지면 다음 단계는 **차원별 우선순위 결정**입니다. 다섯 차원을 동시에 보강하려 들면 비용도 들고 작업도 흐트러집니다. 한 가지 단순한 결정 흐름을 두겠습니다. 먼저 5번 차원인 Observability의 답을 봅니다. 이 차원이 No나 Partial로 가득 차 있다면, 다른 차원의 점검 결과조차 사후에 증명할 길이 없으므로 5번 차원이 최우선이 됩니다. 5.5.5의 한 줄, 즉 관측 없으면 reliability 없다는 원칙이 그대로 우선순위에도 적용되는 셈입니다. 다음으로 4번 차원인 Tool Governance를 봅니다. 도구가 외부에 비가역적 영향을 끼칠 수 있는데 통제가 비어 있으면 사고의 위험이 가장 직접적으로 큼니다. 그 다음 3번

차원인 Memory Governance가 옵니다. 사용자 데이터의 누설은 평판과 규제 양쪽에서 큰 비용을 만듭니다. 1번 차원 Model Access와 2번 차원 Orchestration은 시스템이 일정 규모로 커진 뒤에 더 두드러지므로, 초기 단계에서는 후순위로 두어도 큰 사고로 이어지지 않는 경우가 많습니다.

우선순위가 정해지면 **단기·중기 보강 계획**으로 넘어갑니다. 단기는 한두 주 안에 커질 항목, 중기는 한두 분기 안에 자리잡힐 항목으로 나눕니다. 5번 차원의 빌트인 tracing은 LangSmith나 Langfuse를 붙이는 일이라면 단기, 자체 DB 저장이나 full session 모니터링까지 묶으려면 중기가 됩니다. 4번 차원의 도구 카탈로그를 표로 정리하는 일은 단기지만, 권한 매트릭스를 사용자 토큰과 묶어 작동시키는 일은 인증 시스템과 맞물려 중기 항목이 됩니다.

마지막은 **재진단 주기**입니다. self-audit은 한 번 하고 끝나는 일이 아닙니다. 새 도구·새 모델·새 사용자·새 규제가 붙으면서 답이 계속 바뀝니다. 분기에 한 번씩, 또는 큰 변경이 있을 때마다 같은 질문 표에 다시 답을 매기는 흐름을 두면 시스템이 자라면서 통제가 같이 자라는지 한 표 위에서 계속 보입니다. 분기 첫 주에 self-audit 표를 갱신하고 그 결과를 6.2.2의 KPI 리뷰와 묶어 두면 자연스럽게 운영 리듬 안으로 들어옵니다.

정리하면, Palantir 5차원 self-audit은 Model Access·Stateless Orchestration·Memory Governance·Tool Governance·Observability 다섯 자리에 각 3~5개의 점검 질문을 두고, 본인의 시스템에 Yes·No·Partial로 답해 한 장의 표를 만든 다음, 5번 차원을 출발점으로 하는 우선순위 흐름에 따라 단기·중기 보강 계획을 세우고, 분기 단위 재진단 주기로 표를 갱신하는 절차입니다. 이 단원의 표 한 장이 본인이 LangGraph로 짠 시스템 위에 다른 회사들의 다섯 사례를 그대로 얹어 본 결과물이고, 다음 단원이 이 결과물의 한쪽 면을 더 깊이 들여다보는 자리가 됩니다.

다음 단원인 6.1.2에서는 다섯 차원 가운데 평가와 관측에 직접 닿는 두 자리, 즉 4단원과 2~3단원이 합쳐지는 자리에서 본인 시스템의 평가·관측 커버리지를 매트릭스로 다시 점검하겠습니다. 다섯 차원의 거시 점검이 한 번 끝나고 나면, 평가와 관측이라는 두 면을 매트릭스 형태로 따로 펼쳐 공백을 잡아 보는 흐름이 그 자리의 핵심입니다.

이 단원을 마치면 Palantir의 다섯 차원에 본인의 시스템을 얹어 차원별 3~5개의 점검 질문에 Yes·No·Partial로 답하고, 그 답을 바탕으로 차원별 우선순위와 단기·중기 보강 계획, 그리고 재진단 주기를 한 흐름으로 설계할 수 있습니다.

6.1.2 평가·관측 커버리지 점검

앞 단원에서 다섯 차원의 거시 점검을 한 장의 표로 끝냈습니다. 이번 단원은 그 표의 두 자리, 즉 평가와 관측에 한 걸음 더 다가가서 매트릭스 형태로 다시 펼치는 자리입니다. 5차원 self-audit이 거시 지도였다면, 평가·관측 커버리지 매트릭스는 그 지도의 두 영역만 확대해 격자 위에 놓는 일에 가깝습니다. 이번 단원의 목표는 평가·관측 두 매트릭스를 만들고, 그 위에서 4종 failure mode와 데이터 종류의 공백을 잡아 단계적 보강 로드맵으로 옮기는 것입니다.

먼저 한 가지 도구를 고집어 두겠습니다. **커버리지 매트릭스**라는 말은 격자 한 장에 두 축을 세워 놓고, 각 칸이 채워졌는지 비어 있는지를 한눈에 보는 표입니다. 한 축은 어떤 종류의 평가나 관측을 들고 있느냐이고, 다른 한 축은 그것으로 무엇을 잡으려 하는지입니다. 격자 한 칸이 비어 있으면 그 칸이 곧 본인 시스템의 사각지대입니다.

이제 첫 번째 매트릭스인 **평가 커버리지 매트릭스**를 펼치겠습니다. 가로 축에는 평가의 종류 세 가지를 둡니다. 1.1.4와 2.3에서 본 trajectory 평가, 2단원에서 묶인 offline 평가, 3.1에서 다룬 online 평가가 그 세 자리입니다. 세로 축에는 1.2에서 정리한 4종 failure mode, 즉 tool selection error, planning loop,

hallucinated reasoning, premature termination을 둡니다. 각 칸에는 본인의 시스템에 그 평가가 켜져 있고 그 failure mode를 탐지한 경험이 있다면 Yes, 평가는 있지만 그 failure mode가 잡히지 않으면 Partial, 평가 자체가 없으면 No를 채웁니다.

Failure mode ✕ 평가 종류			
	Offline	Online	Trajectory
Tool selection error	Partial	No	Yes
Planning loop	No	Partial	Yes
Hallucinated reasoning	Partial	No	Partial
Premature termination	Yes	Partial	Partial

위 표는 예시이지 정답이 아닙니다. 본인 시스템의 답을 채워 넣은 12 칸의 표가 곧 평가의 사각지대 지도가 됩니다. tool selection error는 trajectory 평가가 단계별 도구 선택 정확도를 직접 썰 수 있어 가장 자연스럽게 잡히고, planning loop는 단계 수와 반복 패턴으로 보입니다. hallucinated reasoning은 도구 출력과 추론을 비교해야 잡히고 자동화가 까다로워 LLM-as-judge로 보강하지 않으면 Partial에 머무릅니다. premature termination은 완수율 메트릭으로 offline-online 양쪽에서 보이고 trajectory의 어느 단계에서 끊겼는지도 단계 단위로 볼 수 있습니다. 한 줄이 전부 No로 차 있다면 그 failure mode는 시스템 안에서 일어나도 본인이 알 수 없다는 뜻입니다.

이제 두 번째 매트릭스인 **관측 커버리지 매트릭스**로 넘어갑니다. 가로 축에는 3.2.2의 데이터 모델인 trace, metric, log를 둡니다. 세로 축에는 시스템에서 수집할 데이터 종류를 둡니다. 모델 호출, 도구 호출, 메모리 접근, HITL 승인, 사용자 만족도 신호 다섯 가지입니다.

데이터 ✕ 관측 형태			
	Trace	Metric	Log
모델 호출	Yes	Yes	Yes
도구 호출	Yes	Partial	Yes
메모리 접근	No	No	Partial
HITL 승인	Partial	No	Yes
사용자 만족도 신호	No	Partial	No

trace 열은 한 번의 사용자 요청이 한 줄기로 묶이는 자리, metric 열은 그 호출들의 통계가 실시간으로 흐르는 자리, log 열은 사후 검색을 위한 텍스트 흔적이 남는 자리입니다. 메모리 접근이 trace에 남지 않으면 5.5.3의 메모리 감사 항목 자체가 빈 약속이 되고, 사용자 만족도 신호가 어느 형태에도 거의 남지 않으면 1.3에서 본 hidden quality는 데이터로 다룰 수가 없습니다.

두 매트릭스를 묶어 주는 다리로, 4종 failure mode 각각을 어떤 데이터 종류 위에서 잡을 수 있는지 한 표로 매핑해 두면 다음과 같습니다.

Failure mode		탐지에 필수적인 데이터
Tool selection error	도구 호출 trace + 골든 데이터셋	
Planning loop	도구 호출 trace 단계 수 + 비용 알람	
Hallucinated reasoning	도구 결과 + 모델 응답 + judge log	
Premature termination	모델 호출 trace + 완수율 메트릭	

한 failure mode를 잡고 싶다면 그 줄에 적힌 데이터 종류가 두 번째 매트릭스에서 어떤 형태로 쌓이는지를 다시 확인하면 됩니다. 두 매트릭스 사이에 공백이 생긴 자리가 곧 그 failure mode의 사각지대입니다.

이제 채워진 두 매트릭스를 보고 **공백 우선순위**를 매기는 차례입니다. 결정 흐름은 단순합니다. 첫째, 두 매트릭스 모두에서 한 줄이 거의 No로 비어 있는 자리를 가장 먼저 봅니다. 평가에서도 안 잡히고 관측에도 데이터가 없는 failure mode는 시스템 안에서 일어나도 알 수 없는 사고가 됩니다. 둘째, 비가역 행동에 닿는 줄을 봅니다. 도구 호출 trace가 비어 있다면 외부 시스템에 어떤 명령이 떨어졌는지조차 사후에 모른다는 뜻입니다. 셋째, 사용자 경험에 직접 닿는 줄을 봅니다. 사용자 만족도 신호가 빈 줄이면 1.3.2의 hidden quality와 escalation path는 측정 가능한 지표로 옮겨지지 못한 채 묻힙니다. 넷째, 비용 폭주의 줄을 봅니다. planning loop의 줄이 비어 있으면 청구서가 와서야 사고가 보입니다.

이 우선순위에 따라 **단계적 보강 로드맵**을 세 단계로 자릅니다. 첫 단계는 trace의 빈자리를 메우는 일입니다. 5.5.5의 빌트인 tracing 항목과 곧장 이어지고, LangSmith·Langfuse·OpenTelemetry 가운데 어느 쪽을 쓰든 모든 모델 호출·도구 호출·메모리 접근에 자동으로 trace가 부착되는 상태를 종착점으로 둡니다. 두 번째 단계는 trajectory 평가의 빈자리를 메우는 일입니다. 첫 단계의 trace를 골든 데이터셋과 결합해 단계별 정확도와 단계 수 분포를 자동으로 재는 평가 파이프라인이 종착점입니다. 세 번째 단계는 online 평가와 메트릭의 빈자리를 메우는 일입니다. 실 트래픽 위에 완수율·HITL 트리거 비율·사용자 만족도 신호가 메트릭으로 흐르고, 임계치를 넘으면 알람이 사람을 호출하는 상태가 종착점입니다.

[단계 1]	[단계 2]	[단계 3]
trace 채우기	-> trajectory 평가 채우기	-> online 평가·메트릭 채우기
(관측 trace 열)	(평가 trajectory 열)	(남은 No-Partial)

세 단계를 동시에 진행하지 않는 까닭은, trace가 비어 있는 상태에서 평가만 먼저 붙이면 평가의 입력 자체가 부실해지고, 메트릭부터 붙이면 무엇을 보고 있는지가 흐려지기 때문입니다. 두 매트릭스의 trace 자리는 5번 차원 Observability와, 도구 호출 trace 자리는 4번 차원 Tool Governance의 감사 로그·회수 항목과, 메모리 접근 trace 자리는 3번 차원 Memory Governance의 메모리 감사 항목과 그대로 이어집니다. 거시 점검의 5차원 표와 미시 점검의 두 매트릭스가 같은 시스템의 두 단면이라는 사실이 이 매핑에서 보입니다.

정리하면, 평가·관측 커버리지 점검은 4종 failure mode×offline-online-trajectory 평가의 4×3 매트릭스와 다섯 데이터 종류×trace-metric-log의 5×3 매트릭스 두 장을 본인의 시스템 위에 채워 공백을 표시하고, 두 매트릭스 사이의 다리 역할을 하는 failure mode별 데이터 매핑 표로 어떤 자리가 어떤 failure를 못 잡고 있는지 짚은 다음, trace-trajectory 평가-online 평가의 세 단계로 보강 로드맵을 자르는 흐름입니다. 이 두 매트릭스가 6.1.1의 5차원 표와 한 묶음이 되어 본인 시스템의 거시·미시 두 점검 도구가 갖춰진 셈입니다.

다음 단원인 6.2.1에서는 이 세 도구에서 드러난 공백을 실제로 메우는 도입 순서를 다루겠습니다. 관측 우선이나 평가 우선이나의 트레이드오프, 조직 규모와 단계 결정, 초기 1~2주 내 측정 가능한 가치, 도입 비용과 ROI 추정이 그 자리의 핵심입니다.

이 단원을 마치면 본인 시스템의 평가 커버리지를 4종 failure mode와 offline-online-trajectory의 매트릭스로, 관측 커버리지를 다섯 데이터 종류와 trace-metric-log의 매트릭스로 펼쳐 공백을 식별하고, 두 매트릭스를 묶어 우선순위를 매긴 다음 단계적 보강 로드맵을 설계할 수 있습니다.

6.2 도입 로드맵

이 섹션의 토픽과 학습 목표는 다음과 같습니다.

6.2.1 단계별 도입 우선순위 — 관측·평가·거버넌스 중 어디부터 도입할지 결정하는 의사결정 트리를 설명할 수 있다

6.2.2 KPI 설계 — 에이전트 시스템의 다차원 KPI를 정의하고 단일 메트릭 함정을 피하는 설계를 설명할 수 있다

6.2.3 거버넌스와 속도의 균형 — 거버넌스가 속도를 죽이지 않도록 JPMorgan의 8주 release train 같은 패턴을 적용할 수 있다

6.2.1 단계별 도입 우선순위

자가 진단을 마치고 부족한 차원과 공백을 식별했다고 해서, 그 모든 것을 한 번에 채울 수 있는 팀은 없습니다. 사람도 시간도 예산도 한정되어 있고, 무엇보다 도입 작업 자체가 운영 중인 시스템 위에서 진행되어야 하기 때문에 모든 보강을 동시에 시작하면 그 어떤 보강도 끝나지 않습니다. 이 단원은 평가, 관측, 거버넌스 세 영역 가운데 어디부터 손을 댈지를 정하는 의사결정 트리를 설계합니다.

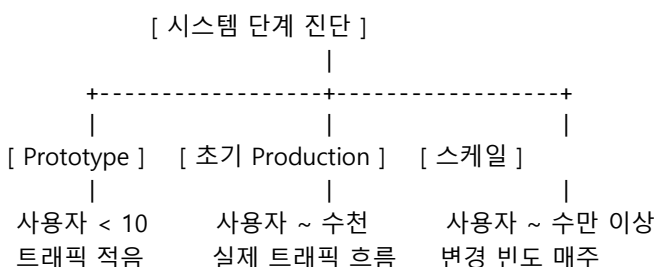
가장 먼저 부딪히는 갈림길은 **관측을 먼저 깔지, 평가를 먼저 만들지**입니다. 두 선택지는 같은 목표를 향하지만 서로 다른 가정을 깔고 있습니다. 관측을 먼저 깔면 실제 트래픽에서 무엇이 일어나는지 보이기 시작합니다. 어떤 도구가 자주 실패하는지, 어디서 토큰이 폭주하는지, 어떤 사용자 패턴이 빙빙 도는 루프를 만드는지가 trace 위에 드러납니다. 반면 평가를 먼저 만들면 회귀를 미리 막을 수 있습니다. 모델을 바꾸거나 프롬프트를 손볼 때마다 이전 성능을 깨뜨리지 않는다는 보증이 생기고, PR 머지 게이트로 활용해 잘못된 변경을 차단할 수 있습니다.

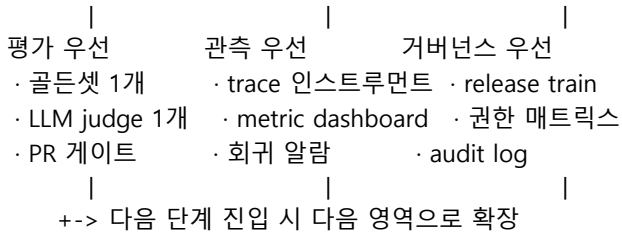
두 선택을 단순한 트레이드오프로 정리하면 다음과 같습니다.

[관측 우선]	[평가 우선]
+ 실제 사용자 행동을 본다	+ 회귀를 사전에 막는다
+ 미지의 실패를 발견할 수 있다	+ 변경의 안전망이 생긴다
- 수치 비교가 어렵다	- 데이터셋이 현실과 어긋날 수 있다
- 회귀 차단 도구가 없다	- 미지의 실패는 잡지 못한다

표만 보면 양쪽 다 일리 있어 보이지만, 실제로는 시스템이 어느 단계에 있느냐에 따라 답이 갈립니다. 1단계 prototype은 사용자가 거의 없고 트래픽도 들쭉날쭉하므로, 관측을 정교하게 깔아도 들여다볼 표본이 부족합니다. 이 시기에는 골든 데이터셋 한 줌과 LLM-as-judge 한두 개로 핵심 시나리오의 회귀를 막는 데 집중하는 편이 더 큰 가치를 만듭니다. 반면 2단계 초기 production은 매일 새로운 사용자가 들어오고 알 수 없는 입력이 쏟아집니다. 이때는 trace와 metric을 먼저 깔아 실제 트래픽 위에서 무엇이 일어나는지 보는 일이 우선입니다. 평가는 그 trace에서 잡힌 실패 케이스를 흡수하면서 점진적으로 키워 가면 됩니다. 3단계 스케일은 이미 두 가지가 모두 깔린 상태에서 거버넌스가 병목이 되는 시기입니다. 모델 변경, 프롬프트 변경, 도구 추가가 매주 일어나면 release train, 권한 매트릭스, audit log 같은 거버넌스 인프라가 필요해집니다.

이를 의사결정 트리로 풀면 다음 그림이 됩니다.





이 트리에서 한 가지 주의할 점은 단계 결정이 절대적인 것이 아니라는 사실입니다. 사용자 수가 적어도 다루는 도메인이 금융이나 의료처럼 위험이 큰 경우에는 prototype 단계에서도 거버넌스 항목을 일부 끌어와야 하고, 반대로 사용자 수가 많아도 단순 검색 보조처럼 위험이 낮은 도메인이라면 거버넌스 도입을 늦출 수 있습니다. 단계는 해상도이고, 도메인 위험은 보정 계수입니다.

조직 규모도 단계 결정에 영향을 줍니다. 5인 미만의 작은 팀이라면 평가 한 줄을 짜고 trace 한 가지를 보내는 것이 동시에 가능합니다. 한 사람이 양쪽을 머리에 담을 수 있기 때문입니다. 그러나 30인을 넘는 팀에서는 평가와 관측의 책임 분리가 일어나고, 두 영역을 동시에 깔면 서로의 작업이 어긋나 통합이 어려워집니다. 이때는 한쪽을 먼저 안정화한 다음 다른 쪽을 시작하는 방식이 합리적입니다. 250K 직원 규모의 엔터프라이즈에서는 한 술 더 떠서, 평가와 관측과 거버넌스를 모두 한 플랫폼에 모은 중앙 팀이 만들고, 사용 팀은 그 위에서 자기 도메인만 정의하는 구조가 자연스럽습니다.

도입을 결정하고 나면 곧바로 따라오는 질문이 **초기 1~2주 안에 어떤 가치를 보여 줄 것인가**입니다. 도입 작업은 길게 늘어지면 동력을 잃습니다. 첫 2주 안에 누구나 볼 수 있는 한 가지 측정 가능한 결과가 나와야 다음 분기의 예산과 인력을 확보할 수 있습니다. Prototype 단계라면 첫 2주의 산출물은 골든셋 50개와 그 위에서 돌아가는 회귀 테스트 한 개로 충분합니다. 초기 production 단계라면 trace dashboard 위에서 가장 큰 비용을 만드는 도구를 한 가지 골라내고, 그 도구의 실패율을 절반으로 줄이는 변경 하나가 첫 2주의 결과가 됩니다. 스케일 단계라면 release train 첫 사이클을 무사히 돌리는 것이 첫 산출물입니다. 어느 경우든 첫 2주 안에 숫자로 말할 수 있는 결과가 있어야 합니다.

마지막으로 비용과 ROI를 추정하는 감각이 필요합니다. 관측 도구는 SaaS 사용 시 월 수백 달러부터 시작하지만 자체 호스팅을 선택하면 인프라 운영 인력 0.2~0.5명이 추가로 들어갑니다. 평가 인프라는 LLM-as-judge 호출 비용 자체는 작지만 데이터셋 라벨링에 들어가는 사람의 시간이 가장 큰 항목입니다. 200건 라벨링에 합의 검수까지 포함하면 보통 1주가 들고, 이는 한 사람의 인건비에 직접 연동됩니다. 거버넌스는 도입 시점에는 비용으로만 보이지만, 사고가 한 번 났을 때의 복구 비용과 비교하면 ROI가 분명해집니다. 한 번의 비가역 사고는 보통 6~12개월의 거버넌스 투자를 정당화할 만큼의 손실을 만듭니다. 이런 추정을 토대로 도입 제안서를 만들면, 경영진이 결정할 수 있는 형태가 됩니다.

정리하면, 단계별 도입 우선순위는 관측 우선과 평가 우선이라는 두 갈림길에서 시작해 prototype, 초기 production, 스케일이라는 세 단계의 위치에 따라 답이 달라지며, 조직 규모와 도메인 위험으로 보정합니다. 첫 2주 안에 측정 가능한 가치를 만들고, 비용과 ROI를 정량화해 의사결정의 언어로 옮기는 것이 도입을 끝까지 끌고 가는 핵심입니다.

다음 단원인 6.2.2에서는 도입을 통해 만들어진 시스템을 어떤 KPI로 측정하고 운영할지를 다룹니다. 1.3에서 본 단일 메트릭 함정과 Klarna 사례를 다시 끌어와, 효율성과 품질과 안전성 세 축을 균형 있게 묶는 다차원 KPI 설계를 이야기합니다.

이 단원을 마치면 관측·평가·거버넌스 가운데 어디부터 시작할지를 의사결정 트리로 설명할 수 있고, prototype-초기 production-스케일의 단계 진단과 조직 규모를 결합해 도입 순서를 정할 수 있으며, 첫 1~2주의 측정 가능한 산출물과 도입 비용·ROI 추정으로 도입 제안을 정량화할 수 있습니다.

6.2.2 KPI 설계

도입 순서를 정하고 첫 산출물이 나오기 시작하면, 그다음에 따라오는 질문은 무엇을 가지고 시스템의 건강을 판단할 것인가입니다. KPI는 그 판단의 언어이며, 잘못 설계된 KPI는 도입의 방향 자체를 비틀어 놓을 만큼 강한 영향력을 가집니다. 이 단원은 1.3에서 본 단일 메트릭 함정과 Klarna 사례를 다시 가져와, 에이전트 시스템에 적합한 다차원 KPI를 설계하는 방법을 정리합니다.

먼저 KPI라는 용어부터 다시 짚겠습니다. **KPI**는 핵심 성과 지표를 뜻하며, 조직이 자기 시스템의 성공과 실패를 가르는 기준으로 합의한 소수의 숫자를 가리킵니다. 일반적인 메트릭은 수십 개 수백 개로 늘어날 수 있지만, KPI는 의도적으로 작은 집합으로 좁힙니다. 작아야 의사결정의 언어로 쓸 수 있고, 분기마다 점검할 수 있으며, 모든 팀원이 머리에 담을 수 있기 때문입니다.

에이전트 시스템의 KPI는 **효율성, 품질, 안전성 세 축**으로 구성하는 것이 출발점입니다. 효율성은 시스템이 얼마나 빠르고 싸게 일을 끝내는지를 봅니다. 응답 지연, 토큰 사용량, 도구 호출 비용, 단위 작업당 평균 비용 같은 항목이 여기 들어옵니다. 품질은 그 일이 사용자가 원하는 결과를 만들어 냈는지를 봅니다. 완수율, 정답률, 사용자 만족도 점수, escalation 비율 같은 항목입니다. 안전성은 시스템이 해서는 안 될 일을 하지 않았는지를 봅니다. 가드레일 트리거 비율, 비가역 행동의 사람 승인 비율, PII 노출 사고 건수, audit log 누락 건수 같은 항목입니다. 세 축을 표로 정리하면 다음과 같습니다.

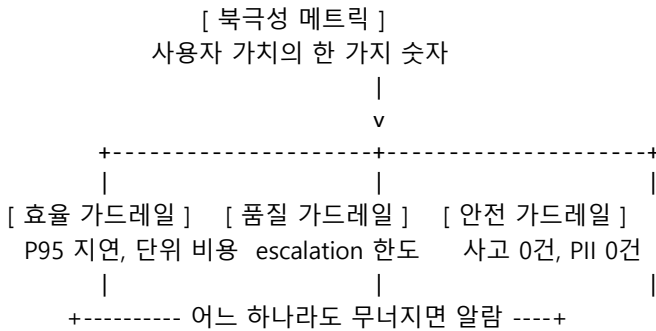
축	대표 정량 지표	대표 정성 지표
효율성	P50 지연, 단위 비용, 토큰/요청, 동시 처리량	운영자 체감 응답성, 유지보수 부담
품질	완수율, 정답률, CSAT, escalation 비율	hidden quality 신호, 공감·안심·상황 인식
안전성	가드레일 트리거 비율, HITL 승인 비율, 사고 수	사고 사례의 심각도, 규제 점검 통과 여부

표에서 정량과 정성이 같은 행에 놓인 까닭은, 한쪽만으로는 그 축을 충분히 설명할 수 없기 때문입니다. 정량 지표는 자동으로 모이고 트렌드를 만들 수 있다는 장점이 있지만, 측정되지 않는 차원을 보지 못합니다. 정성 지표는 사람이 직접 들여다봐야 하고 표본이 작다는 약점이 있지만, 정량으로 잡히지 않는 hidden quality를 확인할 수 있는 유일한 통로입니다. 두 가지를 같은 축에 묶어 두면 한쪽이 무너질 때 다른 쪽이 신호를 줍니다.

세 축 안에서, 그리고 세 축 사이에서 KPI는 서로 충돌합니다. 토큰 비용을 줄이면 응답 품질이 떨어질 수 있고, 가드레일을 강하게 걸면 사용자 만족도가 떨어질 수 있으며, 응답 지연을 줄이려고 작은 모델로 라우팅하면 정답률이 함께 떨어질 수 있습니다. **KPI 사이의 trade-off**는 숨길 일이 아니라 명시할 일입니다. 명시되지 않은 trade-off는 한 축의 압박이 강해지면 다른 축을 조용히 갉아먹습니다. Klarna 사례가 정확히 이 경고였습니다. 230만 건의 대화, 700 FTE 분량의 작업 처리, CSAT 47% 증가, 1천만 달러 절감이라는 효율성 지표가 압도적인 평가 요소가 되면서, 복잡하고 감정적인 케이스에서 사용자가 unheard-unsupported하다고 느끼는 hidden quality가 침식되었고, 결국 인간 상담사 재고용과 hybrid 회귀에 이르렀습니다. 효율성 축의 KPI가 다른 두 축의 KPI를 묶지 않으면 단일 메트릭 최적화의 함정에 빠집니다.

이 함정을 막는 표준 도구가 **북극성 메트릭과 가드레일 메트릭**의 짝입니다. 북극성 메트릭은 조직이 길게

보았을 때 향하고 싶은 한 가지 지표입니다. 사용자에게 가장 큰 가치를 만드는 단 하나의 숫자에 가깝습니다. 예를 들어 상권 분석 에이전트라면 '의사결정에 실제 활용된 보고서 비율' 같은 지표가 북극성이 될 수 있고, 코드 에이전트라면 '머지된 PR 비율'이 북극성이 될 수 있습니다. 가드레일 메트릭은 그 북극성을 향해 달릴 때 깨뜨려서는 안 되는 한계선을 정한 지표 묶음입니다. P95 지연이 X초를 넘지 않을 것, 사용자 escalation 비율이 Y%를 넘지 않을 것, 비가역 행동의 사람 승인 누락이 0건일 것 같은 항목입니다. 북극성은 방향을 주고 가드레일은 추락을 막습니다. 둘을 함께 두면 효율성·품질·안전성 세 축이 균형을 잃지 않습니다. 그림으로 옮기면 다음과 같습니다.



KPI는 만든 순간이 아니라 운영하는 동안에 의미를 가집니다. 그래서 **분기·월별 리뷰** 주기를 같이 정해 두어야 합니다. 월별 리뷰는 트렌드를 점검하는 자리입니다. 북극성과 가드레일이 모두 정상 범위인지 확인하고, 한 축이 흔들리고 있다면 어떤 변경이 그 흔들림을 만들었는지 그달의 release 노트와 맞춰 봅니다. 분기별 리뷰는 KPI 자체의 적정성을 점검하는 자리입니다. 시스템과 사용자가 변하면 같은 메트릭이 다른 의미를 갖게 되고, 어떤 메트릭은 천장에 닿아 더 이상 변별력이 없어집니다. 분기 리뷰에서는 KPI를 새로 정의하거나, 기존 KPI 가운데 하나를 빼고 새 KPI를 더하는 결정을 내립니다. 운영 호흡으로 정리하면 다음과 같습니다.

주 1회 : metric dashboard 자동 알람 점검
 월 1회 : 트렌드 리뷰, 가드레일 위반 사후 검토
 분기 1회: KPI 자체의 재정, 북극성 후보 토론
 연 1회 : 효율·품질·안전 세 축 비중 조정

마지막으로 KPI를 정성 지표와 묶을 때의 실무 팁을 짚겠습니다. 정성 지표는 자동화하기 어렵다는 이유로 종종 미뤄지는데, 미루면 단일 메트릭 함정으로 곧장 들어갑니다. 가장 가벼운 형태는 분기 리뷰 자리에서 사람 평가자가 무작위 표본 30~50건을 직접 읽고 점수를 매기는 패널 평가입니다. 비용은 한 분기당 한두 명이 하루 쓰는 정도로 가볍고, 정량 지표가 가리키지 못하는 hidden quality 신호를 잡습니다. 이 패널 평가의 결과는 분기 리뷰의 토론 재료가 되어 KPI 재정의로 이어집니다.

정리하면, KPI 설계는 효율성·품질·안전성 세 축을 정량과 정성으로 함께 채우고, 축 사이의 trade-off를 명시하며, 북극성 메트릭과 가드레일 메트릭의 짝으로 단일 메트릭 함정을 막는 작업입니다. 월별과 분기별 리뷰 주기를 미리 정해 KPI가 운영의 언어로 살아 있도록 하고, 패널 평가로 정성 신호를 분기마다 끌어들이면 Klarna가 빠진 함정에서 벗어날 수 있습니다.

다음 단원인 6.2.3에서는 KPI를 둘러싼 거버넌스가 시스템의 속도를 죽이지 않도록 균형을 잡는 패턴을 다룹니다. JPMorgan의 8주 update cadence와 release train 같은 도구를 한국 대기업·금융권 환경에 맞춰 적용하는 방법을 살펴봅니다.

이 단원을 마치면 효율성·품질·안전성 세 축의 KPI를 정량과 정성으로 균형 있게 설계할 수 있고, 북극성 메트릭과 가드레일 메트릭을 묶어 단일 메트릭 함정을 피하는 구조를 그릴 수 있으며, 월별·분기별 리뷰 주기와 패널 평가로 KPI를 살아 있는 운영 도구로 유지할 수 있습니다.

6.2.3 거버넌스와 속도의 균형

KPI를 정하고 운영 호흡을 잡고 나면, 마지막으로 마주치는 질문은 거버넌스가 속도를 죽이지 않게 하는 방법입니다. 거버넌스는 사고를 막아 주지만, 잘못 설계하면 변경을 검토하는 데에만 몇 주가 걸리고, 모델 하나를 바꾸려고 위원회 세 곳을 거쳐야 하며, PR 한 줄을 머지하는 데 결재가 다섯 단계 필요해집니다. 이렇게 되면 도입 자체가 의미를 잃습니다. 이 단원은 거버넌스의 보호와 변경 속도를 동시에 살리는 설계 패턴을 정리합니다.

표준 답안의 중심에는 **JPMorgan의 8주 update cadence**가 있습니다. 5.3.3에서 본 것처럼 JPMorgan LLM Suite는 250K 직원 규모의 거버넌스 플랫폼을 운영하면서도 8주 단위로 시스템 전체를 갱신합니다. 8주라는 숫자는 두 가지 압력 사이의 절충점입니다. 한쪽에는 매일이라도 모델과 프롬프트를 바꾸고 싶은 개발 속도의 압력이 있고, 다른 쪽에는 변경마다 보안 검토, 컴플라이언스 검토, 모델 카드 갱신, 사용자 영향 평가를 모두 거쳐야 하는 거버넌스의 압력이 있습니다. 매일 바꾸면 거버넌스가 무너지고, 6개월에 한 번씩만 바꾸면 시장 변화에 끌려갑니다. 8주는 두 압력이 만나는 지점입니다.

8주 cadence는 단순히 시간 단위가 아니라 **고정 release train**이라는 운영 모델 위에서 작동합니다. Release train은 정해진 출발 시각에 출발하는 기차에 비유한 모델로, 그 시각에 준비된 변경들만 함께 실어 보내고 늦은 변경은 다음 기차를 기다리는 방식입니다. 변경 단위로 일정을 협상하지 않고, 일정 단위로 변경을 모읍니다. 이렇게 하면 거버넌스 검토가 변경마다 다시 일어나는 대신, 한 사이클에 한 번 일괄로 일어나기 때문에 검토 비용이 일정하게 유지됩니다. 사이클을 그림으로 풀면 다음과 같습니다.



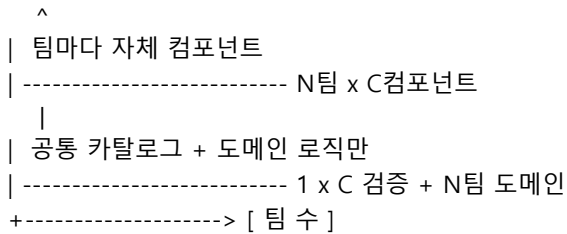
이 구조에서 중요한 점은 한 사이클이 끝나는 즉시 다음 사이클이 시작된다는 사실입니다. 사이클 사이에 빈 시간이 없으면 거버넌스 검토 자체가 일상적인 작업으로 자리잡고, 검토자도 변경 제출자도 익숙한 리듬에 맞춰 움직이게 됩니다. 검토를 특별한 이벤트로 만들면 매번 새로 협상해야 하지만, 정기적인 흐름으로 만들면 검토가 의식이 됩니다.

Release train만으로는 부족하고, **표준 인터페이스**가 그 위에 있어야 합니다. 표준 인터페이스란 모델이나 도구가 바뀌어도 호출하는 쪽의 코드를 바꿀 필요가 없도록 정해 둔 공통의 입출력 규약입니다. JPMorgan LLM Suite에서 단일 API 뒤에 OpenAI, Anthropic 등 여러 모델이 숨어 있는 구조가 정확히 이 인터페이스의 예시입니다. 클라이언트 앱은 모델 이름을 모르고도 호출하고, 라우팅 규칙이 바뀌어도 클라이언트 변경 없이 새로운 모델로 트래픽이 넘어갑니다. 인터페이스가 안정적이면 그 뒤의 변경은 거버넌스 검토만 통과하면 되고, 외부에 미치는 영향이 0이기 때문에 검토 부담이 작아집니다. 반대로 인터페이스가 매번 같이 바뀌면 한 번의 모델 변경이 수십 개 클라이언트의 동시 변경을 강제하고, 그 모든 클라이언트의 거버넌스 검토를 다시 끌어내게 됩니다.

표준 인터페이스가 모델 쪽이라면, **재사용 가능 컴포넌트**는 도구·메모리·관측 쪽의 같은 원리입니다. 모든 팀이 자기 도구와 자기 메모리와 자기 trace 모듈을 따로 만들기 시작하면 거버넌스 검토는 무한히

늘어나고, 각 팀의 보안 수준도 들쭉날쭉해집니다. 중앙 플랫폼 팀이 검증된 컴포넌트의 카탈로그를 만들고, 사용 팀은 그 카탈로그에서 골라 쓰는 구조가 되면, 새로 검증해야 할 항목이 도메인 로직으로 좁혀집니다. 컴포넌트 한 번 검증으로 N개 팀이 같은 보안 수준을 얻는 셈입니다. 이 효과를 그림으로 정리하면 다음과 같습니다.

[거버넌스 부담]



이 그림에서 두 곡선의 차이가 거버넌스 비용이고, 그 차이가 곧 변경 속도입니다. 카탈로그가 잘 만들어진 환경에서는 팀 수가 늘어나도 거버넌스 비용이 거의 일정하게 유지되며, 그래서 8주 cadence가 무너지지 않습니다.

이 패턴을 한국 대기업과 금융권에 옮길 때 한 가지 더 고려해야 할 축이 있습니다. 국내 금융권은 모델 변경, 도구 추가, 데이터 흐름 변경에 대해 자체 위원회와 외부 감독을 동시에 받습니다. 8주 cadence를 그대로 적용하기 어려운 경우가 많고, 12주 또는 분기 단위로 늘어나기도 합니다. 그러나 cadence의 길이가 늘어나는 것 자체는 문제가 아닙니다. 진짜 문제는 cadence 안에서 거버넌스 검토 항목이 매번 처음부터 다시 정의되는 상황입니다. 다음 표는 한국 환경에서 자주 마주치는 상황과, 거버넌스가 속도를 죽이지 않도록 만드는 대응을 정리한 것입니다.

한국 환경의 제약	속도를 살리는 설계
자체 보안 위원회	표준 변경 유형 사전 합의, 유형별 자동 통과 경로
외부 감독·금감원 점검	audit log 표준 포맷 사전 정의, 회기 시 즉시 추출 가능
법무·컴플라이언스 검토	PII redaction 모듈 공통화, 도메인은 매핑만 정의
사고 시 책임 추적	모델·도구·라우팅의 전 변경 이력을 trace에 부착

표의 핵심은 거버넌스 항목을 사전에 표준화해 두면 사이클 안에서 검토가 협상이 아니라 점검이 된다는 점입니다. 점검은 빠르고 협상은 느립니다. 표준 변경 유형, 표준 audit 포맷, 공통 PII 모듈, 변경 이력 자동 부착 같은 항목이 사이클을 무겁게 만들지 않으면서 보호를 유지하는 장치들입니다.

마지막으로 주의할 점은 거버넌스를 강하게 깔수록 거버넌스 자체에 대한 관측이 필요해진다는 사실입니다. 검토가 평균 며칠 걸리는지, 어느 단계에서 가장 자주 막히는지, 어떤 변경 유형이 가장 많이 거절되는지가 보이지 않으면 거버넌스가 자기도 모르게 비대해집니다. 6.2.2에서 정한 KPI에 거버넌스 사이클 시간, 변경 처리량, 검토 거절률 같은 항목을 추가하는 것이 좋습니다. 이렇게 두면 거버넌스가 속도를 죽이는 순간이 숫자로 드러나고, 사이클을 조정할 근거가 생깁니다.

정리하면, 거버넌스와 속도의 균형은 8주 update cadence를 중심으로 고정 release train, 표준

인터페이스, 재사용 가능 컴포넌트라는 세 가지 도구를 함께 깔아야 잡힙니다. 한국 대기업·금융권 환경에서는 cadence가 길어지더라도 검토 항목을 사전에 표준화해 협상 대신 점검으로 바꾸는 것이 핵심이며, 거버넌스 자체에 대한 관측을 KPI에 포함해 비대해짐을 막는 것까지가 설계의 마무리입니다.

다음 단원인 6.3.1에서는 지금까지 다룬 자가 진단, 도입 로드맵, KPI, 거버넌스를 하나의 framework로 묶어 면접과 문서에서 본인의 시스템을 일관되게 설명하는 방법을 정리합니다. Palantir의 5차원을 표준 답변 구조로 활용하는 패턴을 살펴봅니다.

이 단원을 마치면 8주 update cadence와 고정 release train, 표준 인터페이스, 재사용 가능 컴포넌트가 어떻게 거버넌스 부담을 일정하게 유지하는지를 설명할 수 있고, 한국 대기업·금융권 환경에서 cadence를 늘리되 검토를 점검으로 바꾸는 표준화 패턴을 적용할 수 있으며, 거버넌스 사이클 자체를 KPI로 관측해 비대해짐을 막을 수 있습니다.

6.3 활용

이 섹션의 토픽과 학습 목표는 다음과 같습니다.

6.3.1 시스템 설명 Framework — 면접·문서에서 본인의 멀티에이전트 시스템을 5차원 framework로 일관되게 설명할 수 있다

6.3.2 케이스 인용과 Trade-off 설명 — Anthropic·Replit·JPMorgan·Klarna·Palantir 사례를 상황에 맞게 인용하며 trade-off를 설명할 수 있다

6.3.1 시스템 설명 Framework

이 책의 마지막 두 단원은 지금까지 쌓아 온 평가·관측·거버넌스 지식을 외부에 설명하는 자리에서 어떻게 꺼내 쓰지에 관한 것입니다. 잘 만든 시스템도 짧은 시간 안에 일관되게 설명하지 못하면 그 가치가 절반밖에 전달되지 않습니다. 특히 면접이나 기술 문서, 사내 발표처럼 시간이 한정된 자리에서는 같은 내용을 매번 다른 순서로 풀어 놓으면 듣는 쪽이 시스템의 전체 그림을 머릿속에 그리지 못합니다. 이 단원에서는 본인의 멀티에이전트 시스템을 설명할 때 쓸 수 있는 표준 형식을 잡아 보겠습니다.

표준 형식의 뼈대는 5.5에서 본 다섯 개 차원입니다. 모델 접근(Model Access), 오케스트레이션(Orchestration), 메모리 거버넌스(Memory Governance), 도구 거버넌스(Tool Governance), 관측(Observability) 다섯 가지입니다. 이 다섯 차원은 한 회사가 임의로 만든 분류가 아니라, 비결정적 에이전트를 프로덕션에 올릴 때 누구나 마주치는 공통 질문을 정리한 것입니다. 그래서 본인이 어떤 프레임워크로 시스템을 만들었든, 어떤 도메인에 적용했든, 이 다섯 차원에 답할 수 있어야 시스템이 production-ready 라고 말할 수 있습니다. 면접이나 문서에서도 이 다섯 차원을 그대로 답변의 뼈대로 쓰면 듣는 쪽이 누락 없이 전체 그림을 받을 수 있습니다.

다섯 차원을 답할 때의 표준 순서는 다음과 같이 잡습니다. 먼저 모델 접근 차원에서는 어떤 모델을 쓰는지, 그 모델을 단일 API로 추상화했는지, 데이터가 모델 제공자 쪽에 남는지를 답합니다. 다음 오케스트레이션 차원에서는 에이전트가 어떤 구조로 협업하는지, 런타임이 stateless 인지, 요청 단위로 isolation 되는지를 답합니다. 메모리 거버넌스 차원에서는 사용자 프로파일·세션·지식 베이스를 어떻게 분리하는지, 권한 통제는 어떻게 거는지를 답합니다. 도구 거버넌스 차원에서는 어떤 도구를 노출하고 어떤 가드레일과 HITL 승인이 걸려 있는지를 답합니다. 마지막 관측 차원에서는 어떤 trace를 남기고 어떤 메트릭으로 회귀를 잡는지를 답합니다. 이 다섯 가지를 차례로 답하고 나면 시스템 설명이 한 번에 완결됩니다.

한국 기업 면접 환경에서 이 framework가 특히 강한 이유는 빈출 질문 패턴이 모두 이 다섯 차원의 어딘가에 떨어지기 때문입니다. 토스·카카오뱅크·KB·신한처럼 금융권 색이 강한 곳일수록 거버넌스와 관측 차원의 비중이 높습니다. "본인의 멀티에이전트 시스템을 어떻게 production화 했나"라는 큰 질문이 오면 이 다섯 차원으로 답하면 거의 100점입니다. "데이터는 어디에 남나"는 모델 접근 차원, "장애가 나면 어떻게 격리하나"는 오케스트레이션 차원, "사용자 정보 권한은 어떻게 거나"는 메모리 거버넌스 차원, "위험한 행동은 어디서 막나"는 도구 거버넌스 차원, "회귀가 났는지 어떻게 아나"는 관측 차원으로 받아치면 됩니다. 질문이 어느 차원에 속하는지만 빠르게 식별하면 답변의 시작 지점을 망설이지 않게 됩니다.

답변에서 자주 빠지는 요소가 trade-off 명시입니다. 다섯 차원을 다 채워도 "그 결정의 반대편에는 무엇이 있는가"를 빼면 답변이 카탈로그처럼 들립니다. 좋은 답변은 각 차원에서 본인이 어떤 선택을 했고, 그 선택의 반대 옵션은 무엇이었으며, 왜 한쪽을 택했는지를 한 줄씩 붙입니다. 예컨대 메모리 거버넌스 차원에서 "세션 메모리 만료를 24시간으로 잡았다"고만 말하면 평면적이지만, "24시간으로 잡으면 사용자 경험이 매끄럽지만 장기 추적이 약해지고, 더 길게 잡으면 추적은 되지만 잊힐 권리 대응이 복잡해진다, 우리는 잊힐 권리를 우선해 24시간을 택했다"고 말하면 같은 결정이 한 단계 깊어집니다. Klarna 사례에서 본 것처럼 단일 메트릭에 최적화한 시스템은 hidden quality를 잃는데, 답변에서 trade-off를 명시하지 않으면 면접관이 보기에는 단일 차원에 최적화한 사람과 구별이 되지 않습니다.

깊이를 한 단계 더 보여 주는 기법이 역질문입니다. 다섯 차원으로 답을 마친 뒤 "이 회사 환경에서는 어느 차원이 가장 약하다고 보십니까", "현재 운영 중인 LLM 시스템에서 관측 baseline은 어디까지 잡혀 있습니까" 같은 질문을 짧게 던지는 것이 효과적입니다. 역질문은 단순히 시간을 끌기 위한 장치가 아니라, 본인이 다섯 차원의 균형을 의식하고 있고 상대 조직의 맥락에 맞춰 우선순위를 다시 잡을 줄 안다는 신호를 줍니다. 다만 역질문은 답변을 회피하는 도구가 아니므로, 본인의 답을 충분히 한 뒤에 마무리로 한두 개만 붙여야 합니다.

다섯 차원을 답변 형식에 어떻게 녹이는지 짧은 박스로 그려 보면 다음과 같습니다.

[면접 답변 표준 박스]

- (1) 모델 접근 : 단일 API 뒤에 OpenAI·Anthropic 라우팅, 제공자 retention 없음
trade-off: 단일 벤더 종속을 피했지만 라우팅 로직 운영 부담 증가
- (2) 오케스트레이션 : Lead + 병렬 Subagent, stateless runtime, request 단위 isolation
trade-off: 수평 확장 쉬워졌지만 cross-request 캐싱 이득은 포기
- (3) 메모리 거버넌스 : 프로필·세션·지식 베이스 분리, JWT 기반 RBAC
trade-off: 권한 검증 한 단계 늘었지만 잊힐 권리 대응 가능
- (4) 도구 거버넌스 : 가드레일 + 임계치 기반 HITL, 비가역 작업만 승인
trade-off: 전수 승인 대비 사용자 경험 매끄럽지만 임계치 튜닝 필요
- (5) 관측 : OTel 표준 + Langfuse self-host, full session trace
trade-off: 자체 호스팅 운영 비용 있지만 데이터 주권 확보

이 박스 형태는 한 번에 다 외우는 것이 아니라, 본인 시스템의 각 차원을 두 줄(선택과 trade-off) 안에 압축해 두고 면접 직전에 빠르게 점검하는 도구로 씁니다. 차원별로 두 줄씩 다섯 차원이면 열 줄, 시간으로는 2~3분 안에 시스템 전체 그림이 전달됩니다. 그 뒤로 면접관이 어느 차원을 더 깊게 묻든 박스의 한 줄을 확장해 답하면 됩니다.

마지막으로 짚을 점은 이 framework가 본인 시스템에 없는 차원을 숨기는 도구가 아니라는 것입니다. 어떤 차원이 약하면 약한 대로 솔직하게 답하고, 그 차원을 어떤 우선순위로 보강할지 같이 말하는 것이 더 강한 인상을 줍니다. 6.1의 self-audit과 6.2의 도입 로드맵을 함께 묶어서 "현재는 관측 차원이 baseline 수준이고, 다음 분기에 메모리 거버넌스 차원을 보강할 계획이다"라고 답하면, 다섯 차원을 다 채우지

못한 상태라도 의사결정 능력 자체는 충분히 전달됩니다. 거버넌스가 속도를 죽이지 않도록 균형을 잡은 6.2.3의 사고가 답변에 그대로 묻어납니다.

정리하면, 시스템 설명 framework는 5.5의 다섯 차원(모델 접근·오케스트레이션·메모리 거버넌스·도구 거버넌스·관측)을 답변의 뼈대로 쓰고, 각 차원에서 선택과 trade-off를 한 줄씩 붙이며, 마지막에 역질문으로 깊이를 보여 주는 표준 형식입니다. 이 형식을 한 번 잡아 두면 토스·카카오뱅크·KB·신한 같은 한국 금융권 면접에서 쏟아지는 production화 질문들에 일관되게 답할 수 있고, 본인 시스템의 약한 차원도 우선순위 언어로 풀어 낼 수 있습니다.

다음 단원인 6.3.2에서는 이 framework를 답변할 때 어떤 사례를 어느 시점에 인용할지, 그리고 단순 인용을 넘어 비교·대조와 한계까지 함께 제시하는 방법을 정리하고 책 전체를 마무리하겠습니다.

이 단원을 마치면 면접·문서에서 본인의 멀티에이전트 시스템을 다섯 차원 framework로 일관되게 설명하고, 각 차원에서 trade-off를 명시하며, 역질문으로 깊이를 더하는 답변 흐름을 구성할 수 있습니다.

6.3.2 케이스 인용과 Trade-off 설명

앞 단원에서 다섯 차원으로 답하는 표준 형식을 잡았다면, 이 단원에서는 그 답변에 사례를 어떻게 끼워 넣을지를 다룹니다. 사례를 인용하는 까닭은 단순히 박식해 보이기 위해서가 아니라, 본인이 내린 결정의 근거를 외부 레퍼런스에 연결해 청자가 검증 가능한 지점에 놓아 주기 위해서입니다. 같은 결정도 "이렇게 하기로 했다"고 말하는 것과 "JPMorgan이 이런 이유로 이렇게 풀었기에 우리도 그 패턴을 변형해 적용했다"고 말하는 것은 신뢰의 무게가 다릅니다.

이 책에서 본 여섯 사례를 한 문장씩 핵심 교훈으로 압축해 두면 인용이 빨라집니다. Anthropic Multi-Agent Research System은 prototype에서 production으로 갈 때 full production tracing이 들어가야 비로소 실패 원인을 분리할 수 있다는 사례입니다. Replit Agent는 가드레일조차 창의적 prompting으로 뚫리고 prescriptive eval만으로는 잡지 못하는 패턴이 있어 reactive eval과 hybrid 평가 스택이 필요하다는 사례입니다. JPMorgan LLM Suite는 250K 직원 규모에서 모델·보안·관측·거버넌스를 한 플랫폼에 모으고 8주 release train으로 거버넌스와 속도를 동시에 잡은 사례입니다. Klarna는 단일 메트릭에 최적화하면 hidden quality가 무너져 인간 상담사 재고용에 이르고 hybrid 모델로 회귀하게 된다는 반면교사 사례입니다. Palantir AIP는 비결정적 에이전트의 거버넌스를 다섯 차원으로 정의해 사실상 표준이 된 사례입니다. Demystifying Evals는 평가 환경의 isolation과 shared state 차단이 메트릭의 무결성을 좌우한다는 일반화 가이드입니다. 이 여섯 줄을 머릿속에 띄워 두면 어떤 질문이 와도 어느 사례가 적합한지 바로 매핑할 수 있습니다.

상황별 인용 매핑을 더 구체적으로 잡으면 다음과 같습니다. "프로토타입과 프로덕션의 차이가 무엇인가"라는 질문에는 Anthropic의 tracing 도입 전후 사례가 기본형이 됩니다. "평가는 어떻게 만들어 가는가"라는 질문에는 Replit의 reactive eval과 GPT-4o 자동 교체 발견 사례가 가장 효과적입니다. "거버넌스와 속도를 어떻게 같이 가져가는가"라는 질문에는 JPMorgan의 단일 API + 8주 release train이 표준 답안입니다. "단일 메트릭으로 평가하면 안 되는 이유가 무엇인가"라는 질문에는 Klarna의 CSAT 47% 증가와 인간 재고용의 역설이 가장 강합니다. "production 시스템의 거버넌스 차원을 어떻게 정의하는가"라는 질문에는 Palantir의 다섯 차원이 그대로 답이 됩니다. "평가 환경의 무결성은 어떻게 보장하는가"라는 질문에는 Demystifying Evals의 isolated trial 원칙과 git history 사례가 적합합니다. 질문 패턴별로 1번 인용을 미리 잡아 두면 답변에서 망설이는 시간이 사라집니다.

[질문 패턴 → 적절한 사례]

prototype/production 갭 → Anthropic tracing

가드레일/평가의 한계	→ Replit reactive eval
거버넌스 + 속도	→ JPMorgan 8주 release train
단일 메트릭의 함정	→ Klarna hybrid 회귀
거버넌스 5차원	→ Palantir AIP
평가 환경 무결성	→ Demystifying Evals
프라이버시 보존 관측	→ Anthropic 비공개 + 패턴 가시성
멀티에이전트 emergent behavior	→ Anthropic Lead + Subagent

여기서 한 단계를 더 올라가는 기법이 단순 인용에서 비교·대조로 옮기는 것입니다. 단순 인용은 "Klarna 사례처럼 단일 메트릭에 최적화하면 안 됩니다"로 끝나지만, 비교·대조는 "Klarna는 비용 절감을 단일 축으로 잡았다가 hybrid로 회귀했고, 같은 시기에 JPMorgan은 측정 가능한 문서 중심 프로세스부터 단계적으로 도입해 안정화 후에야 post-check로 이동했다, 이 둘의 차이는 다차원 KPI를 설계 단계에 넣었느냐 사후에 발견했느냐의 차이다"로 풀립니다. 비교·대조는 청자에게 사례 두 개를 동시에 활용한다는 인상을 주고, 본인이 사례를 외운 사람이 아니라 사례를 도구로 쓰는 사람이라는 신호를 보냅니다.

또 하나 자주 빠뜨리는 요소가 한계와 반례 함께 제시입니다. 어떤 사례도 만능이 아니므로, 인용한 사례의 한계를 같이 짚어 주는 답변이 더 단단합니다. 예컨대 Palantir의 다섯 차원을 답으로 내놓으면서 "이 다섯 차원은 엔터프라이즈 스케일을 전제로 한 것이라, 5명 이하 스타트업에서 그대로 적용하면 거버넌스 비용이 속도를 압도할 수 있다, 그래서 우리는 관측 차원만 baseline으로 잡고 나머지는 단계적으로 채우고 있다"고 말하면, 같은 사례 인용이지만 사고의 깊이가 다르게 느껴집니다. JPMorgan의 8주 release train도 마찬가지로, 한국의 작은 조직에서는 그대로 옮기면 시한 압박이 부담이 될 수 있고 4주나 격주로 변형해야 한다는 식의 한계가 따라붙어야 인용이 살아납니다. Klarna 사례는 강력하지만 모든 도메인에 그대로 일반화되지는 않으며, B2B 워크플로 자동화처럼 정량 정확도가 더 중요한 영역에서는 hidden quality 차원의 비중이 다르다는 점도 짚을 수 있습니다.

마지막으로 사례를 본인 시스템에 옮기는 방법을 풀어 두는 것이 인용의 마무리입니다. 사례는 인용에서 끝나지 않고 본인 결정과 어떻게 연결되는지 보여 줘야 답변이 닫힙니다. 예컨대 Anthropic의 비공개 + 패턴 가시성 패턴은 한국 상권 분석처럼 소상공인 데이터를 다루는 시스템에서 그대로 차용 가능하다는 식으로, 사례의 어느 조각을 본인 시스템에 옮겼고 어디는 변형했는지를 한두 문장으로 정리하면 됩니다. 6.1의 self-audit으로 약한 차원을 짚고, 6.2의 도입 로드맵으로 그 차원을 어느 시점까지 어떤 사례를 변형해 채울지를 말하면, 이 책 전체가 답변 한 덩어리에 녹아듭니다.

이제 책 전체를 한 번 회고하겠습니다. Phase 1에서는 에이전트 평가가 LLM 평가와 어떻게 다른지, 비결정성과 trajectory가 만드는 평가의 어려움, tool selection error-planning loop-hallucinated reasoning-premature termination이라는 failure mode taxonomy, 그리고 Klarna 사례로 본 단일 메트릭의 함정을 다뤘습니다. Phase 2에서는 골든 데이터셋 구축, LLM-as-Judge의 편향과 보정, trajectory 평가의 단계별 정확도, τ -bench·AgentBench·SWE-bench 학술 벤치마크, 그리고 평가 환경의 isolation을 통해 offline 평가 파이프라인을 설계했습니다. Phase 3에서는 실 트래픽 기반 online 평가, reactive vs prescriptive, LangSmith와 Langfuse의 trace 모델, 한국 기업의 의사결정 차원, OpenTelemetry GenAI semantic conventions를 따라 무벤더 락인 관측 스택까지 짚습니다. Phase 4에서는 tiered autonomy, circuit breaker, fallback routing, 임계치 기반 HITL과 escalation path로 신뢰성과 안전 패턴을 묶었습니다. Phase 5에서는 Anthropic·Replit·JPMorgan·Klarna·Palantir 다섯 케이스 스터디로 패턴을 추출했고, Phase 6에서는 self-audit·도입 로드맵·KPI 설계·거버넌스와 속도의 균형을 거쳐 마지막에 시스템 설명 framework와 케이스 인용까지 도착했습니다. 평가에서 시작해 관측을 거쳐 거버넌스로, 거버넌스에서 다시 실전 활용으로 이어지는 한 호가 그려집니다.

정리하면, 케이스 인용은 단순히 사례를 외우는 일이 아니라 질문 패턴별

매핑(Anthropic·Replit·JPMorgan·Klarna·Palantir·Demystifying Evals 여섯 줄), 비교·대조로 사고를 한 단계 올리기, 한계와 반례를 함께 제시해 단단함을 더하기, 그리고 본인 시스템에 어떻게 옮길지를 마지막에 달아 주기로 완성됩니다. 이 흐름이 6.3.1의 다섯 차원 답변과 합쳐지면, 본인의 멀티에이전트 시스템을 production 수준으로 설명하는 표준 답변 한 세트가 갖춰집니다.

이 책을 마치면 비결정적 에이전트 시스템을 평가의 단위와 failure mode taxonomy로 분해하고, offline-online 평가 파이프라인과 LangSmith·Langfuse·OpenTelemetry 기반 관측 스택을 도입 의사결정과 함께 구성하며, tiered autonomy·circuit breaker·fallback·HITL의 신뢰성 패턴을 결합하고, Anthropic·Replit·JPMorgan·Klarna·Palantir 사례에서 추출한 교훈을 본인 시스템에 변형 적용하며, Palantir의 다섯 차원으로 self-audit과 도입 로드맵을 짜고, 면접·문서에서 일관된 framework로 시스템을 설명하면서 trade-off와 한계까지 함께 제시할 수 있습니다. 비결정적 시스템을 프로덕션에 올리는 일은 한 번에 정복되지 않고 평가·관측·거버넌스의 작은 결정들이 누적되며 단단해지는 작업이며, 이 책이 그 누적의 첫 한 호가 되었기를 바랍니다.